
DATA ANALYST

Série 2 — Volume II

Préparation des données

Nettoyage · Transformation · Fusion
Qualité et réconciliation · Documentation

Fiabiliser avant d'analyser,
avec une boutique pour fil rouge.

Table des matières

Introduction : pourquoi l'essentiel du travail se fait avant l'analyse	1
Trois fichiers, trois questions	1
La mauvaise réponse facile	2
Combien de clients ? Le fichier en compte 7 140	2
Combien le Site a-t-il vendu ? L'export répond 25 millions d'euros	2
Le « 80 % » et ce qu'il recouvre	3
Le cycle : préparer, contrôler, documenter	4
Un nettoyage est une suite de décisions défendables	5
Ne pas embellir : l'éthique de la préparation	5
Ce que contient le volume, et comment il prolonge le volume I	6
Carte du volume, données et environnement	7
Carte du volume	7
Les sources désordonnées du volume	8
Le fichier clients : un CRM alimenté par trois sources	9
L'export du site web	9
Les exports de caisse : douze fichiers, trois réglages	10
Le catalogue du fournisseur et le tableur de stocks	11
Les profils clients et les montants saisis	11
Les fichiers de vérité	11
L'environnement de travail	12
Python, R et SQL	12
Le tableur : Excel, LibreOffice et nos maquettes	12
Les captures d'écran	13
Régénérer les données et refaire les calculs	13
Conventions du volume	13
Chapitre 1 : Nettoyage des données	14
Pourquoi le nettoyage prend tant de temps	14
Le chemin de ce chapitre	15
Les données du chapitre	15
1.1 Valeurs manquantes	16
1.1.1 Repérer ce qui manque	16
1.1.2 Absence, zéro et code spécial	17
1.1.3 Pourquoi une valeur manque : trois mécanismes	18
1.1.4 Ce que coûte une suppression	20
1.1.5 Quatre options, et comment choisir	21
1.2 Valeurs aberrantes	22
1.2.1 Erreur, extrême réel ou cas rare ?	22

1.2.2 Trois méthodes statistiques	22
1.2.3 Les règles métier : regarder la relation, pas la valeur	23
1.2.4 Que faire d'une valeur aberrante ?	25
1.3 Doublons	26
1.3.1 Exact ou approché, clé naturelle ou clé technique	26
1.3.2 Les doublons d'export de la plateforme web	28
1.3.3 Les doublons de scan en caisse : quand la copie est peut-être légitime	29
1.3.4 Quel enregistrement garder ?	30
1.4 Incohérences et erreurs de format	31
1.4.1 Les types : lire d'abord en texte, convertir explicitement	31
1.4.2 Les unités mélangées : le changement de septembre	32
1.4.3 Les dates ambiguës : jour/mois ou mois/jour ?	34
1.4.4 Les catégories écrites de six façons	35
1.4.5 Les règles de cohérence entre colonnes	36
1.4.6 Quand le schéma change d'un fichier à l'autre	37
1.5 ⊕ Pour aller plus loin : texte, dates et heures, encodage, données multilingues	39
1.5.1 Espaces, casse et accents : comparer sans se tromper	40
1.5.2 Unicode : le même caractère de deux façons, et le mojibake	40
1.5.3 Caractères invisibles et expressions régulières	41
1.5.4 Dates et heures : fuseaux et heure d'été	43
1.5.5 Encodage : cp1252, UTF-8 et la marque d'ordre des octets	43
1.5.6 Données multilingues : l'arabe dans une colonne de villes	44
1.6 ⊕ Pour aller plus loin : stratégies d'imputation et leur impact	45
1.6.1 Imputer, c'est prédire : ce qu'on veut préserver	45
1.6.2 Le revenu : une variable peu prévisible	46
1.6.3 La dépense : quand les autres colonnes savent	47
1.6.4 La satisfaction : aucune méthode ne répare un MNAR	48
1.6.5 L'imputation multiple : dire l'incertitude	49
1.6.6 L'indicateur de manquant et la décision finale	50
Bilan du chapitre 1	51
Chapitre 2 : Transformation et fusion des données	53
Quatre gestes, un vocabulaire	53
Le chemin de ce chapitre	54
Les données du chapitre	55
2.1 Création et dérivation de variables	56
2.1.1 Une variable dérivée est une décision	56
2.1.2 Calculs simples : marges, ratios, montants	56
2.1.3 Les dates : en extraire des morceaux	58
2.1.4 Les classes : découper une variable continue	59
2.1.5 Indicateurs, rangs et variables retardées	60
2.1.6 Du texte à la clé	61
2.1.7 Trois pièges, et comment documenter	61
2.2 Fusion et jointure de jeux de données	63
2.2.1 Quatre manières de recoller	63
2.2.2 La cardinalité et le contrôle des effectifs	64
2.2.3 Empiler les douze fichiers de la caisse	65
2.2.4 Lire l'export du site	67
2.2.5 Relier les ventes aux produits : la jointure qui multiplie	70
2.2.6 Harmoniser et empiler : la table des ventes	72
2.2.7 Contrôler : comparer à la base, puis ouvrir la vérité	73
2.2.8 Chercher ce qui manque : les anti-jointures	74

2.2.9 Le même résultat avec trois outils	75
2.3 Agrégation et restructuration	77
2.3.1 Le grain d'une table et le changement de grain	77
2.3.2 groupby et agg : compter sans compter deux fois	77
2.3.3 transform : un agrégat sans perdre les lignes	79
2.3.4 Du grain ligne au grain commande, puis au grain client	79
2.3.5 Table de faits et dimensions	80
2.3.6 Cumuls, moyennes mobiles et comparaisons dans le temps	81
2.3.7 La somme de contrôle : rien n'a bougé	82
2.4 ⊕ Pour aller plus loin : restructuration, pivot et dépivot	83
2.4.1 Trois fonctions à connaître	83
2.4.2 Le tableur de stocks : lire, nettoyer, dépivoter	85
2.4.3 Joindre deux tables du même grain	87
2.4.4 Le même geste dans les autres outils	87
2.5 ⊕ Pour aller plus loin : appariement approximatif et rapprochement d'enregistrements	88
2.5.1 Pourquoi les clés exactes échouent	89
2.5.2 Normaliser d'abord	90
2.5.3 Mesurer une ressemblance	90
2.5.4 Limiter les comparaisons : le blocage	92
2.5.5 Un score, trois zones	92
2.5.6 Le coût des erreurs décide du seuil	94
2.5.7 Rapprocher le catalogue du fournisseur	94
Bilan du chapitre 2	96
Chapitre 3 : Qualité des données et réconciliation	98
Pourquoi un chapitre sur la qualité ?	98
Le chemin de ce chapitre	99
Les données du chapitre	99
3.1 Dimensions de la qualité : exactitude, complétude, cohérence, actualité	100
3.1.1 Une donnée est de qualité « pour un usage »	100
3.1.2 La complétude : ce qui manque	101
3.1.3 La validité et l'unicité : la forme et les copies	102
3.1.4 La cohérence : ne pas se contredire	102
3.1.5 L'exactitude : proche de la réalité	103
3.1.6 L'actualité : à quel point est-ce récent ?	104
3.1.7 Un tableau de bord de qualité	104
3.1.8 Quatre pièges des indicateurs de qualité	105
3.2 Contrôles de validation	106
3.2.1 Un contrôle est une règle qui renvoie ses échecs	106
3.2.2 Les contrôles de forme : type, format, liste de valeurs	107
3.2.3 Les contrôles de plage et de dépendance entre colonnes	108
3.2.4 Les contrôles d'unicité et de comptage : totaux de contrôle	108
3.2.5 Les contrôles temporels : trous, futur et ruptures	109
3.2.6 Un rapport de contrôles	110
3.2.7 Les mêmes contrôles en SQL	111
3.2.8 Que faire quand un contrôle échoue ?	112
3.3 Réconciliation de sources	113
3.3.1 Deux sources, une mesure	113
3.3.2 La méthode en trois temps	114
3.3.3 Le site contre la base : un écart de quarante fois	114
3.3.4 La caisse contre la base : douze fichiers, trois formats	117
3.3.5 Le catalogue contre les produits : fabriquer la clé	119

3.3.6 Un catalogue des causes d'écart	120
3.3.7 Quand s'arrêter ?	120
3.4 ⊕ Pour aller plus loin : règles de réconciliation, seuils de tolérance, rapports d'exceptions	121
3.4.1 Écrire une règle de réconciliation	121
3.4.2 Les seuils de tolérance	122
3.4.3 Appliquer les règles : avant et après nettoyage	122
3.4.4 Le rapport d'exceptions	123
3.4.5 Trier : où est l'argent ?	124
3.4.6 Résoudre et suivre	125
3.5 ⊕ Pour aller plus loin : cadres de validation, pandera et Great Expectations	126
3.5.1 Pourquoi un cadre de validation ?	126
3.5.2 pandera : un schéma de DataFrame	126
3.5.3 Great Expectations : des attentes, des suites, des rapports	128
3.5.4 Les Data Docs	129
3.5.5 Choisir : fonctions maison, pandera ou Great Expectations ?	130
Bilan du chapitre 3	130
Chapitre 4 : Documentation et dictionnaires de données	133
Pourquoi un analyste documente	134
Le chemin de ce chapitre	135
Les données du chapitre	135
4.1 Documenter les jeux de données et les transformations	135
4.1.1 Ce que l'on documente : une fiche par jeu de données	136
4.1.2 Remplir la fiche du CRM	136
4.1.3 Documenter une transformation : le journal des décisions	137
4.1.4 Un journal écrit par le code	138
4.1.5 Le journal face à la vérité : ce qu'il montre, ce qu'il ne dit pas	140
4.1.6 README, commentaires, noms de fichiers : le reste de la documentation	140
4.1.7 La reproductibilité : tout refaire depuis le brut	141
4.1.8 Bonne et mauvaise documentation	142
4.1.9 Combien documenter ? Proportionner à l'enjeu	142
4.2 Construire un dictionnaire de données	143
4.2.1 À quoi sert un dictionnaire, et ce qu'il contient	143
4.2.2 Partir d'un squelette fabriqué par le code	144
4.2.3 L'enrichir à la main	145
4.2.4 Où ranger le dictionnaire : quelques formats	146
4.2.5 Un dictionnaire qui ment : le vérifier automatiquement	147
4.2.6 Dictionnaire et glossaire métier : une définition unique	148
4.2.7 Trois dictionnaires pour la boutique	149
4.2.8 Faire vivre le dictionnaire : versions et changements	149
4.3 ⊕ Pour aller plus loin : lignage des données et pistes d'audit	150
4.3.1 D'où vient ce chiffre ?	151
4.3.2 Un lignage à la main, au niveau des fichiers	151
4.3.3 Le lignage des colonnes	152
4.3.4 Prouver qu'une entrée n'a pas changé : les empreintes	153
4.3.5 Le journal d'audit : qui, quand, quoi	154
4.3.6 Versionner des données, sans outil lourd	154
4.3.7 Les outils existants (non exécutés)	155
4.3.8 Mini-projet : rejouer un chiffre depuis le brut	155
Bilan du chapitre 4	156

Chapitre 5 : Confidentialité et anonymisation des données	159
Le chemin de ce chapitre	160
Les données du chapitre	160
5.1 Données personnelles et principes	160
5.1.1 Ce qu'est une donnée personnelle	161
5.1.2 Identifiants directs, quasi-identifiants, données sensibles	161
5.1.3 Les principes communs aux cadres de protection	162
5.1.4 Le consentement marketing et ses codages	163
5.1.5 Les droits des personnes : supprimer, c'est d'abord retrouver	164
5.2 Pseudonymisation et anonymisation	165
5.2.1 Pseudonymiser : remplacer l'identifiant par un code	165
5.2.2 Le hachage sans clé ne protège pas	165
5.2.3 Le hachage à clé, et la table de correspondance séparée	166
5.2.4 Autres techniques : ce qu'elles protègent, ce qu'elles coûtent	167
5.2.5 Pseudonymisation n'est pas anonymisation	168
5.3 Mesurer le risque de réidentification	169
5.3.1 Quasi-identifiants et unicité	169
5.3.2 Le k-anonymat	171
5.3.3 Généraliser et supprimer pour atteindre k	171
5.3.4 Ce que l'on perd	172
5.3.5 Le recoupement : un montant et une date suffisent	172
5.3.6 La l-diversité : quand un groupe homogène trahit	174
5.3.7 La confidentialité différentielle, en une page	174
5.4 Les bonnes pratiques de l'analyste	176
5.4.1 Minimiser et séparer : préparer le fichier du prestataire	176
5.4.2 Les petits effectifs : ne pas publier un groupe de trois personnes	177
5.4.3 Les sorties : modèles, graphiques et captures d'écran	178
5.4.4 Accès, journaux et conduite à tenir en cas de fuite	178
5.4.5 La liste de contrôle avant d'envoyer un fichier	179
5.4.6 La personne chargée de la protection des données	180
Bilan du chapitre 5	181
Points clés	183
Les points clés, chapitre par chapitre	183
Six idées qui traversent tout le volume	185
Et maintenant ?	185

Introduction : pourquoi l'essentiel du travail se fait avant l'analyse

« Une analyse juste sur des données fausses est une erreur bien rédigée. »

Trois fichiers, trois questions

Le volume I s'est terminé sur une première analyse : un trimestre comparé à un autre, par canal. Nous avons, pour cela, une base propre : une table de commandes, une table de lignes, une table de clients, tout cela cohérent, typé, relié par des identifiants. **La gérante vous annonce un lundi matin que ce n'est pas la situation ordinaire.** Elle pose sur votre bureau l'essentiel de ce que l'entreprise sait vraiment d'elle-même, et ce sont des fichiers qui ne se parlent pas :

- **la caisse** de la boutique, exportée chaque mois par un logiciel qui a changé de réglages au fil de l'année ;
- **le site web**, dont la plateforme fournit un export des commandes, avec ses propres statuts, ses propres formats et quelques surprises ;
- **le fichier clients** (le CRM), alimenté depuis la caisse, le site et des imports, par plusieurs personnes, sans règle commune ;
- **le catalogue du fournisseur**, qui désigne les mêmes articles par d'autres codes et d'autres noms ;
- **un tableur de stocks**, saisi à la main, avec des titres, des cellules fusionnées, des « ND » et des « rupture » dans les colonnes de chiffres.

Elle vous pose trois questions très simples :

1. « **Combien avons-nous de clients ?** »
2. « **Combien le Site a-t-il vendu cette année ?** »
3. « **Quels articles sont en rupture, et d'où viennent-ils ?** »

Ces questions n'ont l'air de rien. Pourtant, **chacune a une mauvaise réponse facile à obtenir**, et c'est précisément ce que ce volume vous apprend à éviter. Nous allons voir deux de ces mauvaises réponses tout de suite : elles montrent mieux que n'importe quel exposé ce qui est en jeu.

La mauvaise réponse facile

Combien de clients ? Le fichier en compte 7 140

Le fichier du CRM est un tableau. La réponse qui vient à l'esprit est le nombre de lignes. Regardons.

```
crm = pd.read_csv("donnees/crm_clients.csv", dtype=str)
print(len(crm), "lignes ;", crm["email"].nunique(), "adresses e-mail différentes")
```

```
7140 lignes ; 6353 adresses e-mail différentes
```

```
clients réels : 6000 | lignes de test : 140 | lignes en double : 1000
part de lignes en trop : 19.0 %
doublons exacts (toutes colonnes sauf l'identifiant, hors lignes de test) : 0
```

La réponse « 7 140 clients » est **fausse**, et compter les adresses de messagerie différentes (6 353) ne la corrige pas : une même personne apparaît parfois **deux ou trois fois**, avec son nom en majuscules, sans accents, avec une faute de frappe, son prénom réduit à une initiale, ou son nom et son prénom inversés ; son e-mail est tantôt présent, tantôt en majuscules, tantôt absent. S'y ajoutent des **lignes de test** laissées par l'équipe du site. La vérité (que nous avons programmée et que nous n'ouvrirons qu'à la fin des études, comme le fait un correcteur) est de **6 000 clients réels** : le fichier compte donc 19 % de lignes en trop. Et aucune des lignes en double n'est **identique** à l'originale : une recherche de doublons exacts n'en trouve aucun parmi les lignes réelles. Il faudra un travail plus fin, celui du chapitre 1 (section 1.3) puis du chapitre 2 (section 2.5).

Combien le Site a-t-il vendu ? L'export répond 25 millions d'euros

L'export de la plateforme web contient une ligne par commande, avec un montant total écrit en texte (« 34,92 € », « 195.40 », « 1 245,00 »). On le convertit en nombre et on additionne.

```
site = pd.read_csv("donnees/site_commandes.csv", dtype=str)
site["montant"] = (site["total"].str.replace("€", "").str.replace(" ", "").str.replace(",", ""))
site["montant"] = site["montant"].astype(float)
print(len(site), "commandes ;", round(site["montant"].sum(), 2), "€")
```

```
6259 commandes ; 25012599.35 €
```

Voilà **25 millions d'euros** de chiffre d'affaires pour le Site en 2025. Or toute l'enseigne, tous canaux confondus, a réalisé environ 1,3 million d'euros cette année-là. Le chiffre est absurde, et c'est heureux : une erreur qui saute aux yeux se corrige. Pour la comprendre, regardons le total mensuel.

```
avant le 15 septembre : médiane d'une commande 82.93 € | après : 7808.0
CA brut par mois (k€) : {1: 42, 2: 35, 3: 43, 4: 44, 5: 49, 6: 51, 7: 55, 8: 38, 9: 3138, 10: 5781,
  11: 6581, 12: 9156}
lignes d'export à partir du 15 septembre : 2518 sur 6259
```

À partir du **15 septembre**, la plateforme exprime le total **en centimes** (le fichier ne le dit pas et personne ne l'a annoncé) : une commande de 78 € devient 7800. Avant cette date, la médiane d'une commande est de 82,93 € ; après, elle est de 7 808, soit près de cent fois plus. En divisant par cent les montants postérieurs au 15 septembre, on retrouve des ordres de grandeur sensés, mais l'histoire n'est pas finie : le fichier contient aussi des **commandes de test**, des **commandes en double** (l'export a été relancé après une panne) et des commandes **annulées**. Rien de tout cela ne ressemble à un défaut d'ordre technique : ce sont des décisions à prendre.

Après avoir converti les centimes, retiré les 60 commandes de test et les 121 lignes d'export en double, il reste **6 078 commandes** pour **617 715,45 €** : très exactement ce que contient la base du volume I pour le Site en 2025. Le chiffre est juste, et on le sait parce qu'on dispose d'une vérité ; dans la vie réelle, on n'en a pas, et c'est pourquoi nous apprendrons à **contrôler** un résultat par recoupement (chapitre 3) plutôt que par la vérité.

Reste une décision : parmi ces 6 078 commandes, **181 sont annulées** (17 551,32 €). Faut-il les compter dans le chiffre d'affaires ? Cela dépend de la question : « ce que les clients ont commandé » (on les garde), « ce que nous avons encaissé » (on les retire : 600 164,13 €). Aucune des deux réponses n'est fausse, et **le chiffre n'a de sens qu'avec sa définition**.

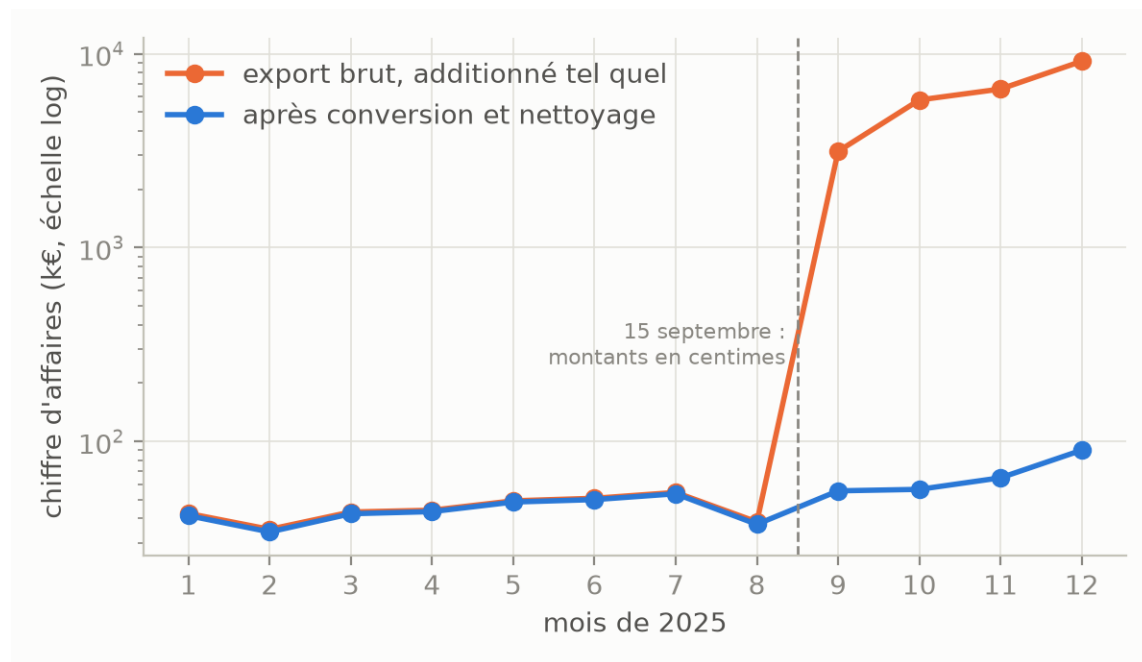


Figure 1: Chiffre d'affaires mensuel du Site en 2025, en échelle logarithmique : la courbe brute (orange) bondit d'un facteur cent à partir de septembre à cause du changement d'unité ; la courbe corrigée (bleue) suit la saison.

Intuition. Les deux erreurs de cette section ne viennent ni d'un calcul faux ni d'une mauvaise fonction : le programme a fait exactement ce qu'on lui demandait. Elles viennent de **données qui ne veulent pas dire ce qu'on croit**. C'est pour cela que la préparation des données n'est pas une corvée qui précède le « vrai » travail : c'est du travail d'analyse, et c'est celui où l'on se trompe le plus facilement sans le voir.

Le « 80 % » et ce qu'il recouvre

On entend souvent que les analystes passent « 80 % de leur temps à préparer les données ». Soyons honnêtes : **ce chiffre n'est pas une mesure**, c'est une règle de pouce de praticiens, et la part réelle dépend de la qualité des sources, de l'outillage et du type de question. Ce qu'il traduit en revanche est solide : dans la pratique, **préparer, contrôler et documenter prend plus de temps que calculer**, et c'est là que se joue la fiabilité du résultat.

Que recouvre ce travail ? Cinq familles d'activités, qui sont les chapitres de ce volume.

Activité	Question que l'on se pose	Chapitre
Nettoyer	Que faire des manques, des valeurs absurdes, des doublons et des formats mêlés ?	1
Transformer et fusionner	Comment fabriquer les variables utiles, relier des tables, changer leur forme ?	2
Contrôler et réconcilier	Les données sont-elles de qualité ? Deux sources racontent-elles la même histoire ?	3
Documenter	Un autre analyste comprendra-t-il d'où vient chaque colonne et chaque décision ?	4
Protéger	Qu'a-t-on le droit de montrer, de garder, de partager ?	5 (⊕)

Le travail est **d'autant plus important qu'il se voit peu** : une analyse brillante sur un fichier mal préparé est pire qu'une analyse modeste sur un fichier sain, parce qu'elle donne une fausse assurance.

Le cycle : préparer, contrôler, documenter

Le travail de préparation suit un cycle qui revient à chaque nouveau jeu de données. On **inspecte** la source brute (que contient-elle vraiment ?), on la **prépare** (nettoyer, transformer, fusionner), on la **contrôle** (validations, recoupements avec une autre source), et l'on **documente** ce qu'on a fait. Quand un contrôle échoue, on revient à la préparation, ou parfois à la source elle-même (on demande à la personne qui l'a produite). C'est seulement ensuite qu'on **analyse**.

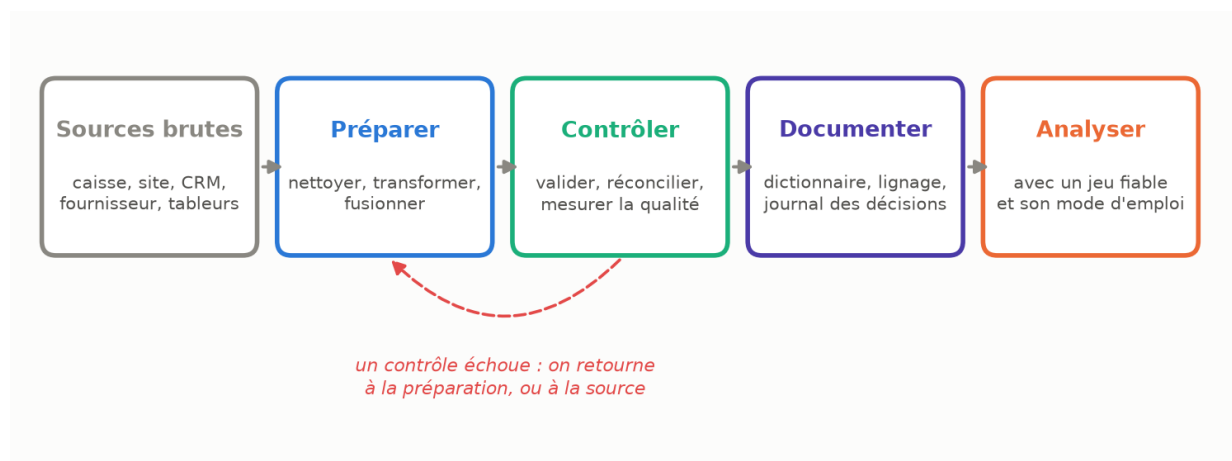


Figure 2: Le cycle de préparation des données : sources brutes, préparer (nettoyer, transformer, fusionner), contrôler (valider, réconcilier), documenter (dictionnaire, lignage), jeu de données fiable, analyse. Quand un contrôle échoue, on revient à la préparation.

Retenez deux propriétés de ce cycle. **Il est itératif** : on ne nettoie pas une fois pour toutes ; chaque contrôle révèle un cas qu'on n'avait pas prévu (c'est ainsi que nous avons trouvé les centimes). **Il est traçable** : chaque transformation doit pouvoir être expliquée et rejouée, sinon le résultat n'est pas reproductible (volume I, introduction).

Un nettoyage est une suite de décisions défendables

Quand on corrige une donnée, on **prend une décision**, même si elle paraît évidente. Retirer les 60 commandes de test est évident. Compter ou non les 181 commandes annulées ne l'est pas. Remplacer une valeur manquante par la moyenne est un choix qui change les résultats (section 1.6). Fusionner deux fiches clients qui se ressemblent à 90 % est un pari.

Une décision est **défendable** quand elle respecte quatre règles, que nous suivrons tout au long du volume.

1. **Elle est écrite.** Un journal des décisions (une ligne par règle : quoi, pourquoi, combien de lignes touchées) vaut plus qu'une mémoire.
2. **Elle est mesurée.** On sait combien de lignes la règle modifie. Une règle qui touche 0,3 % du fichier et une qui touche 30 % n'ont pas la même importance.
3. **Elle est réversible.** On ne modifie jamais le fichier source : on part de lui à chaque fois, avec un programme qui produit la version propre. La version brute reste disponible.
4. **Elle est défendue face à la question.** Elle se justifie par ce que l'on veut mesurer, pas par le goût de l'analyste.

Voici à quoi ressemble, pour notre exemple, un début de journal.

Règle	Pourquoi	Lignes touchées
Diviser par 100 les totaux postérieurs au 15 septembre	la plateforme est passée en centimes	2 518
Retirer les commandes dont l'e-mail est test@example.com	ce sont des essais	60
Retirer les lignes d'export qui répètent une référence déjà vue	l'export a été relancé	121
Garder les commandes annulées dans le « chiffre d'affaires commandé », les retirer dans le « chiffre d'affaires encaissé »	deux définitions, deux indicateurs	181

Piège. La décision la plus risquée est celle que l'on ne sait pas avoir prise. Retirer silencieusement les lignes qui « ne passent pas » dans un calcul est une décision, et elle biaise le résultat si ces lignes ne sont pas au hasard (section 1.1). Ce qu'on écarte doit toujours être **compté et dit**.

Ne pas embellir : l'éthique de la préparation

Une dernière précaution, qui tient de l'honnêteté plus que de la technique. **Nettoyer n'est pas embellir.** On corrige ce qui est manifestement une erreur de saisie ou de format ; on **n'efface pas** une valeur parce qu'elle gêne la conclusion. Trois situations demandent de la vigilance.

- **Les valeurs extrêmes.** Un montant de 9 999 € est probablement un placeholder de saisie ; une grosse commande réelle ne l'est pas. On vérifie auprès de la source avant de retirer quoi que ce soit (section 1.2).
- **Les manques.** Si les clients mécontents répondent moins à l'enquête, en retirer les non-réponses rend le résultat plus flatteur que la réalité (section 1.1).

- **Les personnes.** Fusionner deux fiches clients à tort, c'est attribuer l'historique de quelqu'un à quelqu'un d'autre ; mal anonymiser un fichier, c'est exposer des personnes (chapitre 5).

Dans tous les cas, **la règle est la même que celle du volume I** : un résultat reproductible, sourcé et borné. Le volume II ajoute une exigence : **un résultat qui sait ce qu'on a fait des données.**

Ce que contient le volume, et comment il prolonge le volume I

Le volume I vous a donné des outils : un tableur, SQL, Python et R, quelques notions de statistique, les types de données et la collecte. **Le volume II les utilise sur des données qui ne sont pas propres**, ce qui change tout : le geste (« joindre deux tables ») est le même, mais il faut d'abord savoir si les clés sont les mêmes, si elles sont uniques, si elles ont le même format. Vous retrouverez donc, avec un autre regard, merge, les jointures SQL, les formules de recherche et Power Query. Chaque fois qu'une notion du volume I est nécessaire, nous la citons (« volume I, section 4.3 ») sans la répéter.

Chapitre	Question de la gérante	Contenu
1. Nettoyage des données	« Pourquoi ce fichier de clients ne tombe-t-il pas juste ? »	valeurs manquantes, aberrantes, doublons, formats ; ☉ texte, dates et encodage, imputation
2. Transformation et fusion	« Comment relier la caisse, le site et le fournisseur ? »	variables dérivées, jointures, agrégation, restructuration ; ☉ pivot, rapprochement approximatif
3. Qualité et réconciliation	« Les chiffres de la caisse et du site disent-ils la même chose ? »	dimensions de la qualité, contrôles de validation, réconciliation ; ☉ rapports d'exceptions, pandera et Great Expectations
4. Documentation	« Et si une autre personne reprend ce travail dans six mois ? »	documenter, dictionnaire de données ; ☉ lignage et pistes d'audit
5. Confidentialité (☉)	« Puis-je envoyer ce fichier à un prestataire ? »	données personnelles, pseudonymisation, k-anonymat, bonnes pratiques
Projet (cahier)	« Je veux un fichier clients et un chiffre d'affaires fiables »	nettoyer et réconcilier deux sources désordonnées

À retenir. La préparation des données n'est pas un préalable ennuyeux : c'est l'endroit où l'on décide ce que veulent dire les chiffres. Une donnée n'est « propre » que **par rapport à une question** ; chaque correction est une **décision** qu'il faut écrire, mesurer et pouvoir défendre ; et un nettoyage honnête corrige les erreurs sans embellir la réalité. La section suivante décrit les fichiers désordonnés du volume et l'environnement de travail.

Pour s'entraîner. Cahier, mode d'emploi : un mini-diagnostic de huit questions (repérer des problèmes de qualité, manque ou zéro, doublon exact ou flou, jointure qui multiplie les lignes, date ambiguë, valeur aberrante, encodage, changement d'unité), avec corrigés, pour savoir où revenir avant de commencer.

Carte du volume, données et environnement

Cette section ouvre le volume par quatre choses : la **carte des chapitres**, le **catalogue des sources désordonnées** (avec, pour chacune, ce qui cloche), le mode d'emploi des **fichiers de vérité**, et l'**environnement** nécessaire pour refaire tous les calculs.

Carte du volume

Chaque chapitre répond à une question de la gérante. Les chapitres se lisent dans l'ordre, mais on peut les prendre séparément ; les sections marquées ⊕ sont facultatives, et le chapitre 5 tout entier est complémentaire.

Chapitre	Question posée	Contenu
1. Nettoyage des données	Pourquoi ce fichier ne tombe-t-il pas juste ?	valeurs manquantes, valeurs aberrantes, doublons, incohérences de format ; ⊕ texte, dates, encodage, données multilingues ; ⊕ imputation et son impact
2. Transformation et fusion	Comment relier la caisse, le site, le fournisseur ?	variables dérivées, jointures, agrégation et restructuration ; ⊕ pivot et dépivot ; ⊕ rapprochement approximatif
3. Qualité et réconciliation	Les chiffres de deux sources disent-ils la même chose ?	dimensions de la qualité, contrôles de validation, réconciliation ; ⊕ règles, seuils, rapports d'exceptions ; ⊕ pandera et Great Expectations
4. Documentation	Qui comprendra ce travail dans six mois ?	documenter les jeux de données et les transformations, dictionnaire de données ; ⊕ lignage et pistes d'audit
⊕ 5. Confidentialité et anonymisation	Puis-je partager ce fichier ?	données personnelles, pseudonymisation, k-anonymat, bonnes pratiques
Projet du volume (cahier)	Je veux un fichier clients et un chiffre d'affaires fiables	nettoyer et réconcilier deux sources désordonnées

Les sources désordonnées du volume

Tout est **simulé**, avec des graines fixes, par le script `build/donnees_a2.py`. Il part de la base propre du volume I (la boutique, ses clients, ses commandes) et **fabrique des sources désordonnées**, comme celles que l'on reçoit dans la vie réelle, en y injectant des défauts connus. Les noms de personnes sont inventés, sans origine particulière, et les adresses de messagerie utilisent des domaines réservés aux exemples. Aucune donnée ne vient d'une entreprise réelle.

Fichier	D'où il vient	Ce qui cloche	Lignes	Chapitres
<code>crm_clients.csv</code>	le fichier clients, alimenté par la caisse, le site et des imports	doublons flous, lignes de test, formats de date et de téléphone mêlés, codes postaux amputés, villes écrites de six façons, texte abîmé	7 140	1, 2, 3, 5, projet
<code>site_commandes.csv</code> , <code>site_lignes.csv</code>	export de la plateforme web (Site, 2025)	commandes en double, de test, annulées ; montants en texte ; changement de format de date et d'unité	6 259 ; 13 928	1, 2, 3, projet
<code>caisse/caisse_2025-01.csv ... -12.csv</code>	export mensuel de la caisse de la boutique	dérive de schéma : encodage, séparateur, décimale, noms de colonnes, format de date ; titres, en-têtes répétés, total, montants vides, lignes doublées	12 fichiers	1, 2, 3, projet
<code>catalogue_fournisseurs.csv</code>	catalogue d'un fournisseur	autres codes, désignations réécrites, produits en plus et en moins	118	2, 3
<code>stocks_tableur.xlsx</code>	tableur saisi à la main	titres, cellules fusionnées, sous-totaux, mois en colonnes, « ND », « rupture », nombres en texte	1 feuille	2
<code>profil_clients.csv</code>	fichier de profils (âge, revenu estimé, dépenses, satisfaction)	valeurs manquantes de trois natures différentes	6 000	1
<code>montants_saisis.csv</code>	saisie de montants de lignes de commande	valeurs aberrantes : décimale décalée, signe, zéro, 9 999	6 000	1, 3

Fichier	D'où il vient	Ce qui cloche	Lignes	Chapitres
clients.csv, produits.csv, commandes.csv, lignes_commande.csv	la base propre du volume I	(copies, pour recouper)	6 000 ; 120 ; 36 395 ; 83 905	tous

Le fichier clients : un CRM alimenté par trois sources

Le CRM (*customer relationship management*, la base des contacts) compte 7 140 lignes pour 6 000 clients réels.

Colonne	Contenu	Ce qu'on y trouve
id_crm	numéro de ligne du CRM	pas un identifiant de client : un client peut avoir plusieurs lignes
prenom, nom	identité	majuscules, accents retirés, fautes, initiales, inversions, espaces
email	adresse	absente, en majuscules, malformée
telephone	numéro	cinq présentations
ville, code_postal	adresse	variantes d'orthographe, ville en arabe, code sans son zéro initial, absent
date_naissance, date_inscription	dates	quatre formats de naissance, dont des dates impossibles ; inscription en jj/mm/aaaa
consentement_marketing	accord pour les messages commerciaux	sept façons de l'écrire
source_saisie	caisse, site ou import	peut expliquer la façon d'écrire

```
lignes : 7140 | e-mails absents : 231 (3.2%) | codes postaux absents : 424 | de moins de 5 chiffres :
↳ 199
formats de naissance : {'xx/xx/aaaa': 4674, 'aaaa-mm-jj': 1749, 'texte': 717}
graphies de ville : 119 pour 20 villes ; 191 en arabe
consentement : {'oui': 2411, '(vide)': 2161, 'Oui': 703, '1': 687, '0': 369, 'TRUE': 338, 'OUI': 331,
↳ 'non': 140}
présentations du téléphone : 5 | noms abîmés (« Ã ») : 213
source de saisie : {'caisse': 3507, 'site': 2965, 'import': 668}
```

Les 119 graphies de ville pour 20 villes, les cinq formats de téléphone et les sept écritures du consentement donnent une idée de ce qu'est un champ saisi par plusieurs mains. Notez le nombre de dates écrites xx/xx/aaaa : pour la plupart, on lit « jour/mois », mais **pas pour toutes** (certaines lignes saisies sur le site écrivent « mois/jour »), et c'est un piège que nous rencontrerons à la section 1.4.

L'export du site web

La plateforme exporte une ligne par commande (site_commandes.csv) et une ligne par article (site_lignes.csv), avec des noms de colonnes en anglais.

Colonne	Contenu	Ce qu'on y trouve
order_ref	référence de la commande (WEB-023450)	répétée quand l'export est relancé
created_at	date et heure	2025-03-04 14:22:05 (sans fuseau) jusqu'au 14 septembre, 2025-09-16T14:22:05Z (en UTC) ensuite
status	statut	paid, PAID, Paid, cancelled
customer_email, customer_name	client	en majuscules, avec espaces, ou test@example.com
total, currency	montant et devise	texte (34,92 €, 195.40, 1 245,00), en centimes depuis le 15 septembre ; EUR, eur, €
promo_code, shipping_mode (lignes) sku, qty, unit_price, discount_pct	code promotionnel, livraison article, quantité, prix, remise	vides pour « aucun » le sku s'écrit P043 : le code produit de la boutique, sans le zéro de gauche

commandes (lignes d'export) : 6259 | références distinctes : 6138 | lignes d'articles : 13928
statuts : {'paid': 3629, 'PAID': 1537, 'Paid': 907, 'cancelled': 186} | devises : {'EUR': 4987, 'eur': 643, '€': 629}
présentations du total : {'entier': 2494, 'point': 1510, 'euro': 1476, 'virgule': 779} | promo absente : 84.2%

Les exports de caisse : douze fichiers, trois réglages

L'export de la caisse est produit chaque mois, mais **le logiciel a changé de réglages deux fois dans l'année**. Concaténer les douze fichiers sans précaution échoue ou donne des colonnes mélangées.

Mois	Encodage	Séparateur	Décimale	Colonne des quantités	Date	Colonnes
janvier à juin	cp1252 (Windows)	;	virgule	Qté	jj/mm/aaaa	8
juillet à septembre	UTF-8 avec marque BOM	;	virgule	Quantité	jj/mm/aa	8
octobre à décembre	UTF-8 avec marque BOM	,	point	Qté	jj/mm/aaaa	9 (une colonne Remise (%) en plus)

Chaque fichier commence par trois lignes de titre, répète son en-tête toutes les soixante lignes (le « changement de page »), se termine par une ligne de total, comporte quelques montants vides et, parfois, une ligne répétée par un double passage en caisse.

réglages distincts : 3
('cp1252', ';', 8) → mois 01, 02, 03, 04, 05, 06
('utf-8', ';', 8) → mois 07, 08, 09
('utf-8', ',', 9) → mois 10, 11, 12
lignes de fichier au total : 12943

Le catalogue du fournisseur et le tableau de stocks

Le **catalogue du fournisseur** décrit les articles par un code F-xxxx et une désignation réécrite (casse, accents, abréviations, ordre des mots, quelques mots anglais). Il manque 12 produits de la boutique, et 10 articles du fournisseur ne sont pas à la boutique. Attention à un détail hérité du volume I : **la boutique compte 120 produits mais seulement 60 noms distincts**, parce que chaque nom est porté par deux produits à prix différents. La désignation seule ne suffit donc pas à rapprocher les deux listes : il faudra s'aider du prix (sections 2.2 et 2.5).

Le **tableau de stocks** est une feuille « à la main » : un titre, une phrase de consigne, une ligne d'en-têtes, puis les catégories en cellules fusionnées, les produits (un par ligne, les douze mois en colonnes), un sous-total par catégorie avec une formule, et des cellules qui contiennent « ND », un tiret, « rupture » ou un nombre écrit avec une espace.

```
catalogue : lignes 118 | familles écrites : 12 pour 6 catégories | désignations avec espaces autour :
↳ 19
boutique : produits 120 | noms distincts : 60 | noms portés par deux produits : 60
feuille : A1:N141 | cellules fusionnées : 6 | cellules de données : 1512
« ND » : 49 | tirets : 22 | « rupture » : 120 | nombres écrits en texte avec espace : 131
```

Les profils clients et les montants saisis

Deux petits jeux servent aux chapitres de nettoyage proprement dit. **profil_clients.csv** donne, pour 6 000 clients, l'âge, le revenu annuel estimé, le nombre de commandes et la dépense de 2025, la satisfaction moyenne et le temps passé sur le site ; trois colonnes ont des **trous**, et ils n'ont pas la même origine, ce qui change tout (section 1.1). **montants_saisis.csv** reprend 6 000 lignes de commande de 2024 dans lesquelles ont été glissées des anomalies de saisie, que le chapitre 1 apprend à repérer (section 1.2).

```
profils : manquants par colonne : {'revenu_annuel': 1039, 'depense_2025': 297, 'satisfaction_moy':
↳ 562}
part des manquants : {'revenu_annuel': 0.173, 'depense_2025': 0.05, 'satisfaction_moy': 0.094}
montants saisis : lignes 6000 | médiane 31.43 | minimum -153.8 | maximum 26070.0
```

Les fichiers de vérité

Pour chaque source désordonnée, le script écrit un fichier `verite_*.csv` qui dit **ce qui a été injecté** et ce que la valeur propre aurait dû être.

Fichier	Ce qu'il contient
<code>verite_crm.csv</code>	pour chaque ligne du CRM, le vrai client, l'indicateur de doublon et les défauts injectés
<code>verite_identites.csv</code>	l'identité propre de chaque client (prénom, nom, e-mail)
<code>verite_site.csv</code>	pour chaque référence de l'export du site, la vraie commande, le vrai total et le défaut
<code>verite_caisse.csv</code>	pour chaque ligne des fichiers de caisse, la vraie ligne de la base et l'indicateur de doublon
<code>verite_produits.csv</code>	pour chaque code du fournisseur, le vrai produit de la boutique (ou -1)
<code>verite_stocks.csv</code>	le vrai stock de chaque produit et de chaque mois
<code>profil_clients_verite.csv</code>	les valeurs complètes, sans trous
<code>verite_montants.csv</code>	le type d'anomalie et le vrai montant

Fichier	Ce qu'il contient
---------	-------------------

Piège. Ces fichiers sont des **corrigés**, pas des données : dans la vie réelle, il n'y a pas de fichier de vérité, et c'est tout l'intérêt de ce volume d'apprendre à **contrôler sans lui** (chapitre 3). On les ouvre à la fin d'une étude pour mesurer ce qu'on a bien et mal fait, comme on regarde le corrigé d'un exercice après l'avoir cherché. Un nettoyage qui s'appuie sur eux pour décider n'a rien démontré.

L'environnement de travail

Python, R et SQL

Les calculs du livre sont faits en **Python** (pandas), en **R** (tidyverse) et en **SQL** (SQLite et DuckDB), comme au volume I. Ce volume ajoute des bibliothèques propres à la préparation des données.

Bibliothèque	À quoi elle sert	Chapitres
pandas, polars	lire, nettoyer, transformer	tous
rapidfuzz, jellyfish	mesurer la ressemblance entre deux textes (distance d'édition, Jaro-Winkler)	2.5
unidecode, ftfy	retirer les accents ; réparer un texte abîmé par un mauvais encodage	1.5
pandera, great-expectations	écrire des contrôles de qualité exécutables	3.5
openpyxl, XlsxWriter	lire et écrire des classeurs Excel	1, 2, 3

```
python 3.13.3 | pandas 3.0.6, numpy 2.5.3, polars 2.0.0, openpyxl 3.1.5
préparation : rapidfuzz 3.14.6, jellyfish 1.2.1, Unidecode 1.4.0, ftfy 6.3.1, pandera 0.34.1,
↳ great_expectations 1.24.0
SQLite 3.46.1
R version 4.4.3 (2025-02-28)
```

Le tableur : Excel, LibreOffice et nos maquettes

Même précision d'honnêteté qu'au volume I. **Excel n'est pas installé** sur la machine qui produit ce livre, mais **LibreOffice Calc** l'est : les formules du livre sont **écrites dans des classeurs, recalculées par LibreOffice et comparées aux résultats de pandas**. Les noms de menus et de fonctions varient avec la version et la langue d'Excel : à vérifier dans la vôtre. Ce qui n'est pas exécutable ici (Power Query et son langage M, tableaux croisés dynamiques réels, macros) est signalé « **non exécuté** », avec un équivalent calculé en pandas quand c'est possible.

Les captures d'écran

Les copies d'écran ne sont pas toutes de la même nature, et le livre le dit chaque fois dans la légende :

- les **maquettes** sont dessinées avec matplotlib (comme celles de Power Query ou d'un classeur) ;
- les **captures réelles** sont faites sur des outils libres que l'on exécute ici, avec un navigateur sans interface (par exemple un rapport HTML de Great Expectations) ;
- les **images empruntées** à Internet ne le sont que si leur licence autorise la réutilisation ; elles sont alors créditées dans la légende et dans le fichier `figures/credits.md`. Nous n'utilisons pas de captures prises dans la documentation ou le site d'un éditeur de logiciel commercial.

Régénérer les données et refaire les calculs

Tout le dossier `donnees/` se régénère à l'identique par un script, et les calculs du livre se rejouent par `make check` (qui vérifie que chaque sortie est inchangée).

```
python build/donnees_a2.py      # régénère donnees/ (une dizaine de secondes, graines fixes)
make check                     # rejoue tous les blocs de code et signale toute différence
make pdf                       # reconstruit le livre et le cahier en PDF
```

Conventions du volume

- **Monnaie, villes, canaux** : montants en € ; villes « Ville A » à « Ville T » ; canaux Boutique, Site, Réseaux.
- **Personnes** : tous les noms sont **inventés** ; les adresses de messagerie utilisent des domaines réservés aux exemples (`exemple.org`, `courrier.test`, `mail.example`).
- **Nombres** : à la française dans le texte (espace pour les milliers, virgule décimale), à l'anglaise dans les sorties de programme.
- **Dates** : `aaaa-mm-jj` dans les programmes ; les formats `jj/mm/aaaa` ou `mm/jj/aaaa` sont ceux des fichiers reçus.
- **Aléatoire** : toute simulation fixe sa **graine**.
- **Encadrés** : ■ intuition · ■ formule · ■ expérience · △ piège · ✓ à retenir · ■ repère ou section facultative · ■ pour s'entraîner · ■ données.

À retenir. Les sources du volume sont **simulées et volontairement désordonnées** : un CRM de 7 140 lignes pour 6 000 clients, un export web qui change d'unité en septembre, douze exports de caisse sous trois réglages, un catalogue fournisseur, un tableur saisi à la main, des profils à trous et des montants à anomalies. Les fichiers `verite_*` sont des **corrigés** à n'ouvrir qu'à la fin. Les formules Excel sont vérifiées avec LibreOffice ; les captures d'écran sont des maquettes, des captures d'outils libres ou des images sous licence libre créditées.

Chapitre 1 : Nettoyage des données

« Les données ne sont jamais sales en elles-mêmes : elles sont sales *pour une question donnée*. »

La gérante vous attend avec trois dossiers sous le bras. « Dans le fichier clients, **le revenu manque pour 17 % des personnes** : je les supprime, ou je mets la moyenne à la place ? Dans l'export du site, la somme des commandes de l'année donne **plus de 25 millions d'euros**, alors que le site n'en vend pas un million. Et le logiciel de caisse m'envoie douze fichiers par an qui ne se lisent pas tous de la même façon. Peux-tu me dire ce que je peux croire ? »

Ces trois questions sont le **nettoyage des données** : transformer des fichiers tels que la vie les produit (saisis à la main, exportés par des logiciels qui changent de version, fusionnés sans précaution) en un tableau sur lequel on peut calculer sans se tromper. Les chiffres ci-dessus donnent une idée de l'enjeu : une somme **quarante fois trop grande** n'a rien de subtil, mais d'autres erreurs sont discrètes (un doublon sur quarante, un revenu absent plus souvent chez les jeunes) et faussent un résultat de quelques pour cent **sans que rien ne le signale**.

Pourquoi le nettoyage prend tant de temps

On répète souvent qu'une analyste passe la majeure partie de son temps à préparer les données. Ce n'est pas une corvée accessoire : **c'est là que se prennent les décisions qui déterminent le résultat**. Supprimer ou imputer, plafonner ou garder, fusionner deux écritures d'un même nom : chaque choix change la moyenne, le total ou la répartition que vous allez annoncer. Et ces choix ne se lisent nulle part dans le résultat final.

Intuition. Une donnée est « propre » **par rapport à une question**. Une adresse sans code postal est parfaitement utilisable pour compter les clients par ville, et inutilisable pour calculer une distance de livraison. Il n'existe donc pas de nettoyage universel : on nettoie *pour* un usage, et l'on écrit ce que l'on a fait.

Quatre règles de méthode traversent tout le chapitre.

- **Ne jamais modifier le fichier d'origine.** On lit la source, on produit une copie nettoyée. Le brut sert à prouver, plus tard, ce que l'on a changé.
- **Compter avant et après.** Chaque correction a un effet que l'on mesure : « 121 doublons retirés », « 85 montants corrigés ». Une correction dont on ignore l'ampleur est un risque.
- **Écrire des scripts, pas des clics.** Un nettoyage fait dans le tableur ne se rejoue pas le mois suivant ; une fonction, si.
- **Vérifier par une source indépendante.** Un total affiché, une table de référence, un second fichier : la réconciliation (chapitre 3) est le contrôle qui transforme un nettoyage plausible

en nettoyage prouvé.

Le chemin de ce chapitre

Le parcours essentiel suit les quatre grandes familles de défauts.

- **1.1 Valeurs manquantes** : les repérer, distinguer l'absence d'un zéro ou d'un code spécial, comprendre **pourquoi** une valeur manque (hasard, dépendance à une autre variable, dépendance à la valeur elle-même) et ce que coûte une suppression.
- **1.2 Valeurs aberrantes** : séparer l'erreur de saisie, l'extrême réel et le cas rare ; comparer des méthodes statistiques et des règles métier.
- **1.3 Doublons** : exacts ou approchés, ce qu'est une clé, quel enregistrement garder, quel est l'effet sur les totaux.
- **1.4 Incohérences et erreurs de format** : types, unités mélangées, dates ambiguës, catégories écrites de six façons, schémas qui changent d'un fichier à l'autre.

Deux sections facultatives prolongent ce parcours : ☉ **1.5 Nettoyage de texte, dates et heures, encodage, données multilingues** et ☉ **1.6 Stratégies d'imputation et leur impact**, où l'on compare chaque méthode à la vérité.

Les données du chapitre

Toutes les données sont **simulées** à partir de la base propre du volume I, puis salies de façon contrôlée : nous savons exactement ce qui a été abîmé, ce qui nous permet de **juger** chaque nettoyage (ce que l'on ne peut pas faire dans la vraie vie !). Les fichiers dont le nom commence par `verite_` ou se termine par `_verite` sont cette vérité : on ne s'en sert **qu'à la fin d'une étude**, pour mesurer l'erreur, comme un corrigé.

Fichier	Contenu	Lignes	Sections
<code>profil_clients.csv</code>	profil de 6 000 clients, avec des trous (<code>revenu_annuel</code> , <code>depense_2025</code> , <code>satisfaction_moy</code>)	6 000	1.1, 1.6
<code>profil_clients_verite.csv</code>	les mêmes clients, sans trou	6 000	1.1, 1.6
<code>montants_saisis.csv</code>	montants de lignes de commande de 2024, avec des anomalies injectées	6 000	1.2
<code>site_commandes.csv</code>	export de la plateforme web (commandes du site en 2025)	6 259	1.3, 1.4
<code>crm_clients.csv</code>	le fichier clients du CRM, avec doublons et saisies disparates	7 140	1.3, 1.4, 1.5
<code>caisse/caisse_2025-MM.csv</code>	12 exports mensuels de la caisse de la boutique	12 678	1.3, 1.4

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 à 1.8 (rapport de manquants, mécanismes d'absence, détection d'aberrantes, doublons du site, changement d'unité, douze fichiers de caisse, nettoyage du CRM, imputations comparées à la vérité) et exercices 1.1 à 1.12.

1.1 Valeurs manquantes

Une valeur manquante est une case que l'on s'attendait à trouver remplie. Elle paraît anodine, et c'est pourtant le défaut qui, mal traité, **déforme le plus de résultats** : on supprime des lignes sans le dire, on remplace par la moyenne sans mesurer, on confond l'absence et le zéro. Cette section apprend à les repérer, à comprendre **pourquoi** elles manquent, et à décider en connaissance de cause.

1.1.1 Repérer ce qui manque

Ouvrons le profil des clients (le fichier `profil_clients.csv`) et posons la première question de toute analyse de qualité : **combien manque-t-il, et où ?** Avec pandas, `isna()` renvoie un tableau de vrai/faux, qu'il suffit de sommer ou de moyenner par colonne.

```
profil = pd.read_csv(os.path.join(D, "profil_clients.csv"))
resume = pd.DataFrame({"manquants": profil.isna().sum(), "part_%": (profil.isna().mean() *
↳ 100).round(1)})
print(resume[resume["manquants"] > 0])
```

	manquants	part_%
revenu_annuel	1039	17.3
depense_2025	297	5.0
satisfaction_moy	562	9.4

Trois colonnes sur huit ont des trous : le revenu pour 17,3 % des clients (c'est le chiffre de la gérante), la dépense de 2025 pour 5,0 % et la satisfaction moyenne pour 9,4 %. Mais ces pourcentages par colonne ne disent pas **comment les trous se combinent** : le client dont le revenu manque est-il aussi celui dont la satisfaction manque ? On compte les combinaisons.

```
colonnes = ["revenu_annuel", "depense_2025", "satisfaction_moy"]
combinaisons = profil[colonnes].isna().value_counts()
print(combinaisons.rename("clients").reset_index().to_string(index=False))
print("clients avec au moins un trou :", int(profil[colonnes].isna().any(axis=1).sum()))
```

revenu_annuel	depense_2025	satisfaction_moy	clients
False	False	False	4277
True	False	False	886
False	False	True	435
False	True	False	229
True	False	True	105
True	True	False	46
False	True	True	20
True	True	True	2

clients avec au moins un trou : 1723

Sur 6 000 clients, **4 277 sont complets** ; 1 723 (28,7 %) ont au moins un trou. C'est le chiffre qui compte pour la suite : si l'on supprimait toute ligne incomplète, on perdrait plus d'un client sur quatre, alors que chaque colonne, prise isolément, semble avoir peu de trous. Les trous **s'additionnent** d'une colonne à l'autre.

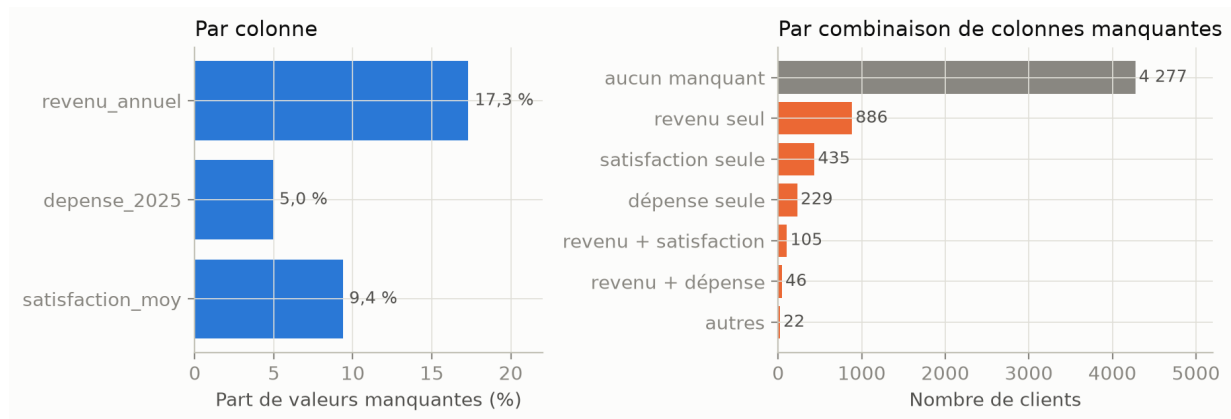


Figure 3: Les valeurs manquantes du profil des clients : à gauche la part par colonne, à droite les combinaisons de colonnes manquantes.

En pratique. Faites ce rituel **avant tout calcul** : compter les manquants par colonne, puis par combinaison. Dans un tableur, NB.VIDE compte les cellules vides ; en SQL, `COUNT(*) - COUNT(colonne)` donne le nombre de NULL d'une colonne ; en R, `colSums(is.na(profil))`. Les trois disent la même chose, avec leur syntaxe.

1.1.2 Absence, zéro et code spécial

Toutes les cases vides ne se ressemblent pas, et toutes les valeurs « bizarres » ne sont pas des trous. Trois situations se confondent facilement.

- **Une vraie absence** : on ne connaît pas la valeur (le revenu d'un client qui ne l'a pas donné).
- **Un zéro** : on connaît la valeur, et elle vaut zéro (un client qui n'a rien acheté a dépensé 0 €, et non « une dépense inconnue »).
- **Un code spécial** : un logiciel écrit ND, -, 999, 9999 ou -1 à la place d'un trou. Lu comme un nombre, ce code fausse tout ; lu comme du texte, il empêche le calcul.

Le tableau suivant donne l'interprétation à retenir pour quelques écritures rencontrées dans les fichiers de la boutique.

Ce qu'on lit	Ce que cela veut dire	Traitement
cellule vide dans <code>revenu_annuel</code>	on ne sait pas	manquant (NaN)
0 dans <code>depense_2025</code>	aucune dépense	vrai zéro, à garder
ND dans un stock	non disponible	manquant
rupture dans un stock	il n'y a plus de produit	zéro , pas un manquant
9999 dans un montant	valeur de remplissage du logiciel	manquant (ou erreur, section 1.2)

Le profil des clients contient justement un cas instructif. Les clients **sans commande en 2025** ont forcément dépensé 0 € : leur dépense n'est pas « inconnue », elle est **déductible**. Vérifions si le fichier le sait.

```
sans_commande = profil["nb_commandes_2025"] == 0
print("clients sans commande :", int(sans_commande.sum()))
print("dont dépense à 0 :", int((profil.loc[sans_commande, "depense_2025"] == 0).sum()), "| dont
↳ dépense manquante :", int(profil.loc[sans_commande, "depense_2025"].isna().sum()))
```

```
clients sans commande : 2125
dont dépense à 0 : 2024 | dont dépense manquante : 101
```

Sur les 2 125 clients sans commande, 2 024 ont bien une dépense de 0 et **101 ont une dépense manquante** : ces 101 trous (sur 297) ne sont pas de vrais manquants, on peut les **remplir avec certitude**. Aucune imputation statistique n'égale une règle logique : avant de sortir un modèle, cherchez ce qui se **déduit**.

```
depense_deduite = profil["depense_2025"].where(~sans_commande, 0.0)      # 0 € pour qui n'a rien
↳ acheté
print("dépenses encore manquantes après déduction :", int(depense_deduite.isna().sum()))
```

```
dépenses encore manquantes après déduction : 196
```

Piège. Remplacer **tous** les trous par 0 est une erreur fréquente : on transforme une absence d'information en information (« ce client a dépensé 0 € »), ce qui tire les moyennes vers le bas. À l'inverse, laisser le mot rupture ou le code 9999 dans une colonne numérique casse la lecture. Pour chaque colonne, écrivez **ce que signifie chaque valeur spéciale** avant de la traiter.

1.1.3 Pourquoi une valeur manque : trois mécanismes

Savoir **combien** il manque ne suffit pas. Ce qui décide du traitement, c'est **pourquoi** cela manque. Les statisticiens distinguent trois mécanismes.

Mécanisme	Définition	Exemple de la boutique
MCAR (<i>missing completely at random</i>)	la probabilité de manquer est la même pour tout le monde	une dépense perdue par un incident technique
MAR (<i>missing at random</i>)	elle dépend d' une autre variable connue	le revenu est plus souvent absent chez les jeunes
MNAR (<i>missing not at random</i>)	elle dépend de la valeur manquante elle-même	un client mécontent ne répond pas à la question de satisfaction

Intuition. Le mot « *at random* » trompe. MAR ne veut pas dire « au hasard » : cela veut dire que, **une fois connue** une autre variable (l'âge, le canal), le trou ne dépend plus de rien d'autre. MNAR est le cas redoutable : ce qui manque est précisément ce qu'il y a de différent dans les cases vides, et rien de ce que l'on a ne permet de le retrouver.

Voyons ce que les données permettent de dire. On teste, pour chacune des trois colonnes, si le fait de manquer dépend de l'âge ou du canal d'acquisition, avec un test du khi-deux (les tests sont détaillés au volume III ; ici la lecture est simple : une **probabilité critique** minuscule signale une dépendance).

```
from scipy.stats import chi2_contingency
profil["classe_age"] = pd.cut(profil["age"], [0, 29, 44, 59, 200], labels=["moins de 30", "30-44",
↳ "45-59", "60 et plus"])
for col in ["depense_2025", "revenu_annuel", "satisfaction_moy"]:
    for var in ["classe_age", "canal_acquisition"]:
        p = chi2_contingency(pd.crosstab(profil[var], profil[col].isna()))[1]
        print(f"{col:17s} selon {var:18s} p = {p:.2g}")
```

```
depense_2025      selon classe_age      p = 0.72
depense_2025      selon canal_acquisition p = 0.53
```

revenu_annuel	selon classe_age	p = 7.2e-49
revenu_annuel	selon canal_acquisition	p = 1.4e-09
satisfaction_moy	selon classe_age	p = 0.76
satisfaction_moy	selon canal_acquisition	p = 0.36

La lecture est nette. **La dépense** ne dépend ni de l'âge ni du canal (probabilités critiques de 0,72 et 0,53 : rien ne s'oppose à l'idée d'un trou au hasard). **Le revenu**, lui, manque selon l'âge et selon le canal, avec des probabilités critiques minuscules : le mécanisme n'est pas MCAR. Les taux parlent d'eux-mêmes.

```
print((profil.groupby("classe_age", observed=True)["revenu_annuel"].apply(lambda s: s.isna().mean() *
↳ 100)).round(1).to_string())
print((profil.groupby("canal_acquisition")["revenu_annuel"].apply(lambda s: s.isna().mean() *
↳ 100)).round(1).to_string())
```

classe_age	
moins de 30	33.7
30-44	14.1
45-59	14.2
60 et plus	13.4
canal_acquisition	
Boutique	16.4
Réseaux	25.8
Site	15.8

Le revenu manque pour **33,7 % des moins de 30 ans** contre environ 14 % ensuite, et pour 25,8 % des clients venus des réseaux contre 16 % ailleurs : c'est un mécanisme **MAR**, qui dépend de deux variables que l'on connaît.

Reste la satisfaction. Les tests ne trouvent **aucune** dépendance à l'âge ni au canal (0,76 et 0,36) : elle ressemble à une absence au hasard. Or ce n'est pas le cas, et nous pouvons le montrer parce que nous avons, exceptionnellement, **la vérité**.

```
verite = pd.read_csv(os.path.join(D, "profil_clients_verite.csv"))
vraie = pd.cut(verite["satisfaction_moy"], [0, 2.5, 3.5, 4.5, 5.01], labels=["≤ 2,5", "2,5-3,5",
↳ "3,5-4,5", "> 4,5"])
print((profil["satisfaction_moy"].isna().groupby(vraie, observed=True).mean() *
↳ 100).round(1).to_string())
```

satisfaction_moy	
≤ 2,5	37.6
2,5-3,5	7.5
3,5-4,5	8.9
> 4,5	7.1

Les clients dont la vraie satisfaction est inférieure ou égale à 2,5 laissent leur case vide **cinq fois plus souvent** (37,6 %) que les autres (autour de 8 %). Le trou dépend de la valeur qui manque : c'est un **MNAR**. Remarquez ce que cela implique : **aucun test sur les données observées ne pouvait le révéler**, puisque la dépendance passe par la valeur que l'on n'a pas.

À retenir. On peut **réfuter** MCAR avec les données (un trou qui dépend de l'âge). On ne peut **pas démontrer** MAR contre MNAR avec les seules données observées : c'est une affirmation sur le monde, qui se défend par la connaissance du métier (« les mécontents ne répondent pas »), pas par un test.

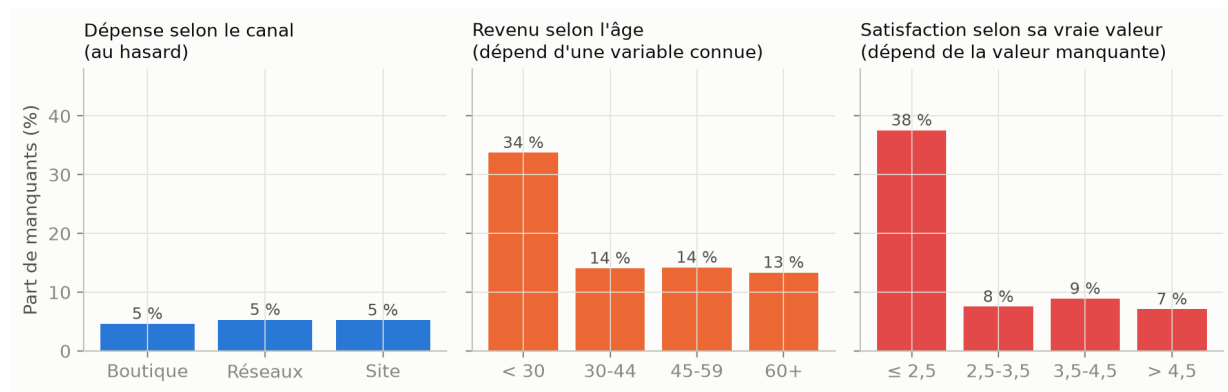


Figure 4: Trois mécanismes d'absence : la dépense manque au hasard, le revenu manque selon l'âge, la satisfaction manque selon sa propre valeur.

1.1.4 Ce que coûte une suppression

La réaction la plus naturelle est de supprimer les lignes incomplètes (*suppression par liste*, ou *cas complets*). Mesurons ce qu'elle fait, **en comparant à la vérité**.

```
complets = profil.dropna(subset=colonnes)
print(f"clients conservés : {len(complets)} sur {len(profil)} ({len(complets) / len(profil) * 100:.1f} %)")
print("part des moins de 30 ans : vérité", round((verite["age"] < 30).mean() * 100, 1), "% | cas complets", round((complets["age"] < 30).mean() * 100, 1), "%")
print("âge moyen : vérité", round(verite["age"].mean(), 2), "| cas complets", round(complets["age"].mean(), 2))
```

```
clients conservés : 4277 sur 6000 (71.3 %)
part des moins de 30 ans : vérité 16.7 % | cas complets 13.4 %
âge moyen : vérité 43.29 | cas complets 44.08
```

On perd 28,7 % des clients, et ceux qui restent sont **plus âgés** : la part des moins de 30 ans passe de 16,7 % à 13,4 %. Cela s'explique : les jeunes manquent plus souvent (c'est le mécanisme MAR vu plus haut), donc ils sont sous-représentés parmi les cas complets. Quel est l'effet sur les indicateurs eux-mêmes ?

```
for col in ["revenu_annuel", "depense_2025", "satisfaction_moy"]:
    print(f"{col:17s} vérité {verite[col].mean():10.2f} | valeurs observées {profil[col].mean():10.2f} | cas complets {complets[col].mean():10.2f}")
bas = lambda s: (s.dropna() <= 2.5).mean() * 100
print("part de satisfactions ≤ 2,5 (%) : vérité", round(bas(verite["satisfaction_moy"]), 2), "| observée", round(bas(profil["satisfaction_moy"]), 2))
```

```
revenu_annuel    vérité    28321.77 | valeurs observées    28540.66 | cas complets    28462.78
depense_2025     vérité     220.79 | valeurs observées     221.47 | cas complets     220.55
satisfaction_moy vérité       3.73 | valeurs observées       3.75 | cas complets       3.75
part de satisfactions ≤ 2,5 (%) : vérité 4.08 | observée 2.81
```

La leçon est **nuancée** : sur les **moyennes**, l'écart reste modeste (+0,8 % pour le revenu, +0,3 % pour la dépense, +0,5 % pour la satisfaction), parce que, dans ces données, le revenu dépend peu de l'âge et que la satisfaction manquante ne représente que 9 % des clients. Mais, sur la **queue** de la distribution, l'erreur est lourde : la part de clients très mécontents est de 4,1 % en réalité et de 2,8 % dans les réponses observées, soit **près d'un tiers de moins**. La suppression donne une image **trop rose**, et personne ne le voit dans la moyenne.

Piège. « La moyenne ne bouge presque pas, donc supprimer est sans risque » est un raisonnement faux. Une moyenne peut rester stable pendant que la composition de l'échantillon change (moins de jeunes) et que la queue de la distribution disparaît (moins de mécontents). Mesurez **ce qui vous intéresse**, pas seulement la moyenne.

On perd aussi de la **précision** : avec 4 277 clients au lieu de 6 000, les intervalles de confiance s'élargissent d'environ 18 % (racine carrée du rapport des effectifs). La suppression n'est donc ni neutre, ni gratuite.

1.1.5 Quatre options, et comment choisir

Face à une colonne avec des trous, quatre stratégies s'offrent à vous.

Option	Principe	Quand elle convient	Risque
Supprimer les lignes (ou la colonne)	on retire ce qui est incomplet	peu de trous, mécanisme MCAR, colonne inutile à la question	biais si MAR ou MNAR, perte de précision
Garder tel quel	on laisse NaN et l'outil l'ignore	moyennes, comptes par groupe, graphiques	oublier que le calcul porte sur un sous-ensemble
Imputer	on remplace par une valeur estimée (section 1.6)	un modèle ou un tableau exigeant des valeurs complètes	fausse la dispersion et les liens entre variables
Marquer	on ajoute une colonne « était manquant »	l'absence elle-même informe (MNAR plausible)	alourdit le tableau, à interpréter avec prudence

Les outils ne se comportent pas pareil devant un trou, et cette différence est une source classique de désaccord entre deux chiffres. pandas, Excel (MOYENNE) et SQL (AVG) **ignorent** les valeurs manquantes ; R, lui, renvoie NA tant qu'on ne lui demande pas explicitement de les ignorer.

```
profil <- read.csv(file.path(Sys.getenv("DONNEES"), "profil_clients.csv"))
print(c(sans_na_rm = mean(profil$revenu_annuel), avec_na_rm = mean(profil$revenu_annuel, na.rm =
  TRUE)))
```

```
sans_na_rm avec_na_rm
NA      28540.66
```

La moyenne de 28 540,66 € est la même que celle de pandas : le chiffre ne dépend pas de l'outil, seule la **demande explicite** (na.rm = TRUE) diffère.

En pratique : comment décider. Posez dans l'ordre quatre questions. (1) *Peut-on déduire la valeur ?* (un zéro logique, une autre colonne) : alors on déduit. (2) *Combien manque-t-il, et selon quel mécanisme probable ?* Moins de 5 % au hasard : supprimer est défendable ; plus, ou dépendant d'une autre variable : évitez. (3) *Que veut-on calculer ?* Une moyenne par groupe supporte les trous ; un modèle ne les supporte pas. (4) *Peut-on le dire ?* Quel que soit le choix, **écrivez-le** : « revenu manquant pour 17,3 % des clients, ignoré dans les moyennes ».

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 et 1.2, exercices 1.1 à 1.3.

1.2 Valeurs aberrantes

Une valeur aberrante est une valeur qui **détonne** : un montant de 10 180 € pour deux articles à 50,90 €, un client à 3 400 € de dépense annuelle dans une clientèle où la médiane est de 98 €. La réaction instinctive, « c'est une erreur, j'enlève », est précisément ce qu'il faut se garder de faire. Cette section apprend à **distinguer** les cas, à comparer des méthodes de détection, et à choisir un traitement.

1.2.1 Erreur, extrême réel ou cas rare ?

Trois situations très différentes produisent la même impression de « valeur bizarre ».

Situation	Exemple dans la boutique	Que faire
Erreur	une décimale décalée : 10 180 € au lieu de 101,80 €	corriger ou écarter, si l'on sait que c'est une erreur
Extrême réel	un client qui dépense 3 382 € en un an	garder : c'est une information vraie, souvent la plus précieuse
Cas rare	un retour massif après une livraison défectueuse	garder, mais le signaler ; l'analyser à part

Intuition. Une méthode statistique ne détecte pas des *erreurs* : elle détecte des valeurs **éloignées des autres**. Que l'éloignement vienne d'une faute de frappe ou d'un très bon client, la formule n'en sait rien. Une valeur aberrante est un **suspect**, pas un coupable : c'est la connaissance du métier qui tranche.

1.2.2 Trois méthodes statistiques

Nous disposons de 6 000 lignes de commande de 2024 saisies à la main, dont **85 anomalies ont été injectées** (décimale décalée de $\times 10$ ou $\times 100$, signe inversé, zéro, valeur de remplissage 9999). Chargeons-les avec leur vérité, que nous n'utiliserons que pour **juger** les méthodes.

```
saisis = pd.read_csv(os.path.join(D, "montants_saisis.csv"))
saisis = saisis.merge(pd.read_csv(os.path.join(D, "verite_montants.csv")), on="id_ligne")
saisis["vraie_anomalie"] = saisis["anomalie"].notna()
print(len(saisis), "lignes dont", int(saisis["vraie_anomalie"].sum()), "anomalies injectées")
print(saisis["montant"].describe().round(1).to_string())
```

```
6000 lignes dont 85 anomalies injectées
count    6000.0
mean       74.7
std       570.0
min      -153.8
25%       17.5
50%       31.4
75%       54.2
max      26070.0
```

La médiane vaut 31,4 € et le troisième quartile 54,2 €, mais le maximum atteint 26 070 € : l'écart-type (570 €) est lui-même gonflé par les anomalies. Testons trois méthodes classiques, en mesurant pour chacune combien de lignes elle **signale**, combien sont de **vraies anomalies** (précision) et combien d'anomalies elle **retrouve** (rappel).

```
def bilan(nom, signal):
    vrais = int((signal & saisis["vraie_anomalie"]).sum())
    print(f"{nom:34s} signalées {int(signal.sum()):4d} | vraies {vrais:3d} | précision {vrais /
    ↪ signal.sum() * 100:5.1f} % | rappel {vrais / saisis['vraie_anomalie'].sum() * 100:5.1f} %")
m = saisis["montant"]
q1, q3 = m.quantile([0.25, 0.75])
bilan("règle 1,5 × EIQ", (m < q1 - 1.5 * (q3 - q1)) | (m > q3 + 1.5 * (q3 - q1)))
bilan("score z > 3 (moyenne, écart-type)", ((m - m.mean()) / m.std()).abs() > 3)
mad = (m - m.median()).abs().median()
bilan("score z robuste (MAD) > 3,5", (0.6745 * (m - m.median()) / mad).abs() > 3.5)
```

règle 1,5 × EIQ	signalées	412		vraies	61		précision	14.8 %		rappel	71.8 %
score z > 3 (moyenne, écart-type)	signalées	24		vraies	24		précision	100.0 %		rappel	28.2 %
score z robuste (MAD) > 3,5	signalées	324		vraies	55		précision	17.0 %		rappel	64.7 %

Les trois méthodes se trompent, **chacune à sa façon**.

- **La règle de l'écart interquartile** (une valeur est suspecte au-delà de $Q_3 + 1,5(Q_3 - Q_1)$, soit ici 109 €) signale 412 lignes, dont **351 sont de vrais montants élevés** (deux ou trois articles, un produit cher) : précision de 14,8 %. Elle retrouve 72 % des anomalies, mais au prix d'un tri fastidieux.
- **Le score z** (distance à la moyenne en écarts-types) ne signale que 24 lignes, **toutes de vraies anomalies**, mais ne retrouve que 28 % d'entre elles : les anomalies **gonflent l'écart-type** (570 € au lieu de 41 € sans elles) et fixent un seuil de 1 785 € que les petites erreurs n'atteignent pas. C'est l'effet de **masquage** : les erreurs cachent les erreurs.
- **Le score z robuste**, qui remplace la moyenne par la médiane et l'écart-type par la déviation absolue médiane (MAD), n'est pas gonflé par les anomalies, mais comme la distribution des montants est **très asymétrique** (beaucoup de petits montants, quelques grands, tous légitimes), il signale lui aussi des centaines de lignes valables.

Pour qui veut la formule. Le score z robuste vaut $0,6745(x - \text{médiane})/\text{MAD}$, où $\text{MAD} = \text{médiane}(|x_i - \text{médiane}|)$; le facteur 0,6745 le rend comparable au score z usuel quand la loi est normale. On le compare à un seuil de 3,5 (règle de Iglewicz et Hoaglin).

1.2.3 Les règles métier : regarder la relation, pas la valeur

La figure ci-dessus donne l'idée qui change tout : on ne regarde plus le montant **seul**, mais sa **relation** avec les autres colonnes. Un montant n'est pas plausible ou non en soi ; il l'est **par rapport à la quantité et au prix**. À la boutique, une ligne vaut quantité × prix unitaire, moins une remise de 0 à 20 % : le montant doit donc se situer entre 80 % et 100 % de ce produit.

```
attendu = saisis["quantite"] * saisis["prix_unitaire"]
rapport = saisis["montant"] / attendu
suspect = ~rapport.between(0.795, 1.005)
bilan("règle métier : 80 % à 100 % de qté × prix", suspect)
```

règle métier : 80 % à 100 % de qté × prix	signalées	85		vraies	85		précision	100.0 %		rappel	100.0 %
---	-----------	----	--	--------	----	--	-----------	---------	--	--------	---------

La règle signale **85 lignes, qui sont exactement les 85 anomalies**. Aucune méthode statistique, même fine, n'a cette précision, parce que la règle utilise une **connaissance du métier** (la remise maximale) que la formule ignore. Les règles métier sont les meilleurs détecteurs d'erreurs, à condition de les connaître et de les écrire.

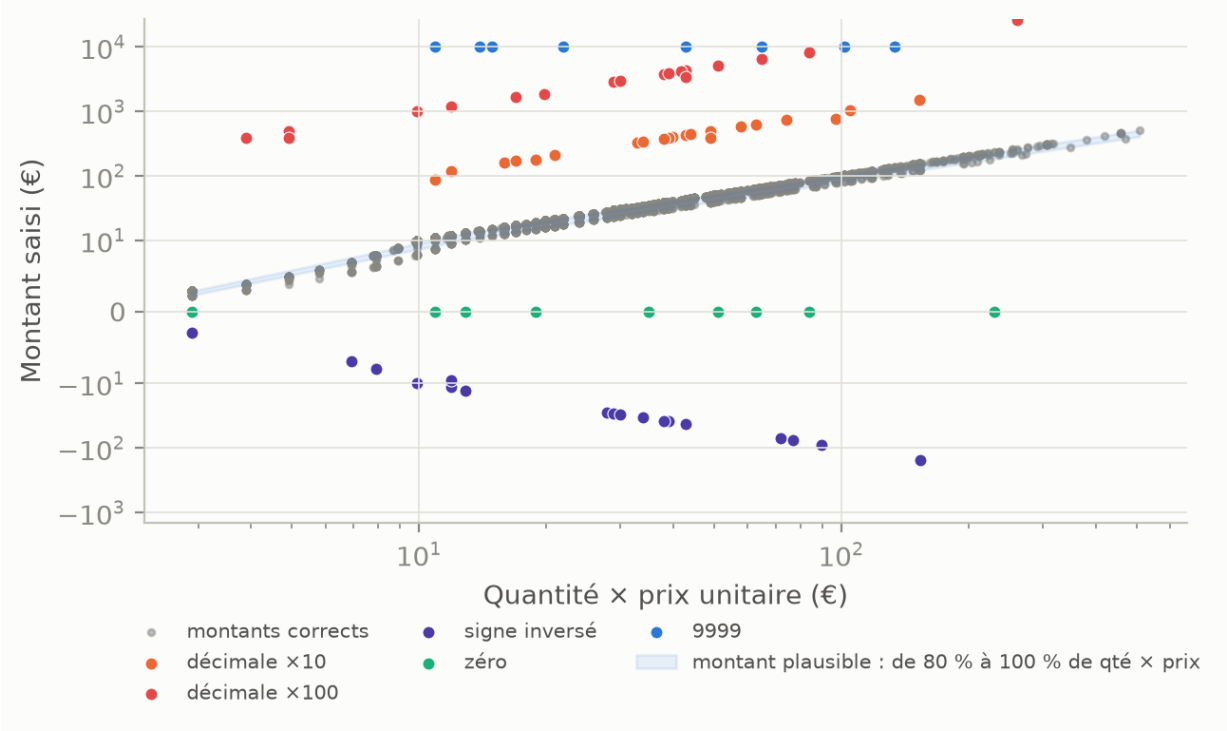


Figure 5: Montant saisi selon la valeur attendue (quantité × prix) : les montants corrects restent dans la bande de 80 % à 100 %, les anomalies s'en écartent.

Type de règle	Exemple	Ce qu'elle détecte
Plage de valeurs	une quantité entre 1 et 100, un montant positif	signe inversé, zéro
Relation entre colonnes	montant = quantité × prix, avec une remise ≤ 20 %	décimale décalée, 9999
Référence externe	le prix figure dans le catalogue	produit inexistant, mauvais prix
Cohérence temporelle	date de retour postérieure à la date de vente	dates inversées

À retenir. Les méthodes statistiques servent à **explorer** (« où regarder ? »), les règles métier à **décider** (« c'est une erreur »). Démarrez par une exploration statistique ; terminez par des règles que l'on peut défendre et rejouer.

Action	Principe	Quand
--------	----------	-------

1.2.4 Que faire d'une valeur aberrante ?

Quatre actions sont possibles. Le choix dépend de ce que l'on **sait** de la valeur et de la **question** posée.

Action	Principe	Quand
Corriger	remplacer par la bonne valeur, issue d'une source fiable	une table de référence existe
Signaler	ajouter une colonne « suspecte », sans modifier	on ne sait pas, ou on veut garder la trace
Plafonner (<i>winsoriser</i>)	ramener les valeurs extrêmes à un seuil (par exemple le 99 ^e centile)	on veut une moyenne moins sensible aux extrêmes, sans perdre de lignes
Supprimer	retirer la ligne	on est sûr que c'est une erreur et qu'aucune correction n'est possible

Mesurons ce que chaque choix fait à un total : celui des 6 000 lignes, dont nous connaissons la vraie valeur (255 631 €).

```
vrai_total = saisis["montant_vrai"].sum()
options = {"laisser tel quel": saisis["montant"].sum(),
           "supprimer les lignes suspectes": saisis.loc[~suspect, "montant"].sum(),
           "remplacer par qté × prix": np.where(suspect, attendu, saisis["montant"]).sum(),
           "reprendre le montant de la source": saisis["montant_vrai"].sum()}
for nom, total in options.items():
    print(f"{nom:34s} {total:10,.0f} € ({(total / vrai_total - 1) * 100:+6.1f} %)".replace(", ", " ")
```

laisser tel quel	447 950 €	(+75.2 %)
supprimer les lignes suspectes	251 897 €	(-1.5 %)
remplacer par qté × prix	255 711 €	(+0.0 %)
reprendre le montant de la source	255 631 €	(+0.0 %)

Laisser les 85 anomalies **gonfle le total de 75 %** : 85 lignes fausses sur 6 000 (1,4 %) suffisent à faire presque doubler le chiffre d'affaires. Supprimer les lignes suspectes **sous-estime** le total de 1,5 % (on retire de vraies ventes avec elles). Remplacer par quantité × prix (en supposant l'absence de remise) donne un écart de 0,03 % seulement : c'est la bonne correction **quand aucune source n'existe**, à condition de la décrire.

Terminons par le piège le plus coûteux. Voici les dépenses annuelles de nos 6 000 clients. Que se passe-t-il si l'on retire les « aberrants » au sens du score z ?

```
depense = pd.read_csv(os.path.join(D, "profil_clients_verite.csv"))["depense_2025"]
gros = (depense - depense.mean()) / depense.std() > 3
print(f"clients à plus de 3 écarts-types : {int(gros.sum())} ({gros.mean() * 100:.1f} %), ils font  
↳ {depense[gros].sum() / depense.sum() * 100:.1f} % du chiffre d'affaires")
print("moyenne :", round(depense.mean(), 1), "->", round(depense[~gros].mean(), 1), "| médiane :",  
↳ round(depense.median(), 1), "->", round(depense[~gros].median(), 1))
print("moyenne après plafonnement au 99e centile (", round(depense.quantile(0.99)), "€ ) :",  
↳ round(depense.clip(upper=depense.quantile(0.99)).mean(), 1))
```

clients à plus de 3 écarts-types : 126 (2.1 %), ils font 14.6 % du chiffre d'affaires
moyenne : 220.8 -> 192.5 | médiane : 98.5 -> 91.2
moyenne après plafonnement au 99e centile (1452 €) : 217.3

Ces 126 « aberrants » sont **les meilleurs clients de la boutique** : 2,1 % des clients, 14,6 % du chiffre d'affaires. Les supprimer fait chuter la dépense moyenne de 220,8 € à 192,5 € (−13 %) et ferait croire à la gérante que ses clients dépensent moins qu'ils ne le font. Le plafonnement au 99^e centile (1 452 €) garde tous les clients et ne déplace la moyenne que de 1,6 % : c'est une option raisonnable quand on veut une moyenne stable **sans nier l'existence** des gros clients.

Piège. Ne supprimez jamais une valeur **parce qu'elle est aberrante**, mais parce que vous **savez** qu'elle est fausse. Une valeur extrême vraie raconte la partie la plus intéressante de l'histoire : un gros client, une grosse commande, un incident. Si vous la retirez, écrivez-le (« 126 clients à plus de 3 écarts-types retirés, 14,6 % du CA ») et montrez le résultat avec et sans.

Pour s'entraîner. Cahier, chapitre 1 : application 1.3, exercices 1.4 et 1.5.

1.3 Doublons

Un doublon est un même fait enregistré deux fois : une commande exportée en double, un ticket scanné deux fois, un client saisi sous deux écritures. Il gonfle les totaux (le chiffre d'affaires, le nombre de clients), et il passe inaperçu, puisqu'une ligne en double ressemble à une ligne ordinaire. Cette section apprend à les **définir**, à les **repérer** même quand ils ne sont pas identiques, et à décider **lequel garder**.

1.3.1 Exact ou approché, clé naturelle ou clé technique

Un doublon **exact** est une ligne identique, caractère pour caractère, à une autre. Un doublon **approché** (ou « flou ») désigne la même réalité écrite autrement : « Mirela Dorvane » et « MIRELA DORVANE », « Mirela » et « M. ». Pour savoir si deux lignes se ressemblent *assez*, il faut d'abord décider **ce qui identifie** un enregistrement : sa **clé**.

- Une **clé technique** est un numéro attribué par le système (`id_crm`, `order_ref`). Elle est unique **par construction**... et donc inutile pour détecter un doublon : deux saisies du même client reçoivent deux numéros différents.
- Une **clé naturelle** est une propriété du monde réel qui identifie la chose : l'e-mail d'un client, le couple (ticket, produit) d'une ligne de caisse. Elle n'est unique que **si le monde le veut bien**.

pandas offre `duplicated()` (qui marque les lignes répétées) et `drop_duplicates()` (qui les retire). Leurs deux arguments à connaître sont `subset` (les colonnes qui forment la clé) et `keep` (garder la première occurrence, la dernière, ou aucune). Appliquons-les au fichier clients du CRM : 7 140 lignes pour 6 000 clients.

```
crm = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)
sans_id = crm.drop(columns="id_crm")
print("lignes :", len(crm), "| doublons exacts (hors identifiant technique) :",
      ↪ int(sans_id.duplicated().sum()))
print(crm[sans_id.duplicated(keep=False)][["prenom", "nom",
      ↪ "email"]].drop_duplicates().to_string(index=False))
```

```
lignes : 7140 | doublons exacts (hors identifiant technique) : 139
prenom nom          email
Test TEST test@example.com
```

Il y a 139 doublons exacts, et ce sont tous des **lignes de test** (« Test TEST », adresse test@example.com) : 140 lignes identiques qu'il faut écarter, mais qui ne sont pas des clients. Les vrais doublons du CRM sont **approchés** : les chercher avec duplicated() sur l'ensemble des colonnes ne trouve rien. Prenons une clé naturelle, l'e-mail, normalisée (sans espaces ni majuscules).

```
crm["email_cle"] = crm["email"].str.strip().str.lower()
reel = crm[crm["email_cle"].notna() & (crm["email_cle"] != "test@example.com")].copy()
doublons_email = reel.duplicated("email_cle", keep="first")
print("lignes avec un e-mail utilisable :", len(reel), "| marquées doublon par l'e-mail normalisé :",
      ↪ int(doublons_email.sum()))
```

```
lignes avec un e-mail utilisable : 6769 | marquées doublon par l'e-mail normalisé : 677
```

L'e-mail normalisé désigne 677 lignes comme des doublons. Combien le sont **vraiment** ? C'est ici que la vérité, que nous n'avons pas dans la vie réelle, permet de juger la clé.

```
vrai_crm = pd.read_csv(os.path.join(D, "verite_crm.csv"))
reel["id_crm"] = reel["id_crm"].astype(int)
reel = reel.merge(vrai_crm[["id_crm", "id_client"]], on="id_crm")
premier = reel.groupby("email_cle")["id_client"].transform("first")
a_tort = doublons_email.values & (reel["id_client"] != premier).values
print("marquées à tort (deux clients différents, même e-mail) :", int(a_tort.sum()))
reels = vrai_crm.loc[vrai_crm["id_client"] > 0, "id_client"]
print("doublons réels dans le fichier (hors test) :", int(reels.duplicated().sum()), "| retrouvés par
      ↪ l'e-mail :", int(doublons_email.sum() - a_tort.sum()))
```

```
marquées à tort (deux clients différents, même e-mail) : 3
doublons réels dans le fichier (hors test) : 1000 | retrouvés par l'e-mail : 674
```

Le bilan de cette clé : **674 vrais doublons retrouvés sur 1 000** (rappel de 67 %), **3 fausses alertes** (précision de 99,6 %). Deux enseignements. D'abord, une clé naturelle n'est jamais parfaite : ici, deux personnes différentes ont reçu la même adresse (des homonymes), et un tiers des doublons échappe à l'e-mail parce que leur e-mail est absent ou mal écrit. Ensuite, aucune clé unique ne suffit : on en **combine plusieurs** (e-mail, puis nom et date de naissance, puis téléphone) ou l'on passe à un **rapprochement approché**, objet de la section 2.5 du chapitre suivant.

Piège. Retirer les doublons sur **toutes** les colonnes ne retire que les copies parfaites. Retirer sur une clé **trop large** (le nom de famille seul) fusionne des personnes différentes. Dans les deux cas, on se trompe sans le voir : mesurez toujours le nombre de lignes retirées, et regardez-en un échantillon.

1.3.2 Les doublons d'export de la plateforme web

Le site de la boutique exporte ses commandes dans `site_commandes.csv`. Un export qu'on relance après un incident réécrit parfois les mêmes commandes : voici ce que cela donne, et ce que cela coûte. Nous ne regardons que la période de janvier à août, avant le changement d'unité de septembre (section 1.4.2), pour que les montants soient comparables.

```
site = pd.read_csv(os.path.join(D, "site_commandes.csv"), dtype=str)
print("lignes :", len(site), "| commandes distinctes :", site["order_ref"].nunique(), "| lignes
↳ strictement identiques :", int(site.duplicated().sum()))
av = site[site["created_at"] < "2025-09-01"].copy()
av["total_n"] = av["total"].str.replace("€", "", regex=False).str.replace(" ", "",
↳ regex=False).str.replace(",", ".", regex=False).astype(float)
```

```
lignes : 6259 | commandes distinctes : 6138 | lignes strictement identiques : 121
```

On compte 121 lignes de trop : toutes les lignes en double sont des **copies exactes**, faciles à retirer. Mais il y a **deux autres types de lignes qui ne doivent pas compter** dans le chiffre d'affaires : les commandes de test (adresse `test@example.com`) et les commandes annulées (statut `cancelled`, écrit aussi `paid` ou `PAID` pour les autres, en trois casses différentes, ce qu'il faudra normaliser). On enchaîne les trois filtres, en comptant à chaque étape.

```
a = av.drop_duplicates("order_ref")
b = a[~a["customer_email"].str.strip().str.lower().eq("test@example.com")]
c = b[b["status"].str.lower() != "cancelled"]
for nom, t in [("lignes brutes", av), ("sans doublons d'export", a), ("sans commandes de test", b),
↳ ("sans commandes annulées", c)]:
    print(f"{nom:26s} {len(t):5d} commandes {t['total_n'].sum():12,.2f} €".replace(",", " "))
```

lignes brutes	3493 commandes	358 029.54 €
sans doublons d'export	3431 commandes	350 778.16 €
sans commandes de test	3397 commandes	350 744.16 €
sans commandes annulées	3295 commandes	340 260.31 €

De janvier à août, le total brut est de 358 029,54 € pour 3 493 lignes. Les doublons en gonflent le nombre de 62 (1,8 %) et le montant de 7 251,38 € (2,1 %), les commandes de test de 34 €, et les commandes annulées de 10 484 € supplémentaires (3,1 %). Au total, **le chiffre d'affaires brut surestime le vrai de 5,2 %**. Le contrôle ultime, c'est la vérité : les commandes valides sont exactement 3 295, pour 340 260,31 €, comme l'enchaînement des trois filtres.

```
vs = pd.read_csv(os.path.join(D, "verite_site.csv"))
dates = site.drop_duplicates("order_ref").set_index("order_ref")["created_at"]
vs = vs[vs["default"].isna() & (vs["order_ref"].map(dates) < "2025-09-01")]
print("vérité : ", len(vs), "commandes valides,", f"{vs['total_vrai'].sum():12,.2f} €".replace(",", "
↳ "))
```

```
vérité : 3295 commandes valides, 340 260.31 €
```

Retenez la **méthode** : trois filtres, appliqués dans un ordre explicite, chacun accompagné de son compte. C'est ce qui permet de répondre à la question « d'où vient la différence entre mon chiffre et celui du site ? ».

1.3.3 Les doublons de scan en caisse : quand la copie est peut-être légitime

La caisse de la boutique pose un problème plus subtil. Quand une caissière scanne deux fois le même article par erreur, on obtient deux lignes identiques dans le même ticket. Mais **un client peut aussi acheter deux fois le même article** : deux lignes identiques légitimes. Rien, dans le fichier, ne distingue les deux cas. Lisons les douze fichiers (la fonction `lire_caisse`, qui absorbe leurs formats différents, est écrite en 1.4.6) et cherchons les lignes identiques.

```
cais = pd.concat([C.lire_caisse(f)[0] for f in sorted(glob.glob(os.path.join(D, "caisse",
↳ "*.csv")))], ignore_index=True)
cais["article"] = cais["article"].str.lower()
identiques = cais.duplicated(["ticket", "article", "quantite", "prix_unitaire", "montant"],
↳ keep="first")
print("lignes dans les fichiers :", len(cais), "| lignes identiques à une précédente :",
↳ int(identiques.sum()))
```

```
lignes dans les fichiers : 12678 | lignes identiques à une précédente : 163
```

163 lignes sont identiques à une ligne précédente du même ticket. Faut-il toutes les retirer ? Pour trancher, il faut une **référence** : la base de données de la boutique sait, elle, ce que contient chaque ticket. On rapproche chaque ligne de la caisse d'une ligne de la base, avec une clé (ticket, article, quantité, prix) complétée d'un **rang** : la première ligne « Plaid, 1, 22,56 € » d'un ticket correspond à la première de la base, la deuxième à la deuxième, et ainsi de suite. Une ligne de la caisse **sans correspondance** est une ligne en trop.

```
produits = pd.read_csv(os.path.join(D, "produits.csv"))[["id_produit", "nom_produit"]]
base = pd.read_csv(os.path.join(D, "lignes_commande.csv")).merge(pd.read_csv(os.path.join(D,
↳ "commandes.csv"))[["id_commande", "canal", "date_commande"]], on="id_commande").merge(produits,
↳ on="id_produit")
base = base[(base["canal"] == "Boutique") & (base["date_commande"] >= "2025-01-01")].copy()
base["article"] = base["nom_produit"].str.lower()
cle = ["id_commande", "article", "quantite", "prix_unitaire"]
for t in (cais, base):
    t["rang"] = t.groupby(cle).cumcount()
rapproche = cais.merge(base[cle + ["rang", "montant"]].rename(columns={"montant": "montant_base"}),
↳ on=cle + ["rang"], how="left", indicator=True)
en_trop = rapproche["_merge"] == "left_only"
print("lignes dans la base :", len(base), "| lignes de la caisse sans correspondance :",
↳ int(en_trop.sum()))
```

```
lignes dans la base : 12611 | lignes de la caisse sans correspondance : 67
```

La base compte 12 611 lignes pour les mêmes tickets : il y a **67 lignes en trop**, et non 163. Les **96 autres lignes « identiques »** sont de vraies ventes de deux exemplaires d'un même article. Mesurons ce que serait l'erreur d'un dédoublonnage sans référence.

```
fr = lambda x: f"{x:,.2f} €".replace(",", " ")
print("avec la référence :", int(en_trop.sum()), "lignes retirées, montant",
↳ fr(rapproche.loc[en_trop, "montant"].sum()))
print("sans la référence :", int(identiques.sum()), "lignes retirées, montant",
↳ fr(cais.loc[identiques, "montant"].sum()))
```

```
avec la référence : 67 lignes retirées, montant 3 126.29 €
sans la référence : 163 lignes retirées, montant 7 576.01 €
```

Avec la référence, on retire exactement 67 lignes (3 126,29 €). Sans elle, on en retire 163 (7 576,01 €) : **4 449,72 € de vraies ventes** auraient disparu. La réconciliation (chapitre 3) est la généralisation de cette idée : un doublon se prouve contre une **source indépendante**.

Remarque. Le fichier de caisse contient, à la dernière ligne de chaque mois, un **total**. La somme de ces douze totaux (560 973,91 €) est **exactement** le chiffre d'affaires de la base pour ces tickets : la caisse avait raison, c'est l'export qui a perdu des montants et ajouté des doublons (section 1.4.6). Un total affiché est un excellent point de contrôle à conserver.

1.3.4 Quel enregistrement garder ?

Une fois les doublons identifiés, il faut en garder **un**. Quatre critères sont courants.

Critère	Principe	Convient quand
Le plus récent	on garde la dernière saisie	les données changent (adresse, téléphone)
Le plus complet	on garde la ligne avec le moins de trous	les copies diffèrent par ce qu'elles contiennent
La source la plus fiable	on garde l'enregistrement du système de référence	une source fait autorité (le logiciel de caisse plutôt que le tableur)
La fusion	on garde, colonne par colonne, la première valeur renseignée	chaque copie a des morceaux utiles

Regardons une paire du CRM, retrouvée par l'e-mail normalisé : les clients 3921 et 3922.

```
paire = reel[reel["id_crm"].isin([3921, 3922])].sort_values("id_crm")[["id_crm", "prenom", "nom",
↪ "ville", "code_postal", "date_naissance"]]
print(paire.to_string(index=False))
```

```
id_crm prenom      nom  ville code_postal date_naissance
3921  Ardare Brentier  Vile A      NaN      05/02/1960
3922  ARDARE BRENTIER  Ville A    01601      1960-05-02
```

Les deux lignes décrivent la même personne. La première a une faute dans la ville (« Vile A ») et pas de code postal ; la seconde a tout, mais le nom en majuscules ; les dates de naissance ne sont pas écrites de la même façon (jour/mois/année et année-mois-jour). Garder **la plus complète** et combler ses trous avec l'autre est ce qui donne le meilleur enregistrement. En pandas, `groupby(...).first()` fait exactement cela : il prend, colonne par colonne, la première valeur **non vide**.

```
reel["manquants"] = reel.isna().sum(axis=1)
fusion = reel.sort_values(["email_cle", "manquants"]).groupby("email_cle", as_index=False).first()
print("lignes avant :", len(reel), "| après fusion par e-mail :", len(fusion))
print("codes postaux manquants avant :", int(reel["code_postal"].isna().sum()), "| après :",
↪ int(fusion["code_postal"].isna().sum()))
```

```
lignes avant : 6769 | après fusion par e-mail : 6092
codes postaux manquants avant : 408 | après : 328
```

On passe de 6 769 lignes à 6 092 fiches, et le nombre de codes postaux manquants de 408 à 328 : la fusion a **comblé 80 trous** que chaque copie, prise seule, ne comblait pas. Les 231 lignes sans e-mail ne sont pas fusionnées (elles échappent à cette clé) : elles seront rapprochées autrement, au chapitre suivant (section 2.5).

À retenir. Un traitement de doublons comprend **quatre actes** : (1) définir la clé, (2) repérer, **en mesurant** ce qui est retrouvé et ce qui est faussement signalé, (3) choisir l'enregistrement

à garder, (4) **journaliser** ce que l'on a retiré (garder une table des lignes écartées avec la raison). Un total qui change après dédoublonnage doit pouvoir s'expliquer ligne à ligne.

Pour s'entraîner. Cahier, chapitre 1 : application 1.4, exercices 1.6 et 1.7.

1.4 Incohérences et erreurs de format

Dernière famille de défauts, et la plus variée : tout ce qui est *écrit* correctement... dans un autre format que celui que l'on attendait. Un nombre qui contient un symbole monétaire, une unité qui change sans prévenir, une date que l'on peut lire de deux façons, une ville écrite de six manières, un logiciel qui modifie le format de ses exports d'un mois à l'autre. Ces défauts ne se voient pas dans un tableau de comptes ; il faut aller **regarder les valeurs**.

1.4.1 Les types : lire d'abord en texte, convertir explicitement

Un fichier CSV ne contient que du texte : c'est le logiciel de lecture qui **devine** les types. Cette devinette est la première source de surprises. L'export du site, par exemple, écrit le total des commandes de trois façons différentes. Lisons-le **sans rien deviner** (`dtype=str`), puis demandons à pandas ce qu'il sait convertir directement.

```
site = pd.read_csv(os.path.join(D, "site_commandes.csv"), dtype=str)
direct = pd.to_numeric(site["total"], errors="coerce")
print("montants convertis directement :", int(direct.notna().sum()), "sur", len(site))
print("exemples de montants qui résistent :", site.loc[direct.isna(), "total"].head(4).tolist())
```

```
montants convertis directement : 4004 sur 6259
exemples de montants qui résistent : ['34,92 €', '63,66 €', '195,40 €', '81,07 €']
```

Seuls 4 004 totaux sur 6 259 se convertissent tels quels : les autres portent un symbole « € », une virgule décimale ou un espace de milliers (« 1 245,00 »). Ils ne sont pas faux, ils sont **écrits pour des humains**. On écrit une petite fonction de conversion, et l'on **vérifie** qu'aucune valeur ne lui échappe.

```
def nombre(texte):
    """« 1 245,00 € » -> 1245.0 ; « 45.9 » -> 45.9"""
    return float(texte.replace("€", "").replace(" ", "").replace(",", ".").strip())
site["total_n"] = site["total"].map(nombre)
print(site["total_n"].describe().round(1).to_string())
```

```
count    6259.0
mean     3996.3
std      6926.6
min       1.0
25%      68.9
50%     170.6
75%    5954.0
max    62883.0
```

Aucune erreur de conversion, mais **ce résumé est suspect** : une commande médiane de 171 € tandis que le troisième quartile vaut 5 954 € et le maximum 62 883 € ? La boutique ne vend pas de commandes à 66 000 €. La conversion a réussi sur le plan technique ; il reste un défaut d'**unité**, que la section suivante traque.

Piège. Deux erreurs de type sont particulièrement coûteuses. **Les identifiants lus comme des nombres** : le code postal 01601 devient 1601.0 (le zéro de tête disparaît, et la colonne passe en nombre décimal si elle contient des trous). **Les dates lues comme du texte** : le tri alphabétique range « 10/02/2025 » avant « 2/03/2025 ». Les identifiants se lisent en **texte**, les dates se **convertissent explicitement avec un format**.

```
crm_default = pd.read_csv(os.path.join(D, "crm_clients.csv"))
zero = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)["code_postal"].str.startswith("0",
↳ na=False)
print("type deviné :", crm_default["code_postal"].dtype, "| codes commençant par 0 lus ainsi :",
↳ crm_default.loc[zero, "code_postal"].head(3).tolist())
```

```
type deviné : float64 | codes commençant par 0 lus ainsi : [5723.0, 9321.0, 5723.0]
```

1.4.2 Les unités mélangées : le changement de septembre

Revenons à la médiane suspecte. Une unité qui change **au milieu** d'un fichier se détecte en regardant le chiffre **dans le temps**. Calculons, mois par mois, le montant médian d'une commande du site.

```
site["dt"] = pd.to_datetime(site["created_at"].str.replace("Z", "", regex=False), format="mixed")
site["mois"] = site["dt"].dt.strftime("%Y-%m")
print(site.groupby("mois")["total_n"].median().round(0).rename_axis(None).to_string())
```

```
2025-01      74.0
2025-02      82.0
2025-03      84.0
2025-04      77.0
2025-05      80.0
2025-06      86.0
2025-07      90.0
2025-08      86.0
2025-09    1641.0
2025-10    8396.0
2025-11    7087.0
2025-12    7808.0
```

De janvier à août, la commande médiane vaut entre 74 € et 90 €. En septembre elle bondit à 1 641 €, puis oscille autour de 7 000 à 8 400 € jusqu'en décembre : un facteur de **près de cent**. Septembre est un mois « de transition », avec une médiane intermédiaire : cela signale un changement **en cours de mois**. On cherche le jour exact.

```
quotidien = site.set_index("dt")["total_n"].resample("D").median()
for jour, valeur in quotidien["2025-09-12":"2025-09-17"].items():
    print(jour.date(), round(valeur))
```

```
2025-09-12  97
2025-09-13  79
2025-09-14  49
2025-09-15 11676
2025-09-16 6979
2025-09-17 4419
```

La rupture est nette : **le 15 septembre**, le montant d'une commande est multiplié par cent environ. La plateforme a commencé à exporter les totaux **en centimes**, sans que personne l'annonce. Cette erreur est la plus dangereuse de toutes : aucune valeur n'est invalide, aucun contrôle de plage simple ne sonne, et la somme annuelle du site (plus de 25 millions) est faussée d'un facteur quarante.

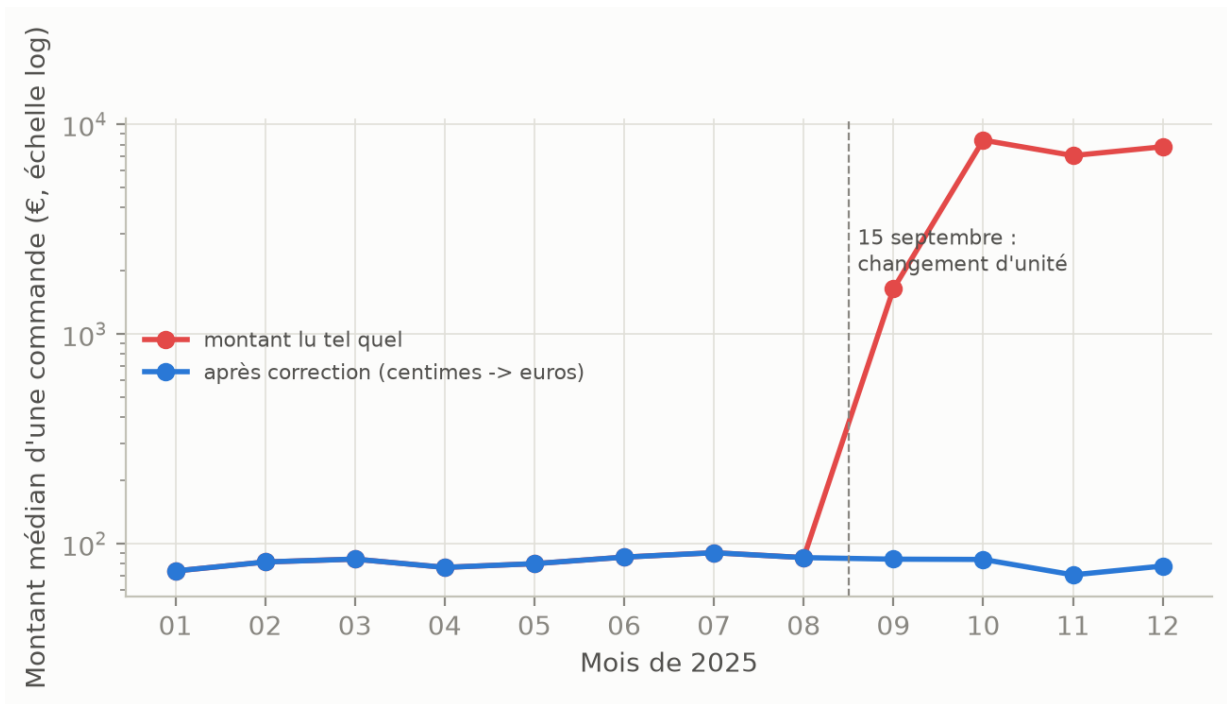


Figure 6: Montant médian d'une commande du site, par mois, avant et après correction de l'unité (échelle logarithmique).

Pour corriger, on applique la division par cent à **partir de la date de rupture**, puis on vérifie. La correction est justifiée par trois indices : la rupture est datée, son rapport est de cent, et elle touche **toutes** les lignes après le 15 septembre.

```
site["total_corrige"] = np.where(site["dt"] >= "2025-09-15", site["total_n"] / 100, site["total_n"])
verite_site = pd.read_csv(os.path.join(D, "verite_site.csv")).drop_duplicates("order_ref")
verifie = site.merge(verite_site[verite_site["default"] != "test"], on="order_ref")
print("lignes comparées à la vérité :", len(verifie), "| écart maximal :",
      round((verifie["total_corrige"] - verifie["total_vrai"]).abs().max(), 2), "€")
```

lignes comparées à la vérité : 6199 | écart maximal : 0.0 €

Sur les 6 199 commandes comparables, l'écart maximal est de **0,00 €** : la correction retrouve exactement les montants d'origine. Reprenons maintenant le chiffre d'affaires du site, en appliquant dans l'ordre les corrections vues depuis 1.3 : doublons, tests, annulées, unité.

```
propre = site.drop_duplicates("order_ref")
propre = propre[~propre["customer_email"].str.strip().str.lower().eq("test@example.com") &
               (propre["status"].str.lower() != "cancelled")]
vraie = verite_site[verite_site["default"].isna()]
print("chiffre d'affaires propre :", f"{propre['total_corrige'].sum():.2f}".replace(",", " "), "€
      sur", len(propre), "commandes | vérité :", f"{vraie['total_vrai'].sum():.2f}".replace(",", " "),
      "€ sur", len(vraie))
```

chiffre d'affaires propre : 600 164.13 € sur 5897 commandes | vérité : 600 164.13 € sur 5897

Après les quatre corrections, le chiffre d'affaires du site est de **600 164,13 € sur 5 897 commandes**, **identique** à la vérité, alors que la somme brute valait plus de 25 millions : un facteur quarante.

À retenir. Pour détecter un changement d'unité ou de format : (1) **regardez la distribution dans le temps** (médiane par mois ou par jour) ; (2) cherchez une **rupture datée** et un **rapport**

simple (100, 1 000, 1,2 pour une TVA) ; (3) corrigez à **partir de la date**, jamais « là où la valeur est grande » ; (4) **vérifiez** par une source indépendante. Et prévenez l'équipe qui exploite la plateforme : l'erreur se reproduira.

1.4.3 Les dates ambiguës : jour/mois ou mois/jour ?

Les dates sont le champ de mines des formats. Dans le CRM, les dates de naissance sont écrites sous trois formes : 1960-05-02 (année-mois-jour, sans ambiguïté), 2 mai 1960 (en toutes lettres, sans ambiguïté non plus) et 05/02/1960, qui est **ambiguë** : le 5 février, ou le 2 mai ? Voyons ce que l'on peut déduire des valeurs elles-mêmes.

```
crm = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)
crm["id_crm"] = crm["id_crm"].astype(int)
crm = crm.merge(pd.read_csv(os.path.join(D, "verite_crm.csv")), on="id_crm")
crm = crm[crm["id_client"] > 0].copy() # on met de côté les 140 lignes de test
slash = crm["date_naissance"].str.fullmatch(r"\d\d\d\d\d\d{4}")
champ1, champ2 = (pd.to_numeric(crm.loc[slash, "date_naissance"].str[a:b]) for a, b in ((0, 2), (3,
↪ 5)))
print("dates avec des « / » :", int(slash.sum()), "sur", len(crm))
print("jour/mois certain (1er champ > 12) :", int((champ1 > 12).sum()), "| mois/jour certain (2e champ
↪ > 12) :", int((champ2 > 12).sum()), "| ambiguës :", int(((champ1 <= 12) & (champ2 <= 12)).sum()))
```

```
dates avec des « / » : 4534 sur 7000
jour/mois certain (1er champ > 12) : 2083 | mois/jour certain (2e champ > 12) : 634 | ambiguës : 1829
```

Sur 4 534 dates avec des « / », 2 083 sont **certainement** écrites jour/mois (le premier nombre dépasse 12), 634 **certainement** mois/jour (le deuxième dépasse 12), et **1 829 sont ambiguës**. Une partie de la saisie du CRM a donc été faite à l'américaine, et rien dans le fichier ne l'annonce. Peut-on s'aider d'une autre colonne ? Regardons si la source de saisie (caisse, site, import) explique le format, **en utilisant la vérité** pour savoir quelles lignes sont vraiment américaines.

```
americaine = crm["defauts"].fillna("").str.contains("américain")
print((americaine.groupby(crm["source_saisie"]).mean() * 100).round(1).to_string())
ambigues_fausses = americaine[slash] & (champ1 <= 12) & (champ2 <= 12) & (champ1 != champ2)
print("dates américaines lues à tort en jour/mois, sans erreur visible :",
↪ int(ambigues_fausses.sum()), "sur", len(crm), "lignes")
```

```
source_saisie
caisse    14.8
import    15.9
site      15.0
dates américaines lues à tort en jour/mois, sans erreur visible : 398 sur 7000 lignes
```

La part de dates à l'américaine est la même (environ 15 %) quelle que soit la source : **aucune colonne ne permet de trancher**. Si l'on suppose partout « jour/mois », les dates américaines dont le deuxième nombre dépasse 12 deviennent impossibles (« 05/27/1970 » donne un mois 27) et se détectent ; en revanche, celles dont les deux nombres sont inférieurs ou égaux à 12 se lisent **sans erreur... mais fausses**. On en compte **398**, soit 5,7 % des 7 000 lignes : près d'une date de naissance sur dix-huit est fautive, sans le moindre message d'erreur.

Intuition. Une date ambiguë n'est pas un défaut de la donnée, c'est un défaut de **l'information** : le format n'a pas été transmis. Il se règle à la source (demander au service qui a saisi, imposer le format ISO AAAA-MM-JJ dans les exports), pas par un algorithme. L'algorithme ne peut que **mesurer** le risque : combien de lignes sont indécidables ?

Un choix défendable consiste à lire en jour/mois (hypothèse majoritaire : plus des trois quarts

des dates certaines le sont), à **basculer** en mois/jour quand le jour est impossible, et à **signaler** le risque pour les dates ambiguës.

```
def lire_naissance(s):
    if re.fullmatch(r"\d\d/\d\d/\d{4}", s) and int(s[3:5]) > 12:      # 2e champ impossible comme
        ↪ mois : c'est mm/jj
        return C.date_mixte(s, americain=True)
    return C.date_mixte(s)                                           # sinon jj/mm (pari, risqué si
        ↪ ambiguë)
crm["naissance"] = crm["date_naissance"].map(lire_naissance)
print("dates illisibles (NaT) :", int(crm["naissance"].isna().sum()), "| lues :",
    ↪ int(crm["naissance"].notna().sum()))
```

```
dates illisibles (NaT) : 44 | lues : 6956
```

Il reste 44 dates **illisibles** (le 31 février, le « 00/00/0000 », le mois 13) : elles ne se corrigent pas, elles se **signalent**. Ce sont des vraies erreurs de saisie, que la section 1.4.5 traitera par des règles de cohérence.

Piège. Une lecture « qui marche » n'est pas une lecture juste. `pd.to_datetime("05/02/1960")` lit par défaut en mois/jour ; `dayfirst=True` lit en jour/mois ; dans les deux cas, **aucune erreur n'est levée**. Écrivez toujours le **format attendu** (`format="%d/%m/%Y"`) : il échoue bruyamment sur ce qu'il ne comprend pas, c'est précisément ce que l'on souhaite.

1.4.4 Les catégories écrites de six façons

Une colonne catégorielle (la ville, le canal, le consentement) ne devrait contenir qu'une poignée de valeurs. Les saisies libres en produisent des dizaines. Combien d'écritures distinctes du nom de la ville dans le CRM ?

```
print("écritures distinctes de la ville :", crm["ville"].nunique())
print(crm["ville"].value_counts().head(6).to_string())
```

```
écritures distinctes de la ville : 119
ville
Ville A      618
Ville B      507
Ville C      440
Ville D      391
Ville E      331
Ville F      261
```

On compte 119 écritures pour 20 villes. Les plus fréquentes sont propres ; la longue traîne contient des **majuscules** (« VILLE A »), des **minuscules** (« ville a »), des **fautes** (« Vile A »), un **point final** (« Ville A. ») et, pour environ 2,7 % des lignes, l'**arabe** (« أ المدينة » : « la ville A »). Regardons les écritures de la ville A.

```
print(sorted(crm.loc[crm["ville"].str.strip().str.lower().str.rstrip(".").isin(["ville a", "vile a"])]
    ↪ | crm["ville"].str.endswith("أ"), "ville"].unique()))
```

```
['VILLE A', 'Vile A', 'Ville A', 'Ville A.', 'ville a', 'أ المدينة']
```

Six écritures pour une seule ville. La méthode est toujours la même : une **fonction de normalisation** (retirer les espaces et le point, mettre la casse, corriger la faute connue, traduire l'arabe) et, surtout, une **table de correspondance** conservée avec le traitement, pour que l'on sache d'où vient chaque valeur.

```
def ville_propre(s):
    s = s.strip()
    if s.startswith("المدينة"):
        return "Ville " + chr(65 + C.AR.index(s.split()[-1]))
    return C.normaliser_ville(s)
crm["ville_propre"] = crm["ville"].map(ville_propre)
correspondance = crm.groupby(["ville_propre", "ville"]).size().rename("lignes").reset_index()
print("écritures distinctes après nettoyage :", crm["ville_propre"].nunique(), "| lignes dans la
↳ table de correspondance :", len(correspondance))
```

écritures distinctes après nettoyage : 20 | lignes dans la table de correspondance : 119

On passe de 119 à 20 valeurs, avec une table de 119 lignes qui documente chaque substitution. Reste à **vérifier** : la ville nettoyée est-elle la vraie ville du client ?

```
vraie_ville = crm.merge(pd.read_csv(os.path.join(D, "clients.csv"))[["id_client",
↳ "ville"]].rename(columns={"ville": "ville_vraie"}), on="id_client")
print("villes nettoyées identiques à la vérité :", round((vraie_ville["ville_propre"] ==
↳ vraie_ville["ville_vraie"]).mean() * 100, 1), "% de", len(vraie_ville), "lignes")
```

villes nettoyées identiques à la vérité : 100.0 % de 7000 lignes

Le consentement marketing est un autre exemple, plus délicat : sept écritures (oui, Oui, OUI, 0, 1, TRUE, vide).

```
consentement = {"oui": True, "o": True, "1": True, "true": True, "non": False}
crm["consentement_propre"] = crm["consentement_marketing"].str.strip().str.lower().map(consentement)
print(crm["consentement_propre"].value_counts(dropna=False).to_string())
```

```
consentement_propre
True      4839
NaN       2161
```

Parmi les 7 000 lignes, 4 839 expriment un consentement ; **2 161 sont vides**. Le point important : un vide n'est **pas un « non »**. Ce n'est pas non plus un « oui ». C'est l'absence de réponse, qui se traite comme un **manquant**, avec une conséquence concrète : sans consentement explicite, on n'envoie pas de message promotionnel (chapitre 5 sur la confidentialité). La valeur False n'apparaît pas dans ces lignes : le seul « non » du fichier d'origine se trouve dans les lignes de test.

1.4.5 Les règles de cohérence entre colonnes

Une valeur peut être correcte **seule** et absurde **avec une autre** : une date d'inscription antérieure à la naissance, un code postal de quatre chiffres, une adresse électronique sans arobase. Ces **règles de cohérence** s'écrivent comme de petites expressions logiques ; on compte les lignes en infraction.

```
ins = pd.to_datetime(crm["date_inscription"], format="%d/%m/%Y")
lisible = crm["naissance"].notna()
regles = {"naissance_lisible": lisible,
    "naissance entre 1920 et 2010 (parmi les lisibles)": ~lisible |
    ↳ crm["naissance"].between("1920-01-01", "2010-12-31"),
    "inscription après la naissance": ~lisible | (ins >= crm["naissance"]),
    "code postal de 5 chiffres (quand il est renseigné)": crm["code_postal"].isna() |
    ↳ crm["code_postal"].str.fullmatch(r"\d{5}"),
    "e-mail de forme valide (quand il est renseigné)": crm["email"].isna() |
    ↳ crm["email"].str.fullmatch(r"(?!.*\.\.)([w.+-]+@[w-]+\.\.[w.]+)")}
for nom, ok in regles.items():
    print(f"{nom:52s} {int((~ok).sum()):4d} lignes en infraction")
```

naissance lisible	44 lignes en infraction
naissance entre 1920 et 2010 (parmi les lisibles)	24 lignes en infraction
inscription après la naissance	11 lignes en infraction
code postal de 5 chiffres (quand il est renseigné)	199 lignes en infraction
e-mail de forme valide (quand il est renseigné)	95 lignes en infraction

Chaque règle compte ses infractions, et chacune se **vérifie** contre la vérité : les 44 dates illisibles et les 24 dates hors de l'intervalle 1920-2010 font **68 dates impossibles**, exactement le nombre injecté. Les 199 codes postaux de quatre chiffres sont ceux dont le zéro initial a été perdu (on les **répare** en les complétant à gauche : `.str.zfill(5)`), et les 95 adresses invalides viennent de doubles arobases ou de points consécutifs (une expression régulière trop permissive laisse passer ces derniers : on les interdit explicitement avec `(?!.*\.\. )`). Quant aux 11 infractions de la règle « inscription après la naissance », ce sont les dates de naissance fixées en 2030.

```
cp_repare = crm["code_postal"].str.zfill(5)
print("codes postaux de 5 chiffres après réparation :", int(cp_repare.str.fullmatch(r"\d{5}").sum()),
      ↪ "sur", int(cp_repare.notna().sum()), "renseignés")
```

codes postaux de 5 chiffres après réparation : 6576 sur 6576 renseignés

En pratique. Écrivez vos règles **dans une table** (nom, formule, nombre d'infractions, décision) et rejouez-la à chaque nouvelle livraison de données. C'est le début d'un contrôle de qualité (chapitre 3). Et distinguez **ce qui se répare** (un zéro perdu), **ce qui se signale** (une date impossible) et **ce qui se demande** : combien de clients ont moins de 18 ans à l'inscription ? La boutique accepte-t-elle les mineurs ? Une règle métier **inventée** vaut moins qu'une question posée à la gérante.

1.4.6 Quand le schéma change d'un fichier à l'autre

Le logiciel de caisse envoie un fichier par mois. Il semble identique d'un mois à l'autre ; il ne l'est pas. Regardons l'en-tête et la première ligne de trois mois.

```
for mois in ("01", "07", "10"):
    brut = open(os.path.join(D, "caisse", f"caisse_2025-{mois}.csv"), "rb").read()
    texte = (brut.decode("cp1252") if mois == "01" else brut.decode("utf-8-sig")).splitlines()
    print(mois, "|", texte[3][:78], "\n   |", texte[4][:78])
```

```
01 | N° ticket;Date;Heure;Article;Catégorie;Qté;Prix unitaire;Montant
   | T23468;01/01/2025;11:18;Poêle mat;Cuisine;1;40,07;40,07
07 | N° ticket;Date;Heure;Article;Catégorie;Quantité;Prix unitaire;Montant
   | T29050;01/07/25;10:59;Jardinière design;Jardin;1;51,40;51,40
10 | Ticket,Date,Heure,Article,Catégorie,Qté,Prix unitaire,Remise (%),Montant
   | "T31901","01/10/2025","10:05","Jardinière design","Jardin","3","51.40","0","15
```

Les trois mois ont **trois formats** : encodage cp1252 puis UTF-8, séparateur ; puis ,, virgule puis point décimal, « Qté » puis « Quantité », date 01/07/25 à deux chiffres pour l'année, une colonne « Remise (%) » ajoutée, des champs entre guillemets. On ne lit pas douze fichiers avec douze scripts : on écrit **une fonction de lecture qui détecte** le format, en trois petites étapes. La première **ouvre** le fichier en essayant UTF-8 puis l'ancien encodage Windows.

```
def ouvrir(fichier):
    """lignes du fichier, en essayant UTF-8 puis l'ancien encodage Windows"""
    brut = open(fichier, "rb").read()
    try:
        return brut.decode("utf-8-sig").splitlines(), "utf-8"
    except UnicodeDecodeError:
        return brut.decode("cp1252").splitlines(), "cp1252"
```

La deuxième **découpe** : elle saute les lignes de titre, repère l'en-tête, déduit le séparateur, retire les en-têtes répétés à chaque « page » et lit le **total affiché** à la dernière ligne.

```
est_entete = lambda l: re.match(r'^"? (N° ticket|Ticket)', l) is not None
def decouper(lignes):
    """(en-tête, lignes de données, séparateur, total affiché)"""
    i0 = next(i for i, l in enumerate(lignes) if est_entete(l))
    sep = ";" if lignes[i0].count(";") > lignes[i0].count(",") else ","
    total = float(lignes[-1].split(sep)[-1].strip(' ').replace(",", "."))
    return lignes[i0], [l for l in lignes[i0 + 1:-1] if not est_entete(l)], sep, total
```

La troisième **uniformise** les noms de colonnes, convertit les nombres et les dates, et assemble le tout.

```
RENOMMER = {"N° ticket": "ticket", "Ticket": "ticket", "Qté": "quantite", "Quantité": "quantite",
            "Date": "date", "Heure": "heure", "Article": "article",
            "Catégorie": "categorie", "Prix unitaire": "prix_unitaire", "Remise (%)": "remise_pct",
            "Montant": "montant"}
def lire_caisse(fichier):
    lignes, enc = ouvrir(fichier)
    entete, corps, sep, total = decouper(lignes)
    t = pd.read_csv(io.StringIO("\n".join([entete] + corps)), sep=sep,
                    dtype=str).rename(columns=RENOMMER)
    for c in ("prix_unitaire", "montant"):
        t[c] = pd.to_numeric(t[c].str.replace(",", "."), errors="coerce")
    t["date"] = pd.to_datetime(t["date"], format="%d/%m/%y" if len(t["date"].iloc[0]) == 8 else
                              "%d/%m/%Y")
    t["quantite"], t["id_commande"] = t["quantite"].astype(int), t["ticket"].str[1:].astype(int)
    return t, total, f"{enc}, séparateur « {sep} »"
```

Reste le plus important : **vérifier**. Chaque fichier se termine par un total affiché. Si notre lecture est fidèle, la somme des montants lus doit retrouver ce total, ou bien l'écart doit **s'expliquer**.

```
fichiers = sorted(glob.glob(os.path.join(D, "caisse", "*.csv")))
for f in fichiers:
    t, total, desc = lire_caisse(f)
    print(f"{os.path.basename(f)[7:14]} {desc:26s} {len(t):5d} lignes
          {int(t['montant'].isna().sum()):3d} vides somme {t['montant'].sum():9.2f} total
          {total:9.2f} écart {total - t['montant'].sum():8.2f}")
```

2025-01 cp1252, séparateur « ; »	955 lignes	36 vides	somme	37826.31	total	38882.41	écart
↳ 1056.10							
2025-02 cp1252, séparateur « ; »	770 lignes	22 vides	somme	32273.46	total	33079.41	écart
↳ 805.95							
2025-03 cp1252, séparateur « ; »	916 lignes	28 vides	somme	37863.67	total	39038.90	écart
↳ 1175.23							
2025-04 cp1252, séparateur « ; »	1013 lignes	31 vides	somme	45426.14	total	45832.57	écart
↳ 406.43							
2025-05 cp1252, séparateur « ; »	1028 lignes	31 vides	somme	43381.31	total	44905.92	écart
↳ 1524.61							
2025-06 cp1252, séparateur « ; »	973 lignes	30 vides	somme	42277.42	total	43118.16	écart
↳ 840.74							
2025-07 utf-8, séparateur « ; »	877 lignes	21 vides	somme	41035.68	total	41595.82	écart
↳ 560.14							
2025-08 utf-8, séparateur « ; »	814 lignes	20 vides	somme	42350.83	total	42873.50	écart
↳ 522.67							
2025-09 utf-8, séparateur « ; »	1048 lignes	31 vides	somme	45600.67	total	46354.25	écart
↳ 753.58							
2025-10 utf-8, séparateur « , »	1138 lignes	38 vides	somme	49839.95	total	51321.48	écart
↳ 1481.53							
2025-11 utf-8, séparateur « , »	1452 lignes	48 vides	somme	59096.64	total	60586.62	écart
↳ 1489.98							
2025-12 utf-8, séparateur « , »	1694 lignes	63 vides	somme	70924.34	total	73384.87	écart
↳ 2460.53							

La fonction lit les douze fichiers, quel que soit leur format. Chaque mois présente un **écart positif** (le total affiché dépasse la somme lue) et un nombre de montants **vides** : c'est la signature de la perte de montants. Mais les lignes en double, elles, font l'inverse (elles ajoutent des montants que le total n'inclut pas). L'écart est-il donc **entièrement expliqué** ? C'est le moment de reprendre le tableau rapproche de la section 1.3.3, qui rapproche chaque ligne de la caisse d'une ligne de la base.

```
lue = rapproche["montant"].sum()
vides_retrouves = rapproche.loc[rapproche["montant"].isna() & ~en_trop, "montant_base"].sum()
doubles = rapproche.loc[en_trop, "montant"].sum()
affiche = sum(lire_caisse(f)[1] for f in fichiers)
print(f"somme lue {lue:,.2f} - lignes en double {doubles:,.2f} + montants vides retrouvés dans la base
↳ {vides_retrouves:,.2f} = {lue - doubles + vides_retrouves:,.2f} €".replace(",", " "))
print("total affiché par la caisse (somme des douze fichiers) :", f"{affiche:,.2f} €".replace(",", "
↳ "))
```

```
somme lue 547 896.42 - lignes en double 3 126.29 + montants vides retrouvés dans la base 16 203.78 =
↳ 560 973.91 €
total affiché par la caisse (somme des douze fichiers) : 560 973.91 €
```

L'écart global (13 077,49 €) s'explique **au centime près** : les montants vides en retranchent 16 203,78 €, les lignes en double en ajoutent 3 126,29 €. C'est la forme la plus satisfaisante du contrôle : non pas « ça a l'air bon », mais « l'écart est expliqué, ligne à ligne ».

À retenir. Une lecture robuste d'un fichier qui évolue suit quatre principes : **détecter** (encodage, séparateur, en-tête) plutôt que supposer ; **tout lire en texte** puis convertir explicitement ; **uniformiser** les noms de colonnes dans une table de correspondance ; **contrôler** chaque fichier à l'aide d'un point fixe (le total affiché) et expliquer les écarts. Et gardez le **fichier d'origine** intact.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.5 et 1.6, exercices 1.8 et 1.9.

1.5 ⊕ Pour aller plus loin : texte, dates et heures, encodage, données multilingues

Section complémentaire. Elle prolonge la section 1.4 par les défauts propres au **texte** : espaces, casse, accents, caractères invisibles, encodage, et l'arabe qui s'invite dans une colonne de villes. Le reste du chapitre ne la suppose pas.

Un tableau de nombres se nettoie avec des comparaisons ; un tableau de **texte** se nettoie avec des conventions, parce que deux chaînes qui paraissent identiques à l'écran peuvent être différentes pour l'ordinateur. Cette section en donne les quelques règles qui évitent 90 % des mauvaises surprises.

1.5.1 Espaces, casse et accents : comparer sans se tromper

Deux fichiers parlent des mêmes produits : le catalogue de la boutique et celui du fournisseur. Pour savoir combien de désignations du fournisseur correspondent à un produit de la boutique, on les compare. Voyons ce que donne une comparaison naïve, puis une comparaison **normalisée**.

```
cat = pd.read_csv(os.path.join(D, "catalogue_fournisseur.csv"), dtype=str)
noms = set(pd.read_csv(os.path.join(D, "produits.csv"))["nom_produit"])
print("désignations du fournisseur :", len(cat), "| noms distincts à la boutique :", len(noms))
print("correspondance exacte :", int(cat["designation"].isin(noms).sum()))
print(cat["designation"].head(4).tolist())
```

```
désignations du fournisseur : 118 | noms distincts à la boutique : 60
correspondance exacte : 34
[' Pochette compact ', 'PLANCHE COMPACT', 'COUSSIN MAT', 'Diffu. design']
```

Sur 118 désignations, seules 34 correspondent exactement à un nom de la boutique. Les exemples montrent pourquoi : des espaces superflus (' Pochette compact '), des majuscules (' PLANCHE COMPACT '), des abréviations ('Diffu. design'). Les deux premiers défauts se corrigent par **normalisation** ; le troisième exige un rapprochement approché (section 2.5).

On construit une **clé de comparaison** : on retire les espaces aux extrémités, on réduit les espaces multiples, on passe en minuscules et on supprime les accents. Cette clé ne remplace pas le texte d'origine (on garde l'original pour l'affichage) : elle sert uniquement à **comparer**.

```
from unidecode import unidecode
def cle(texte):
    return re.sub(r"\s+", " ", unidecode(texte).strip().lower())
cles = {cle(n) for n in noms}
print("après strip et minuscules :", int(cat["designation"].str.strip().str.lower().isin({n.lower()
↪ for n in noms}).sum()), "| après clé complète (accents et espaces) :",
↪ int(cat["designation"].map(cle).isin(cles).sum()))
```

```
après strip et minuscules : 74 | après clé complète (accents et espaces) : 78
```

On passe de 34 à 74 correspondances en retirant espaces et majuscules, puis à 78 en supprimant les accents (Étagère devient etagere). Les 40 désignations restantes sont des abréviations, des mots dans un autre ordre ou des produits absents de la boutique : un problème de rapprochement, non de normalisation.

Piège. Supprimer les accents est **une opération destructrice** : cote et côté, ou et où, deviennent identiques. Faites-le pour **comparer**, jamais pour **stocker**. Même précaution pour la casse : `str.title()` transforme « d'Alembert » en « D'Alembert » et « McDonald » en « Mcdonald ». Normalisez dans une colonne clé, gardez l'original à côté.

1.5.2 Unicode : le même caractère de deux façons, et le mojibake

Un caractère accentué peut s'écrire de deux manières en Unicode : en **un seul signe** (é, forme composée, NFC) ou en **deux signes** (la lettre e suivie d'un accent combinant, forme décomposée, NFD). À l'écran, c'est identique. Pour l'ordinateur, ce sont deux chaînes différentes.

```
a, b = "Zoé", "Zoé" # « Zoé » : accent combinant, puis caractère composé
print(a == b, "| longueurs :", len(a), len(b), "| égales après normalisation NFC :",
↪ unicodedata.normalize("NFC", a) == unicodedata.normalize("NFC", b))
```

```
False | longueurs : 4 3 | égales après normalisation NFC : True
```

Deux chaînes qui s'affichent pareil mais ne sont pas égales, de longueurs différentes : c'est le

genre de défaut qui fait **échouer une jointure sans raison apparente**. Le remède est de normaliser en NFC dès la lecture (`unicodedata.normalize("NFC", texte)`, ou `.str.normalize("NFC")` en pandas).

Le défaut le plus visible de ce genre est le **mojibake** : du texte lu avec le mauvais encodage. Le CRM en contient. Le caractère é s'écrit, en UTF-8, avec **deux octets** ; si un logiciel les lit comme deux caractères de l'ancien encodage Windows (cp1252), il affiche Ã©.

```
crm = pd.read_csv(os.path.join(D, "crm_clients.csv"), dtype=str)
mojibake = crm["nom"].str.contains("Ã|Ã", regex=True)
print("é en UTF-8 :", "é".encode("utf-8"), "| lu comme cp1252 :",
      ↪ "é".encode("utf-8").decode("cp1252"))
print("noms abîmés dans le CRM :", int(mojibake.sum()), "|", crm.loc[mojibake,
      ↪ "nom"].head(3).tolist())
```

```
é en UTF-8 : b'\xc3\xa9' | lu comme cp1252 : Ã©
noms abîmés dans le CRM : 213 | ['TarvaneÃ©', 'SomariÃ©', 'RavtierÃ©']
```

Une bibliothèque, `ftfy` (*fixes text for you*), détecte ces enchaînements typiques et **inverse l'erreur** : elle devine que Ã© est un é mal décodé.

```
import ftfy
corrige = crm.loc[mojibake, "nom"].map(ftfy.fix_text)
print(list(zip(crm.loc[mojibake, "nom"].head(3), corrige.head(3))))
print("noms intacts modifiés à tort par ftfy :", int((crm.loc[~mojibake, "nom"].map(ftfy.fix_text) !=
      ↪ crm.loc[~mojibake, "nom"]).sum()))
```

```
[('TarvaneÃ©', 'Tarvaneé'), ('SomariÃ©', 'Somarié'), ('RavtierÃ©', 'Ravtieré')]
noms intacts modifiés à tort par ftfy : 0
```

La correction rend le texte **tel qu'il a été écrit** (le é final fait bien partie de la saisie abîmée) et ne modifie aucun des noms corrects : c'est ce que l'on attend d'un bon outil de réparation, qui doit être **conservateur**. Mais la meilleure réparation est de ne pas casser le texte : lire le fichier avec le **bon encodage** dès le départ (section 1.5.5).

1.5.3 Caractères invisibles et expressions régulières

Deux défauts échappent à l'œil : l'**espace insécable** (le caractère , qui ressemble à une espace) et les caractères de largeur nulle, qui n'occupent aucune place à l'écran. Ils font échouer comparaisons et jointures.

```
v1, v2 = "Ville A", "Ville A"
print(v1 == v2, "|", repr(v2), "| avec \s dans une expression régulière :", re.sub(r"\s+", " ", v2)
      ↪ == v1)
```

```
False | 'Ville\xa0A' | avec \s dans une expression régulière : True
```

Dans une **expression régulière**, `\s` désigne toute espace (y compris l'insécable) : `re.sub(r"\s+", " ", texte)` est donc un bon réflexe de nettoyage. Les expressions régulières sont l'outil naturel de tout ce qui a une **forme** : téléphone, code postal, adresse électronique, référence produit. Voici les motifs les plus utiles.

Besoin	Motif	Exemple
un chiffre, des chiffres	<code>\d, \d+</code>	<code>\d{5}</code> : exactement cinq chiffres
tout sauf un chiffre	<code>\D</code>	<code>re.sub(r"\D", "", "0219.86")</code> retire les séparateurs
espace(s)	<code>\s, \s+</code>	<code>re.sub(r"\s+", " ", t)</code>

Besoin	Motif	Exemple
un ensemble de caractères	[a-z], [\w.+-]	[\w.+-]+@ : début d'une adresse
début, fin de chaîne	^, \$	str.fullmatch impose que tout le texte corresponde

Appliquons-les aux **numéros de téléphone** du CRM, écrits de cinq façons différentes. Classons d'abord les formats (sans afficher les numéros eux-mêmes : on ne diffuse pas de données personnelles pour illustrer un nettoyage).

```
def forme(s):
    if s.startswith("+"):
        return "préfixe international"
    return "parenthèses" if s.startswith("(") else "points" if "." in s else "espaces" if " " in s
    ↪ else "chiffres collés"
print(crm["telephone"].map(forme).value_counts().to_string())
```

```
telephone
chiffres collés      1516
points               1469
préfixe international 1420
parenthèses          1408
espaces              1327
```

Cinq formes, à peu près également réparties. La normalisation retient les **chiffres** et traite à part le préfixe international (dont on remplace l'indicatif par le zéro initial, dans la convention locale). On accepte le résultat s'il a la forme attendue (dix chiffres commençant par 0), sinon on **rejette** : une valeur invalide vaut mieux qu'une valeur inventée.

```
def telephone_propre(s):
    chiffres = re.sub(r"\D", "", s)
    if s.startswith("+"):
        ↪ chiffres ici) devient 0 # préfixe international : l'indicatif (2
        chiffres = "0" + chiffres[2:]
    return chiffres if re.fullmatch(r"0\d{9}", chiffres) else None
crm["tel"] = crm["telephone"].map(telephone_propre)
print("numéros invalides :", int(crm["tel"].isna().sum()), "| formes distinctes après nettoyage :",
    ↪ int(crm["tel"].str.len().nunique()))
```

```
numéros invalides : 0 | formes distinctes après nettoyage : 1
```

Les 7 140 numéros se ramènent à une forme unique. Reste à **vérifier** que la normalisation n'invente rien : si deux lignes décrivent le même client, elles doivent avoir **le même numéro normalisé**.

```
verite = pd.read_csv(os.path.join(D, "verite_crm.csv"))
crm["id_crm"] = crm["id_crm"].astype(int)
x = crm.merge(verite, on="id_crm")
x = x[x["id_client"] > 0]
print("clients dont les lignes ont plusieurs numéros normalisés différents :",
    ↪ int((x.groupby("id_client")["tel"].nunique() > 1).sum()), "sur", x["id_client"].nunique())
```

```
clients dont les lignes ont plusieurs numéros normalisés différents : 0 sur 6000
```

À retenir. Quatre gestes pour normaliser un texte : **couper** les espaces, **réduire** les espaces multiples (\s+), **uniformiser** la casse et, si l'on compare, les accents, puis **vérifier** la forme avec fullmatch. Et toujours **conserver l'original** à côté de la valeur nettoyée.

1.5.4 Dates et heures : fuseaux et heure d'été

Une heure sans fuseau est ambiguë. L'export du site écrit les dates de deux façons : 2025-03-04 14:22:05 jusqu'au 14 septembre (heure sans indication) et 2025-09-15T14:22:05Z à partir du 15 (le Z signifie « heure UTC », l'heure du méridien de référence). Si ce Z était vrai, les commandes du jour se répartiraient autrement : une boutique située à une heure ou deux de UTC verrait ses heures de commande **décalées d'autant**. Regardons.

```
site = pd.read_csv(os.path.join(D, "site_commandes.csv"), dtype=str)
site["utc"] = site["created_at"].str.endswith("Z")
site["heure"] = pd.to_datetime(site["created_at"].str.replace("Z", "", regex=False),
    ↪ format="mixed").dt.hour
print(site.groupby("utc")["heure"].agg(["size", "mean"]).round(2).rename(index={False: "sans
    ↪ indication", True: "avec Z"}).to_string())
```

	size	mean
utc		
sans indication	3741	15.45
avec Z	2518	15.37

L'heure moyenne d'une commande est de 15,45 avant le 15 septembre et de 15,37 après : **aucun décalage** d'une ou deux heures. Le Z n'a donc **pas** accompagné un changement réel de fuseau : soit la plateforme étiquette « UTC » des heures locales (le plus probable), soit les clients ont brusquement changé d'habitude. On ne tranche pas par un calcul : on **pose la question** à l'équipe qui gère la plateforme. Retenez le raisonnement : **comparer une distribution avant et après un changement de format** est un bon test de cohérence.

Le passage à l'heure d'été ajoute une difficulté. Quand l'horloge avance (au printemps), une heure locale **n'existe pas** ; quand elle recule (en automne), une heure locale existe **deux fois**. Le même texte 02:30 désigne alors deux instants différents, ce que seul le décalage par rapport à UTC distingue.

```
avant = pd.Timestamp("2025-10-26 02:30:00+02:00").tz_convert("UTC")
apres = pd.Timestamp("2025-10-26 02:30:00+01:00").tz_convert("UTC")
print("02:30 locale, première fois :", avant, "| deuxième fois :", apres, "| écart :", apres - avant)
```

```
02:30 locale, première fois : 2025-10-26 00:30:00+00:00 | deuxième fois : 2025-10-26 01:30:00+00:00 |
    ↪ écart : 0 days 01:00:00
```

En pratique. Pour les dates : stockez en ISO 8601 (AAAA-MM-JJTHH:MM:SS) ; **indiquez le fuseau** (ou stockez en UTC) dès que des données viennent de plusieurs sources ; écrivez toujours le **format** à la lecture (format="%d/%m/%Y") ; et méfiez-vous des calculs de durée à travers un changement d'heure.

1.5.5 Encodage : cp1252, UTF-8 et la marque d'ordre des octets

Un fichier texte n'est qu'une suite d'octets ; l'**encodage** dit comment les transformer en caractères. Les deux que vous rencontrerez : **cp1252** (ancien encodage de Windows pour l'Europe occidentale) et **UTF-8** (le standard actuel, qui sait tout écrire, de l'accent au caractère arabe). Un fichier UTF-8 peut commencer par trois octets invisibles, la **marque d'ordre des octets** (BOM), ajoutée par certains logiciels. Les douze fichiers de caisse en fournissent une démonstration.

```
janvier = open(os.path.join(D, "caisse", "caisse_2025-01.csv"), "rb").read()
juillet = open(os.path.join(D, "caisse", "caisse_2025-07.csv"), "rb").read()
print("début de janvier :", janvier[:3], "| début de juillet :", juillet[:3])
print("juillet lu en cp1252 :", repr(juillet.decode("cp1252")[:27]))
try:
```

```
janvier.decode("utf-8")
except UnicodeDecodeError as erreur:
    print("janvier lu en UTF-8 :", str(erreur))
```

```
début de janvier : b'Exp' | début de juillet : b'\xef\xbb\xbf'
juillet lu en cp1252 : 'i»¿Export caisse - Boutique'
janvier lu en UTF-8 : 'utf-8' codec can't decode byte 0xe9 in position 33: invalid continuation byte
```

Le fichier de juillet commence par les trois octets ef bb bf du BOM : lu comme du cp1252, ils s'affichent i»¿ devant le premier mot (c'est ce « i»¿ » que l'on voit parfois en tête d'une colonne mal lue). Le fichier de janvier, lui, n'est **pas** de l'UTF-8 : le é est codé par un seul octet (0xe9), ce qui est interdit en UTF-8, d'où l'erreur. La règle de lecture est celle de la fonction écrite en 1.4.6 : **essayer UTF-8 d'abord** (qui échoue bruyamment sur un fichier qui n'en est pas), puis cp1252 ; avec utf-8-sig, le BOM est avalé.

Piège. Lire un fichier cp1252 en UTF-8 plante (bruyant, donc tant mieux) ; lire un fichier UTF-8 en cp1252 ne plante **jamais** et produit du mojibake (silencieux, donc dangereux). Quand vous voyez Å© dans vos données, c'est un UTF-8 lu en cp1252 : relisez le fichier avec le bon encodage plutôt que de réparer le texte après coup.

1.5.6 Données multilingues : l'arabe dans une colonne de villes

La boutique a des clients arabophones, et certains ont saisi leur ville dans leur langue : la ville A s'écrit المدينة أ (« la ville A »). Sur 7 000 lignes, 191 (2,7 %) sont dans ce cas. Trois choses méritent d'être comprises avant de traiter ce texte, sans être spécialiste de la langue.

L'ordre logique et l'ordre d'affichage. L'arabe s'écrit de droite à gauche. Mais le fichier stocke les caractères **dans l'ordre où on les lit** (ordre logique) ; c'est l'affichage qui les range de droite à gauche. Le dernier caractère de la chaîne est donc bien la dernière lettre lue, quelle que soit sa position à l'écran.

```
ville = "المدينة أ"
print("longueur :", len(ville), "| dernier caractère :", ville[-1], unicodedata.name(ville[-1]), "|
↳ sens d'écriture :", unicodedata.bidirectional(ville[-1]), "(AL = arabe, L = latin)")
```

```
longueur : 9 | dernier caractère : أ ARABIC LETTER ALEF WITH HAMZA ABOVE | sens d'écriture : AL (AL =
↳ arabe, L = latin)
```

Les signes qui varient. L'arabe s'écrit avec ou sans **voyelles brèves** (signes combinants placés au-dessus ou au-dessous des lettres, par exemple dans un texte vocalisé), et peut étirer les lettres par un trait horizontal décoratif (le *tatweel*). Deux écritures qui se lisent pareil sont alors deux chaînes différentes, que l'on **normalise** en retirant ces signes.

```
vocalise = "المدينة"
simple = re.sub("[\u064b-\u065f\u0640]", "", vocalise) # voyelles brèves et tatweel
print("caractères avant :", len(vocalise), "| après :", len(simple), "| résultat :", simple)
```

```
caractères avant : 11 | après : 7 | résultat : المدينة
```

Une normalisation qui détruit. Ici, le piège est subtil : la lettre qui distingue les villes (أ, alef avec hamza) a des variantes proches (ا, إ, ئ) que beaucoup de recettes de normalisation **fusionnent**. Appliquée à المدينة أ, une telle recette donnerait المدينة ا : la ville A et d'éventuelles autres lettres se confondraient, ce qui casserait notre table de correspondance. Notre fonction ville_propre (section 1.4.4) s'appuie sur la lettre **exacte**, et la **table de correspondance** garde la trace de chaque substitution.

Remarque. Il n'existe pas de recette universelle pour le texte multilingue. Trois habitudes simples : **conserver l'original** et ajouter une colonne normalisée ; **ne normaliser que ce que l'on sait** (retirer les voyelles brèves est sûr, fusionner des lettres ne l'est pas) ; **faire valider** par un lecteur de la langue les correspondances qui comptent. Un français tiré de l'arabe, ou l'inverse, ne se « transcrit » pas par une simple substitution de caractères.

Pour s'entraîner. Cahier, chapitre 1 : application 1.7, exercice 1.10.

1.6 ⊕ Pour aller plus loin : stratégies d'imputation et leur impact

Section complémentaire. Elle suppose la section 1.1 (les mécanismes d'absence). On y compare des méthodes d'imputation **à la vérité**, ce qui est le seul moyen honnête d'en juger.

Imputer, c'est remplacer une valeur manquante par une valeur estimée. C'est tentant (les modèles aiment les tableaux complets) et risqué (on fabrique des données). La bonne question n'est pas « quelle méthode est la meilleure ? » mais : **que veut-on préserver, et qu'est-ce que les autres colonnes permettent de savoir ?** Les données du chapitre permettent, pour une fois, de mesurer ce que chaque méthode abîme, puisque nous connaissons la vérité.

1.6.1 Imputer, c'est prédire : ce qu'on veut préserver

Une imputation est une **prédiction** de la valeur manquante à partir de ce que l'on sait par ailleurs. On peut la juger sur trois critères, qui ne vont pas toujours ensemble.

On veut préserver...	Pourquoi	Méthode qui le fait mal
la moyenne	le chiffre annoncé ne doit pas se déplacer	supprimer les lignes (si le mécanisme n'est pas MCAR)
la dispersion	un intervalle de confiance, un écart-type, une part de cas extrêmes	remplacer par la moyenne : les valeurs imputées sont toutes identiques
les liaisons entre variables	un modèle, une corrélation, un tableau croisé	remplacer par la moyenne : le lien est atténué

Quatre méthodes simples couvrent l'essentiel. La **moyenne** (ou la **médiane**, moins sensible aux extrêmes) remplace par une valeur centrale unique. La **moyenne par groupe** remplace par la moyenne d'un groupe proche (âge et canal). La **régression** prédit la valeur par un modèle linéaire des autres colonnes. Les **k plus proches voisins** (en anglais *k-nearest neighbours*) prennent la moyenne des dix clients les plus semblables, d'après leurs autres colonnes. On les écrit en une fonction.

```
from sklearn.linear_model import LinearRegression
from sklearn.impute import KNNImputer
profil = pd.read_csv(os.path.join(D, "profil_clients.csv"))
verite = pd.read_csv(os.path.join(D, "profil_clients_verite.csv"))
profil["classe_age"] = pd.cut(profil["age"], [0, 29, 44, 59, 200], labels=["moins de 30", "30-44",
↪ "45-59", "60 et plus"])
X = pd.get_dummies(profil[["age", "canal_acquisition", "nb_commandes_2025", "minutes_site"]],
↪ drop_first=True).astype(float)
def imputations(col):
```

```

m, obs = profil[col].isna(), profil[col]
res = {"moyenne": pd.Series(obs.mean(), index=obs.index[m]), "médiane": pd.Series(obs.median(),
↪ index=obs.index[m])}
res["moyenne par groupe"] = profil.groupby(["classe_age", "canal_acquisition"],
↪ observed=True)[col].transform("mean")[m]
res["régression"] = pd.Series(LinearRegression().fit(X[~m], obs[~m]).predict(X[m]),
↪ index=obs.index[m])
A = X.join(obs); Z = (A - A.mean()) / A.std()
res["k plus proches voisins"] = pd.Series(KNNImputer(n_neighbors=10).fit_transform(Z)[:,
↪ -1][m.values] * obs.std() + obs.mean(), index=obs.index[m])
return res

```

Pour **juger** chaque méthode, on compare les valeurs imputées aux vraies valeurs : l'erreur typique (racine de l'erreur quadratique moyenne, sur les seules cases imputées), le **biais** de la moyenne obtenue après imputation, et l'écart-type du tableau complété. La dernière ligne rappelle la **suppression** des lignes incomplètes (on garde les valeurs observées, on ignore le reste).

```

def tableau(col):
    m = profil[col].isna(); vrai = verite.loc[m, col]; lignes = []
    for nom, imp in imputations(col).items():
        complet = profil[col].copy(); complet[m] = imp
        lignes.append((nom, np.sqrt(((imp - vrai) ** 2).mean()), complet.mean() - verite[col].mean(),
↪ complet.std()))
    lignes.append(("suppression des manquants", np.nan, profil[col].mean() - verite[col].mean(),
↪ profil[col].std()))
    print(f"vérité : moyenne {verite[col].mean():.2f}, écart-type {verite[col].std():.2f}")
    print(pd.DataFrame(lignes, columns=["méthode", "erreur_typique", "biais_moyenne",
↪ "écart_type"]).round(2).to_string(index=False))

```

1.6.2 Le revenu : une variable peu prévisible

Commençons par le revenu annuel, qui manque pour 17 % des clients selon un mécanisme MAR (section 1.1.3) : il manque plus souvent chez les moins de 30 ans et pour le canal réseaux.

```
tableau("revenu_annuel")
```

```

vérité : moyenne 28321.77, écart-type 11212.22
      méthode  erreur_typique  biais_moyenne  écart_type
      moyenne      11188.57        218.89    10201.99
      médiane      11137.51       -117.17    10228.39
    moyenne par groupe      10180.03       -24.14    10364.80
      régression      10079.84       -23.01    10369.30
    k plus proches voisins      10457.96       -20.77    10433.35
suppression des manquants           NaN        218.89    11219.76

```

Trois enseignements. **Premier : la moyenne est bien retrouvée par les méthodes qui tiennent compte de l'âge et du canal** (biais d'environ -20 à -24 €, contre +219 € pour la moyenne simple et pour la suppression). Les premières corrigent le mécanisme MAR, les secondes l'ignorent. **Deuxième : l'erreur individuelle reste énorme** (environ 10 000 € sur une valeur moyenne de 28 000 €), parce que l'âge, le canal et le comportement d'achat disent peu de chose sur le revenu : aucune méthode ne retrouve la valeur d'un client, elles ne font que respecter la moyenne d'un groupe. **Troisième : la moyenne simple écrase la dispersion** : l'écart-type tombe à 10 202 € au lieu de 11 212 € (-9 %), parce que 1 039 clients reçoivent exactement la même valeur.

Piège. Un tableau imputé a l'air **complet** et donc plus fiable ; il est en réalité plus **trompeur** qu'un tableau avec des trous honnêtes, parce que les valeurs fabriquées n'ont pas la variabilité des vraies. Un écart-type, un intervalle de confiance ou une part de clients « à risque »

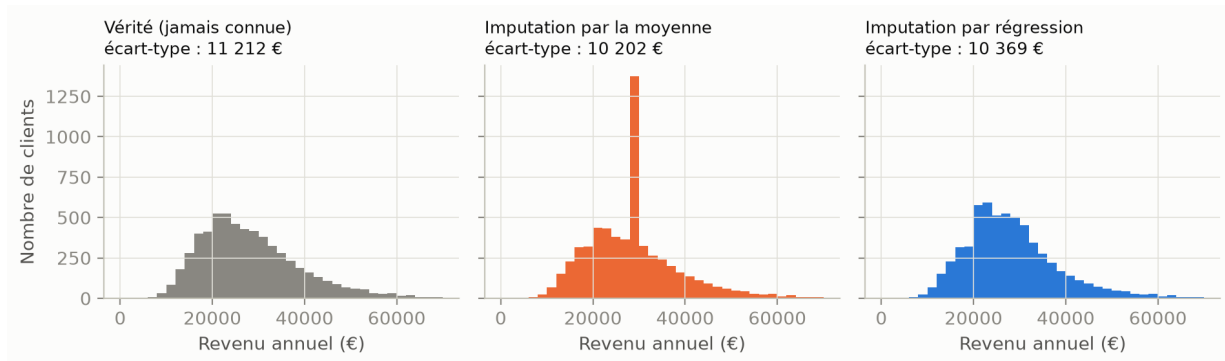


Figure 7: Distribution du revenu : la vérité, puis après imputation par la moyenne (un pic artificiel) et par la régression (une distribution plus étroite).

calculés après une imputation par la moyenne sont **sous-estimés**.

1.6.3 La dépense : quand les autres colonnes savent

La dépense 2025 manque pour 5 % des clients, **au hasard** (MCAR). Mais, contrairement au revenu, elle est très prévisible à partir d'une autre colonne connue : le nombre de commandes (corrélation de 0,92 dans la vérité).

```
tableau("depense_2025")
```

vérité : moyenne 220.79, écart-type 317.98

	méthode	erreur_typique	biais_moyenne	écart_type
	moyenne	286.54	0.68	311.53
	médiane	307.15	-5.52	312.71
	moyenne par groupe	286.11	0.66	311.55
	régression	106.83	0.27	316.67
	k plus proches voisins	118.76	-0.17	316.17
	suppression des manquants	NaN	0.68	319.54

Le contraste avec le revenu est frappant. Les méthodes qui exploitent le nombre de commandes **divisent l'erreur par près de trois** (107 € pour la régression, 119 € pour les voisins, contre 287 € pour la moyenne) et conservent l'écart-type (317 € et 316 €, contre 318 € en vérité). Quand une autre colonne **sait** quelque chose, l'imputation par modèle est précieuse ; quand aucune ne sait rien, elle n'apporte presque rien. Une imputation ne crée jamais d'information : elle **redistribue** celle qui existe.

Regardons enfin ce que l'imputation fait aux **liaisons** : la corrélation entre dépense et nombre de commandes vaut 0,923 en vérité.

```
md = profil["depense_2025"].isna()
print("corrélation vraie :", round(verite["depense_2025"].corr(verite["nb_commandes_2025"]), 3), "|
↳ valeurs observées seules :", round(profil["depense_2025"].corr(profil["nb_commandes_2025"]), 3))
for nom, imp in imputations("depense_2025").items():
    complet = profil["depense_2025"].copy(); complet[md] = imp
    print(f"{nom:24s} {complet.corr(profil['nb_commandes_2025']):.3f}")
```

corrélation vraie : 0.923 | valeurs observées seules : 0.922

moyenne	0.905
médiane	0.902
moyenne par groupe	0.905
régression	0.925
k plus proches voisins	0.924

La moyenne **atténue** la corrélation (0,905 au lieu de 0,923 : les valeurs imputées n'ont aucun lien avec le nombre de commandes), la régression et les voisins la **préservent** (0,925 et 0,924). Attention au revers : une imputation par régression **fabrique** une relation (elle utilise la relation pour prédire) ; si l'on calcule ensuite un modèle entre ces deux variables, la relation est en partie **circulaire**.

1.6.4 La satisfaction : aucune méthode ne répare un MNAR

Dernier cas, le plus instructif : la satisfaction moyenne, qui manque pour 9 % des clients **selon sa propre valeur** (section 1.1.3). Aucune autre colonne ne sait ce que pense le client.

```
tableau("satisfaction_moy")
```

```
vérité : moyenne 3.73, écart-type 0.68
```

	méthode	erreur_typique	biais_moyenne	écart_type
	moyenne	0.83	0.02	0.64
	médiane	0.83	0.02	0.64
	moyenne par groupe	0.82	0.02	0.64
	régression	0.82	0.02	0.64
	k plus proches voisins	0.84	0.02	0.64
	suppression des manquants	NaN	0.02	0.67

Toutes les méthodes se valent : l'erreur individuelle (0,83 point) est celle que l'on commettrait en devinant la moyenne, et **toutes conservent le même biais** de +0,02 point. Sur la moyenne, le dégât est faible. Sur la **part de clients très mécontents** (satisfaction inférieure ou égale à 2,5), il est important.

```
ms = profil["satisfaction_moy"].isna()
print("part de satisfactions ≤ 2,5 : vérité", round((verite["satisfaction_moy"] <= 2.5).mean() * 100,
↵ 2), "% | valeurs observées seules", round((profil["satisfaction_moy"].dropna() <= 2.5).mean() *
↵ 100, 2), "%")
for nom, imp in imputations("satisfaction_moy").items():
    complet = profil["satisfaction_moy"].copy(); complet[ms] = imp
    print(f"après imputation ({nom}) : {(complet <= 2.5).mean() * 100:.2f} %")
```

```
part de satisfactions ≤ 2,5 : vérité 4.08 % | valeurs observées seules 2.81 %
après imputation (moyenne) : 2.55 %
après imputation (médiane) : 2.55 %
après imputation (moyenne par groupe) : 2.55 %
après imputation (régression) : 2.55 %
après imputation (k plus proches voisins) : 2.55 %
```

Les clients très mécontents représentent 4,08 % de la clientèle en vérité. Les valeurs observées seules en montent 2,81 %, parce que les mécontents répondent moins. Et **chaque imputation donne 2,55 %, pire** que de ne rien faire : on remplit les trous avec des valeurs centrales, qui ne sont jamais « très mécontentes ». Le MNAR a effacé l'information, et l'imputation ne l'a pas retrouvée.

Que peut-on faire ? On ne peut pas **corriger** un MNAR, mais on peut en mesurer l'**importance** par une **analyse de sensibilité** : supposer que les non-répondants sont moins satisfaits que les répondants d'un certain écart δ , et regarder ce que devient la conclusion.

```
obs = profil["satisfaction_moy"]
for delta in (0, 0.1, 0.2, 0.3):
    complet = obs.copy(); complet[ms] = obs.mean() - delta
    print(f"si les non-répondants sont {delta:.1f} point moins satisfaits : moyenne
↵ {complet.mean():.3f}")
print("vérité :", round(verite["satisfaction_moy"].mean(), 3), "| écart réel entre répondants et
↵ non-répondants :", round(obs.mean() - verite.loc[ms, "satisfaction_moy"].mean(), 3))
```

```

si les non-répondants sont 0.0 point moins satisfaits : moyenne 3.747
si les non-répondants sont 0.1 point moins satisfaits : moyenne 3.738
si les non-répondants sont 0.2 point moins satisfaits : moyenne 3.728
si les non-répondants sont 0.3 point moins satisfaits : moyenne 3.719
vérité : 3.729 | écart réel entre répondants et non-répondants : 0.195

```

Dans la vraie vie, on ne connaît pas l'écart (0,195 point ici, que seule la vérité programmée révèle) ; on essaie une plage plausible et l'on **dit ce que devient la conclusion** : « avec un écart entre 0 et 0,3 point, la satisfaction moyenne se situe entre 3,72 et 3,75 ». Si la décision change selon l'hypothèse, il faut collecter des données, pas imputer.

À retenir. Aucune imputation ne répare un **MNAR**. Pour **MCAR**, presque toutes les méthodes préservent la moyenne ; pour **MAR**, il faut des méthodes qui utilisent les variables dont dépend l'absence ; pour **MNAR**, il reste l'**analyse de sensibilité** et la collecte d'information supplémentaire.

1.6.5 L'imputation multiple : dire l'incertitude

Une imputation unique a un défaut fondamental : on traite la valeur imputée comme **si elle était vraie**. Les intervalles de confiance calculés ensuite sont donc **trop étroits** : ils ignorent que 17 % des revenus sont des estimations. L'**imputation multiple** corrige cela en produisant non pas un tableau complété, mais **plusieurs** (par exemple vingt), chacun avec des valeurs imputées **tirées au hasard** dans l'incertitude du modèle. On analyse chaque tableau, puis on **combine** les résultats : la moyenne des moyennes est l'estimation, et l'incertitude additionne la variabilité à l'intérieur de chaque tableau et celle **entre** les tableaux.

Règles de combinaison (Rubin). Avec m tableaux imputés, d'estimations $\hat{Q}_1, \dots, \hat{Q}_m$ et de variances estimées $U_1, \dots, U_m$: l'estimation finale est $\bar{Q} = \frac{1}{m} \sum \hat{Q}_i$; la variance intra est $\bar{U} = \frac{1}{m} \sum U_i$; la variance inter est $B = \frac{1}{m-1} \sum (\hat{Q}_i - \bar{Q})^2$; la variance totale est $T = \bar{U} + (1 + \frac{1}{m}) B$.

```

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
A = X.join(profil["revenu_annuel"]); estim, variances = [], []
for graine in range(20):
    complet = pd.Series(IterativeImputer(sample_posterior=True, random_state=graine,
    ↪ max_iter=10).fit_transform(A[:, -1], index=profil.index)
    estim.append(complet.mean()); variances.append(complet.var() / len(complet))
estim, variances = np.array(estim), np.array(variances)
intra, inter = variances.mean(), estim.var(ddof=1)
print("moyenne combinée :", round(estim.mean(), 1), "| erreur type totale :", round(np.sqrt(intra +
    ↪ (1 + 1 / 20) * inter), 1), "(intra", round(np.sqrt(intra), 1), ", inter", round(np.sqrt(inter),
    ↪ 1), ")")

```

```
moyenne combinée : 28303.9 | erreur type totale : 159.3 (intra 145.4 , inter 63.4 )
```

Comparons à ce qu'aurait donné l'imputation par la moyenne : le calcul naïf trouve une **erreur type de 131,7 €**, soit **17 % de moins** que les 159,3 € de l'imputation multiple. Le calcul naïf **se croit plus précis qu'il ne l'est**. L'estimation combinée (28 304 €) est aussi plus proche de la vérité (28 322 €) que la suppression des lignes (28 541 €). L'imputation multiple est la méthode de référence quand on a besoin d'intervalles honnêtes ; elle est plus lourde, et rarement nécessaire pour une simple moyenne par groupe.

```

complet = profil["revenu_annuel"].fillna(profil["revenu_annuel"].mean())
print("erreur type après imputation par la moyenne :", round(complet.std() / np.sqrt(len(complet)),
    ↪ 1), "| avec les seules valeurs observées :", round(profil["revenu_annuel"].std() /
    ↪ np.sqrt(profil["revenu_annuel"].notna().sum(), 1))

```

erreur type après imputation par la moyenne : 131.7 | avec les seules valeurs observées : 159.3

1.6.6 L'indicateur de manquant et la décision finale

Une dernière technique, simple, évite de choisir : **garder trace de l'absence**. On ajoute à côté de la colonne une colonne `...manquant` (1 si la valeur manquait, 0 sinon) et l'on peut ensuite imputer ce que l'on veut. L'indicateur laisse à un modèle la possibilité de **voir** l'absence, qui est elle-même une information. Elle l'est ici : les clients dont la satisfaction manque sont, en vérité, **moins satisfaits** que les répondants (3,55 en moyenne contre 3,75).

```
profil["satisfaction_manquante"] = profil["satisfaction_moy"].isna().astype(int)
print("satisfaction vraie moyenne des clients dont la valeur manque :", round(verite.loc[ms,
↳ "satisfaction_moy"].mean(), 2), "| de ceux qui ont répondu :", round(verite.loc[~ms,
↳ "satisfaction_moy"].mean(), 2))
```

satisfaction vraie moyenne des clients dont la valeur manque : 3.55 | de ceux qui ont répondu : 3.75

Voici le tableau de décision que l'on peut retenir pour la préparation d'une colonne avec trous.

Situation	Que faire	Méthode adaptée
Une règle logique donne la valeur	déduire (section 1.1.2)	zéro logique, valeur d'une autre colonne
Peu de trous (moins de 5 %), au hasard	supprimer ou laisser	moyennes qui ignorent les manquants
Trous qui dépendent d'une variable connue (MAR)	imputer avec cette variable	moyenne par groupe, régression, voisins
Une autre colonne prédit bien la valeur	imputer par modèle	régression, voisins, imputation multiple
On a besoin d'intervalles de confiance	imputer plusieurs fois	imputation multiple
Le trou dépend de la valeur manquante (MNAR)	mesurer la sensibilité , collecter	indicateur de manquant, hypothèses explicites

Piège. Ne jamais imputer **avant** de séparer l'entraînement du test, quand l'imputation sert à un modèle de prédiction : calculer la moyenne sur tout le tableau fait fuir de l'information du test vers l'entraînement. L'imputation s'apprend sur l'entraînement et s'applique au test (c'est ce que fait un *pipeline*).

Pour s'entraîner. Cahier, chapitre 1 : application 1.8, exercices 1.11 et 1.12.

Bilan du chapitre 1

Vous savez maintenant :

- **repérer** les valeurs manquantes par colonne **et par combinaison**, distinguer une vraie absence d'un zéro logique ou d'un code spécial (ND, 9999, rupture), **déduire** ce qui peut l'être (101 dépenses sur 297), et comprendre les trois mécanismes d'absence (**MCAR**, **MAR**, **MNAR**) en sachant que seul le premier se teste et que le dernier ne se voit pas dans les données observées ;
- **mesurer ce que coûte une suppression** : 28,7 % de clients perdus, des jeunes sous-représentés (16,7 % → 13,4 %), une part de clients très mécontents qui passe de 4,1 % à 2,8 % alors que la moyenne ne bouge presque pas ;
- **séparer** l'erreur, l'extrême réel et le cas rare ; comparer les méthodes statistiques (écart interquartile, score z, score z robuste) **aux règles métier**, et ne jamais supprimer une valeur parce qu'elle est extrême (126 clients à plus de trois écarts-types font 14,6 % du chiffre d'affaires) ;
- **définir une clé** pour détecter les doublons, exacts ou approchés, mesurer ce qu'une clé retrouve et ce qu'elle signale à tort, **choisir l'enregistrement à garder** (le plus récent, le plus complet, le plus fiable, la fusion) et **journaliser** ce qu'on retire ;
- **détecter** un changement d'unité par la distribution dans le temps (les centimes du 15 septembre), une date ambiguë (jour/mois contre mois/jour), une catégorie écrite de six façons, une règle de cohérence enfreinte, et **lire une série de fichiers dont le format change** avec une fonction qui détecte au lieu de supposer ;
- (en option) **normaliser du texte** (espaces, casse, accents, Unicode NFC, caractères invisibles), réparer le **mojibake**, lire avec le bon **encodage**, comprendre les fuseaux et l'heure d'été, et manipuler un peu d'**arabe** sans le détruire ;
- (en option) **comparer des imputations à la vérité**, connaître leurs effets sur la moyenne, la dispersion et les liaisons, **mesurer la sensibilité** quand le mécanisme est MNAR, et dire l'incertitude par l'**imputation multiple**.

Le chapitre a mis des chiffres sur des défauts que l'on sous-estime d'ordinaire :

Défaut	Ce que nous avons mesuré
Valeurs manquantes	17,3 % de revenus manquants ; 1 723 clients (28,7 %) avec au moins un trou
Suppression des lignes incomplètes	moins de 30 ans : 16,7 % → 13,4 % ; clients très mécontents : 4,08 % → 2,81 %
Valeurs aberrantes	85 lignes fausses sur 6 000 (1,4 %) gonflent le total de 75 %
Détection par la statistique	écart interquartile : précision 14,8 % ; score z : rappel 28,2 % ; règle métier : 100 % et 100 %
Doublons du site, tests, annulées	chiffre d'affaires de janvier à août surestimé de 5,2 % (358 030 € contre 340 260 €)
Doublons de caisse	67 lignes en trop ; sans référence on en retirerait 163 et 4 450 € de vraies ventes
Doublons approchés du CRM	l'e-mail normalisé en retrouve 674 sur 1 000, avec 3 fausses alertes
Changement d'unité (centimes)	somme brute du site : 25,0 M€ ; chiffre d'affaires propre : 600 164 € (identique à la vérité)
Dates ambiguës	398 dates de naissance faussement lues, sans le moindre message d'erreur
Catégories	119 écritures de la ville pour 20 villes réelles

Défaut	Ce que nous avons mesuré
Douze fichiers de caisse	écart de 13 077 € entre la somme lue et le total affiché, expliqué au centime près
Imputation d'une variable peu prévisible (revenu)	erreur individuelle d'environ 10 000 € quelle que soit la méthode ; écart-type écrasé de 9 % par la moyenne
Imputation d'une variable prévisible (dépense)	erreur de 107 € par régression contre 287 € par la moyenne
Imputation d'un MNAR (satisfaction)	2,55 % de clients très mécontents après imputation, contre 4,08 % en vérité : pire que de ne rien faire

Le fil conducteur du chapitre tient en une phrase : **un nettoyage est une suite de décisions que l'on mesure, que l'on écrit et que l'on vérifie contre une source indépendante**. Compter avant et après, garder le brut intact, écrire des fonctions plutôt que des clics, et préférer la règle métier à la formule : ces habitudes rendent un nettoyage défendable, c'est-à-dire refaisable et discutable.

Le chapitre 2 prend la suite : maintenant que les données sont propres, il faut les **transformer et les fusionner** : créer des variables dérivées, joindre des sources qui n'ont pas la même clé (le CRM, le site, la caisse, le catalogue du fournisseur) et rapprocher des enregistrements qui parlent de la même chose sans s'écrire pareil, ce que la section 1.3 n'a fait qu'effleurer.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 à 1.8 (rapport de manquants, mécanismes d'absence, aberrantes, doublons du site, changement d'unité, douze fichiers de caisse, nettoyage du CRM, imputations comparées à la vérité) et exercices 1.1 à 1.12.

Chapitre 2 : Transformation et fusion des données

« Une analyse commence le jour où toutes les données tiennent dans une seule table, et où l'on sait ce que chaque ligne représente. »

Un matin, la gérante de la boutique vous apporte une demande qui a l'air simple : « **Je voudrais une seule table de toutes les ventes de l'année, la caisse et le site ensemble, avec le nom du produit et la marge sur chaque ligne.** Ensuite, je pourrai répondre moi-même à mes questions avec un tableau croisé. » Elle ajoute, un peu gênée, que la caisse lui envoie un fichier par mois, que le site web fournit « un export », et qu'elle ne sait plus très bien quand le format a changé.

Vous ouvrez les fichiers. Les douze exports de la caisse n'ont pas tous les mêmes colonnes ni le même codage ; l'export du site exprime ses montants en texte, avec un symbole monétaire, et, à partir de la mi-septembre, **en centimes** ; aucun des deux ne contient le coût d'achat des produits, qui est dans une autre table, ni de numéro de produit utilisable pour la caisse. Tout est là pour répondre à la gérante, mais **rien n'est prêt** : avant l'analyse, il faut **transformer** (créer des variables), **fusionner** (relier des tables), **empiler** (mettre des fichiers bout à bout) et **agréger** (changer le niveau de détail).

Ce chapitre vous apprend ces quatre gestes, et surtout **comment s'assurer qu'on ne s'est pas trompé**. Car chacun de ces gestes peut faire perdre des lignes, en inventer, ou fausser un total **sans qu'aucune erreur ne s'affiche**. Une jointure qui double les lignes donne un chiffre d'affaires deux fois trop grand ; un fichier lu avec la mauvaise décimale divise les montants par cent ; une agrégation faite au mauvais niveau compte deux fois la même commande. La méthode qui traverse tout le chapitre tient en une phrase : **à chaque étape, comparer les effectifs et les totaux avant et après, et expliquer chaque écart.**

Intuition. Préparer des données, c'est comme **monter un meuble** à partir de plusieurs cartons : chaque pièce est correcte, mais l'ensemble n'est utilisable qu'une fois les pièces assemblées dans le bon ordre. Et, comme pour un meuble, il vaut mieux **vérifier à chaque vissage** que la pièce est bien droite plutôt que de s'en apercevoir à la fin, quand l'étagère penche.

Quatre gestes, un vocabulaire

Pour que la suite soit lisible, fixons le vocabulaire. Les quatre gestes se distinguent par ce qu'ils font du **nombre de lignes** et du **nombre de colonnes**.

Geste	Ce qu'il fait	Lignes	Colonnes	Exemple de la boutique
Dériver	calculer de nouvelles variables à partir des colonnes existantes	inchangées	augmentent	la marge d'une ligne de vente, le trimestre d'une date
Joindre	relier deux tables par une clé commune	ne devraient pas changer (en cas de 1-n)	augmentent	ajouter le coût d'achat à chaque ligne de vente
Empiler	mettre bout à bout des tables de même structure	s'additionnent	inchangées	les douze fichiers mensuels de la caisse
Agréger	résumer par groupes, en changeant le niveau de détail	diminuent	changent	le chiffre d'affaires par mois et par canal

Un cinquième geste, le **pivot** (passer d'un tableau « large » à un tableau « long » et inversement), est traité dans une section facultative, ainsi que le **rapprochement approximatif** (relier deux enregistrements qui désignent la même chose sans être écrits de la même façon).

Le chemin de ce chapitre

Le parcours essentiel suit les trois premiers gestes, dans l'ordre où la gérante en a besoin.

- **2.1 Création et dérivation de variables** : calculer une marge, extraire des morceaux de date, fabriquer des classes et des indicateurs, mesurer un délai entre deux commandes, normaliser du texte pour en faire une clé ; et repérer les pièges (division par zéro, valeurs manquantes qui se propagent, fuite d'information).
- **2.2 Fusion et jointure de jeux de données** : comprendre les types de jointure et la **cardinalité**, contrôler les effectifs avant et après, empiler les douze fichiers de la caisse, lire l'export du site, **harmoniser** les deux sources en une table de ventes unique, et chercher ce qui manque.
- **2.3 Agrégation et restructuration** : changer de niveau de détail (ligne, commande, client), distinguer table de faits et dimensions, calculer des parts, des cumuls et des fenêtres, et vérifier par une **somme de contrôle** que rien n'a été perdu.

Deux sections facultatives prolongent ce parcours : **2.4 Restructuration : pivot et dépivot** (le tableur de stocks de la boutique, saisi à la main) et **2.5 Appariement approximatif et rapprochement d'enregistrements** (dédoublonner le fichier clients, rapprocher le catalogue du fournisseur).

Les données du chapitre

Les fichiers de ce chapitre sont **simulés** : ils ont été fabriqués à partir de la base propre du volume I, puis abîmés de façon contrôlée. Des fichiers de **vérité** (`verite_*.csv`) disent ce qui a été injecté : nous ne les ouvrirons qu'en fin d'étude, pour **juger** notre travail, comme on corrige un exercice. Dans la vie réelle, on ne dispose jamais de cette vérité : c'est précisément pourquoi les contrôles de ce chapitre sont indispensables.

Fichier	Contenu	Particularités
caisse/caisse_2025-01.csv ... caisse_2025-12.csv	ventes du canal Boutique en 2025, un fichier par mois	le format change trois fois dans l'année (codage, séparateur, décimale, noms de colonnes, format des dates) ; lignes de titre, en-têtes répétés, ligne de total, montants vides, lignes doublées
site_commandes.csv, site_lignes.csv	commandes du canal Site en 2025 (en-têtes puis lignes)	montants en texte, statuts écrits de plusieurs façons, doublons d'export, commandes de test, commandes annulées, changement de format et d'unité à la mi-septembre
catalogue_fournisseur.csv	catalogue d'un fournisseur : code, désignation, prix d'achat	désignations réécrites ; certains produits manquent ; d'autres n'existent pas à la boutique
crm_clients.csv	le fichier clients du CRM	plusieurs lignes pour un même client, noms et e-mails mal écrits
stocks_tableur.xlsx	stock mensuel saisi à la main dans un tableur	cellules fusionnées, sous-totaux, valeurs textuelles
clients.csv, produits.csv, commandes.csv, lignes_commande.csv	la base propre du volume I	servent de référence (produits, coût d'achat) et de contrôle

Pour fixer les idées : les douze fichiers de la caisse contiennent 12 678 lignes de vente (une fois retirés les titres et les en-têtes répétés), l'export du site compte 6 259 lignes d'en-têtes de commande et 13 928 lignes de détail, et la base propre du volume I contient 36 395 commandes et 83 905 lignes de commande sur trois ans. Retenez ces ordres de grandeur : vous les retrouverez en fil de chapitre, sous forme de **contrôles**.

Ce que ce chapitre suppose. Vous savez lire un fichier CSV avec pandas, filtrer, regrouper et faire une jointure simple (volume I, chapitres 3 et 4). Nous reprendrons ces notions en les mettant à l'épreuve de données réelles dans leur désordre. Les tables sont reliées par des clés (`id_commande`, `id_produit`, `id_client`) : le schéma de la base est rappelé dans le volume I, section 3.1.1.

2.1 Création et dérivation de variables

Les tables que l'on reçoit contiennent rarement **les colonnes dont on a besoin** : la caisse note un prix et une quantité, mais pas la marge ; la table des commandes porte une date, mais ni le trimestre ni le jour de la semaine ; le fichier clients a une année de naissance, mais pas la tranche d'âge. Une **variable dérivée** est une colonne que l'on **calcule** à partir des autres. Cette section apprend à en fabriquer de cinq sortes (des calculs, des morceaux de date, des classes, des indicateurs et des délais), à les **documenter**, et à éviter les trois pièges qui les abîment : la division par zéro, la valeur manquante qui se propage et la fuite d'information.

2.1.1 Une variable dérivée est une décision

Calculer « la marge d'une ligne de vente » paraît mécanique. Pourtant, avant d'écrire la première formule, **quatre décisions** sont déjà prises, et elles changent le chiffre :

1. **Quelle définition ?** La marge est-elle calculée avant ou après remise ? Sur le prix toutes taxes comprises, ou hors taxe ? Avec le coût d'achat seul, ou avec les frais de transport ?
2. **Quelle unité ?** Des euros, un pourcentage du prix de vente (le **taux de marge**) ou du coût d'achat (le **taux de marque**) ? Les trois se confondent souvent à l'oral, et ne valent pas la même chose (volume I, section 1.5.4).
3. **Quelle date de référence ?** Le coût d'achat d'aujourd'hui, ou celui de la date de la vente ?
4. **Que faire des cas limites ?** Une vente à zéro euro, un produit sans coût d'achat, un retour.

Chacune de ces décisions est un **choix de métier**, pas de technique. C'est pourquoi on **écrit** la définition de chaque variable créée (nous y reviendrons en 2.1.7), et pourquoi deux analystes qui calculent « la marge » sans s'être parlé obtiennent deux chiffres. Par convention, dans ce chapitre : la **marge brute** d'une ligne est son **chiffre d'affaires hors taxe** moins sa **quantité multipliée par le coût d'achat** ; la **TVA** est fixée à 20 % **pour l'illustration**, et le **taux de marge** est la marge divisée par le chiffre d'affaires hors taxe.

Intuition. Une variable dérivée est une **phrase** qu'on a figée dans une colonne. Si la phrase est ambiguë (« la marge »), la colonne l'est aussi, et personne ne s'en apercevra avant qu'un chiffre ne soit contesté en réunion.

2.1.2 Calculs simples : marges, ratios, montants

Commençons par la demande de la gérante : une marge par ligne de vente. La table des lignes contient le **montant payé** (toutes taxes comprises, après remise) ; le coût d'achat est dans la table des produits. Il faut donc d'abord **relier** les deux tables (nous détaillerons la jointure en 2.2), puis calculer.

```
lig = pd.read_csv("donnees/lignes_commande.csv")
prod = pd.read_csv("donnees/produits.csv")
x = lig.merge(prod[["id_produit", "nom_produit", "categorie", "cout_achat"]], on="id_produit",
              validate="m:1")
x["ca_ht"] = x["montant"] / 1.20
x["marge_ht"] = x["ca_ht"] - x["quantite"] * x["cout_achat"]
x["taux_marge"] = x["marge_ht"] / x["ca_ht"]
cols = ["id_ligne", "nom_produit", "quantite", "montant", "marge_ht", "taux_marge"]
print(x[cols].head(4).round(3).to_string(index=False))
```

id_ligne	nom_produit	quantite	montant	marge_ht	taux_marge
1	Étagère compact	1	33.9	13.040	0.462
2	Set de table rustique	1	49.9	18.523	0.445

3	Bougie nordique	1	28.9	11.203	0.465
4	Miroir rustique	2	51.8	18.627	0.432

Vérifions la première ligne à la main, comme on le ferait avant de faire confiance à une colonne. Un article payé 33,90 € toutes taxes comprises vaut $33,90 / 1,20 = 28,25$ € hors taxe ; son coût d'achat est de 15,21 € ; la marge est donc $28,25 - 15,21 = 13,04$ €, soit 46,2 % du chiffre d'affaires hors taxe. Le code a donné la même valeur : on peut généraliser.

Le contrôle essentiel d'une jointure est visible dans ce que le code vérifie lui-même : `validate="m:1"` demande à pandas de s'arrêter si un produit apparaissait deux fois dans la table des produits, ce qui multiplierait les lignes. Ici la jointure est sans histoire : on part de 83 905 lignes et l'on en retrouve 83 905. Passons à ce que la gérante demande vraiment, une vue par catégorie.

```
par_cat = x.groupby("categorie").agg(ca_ht=("ca_ht", "sum"), marge=("marge_ht", "sum"),
    ↪   taux_moyen_des_lignes=("taux_marge", "mean"))
par_cat["taux_marge"] = par_cat["marge"] / par_cat["ca_ht"]
print(par_cat.round(3).to_string())
```

	ca_ht	marge	taux_moyen_des_lignes	taux_marge
catégorie				
Bien-être	270031.033	91815.643	0.344	0.340
Cuisine	549163.400	199010.450	0.359	0.362
Décoration	593853.075	230078.385	0.374	0.387
Jardin	808649.508	299547.988	0.375	0.370
Maison	690389.717	254326.397	0.373	0.368
Papeterie	132211.000	47060.550	0.364	0.356

Deux colonnes se ressemblent mais ne disent pas la même chose. Le **taux de marge** d'un groupe est le rapport des **sommes** (la marge totale divisée par le chiffre d'affaires hors taxe total) ; la **moyenne des taux** des lignes donne à une ligne de 5 € le même poids qu'une ligne de 150 €. Pour la décoration, la première vaut 38,7 % et la seconde 37,4 %. **Un ratio d'agrégat se calcule toujours à partir des sommes**, jamais en faisant la moyenne de ratios ; c'est la même règle que pour le panier moyen du volume I. Sur l'ensemble des ventes, le taux de marge brute est de 36,9 %, avec des écarts d'une catégorie à l'autre : de 34,0 % pour le bien-être à 38,7 % pour la décoration.

Piège : la remise grignote la marge. Une remise de 20 % ne coûte pas 20 % **de marge**, mais 20 % **du prix**, c'est-à-dire une part bien plus grande de la marge. Sur nos données, le taux de marge tombe de 38,2 % pour les lignes sans remise à 22,7 % pour celles qui bénéficient de 20 % de remise. Une variable marge calculée **avant** remise aurait caché cette érosion.

2.1.3 Les dates : en extraire des morceaux

Une date contient plusieurs informations utiles que l'on ne peut pas exploiter tant qu'elles ne sont pas **séparées** : l'année, le trimestre, le mois, la semaine, le jour de la semaine, le week-end. On les extrait avec l'accessor `.dt`, après avoir **vérifié que la colonne est bien une date** (et non du texte : volume I, section 4.1.3).

```
cmd = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
d = cmd["date_commande"].dt
cmd["annee"], cmd["trimestre"], cmd["mois"] = d.year, d.quarter, d.month
cmd["semaine_iso"], cmd["jour_semaine"] = d.isocalendar().week, d.dayofweek + 1
cmd["week_end"] = d.dayofweek >= 5
cols = ["id_commande", "date_commande", "annee", "trimestre", "semaine_iso", "jour_semaine",
        ↪ "week_end"]
print(cmd[cols].head(4).to_string(index=False))
```

id_commande	date_commande	annee	trimestre	semaine_iso	jour_semaine	week_end
1	2023-01-01	2023	1	52	7	True
2	2023-01-01	2023	1	52	7	True
3	2023-01-01	2023	1	52	7	True
4	2023-01-01	2023	1	52	7	True

Regardez la première ligne du tableau ci-dessus : le 1er janvier 2023 est rangé dans la **semaine 52**, alors que l'année civile affiche 2023 (c'est la semaine 52 de l'année ISO 2022). Deux conventions méritent d'être **écrites** : `dayofweek` numérote les jours de 0 (lundi) à 6 (dimanche), d'où le + 1 pour obtenir 1 à 7 ; et la **semaine ISO** commence le lundi, la semaine 1 étant celle qui contient le premier jeudi de l'année. Cette dernière règle crée un piège redoutable : **les derniers jours de décembre peuvent appartenir à la semaine 1 de l'année suivante**. Le 29 décembre 2025 est un lundi ; il est dans la semaine 1 de l'année 2026.

```
jour = pd.Timestamp("2025-12-29")
print(jour.year, jour.isocalendar().year, jour.isocalendar().week)
```

```
2025 2026 1
```

Sur nos 36 395 commandes, 254 ont une année ISO différente de leur année civile. Un tableau « chiffre d'affaires par année et par semaine » construit avec `semaine_iso` et `annee` mettrait donc les derniers jours de décembre dans la semaine 1 de l'**année précédente** si l'on n'y prend garde. **Règle** : quand on parle de semaines, on garde l'**année ISO avec la semaine ISO** (`isocalendar().year`), jamais l'année civile.

Les morceaux de date servent ensuite à regrouper. Voici le chiffre d'affaires par jour de la semaine : il suffit de relier les lignes à leur date, puis de regrouper.

```
x = x.merge(cmd[["id_commande", "date_commande", "id_client", "cana1", "jour_semaine"]],
            ↪ on="id_commande", validate="m:1")
ca_jour = x.groupby("jour_semaine")["montant"].sum().round(0)
print(ca_jour.to_string())
```

```
jour_semaine
1    508828.0
2    461784.0
3    487038.0
4    517374.0
5    611017.0
6    722969.0
7    344146.0
```

Le samedi (jour 6) est le jour le plus fort (723 k€ sur trois ans) et le dimanche (jour 7) le plus faible (344 k€) : un rapport de 2,1 entre les deux. Ce motif de semaine, invisible dans la colonne

de dates, apparaît dès que l'on a dérivé le jour.

On obtient les mêmes morceaux de date en **SQL**, ce qui est utile quand les données restent dans une base. Voici le comptage des commandes par jour de la semaine avec **DuckDB**, un moteur SQL qui lit directement les fichiers CSV et qui parle un dialecte très proche de PostgreSQL ; on vérifie ensuite qu'il donne le même résultat que pandas.

```
import duckdb
q = "select date_part('isodow', date_commande) as jour, count(*) as n from 'donnees/commandes.csv'
↳ group by 1 order by 1"
n_sql = duckdb.sql(q).df().set_index("jour")["n"]
n_pd = cmd["jour_semaine"].value_counts().sort_index()
print("même résultat que pandas :", (n_sql.values == n_pd.values).all())
```

même résultat que pandas : True

2.1.4 Les classes : découper une variable continue

Un montant de panier est une variable **continue** : il en existe des milliers de valeurs différentes. Pour un tableau de bord, une segmentation commerciale ou un tableau croisé, on le découpe en **classes** (volume I, section 5.1). Deux outils, deux philosophies :

- **pd.cut** découpe selon des **bornes que l'on choisit** (0, 50, 100, 200...) : les classes ont un sens **métier** (« petit panier », « gros panier »), mais des effectifs inégaux ;
- **pd.qcut** découpe selon des **quantiles** : chaque classe contient autant de lignes (dix déciles de 3 640 commandes environ), mais les bornes sont des nombres peu parlants.

```
paniers = x.groupby("id_commande", as_index=False).agg(panier=("montant", "sum"),
↳ id_client=("id_client", "first"), date=("date_commande", "first"))
bornes = [0, 50, 100, 200, np.inf]
paniers["classe"] = pd.cut(paniers["panier"], bornes, right=False, labels=["< 50", "50 à 100", "100 à
↳ 200", "200 et +"])
paniers["decile"] = pd.qcut(paniers["panier"], 10, labels=False) + 1
print(paniers["classe"].value_counts().sort_index().to_string())
print(paniers.groupby("decile")["panier"].agg(["min", "max"]).round(0).head(3).to_string())
```

```
classe
< 50      10874
50 à 100   11261
100 à 200  10445
200 et +   3815
         min  max
decile
1       2.0  22.0
2      22.0  36.0
3      36.0  50.0
```

Le paramètre `right=False` mérite attention : il rend chaque classe **fermée à gauche et ouverte à droite** (`[50 ; 100[`). Sans lui, un panier de **exactement** 100 € irait dans la classe « 50 à 100 » ; avec lui, dans « 100 à 200 ». Les deux conventions sont défendables ; l'important est **d'en choisir une et de l'écrire**, sinon deux tableaux du même fichier ne tombent pas d'accord pour quelques lignes. Les classes obtenues donnent 10 874 paniers de moins de 50 € et 3 815 de 200 € et plus, soit 10 % des commandes. Quant aux déciles, le premier décile s'arrête à 22 € et le cinquième à 80 € : la médiane des paniers.

L'équivalent en SQL s'écrit avec CASE, qui évalue les conditions **dans l'ordre** et s'arrête à la première vraie. On vérifie que les deux méthodes classent les paniers de la même façon.

```
con = duckdb.connect(); con.register("paniers", paniers)
q = "select case when panier < 50 then '< 50' when panier < 100 then '50 à 100' when panier < 200 then
↳ '100 à 200' else '200 et +' end as classe, count(*) as n from paniers group by 1"
n_case = con.sql(q).df().set_index("classe")["n"]
print("CASE = cut :", (n_case.reindex(vc.index.astype(str)).values == vc.values).all())
```

```
CASE = cut : True
```

En pratique : choisir les bornes. Partez de l'usage : si les classes servent à une décision (« remise à partir de 200 € »), les bornes viennent du métier. Si elles servent à comparer des groupes de taille comparable, prenez des quantiles. Dans les deux cas, **gardez la variable continue d'origine** à côté de la classe : on peut toujours reclasser, on ne peut pas « déclasser ».

2.1.5 Indicateurs, rangs et variables retardées

Un **indicateur** (ou variable binaire) vaut vrai ou faux : « la commande a eu lieu un week-end », « le panier dépasse 200 € », « la ligne a bénéficié d'une remise ». C'est la variable dérivée la plus simple, et l'une des plus utiles, parce qu'une moyenne d'indicateur **est une proportion** : la moyenne de `week_end` est la part des commandes passées le week-end.

Les variables les plus délicates sont celles qui **regardent une autre ligne** : « le délai depuis la commande précédente du même client », « le rang de la commande dans la vie du client ». Elles demandent deux gestes : **trier** (le « précédent » n'a de sens que dans un ordre) et **regrouper** (le précédent d'une commande est une commande **du même client**). En pandas, `groupby` puis `shift` décale chaque groupe d'une ligne ; `cumcount` numérote les lignes d'un groupe.

```
paniers = paniers.sort_values(["id_client", "date", "id_commande"]).reset_index(drop=True)
g = paniers.groupby("id_client")
paniers["rang_commande"] = g.cumcount() + 1
paniers["delai_jours"] = (paniers["date"] - g["date"].shift()).dt.days
paniers["dormant_180"] = paniers["delai_jours"] > 180
un = paniers[paniers["id_client"] == 2]
print(un[["id_commande", "date", "panier", "rang_commande", "delai_jours",
↳ "dormant_180"]].to_string(index=False))
```

id_commande	date	panier	rang_commande	delai_jours	dormant_180
434	2023-01-18	7.90	1	NaN	False
2517	2023-04-11	18.90	2	83.0	False
7304	2023-09-27	310.20	3	169.0	False
17755	2024-08-05	135.70	4	313.0	True
19508	2024-10-05	159.40	5	61.0	False
29212	2025-07-05	24.62	6	273.0	True
30003	2025-08-01	75.82	7	27.0	False
35090	2025-12-09	21.43	8	130.0	False
35460	2025-12-16	89.31	9	7.0	False

La première commande du client n°2 a un délai **vide** (NaN), et c'est exact : elle n'a pas de précédente. Une erreur fréquente consisterait à remplacer ce vide par zéro, ce qui dirait « ce client a recommandé le jour même ». Sur nos 4 806 clients, 4 806 commandes n'ont pas de précédente (une par client), et le délai **médian** entre deux commandes consécutives d'un même client est de 47 jours. L'indicateur `dormant_180` marque les commandes passées plus de six mois après la précédente : 13,2 % des commandes qui **ont** une précédente. Notez que `NaN > 180` vaut `False` : l'indicateur, calculé sur toutes les lignes, dilue cette proportion à 11,4 %. **Le choix du dénominateur est une décision** : on documente « sur les commandes qui ont une précédente ».

Le décalage fait aussi apparaître la forme du comportement d'achat. Un client qui revient 83 jours,

puis 169 jours, puis 313 jours après sa commande précédente s'éloigne ; la variable `delai_jours`, que la colonne `date` ne contenait pas, le rend **visible** et **mesurable**.

Expérience : le même délai en SQL. En SQL, la fonction fenêtre `LAG(date) OVER (PARTITION BY id_client ORDER BY date)` joue le rôle de `groupby + shift` (volume I, section 3.3.4). Retenez l'équivalence : `partition by` $\leftrightarrow$ `groupby`, `order by` $\leftrightarrow$ `sort_values`, `lag` $\leftrightarrow$ `shift`.

2.1.6 Du texte à la clé

Pour relier deux sources, il faut une **clé** qui s'écrive de la même façon des deux côtés. Or le texte est la matière la moins fiable : la caisse écrit « BOÎTE RUSTIQUE », le catalogue « Boîte rustique », le CRM « boite rustique ». Trois écritures, un seul objet. La première transformation, avant toute jointure sur du texte, est de **normaliser** : tout passer en minuscules, retirer les accents, remplacer la ponctuation par des espaces, compresser les espaces multiples.

```
import unicodedata, re
def cle_texte(s):
    s = "".join(c for c in unicodedata.normalize("NFD", str(s)) if unicodedata.category(c) != "Mn")
    return re.sub(r"\s+", " ", re.sub(r"[^a-z0-9 ]", " ", s.lower())).strip()
exemples = ["BOÎTE RUSTIQUE", "Boîte rustique", "boite rustique", "Boîte-rustique."]
print([cle_texte(e) for e in exemples])
```

```
['boite rustique', 'boite rustique', 'boite rustique', 'boite rustique']
```

La décomposition en Unicode « NFD » sépare chaque lettre accentuée en une lettre et un **accent combinant** ; on supprime ensuite les accents. Les quatre écritures donnent 1 seule clé. Mais attention : **normaliser n'est pas dédoubler**. Dans notre catalogue de 120 produits, il n'y a que 60 noms distincts, et la normalisation n'en retire aucun (60 clés) : **chaque nom est porté par deux produits** différents. Une clé de texte rend deux écritures comparables ; elle ne dit pas si deux objets distincts portent le même nom. Cette ambiguïté est l'un des fils de la section suivante.

2.1.7 Trois pièges, et comment documenter

Piège 1 : la division par zéro. Un ratio dont le dénominateur peut valoir zéro doit prévoir ce cas. En pandas, `x / 0` donne `inf` (l'infini) et `0 / 0` donne `NaN`, sans erreur : un `inf` dans une moyenne la rend infinie, et personne n'est prévenu. Calculons le panier moyen par client sur 2025. Les clients qui n'ont pas commandé cette année-là n'ont ni chiffre d'affaires ni commande : leur panier moyen n'est **pas défini**.

```
x25 = x[x["date_commande"].dt.year == 2025]
cl = pd.read_csv("donnees/clients.csv")[["id_client"]]
cl = cl.merge(x25.groupby("id_client").agg(ca=("montant", "sum"), nb=("id_commande", "nunique")),
             on="id_client", how="left")
cl["panier_moyen"] = cl["ca"] / cl["nb"]
print("clients sans commande en 2025 :", int(cl["nb"].isna().sum()), "| paniers moyens indéfinis :",
      int(cl["panier_moyen"].isna().sum()))
print("moyenne des paniers moyens (acheteurs) :", round(cl["panier_moyen"].mean(), 2), "| avec des
      zéros :", round(cl["panier_moyen"].fillna(0).mean(), 2))
```

```
clients sans commande en 2025 : 2125 | paniers moyens indéfinis : 2125
moyenne des paniers moyens (acheteurs) : 101.14 | avec des zéros : 65.32
```

Sur 6 000 clients, 2 125 n'ont rien acheté en 2025. Les laisser en `NaN` donne un panier moyen de **101,14 €** (celui des acheteurs) ; les remplacer par zéro l'écrase à 65,32 €. Aucun des deux calculs n'est « faux » : ils répondent à deux questions différentes. Ce qui serait faux serait de **ne pas savoir lequel on a fait**.

Piège 2 : la valeur manquante qui se propage. Une opération arithmétique avec un NaN donne un NaN. Si l'on calcule le chiffre d'affaires total d'un client en **additionnant** ses chiffres par canal, tous ceux qui n'ont pas acheté dans l'un des trois canaux ont un total vide.

```
par_canal = x25.pivot_table(index="id_client", columns="canal", values="montant", aggfunc="sum")
mauvais = par_canal["Boutique"] + par_canal["Site"] + par_canal["Réseaux"]
bon = par_canal.sum(axis=1)
print("totaux vides avec + :", int(mauvais.isna().sum()), "sur", len(par_canal), "| avec sum(axis=1)
↳ :", int(bon.isna().sum()))
```

```
totaux vides avec + : 3224 sur 3875 | avec sum(axis=1) : 0
```

Avec l'opérateur +, 3 224 totaux sur 3 875 (soit 83 %) sont perdus, alors qu'axis=1 avec sum **ignore** les vides. Retenez la règle : **sum ignore les manquants, + les propage**. L'une n'est pas meilleure que l'autre, mais il faut choisir en connaissance de cause.

Piège 3 : la fuite d'information. Une variable dérivée **fuit** quand elle utilise une information qui n'était pas connue à la date où l'on veut s'en servir. Imaginons que la gérante veuille repérer, au 30 juin 2025, les clients qui commanderont encore au second semestre. Parmi les clients qui avaient déjà commandé, nous calculons deux versions de « nombre de commandes » : l'une **avant** le 30 juin, l'autre **au total**, futur compris.

```
coupure = pd.Timestamp("2025-06-30")
avant = paniers[paniers["date"] <= coupure].groupby("id_client").size().rename("avant")
total = paniers.groupby("id_client").size().rename("total")
cible = paniers[(paniers["date"] > coupure) & (paniers["date"].dt.year ==
↳ 2025)].groupby("id_client").size().rename("s2")
t = pd.concat([avant, total, cible], axis=1).dropna(subset=["avant"]).fillna(0)
from sklearn.metrics import roc_auc_score
y = (t["s2"] > 0).astype(int)
print("AUC avec 'avant' :", round(roc_auc_score(y, t["avant"]), 3), "| avec 'total' :",
↳ round(roc_auc_score(y, t["total"]), 3))
```

```
AUC avec 'avant' : 0.776 | avec 'total' : 0.875
```

Sur 4 409 clients ayant déjà commandé au 30 juin, 63 % commandent au second semestre. La variable honnête (avant) a un pouvoir de classement modeste (une AUC de 0,78 : 0,5 correspondrait au hasard, 1 à un classement parfait), alors que la version qui regarde le futur (total, qui **contient** les commandes du second semestre) paraît bien meilleure : 0,87. Cette différence n'a aucune valeur : c'est la **cible** qui se glisse dans la prédiction. Règle : **toute variable qui sert à prévoir doit être calculée avec les seules données antérieures à la date de prévision**. Quand on dérive une variable, on se demande donc : « à quelle date aurais-je pu la calculer ? ». (Le volume III revient en détail sur la fuite d'information et l'évaluation des modèles.)

Documenter chaque variable créée. Une variable dérivée sans définition est une dette. Une **fiche** de quelques lignes suffit ; le chapitre 4 de ce volume en fait un dictionnaire de données complet.

Champ	Exemple pour marge_ht
Nom	marge_ht
Définition	chiffre d'affaires hors taxe de la ligne moins quantité × coût d'achat
Unité	euros
Formule	montant / 1,20 - quantite × cout_achat
Hypothèses	TVA de 20 % (fictive) ; coût d'achat actuel du catalogue, sans frais de transport
Cas particuliers	un produit sans coût d'achat donne une marge vide (jamais zéro)

Champ	Exemple pour marge_ht
Auteur, date	l'analyste, date de création

À retenir.

- Une variable dérivée est une **décision de définition** : unité, base de calcul, cas limites. On l'**écrit**.
- Un **ratio d'agrégat** se calcule à partir des **sommes**, jamais en faisant la moyenne de ratios.
- Pour les dates : on extrait année, trimestre, mois, jour ; on garde **l'année ISO avec la semaine ISO**.
- Les classes se choisissent (cut, bornes du métier) ou se calculent (qcut, quantiles) ; on **écrit la convention des bornes** et l'on garde la variable d'origine.
- Un délai ou un rang demande de **trier** puis de **regrouper** ; le vide d'un premier délai n'est **pas** un zéro.
- Trois pièges : la division par zéro, le NaN qui se propage ($\text{sum} \neq +$), la **fuite d'information**.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 et 2.2, exercices 2.1 à 2.4.

2.2 Fusion et jointure de jeux de données

La demande de la gérante tient en une table, et les données arrivent en pièces détachées : douze fichiers de caisse, un export du site, une table de produits. Cette section apprend à **recoller** ces pièces sans en perdre ni en inventer : comprendre les types de jointure et la **cardinalité**, **contrôler** les effectifs avant et après chaque opération, **empiler** les fichiers de la caisse malgré leurs formats changeants, **lire** l'export du site et **harmoniser** les deux sources en une table de ventes unique, puis **chercher ce qui manque**. À la fin, on retrouve le même résultat avec trois outils (pandas, SQL, R), et l'on ouvre enfin le fichier de vérité pour juger le travail.

2.2.1 Quatre manières de recoller

Il existe deux familles d'opérations, qui ne répondent pas à la même question :

- **Joindre** (*merge, join*) relie deux tables **côte à côte** : chaque ligne de la première reçoit, de la seconde, les colonnes qui lui correspondent, selon une **clé**. On ajoute des **colonnes**.
- **Empiler** (*concat, union*) met deux tables **l'une sous l'autre** : elles doivent avoir la même structure. On ajoute des **lignes**.

Pour une jointure, la question qui change tout est : **que faire des lignes sans correspondance ?** Quatre réponses, que l'on retrouve sous le même nom en SQL, en R et dans Power Query.

Type (how=)	Garde...	Lignes sans correspondance
inner	seulement les lignes présentes des deux côtés	perdues
left	toutes les lignes de la table de gauche	à droite : colonnes vides (NaN)
right	toutes celles de la table de droite	à gauche : colonnes vides

Type (how=)	Garde...	Lignes sans correspondance
outer	toutes les lignes des deux tables	colonnes vides de l'un ou l'autre côté

Sur trois lignes de produits et trois lignes de stocks, l'option `indicator=True` ajoute une colonne qui dit **d'où vient chaque ligne** : c'est le meilleur outil de diagnostic d'une jointure.

```
a = pd.DataFrame({"id_produit": [1, 2, 3], "nom": ["Bol", "Vase", "Plaid"]})
b = pd.DataFrame({"id_produit": [2, 3, 4], "stock": [10, 0, 7]})
print(a.merge(b, on="id_produit", how="outer", indicator=True).to_string(index=False))
```

```
id_produit  nom  stock  _merge
1      Bol   NaN   left_only
2      Vase  10.0    both
3     Plaid   0.0    both
4       NaN   7.0  right_only
```

Le produit 1 n'a pas de stock (`left_only`), le produit 4 n'a pas de nom (`right_only`), les produits 2 et 3 sont présents des deux côtés (`both`). Une jointure `inner` n'aurait gardé que ces deux derniers ; une jointure `left`, les produits 1, 2 et 3. **Le choix du type de jointure est un choix d'analyse** : si l'on veut le chiffre d'affaires de **tous** les produits du catalogue, y compris ceux qui n'ont rien vendu, il faut un `left` depuis le catalogue ; un `inner` ferait **disparaître** ces produits sans le moindre message.

Piège : la jointure `inner` qui fait disparaître des lignes. Par défaut, `merge` fait une jointure `inner`. Si la clé est mal écrite d'un côté (une majuscule, un espace), les lignes concernées **disparaissent** sans erreur. On vérifie donc toujours le nombre de lignes avant et après.

2.2.2 La cardinalité et le contrôle des effectifs

La **cardinalité** d'une jointure dit combien de lignes de droite correspondent à une ligne de gauche.

- **1-1** : chaque clé apparaît au plus une fois des deux côtés (un client, sa fiche d'identité).
- **1-n** (ou **n-1** vu de l'autre côté) : une clé de la table « un » correspond à plusieurs lignes de l'autre (un produit, ses lignes de vente). C'est le cas le plus courant, et le plus sain : on **ajoute des colonnes** sans changer le nombre de lignes de la table « n ».
- **n-n** : la clé est répétée **des deux côtés**. Chaque ligne de gauche est associée à **toutes** les lignes de droite de même clé : le nombre de lignes **se multiplie**. C'est presque toujours un accident.

On teste la cardinalité avec le paramètre `validate` ("`1:1`", "`1:m`", "`m:1`", "`m:m`") : `pandas` lève une erreur si la réalité ne correspond pas à ce que l'on a **déclaré**. C'est la façon la plus simple de transformer une hypothèse tacite en vérification. Et quatre contrôles, à faire après **chaque** jointure, résument la méthode du chapitre :

1. le **nombre de lignes** avant et après (un `1-n` ne doit rien changer) ;
2. la **somme** d'une colonne de la table « n » avant et après ;
3. le **nombre de lignes sans correspondance** (`indicator`) ;
4. l'**unicité** de la clé du côté « un ».

```
def controler(gauche, droite, cle, how="left"):
    r = gauche.merge(droite, on=cle, how=how, indicator=True)
    return {"lignes avant": len(gauche), "lignes après": len(r),
```

```

    "clé dupliquée à droite": int(droite.duplicated(cle).sum()),
    "sans correspondance": int((r["_merge"] == "left_only").sum())}
print(controller(lig, prod[["id_produit", "cout_achat"]], "id_produit"))

```

```
{'lignes avant': 83905, 'lignes après': 83905, 'clé dupliquée à droite': 0, 'sans correspondance': 0}
```

Pour la jointure des lignes de vente avec les produits, les quatre contrôles passent : 83 905 lignes avant et 83 905 après, aucune clé dupliquée du côté des produits (0), aucune ligne sans correspondance (0). **On peut avancer.** Gardez cette petite fonction : vous allez voir, dans la suite, des jointures qui ne passent pas.

2.2.3 Empiler les douze fichiers de la caisse

La caisse de la boutique envoie **un fichier par mois**. Pour répondre à la gérante, il faut les mettre bout à bout, et c'est là que les ennuis commencent : **le format a changé en cours d'année**. Ce phénomène porte un nom, la **dérive de schéma** : la source a évolué (mise à jour du logiciel, changement de paramètres régionaux) et n'a prévenu personne.

Mois	Codage du fichier	Séparateur	Décimale	Nom de la colonne des quantités	Date	Autres différences
janvier à juin	cp1252 (ancien Windows)	;	,	« Qté »	jj/mm/aaaa	
juillet à septembre	UTF-8 avec « BOM »	;	,	« Quantité »	jj/mm/aa (année sur 2 chiffres)	
octobre à décembre	UTF-8 avec « BOM »	,	.	« Qté »	jj/mm/aaaa	colonne « Remise (%) » en plus, champs entre guillemets

Chaque fichier commence par **trois lignes de titre** (dont une vide), répète son **en-tête** toutes les soixante lignes environ (comme à chaque « page » imprimée), et se termine par une **ligne de total**. Aucune de ces particularités n'est une erreur ; ce sont des **conventions d'édition** qu'un programme doit connaître. L'approche la plus robuste est de ne **rien deviner à la main** : on **détecte** le format de chaque fichier, puis on applique les mêmes étapes. D'abord, ouvrir le fichier en octets et choisir le codage : on essaie UTF-8, et l'on retombe sur cp1252 si le décodage échoue.

```

import io, glob, os
def lire_brut(chemin):
    brut = open(chemin, "rb").read()
    try:
        texte = brut.decode("utf-8-sig")          # « sig » : retire le BOM, marque d'ordre des octets
    except UnicodeDecodeError:
        texte = brut.decode("cp1252")
    lignes = texte.splitlines()
    sep = ";" if lignes[3].count(";") > lignes[3].count(",") else ","
    return lignes, sep

```

Le séparateur se devine sur la ligne d'en-tête (la quatrième) : il y en a plus de points-virgules que de virgules dans un fichier à points-virgules, et réciproquement (les décimales à virgule

n'apparaissent pas dans l'en-tête). Ensuite, lire le corps en **texte** (jamais de conversion automatique : on convertit explicitement), retirer le total et les en-têtes répétés, et harmoniser le nom des colonnes.

```
RENOM = {"N° ticket": "ticket", "Ticket": "ticket", "Qté": "quantite", "Quantité": "quantite",
↪ "Date": "date", "Heure": "heure",
        "Article": "article", "Catégorie": "categorie", "Prix unitaire": "prix_unitaire", "Remise
↪ (%)": "remise_pct", "Montant": "montant"}

def ouvrir_caisse(chemin):
    lignes, sep = lire_brut(chemin)
    dec = "," if sep == ";" else "."
    total = float(lignes[-1].split(sep)[-1].replace(dec, "."))
    df = pd.read_csv(io.StringIO("\n".join(lignes[3:-1])), sep=sep, dtype=str,
↪ keep_default_na=False).rename(columns=RENOM)
    return df[~df["ticket"].isin(["N° ticket", "Ticket"])].copy(), total, dec
```

Reste à **typer** : nombres (en remplaçant la décimale locale par le point), quantité entière, date (le format dépend de la longueur du texte : dix caractères pour l'année sur quatre chiffres, huit pour l'année sur deux).

```
def typer_caisse(df, dec, nom):
    for c in ["prix_unitaire", "montant", "remise_pct"]:
        df[c] = pd.to_numeric(df[c].str.replace(dec, "."), errors="coerce") if c in df else np.nan
    df["quantite"] = df["quantite"].astype(int)
    df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y" if len(df["date"].iloc[0]) == 10 else
↪ "%d/%m/%y")
    df["fichier"] = nom
    df["id_commande"] = df["ticket"].str.lstrip("T").astype(int)
    return df
```

On applique alors la même fonction aux douze fichiers, on empile avec `pd.concat`, et l'on construit **en même temps** un tableau de contrôle : pour chaque fichier, le **total affiché en pied** et la **somme des montants lus**. C'est la ligne de total du fichier qui sert de **somme de contrôle**.

```
morceaux, ctrl = [], []
for f in sorted(glob.glob("donnees/caisse/caisse_2025-*.csv")):
    df, total, dec = ouvrir_caisse(f)
    df = typer_caisse(df, dec, os.path.basename(f))
    morceaux.append(df)
    ctrl.append((os.path.basename(f), len(df), total, round(df["montant"].sum(), 2),
↪ int(df["montant"].isna().sum())))
caisse = pd.concat(morceaux, ignore_index=True)
ctrl = pd.DataFrame(ctrl, columns=["fichier", "lignes", "total_affiche", "somme_lue",
↪ "montants_vides"])
ctrl["ecart"] = (ctrl["total_affiche"] - ctrl["somme_lue"]).round(2)
print(len(caisse), "lignes |", caisse["fichier"].nunique(), "fichiers")
print(ctrl.iloc[[0, 3, 6, 9, 11]].to_string(index=False))
```

12678 lignes 12 fichiers						
	fichier	lignes	total_affiche	somme_lue	montants_vides	ecart
caisse_2025-01.csv		955	38882.41	37826.31	36	1056.10
caisse_2025-04.csv		1013	45832.57	45426.14	31	406.43
caisse_2025-07.csv		877	41595.82	41035.68	21	560.14
caisse_2025-10.csv		1138	51321.48	49839.95	38	1481.53
caisse_2025-12.csv		1694	73384.87	70924.34	63	2460.53

Les douze fichiers, empilés, donnent 12 678 lignes de vente. Mais le tableau de contrôle est **net** : pour chaque fichier, la somme des montants lus est **inférieure** au total affiché par la caisse, d'un écart qui va de 406 € à 2 461 € selon les mois. Sur l'année, la caisse annonce 560 973,91 €, et notre table n'en contient que 547 896,42 €, soit **13 077,49 € de moins**. Une jointure ou une lecture qui « marche » sans erreur peut ainsi perdre de l'argent, et c'est la **ligne de total du fichier**, que l'on aurait pu jeter avec les titres, qui l'a révélé.

D'où vient l'écart ? La colonne `montants_vides` donne la piste : **399 lignes (3,1 %) n'ont pas de montant** (cellule vide dans l'export). Une ligne de caisse contient pourtant de quoi **reconstituer** son montant : quantité × prix unitaire, moins la remise. La remise n'est écrite que dans les fichiers d'octobre à décembre ; avant, on ne la connaît pas, et l'on suppose zéro (c'est une **hypothèse** à documenter, que nous testerons plus loin). Les montants reconstitués sont **marqués**, jamais mélangés en silence avec les montants lus.

```
caisse["montant_vide"] = caisse["montant"].isna()
remise = caisse["remise_pct"].fillna(0)
caisse["montant_corrige"] = caisse["montant"].fillna((caisse["quantite"] * caisse["prix_unitaire"] *
↳ (1 - remise / 100)).round(2))
caisse["identique_precedente"] = caisse.duplicated(["ticket", "date", "heure", "article",
↳ "categorie", "quantite", "prix_unitaire", "montant"])
par_fichier = caisse.groupby("fichier")["montant_corrige"].sum().round(2).values
print("écart restant sur l'année :", round(par_fichier.sum() - ctrl["total_affiche"].sum(), 2), "€")
print("lignes strictement identiques à une ligne précédente :",
↳ int(caisse["identique_precedente"].sum()))
```

```
écart restant sur l'année : 3498.68 €
lignes strictement identiques à une ligne précédente : 154
```

Après reconstitution, l'écart ne disparaît pas : il **change de signe**. Au lieu de 13 077 € **manquants**, il y a maintenant 3 499 € **en trop** (0,62 % du total). La cause est la seconde anomalie du fichier : des lignes **strictement identiques** à une ligne précédente, au nombre de 154, pour 7 009 € : probablement des **doubles scans** à la caisse.

Faut-il les supprimer ? Le total de contrôle dit **combien** d'euros sont en trop, pas **lesquelles** des lignes le sont, car deux lignes identiques peuvent aussi être **légitimes** : un client qui achète deux fois le même article, enregistré en deux passages. Voici l'arithmétique des deux décisions possibles.

Décision	Écart au total de la caisse
garder toutes les lignes (et les signaler)	+3 499 € (+0,62 %)
supprimer toutes les lignes identiques	-3 511 € (-0,63 %)

Les deux erreurs sont du même ordre de grandeur et de signe opposé : aucune décision n'est exacte, et la bonne réponse est de **ne rien supprimer sans preuve** : on **garde** les lignes, on les **marque** (`identique_precedente`), et l'on **écrit** l'incertitude résiduelle (de l'ordre de 0,6 % du chiffre d'affaires de la caisse). En fin de section, nous ouvrirons le fichier de vérité pour savoir ce qu'il en était vraiment.

2.2.4 Lire l'export du site

L'export du site est d'une autre nature : un seul fichier d'en-têtes de commande (`site_commandes.csv`), un fichier de lignes (`site_lignes.csv`), liés par la référence `order_ref`. Tout est écrit **en texte**, et la méthode est la même : ne rien convertir automatiquement, **regarder** d'abord.

```
site = pd.read_csv("donnees/site_commandes.csv", dtype=str, keep_default_na=False)
lignes_site = pd.read_csv("donnees/site_lignes.csv")
print(site[["order_ref", "created_at", "status", "total",
↳ "currency"]].head(4).to_string(index=False))
print(site["status"].value_counts().to_dict())
print(site["currency"].value_counts().to_dict())
```

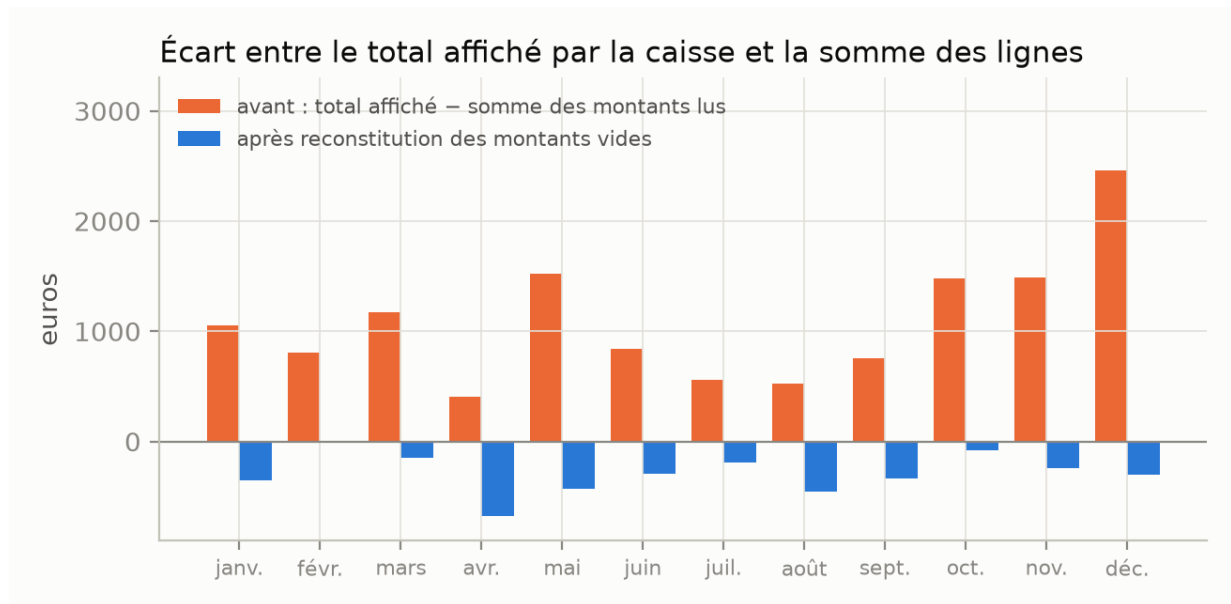


Figure 8: Écart mensuel entre le total affiché en pied de chaque fichier de la caisse et la somme des montants lus : avant correction, les montants vides font « manquer » de l'argent (barres orange, positives) ; après reconstitution, les lignes doublées en font apparaître en trop (barres bleues, négatives).

```
order_ref      created_at status  total currency
WEB-023473 2025-01-01 09:27:00 PAID  34,92 €      EUR
WEB-023450 2025-01-01 11:44:00 paid  63,66 €      eur
WEB-023464 2025-01-01 13:14:00 paid 195,40 €      EUR
WEB-023465 2025-01-01 14:22:00 paid  81,07 €      EUR
{'paid': 3629, 'PAID': 1537, 'Paid': 907, 'cancelled': 186}
{'EUR': 4987, 'eur': 643, '€': 629}
```

Trois remarques s'imposent. Le **statut** est écrit de trois façons (paid, PAID, Paid) : une comparaison avec `== "paid"` laisserait de côté 40 % des commandes payées. La **devise** est écrite de trois façons aussi (EUR, eur, €) : une seule devise en réalité, mais un groupby sur la colonne brute en ferait trois groupes. Et le **montant** est un texte, avec un symbole, une virgule ou un point selon la ligne : "34,92 €", "158.42". Le plus sournois est ailleurs : regardons l'ordre de grandeur du montant mois par mois, après une conversion qui ignore pour l'instant tout format exotique.

```
site["brut"] = pd.to_numeric(site["total"].str.replace("€", "").str.replace(" ", "").str.replace(",",
    ↪ "."), errors="coerce")
site["date_heure"] = pd.to_datetime(site["created_at"].str.replace("Z", "", regex=False),
    ↪ format="ISO8601")
site["mois"] = site["date_heure"].dt.month
mediane = site[site["customer_email"].str.lower() !=
    ↪ "test@example.com"].groupby("mois")["brut"].median().round(0)
print(mediane.loc[7:11].to_string())
```

```
mois
7      91.0
8     87.0
9    1741.0
10   8426.0
11   7190.0
```

Voilà une anomalie que **seul un contrôle d'ordre de grandeur** détecte : la **médiane du montant** vaut 87 € en août, 1 741 € en septembre et 8 426 € en octobre. Personne n'a vendu pour 8 000

€ de bougies : à partir d'une certaine date, la plateforme exporte les montants **en centimes**. La rupture coïncide avec un autre changement, discret : à partir du 15 septembre, le texte de la date se termine par un Z (ISO 8601, heure UTC), alors qu'il n'en avait pas auparavant. Une mise à jour du site a changé **deux choses à la fois**, et n'en a annoncé aucune. L'hypothèse à tester : *les montants sont en centimes exactement quand la date porte le Z*. La médiane brute vaut 83 € sans Z et 7901 € avec Z.

On teste l'hypothèse de la seule manière convaincante : on corrige, puis on **compare avec une autre source**, ici la somme des lignes de détail de chaque commande, qui est indépendante du montant total de l'en-tête.

```
site["en_utc"] = site["created_at"].str.endswith("Z")
site["total_num"] = np.where(site["en_utc"], site["brut"] / 100, site["brut"])
lignes_site["montant"] = (lignes_site["qty"] * lignes_site["unit_price"] * (1 -
↳ lignes_site["discount_pct"] / 100)).round(2)
somme_lignes = lignes_site.groupby("order_ref")["montant"].sum().rename("somme_lignes")
test = site[site["customer_email"].str.lower() !=
↳ "test@example.com"].drop_duplicates("order_ref").merge(somme_lignes, on="order_ref", how="left")
print("commandes dont le total égale la somme des lignes :", round(((test["total_num"] -
↳ test["somme_lignes"]).abs() < 0.011).mean() * 100, 1), "%")
```

```
commandes dont le total égale la somme des lignes : 100.0 %
```

Sur 6 078 commandes, **100 %** ont un total égal à la somme de leurs lignes : l'hypothèse « centimes si et seulement si Z » est confirmée, au centime près. Reste à regarder la **date** elle-même. Un Z signifie « heure UTC » : si elle l'était vraiment, les heures de commande devraient se décaler d'une ou deux heures par rapport à celles d'avant la mise à jour, puisque la boutique n'est pas à l'heure UTC. Comparons l'heure moyenne de commande avant et après.

```
heure_moyenne = site.assign(h=site["date_heure"].dt.hour).groupby("en_utc")["h"].mean().round(2)
print(heure_moyenne.to_dict())
```

```
{False: 15.45, True: 15.37}
```

L'heure moyenne est de 15,45 avant et de 15,37 après : **aucun décalage**. Le Z est donc une étiquette sans effet sur les heures, ou les horodatages sont restés locaux malgré l'étiquette : nous les traitons comme des heures locales, en **documentant** l'hypothèse (et en la signalant au responsable du site). Ce raisonnement, tester une hypothèse sur la distribution plutôt que de la croire, vaut mieux que n'importe quelle conversion de fuseau faite « au cas où ».

Il reste à **filtrer**. Trois catégories de lignes ne sont pas des ventes : les **commandes de test** (adresse test@example.com), les **doublons d'export** (même référence deux fois : l'export a été relancé) et les **commandes annulées**. On les retire **dans cet ordre, en comptant ce que l'on retire**, pour pouvoir en rendre compte.

```
site["email_norm"] = site["customer_email"].str.strip().str.lower()
n0 = len(site)
s1 = site[site["email_norm"] != "test@example.com"]
s2 = s1.drop_duplicates("order_ref")
s3 = s2[s2["status"].str.lower() != "cancelled"]
print("export brut :", n0, "| sans tests :", len(s1), "| sans doublons :", len(s2), "| sans annulées
↳ :", len(s3))
cmd_site = site.assign(est_test=site["email_norm"] == "test@example.com",
↳ est_double=site.duplicated("order_ref"), statut=site["status"].str.lower())
```

```
export brut : 6259 | sans tests : 6199 | sans doublons : 6078 | sans annulées : 5897
```

L'export contient 6 259 lignes ; on en retire 60 de test, 121 doublons et 181 annulées, pour **5 897 commandes** (600 164,13 € de chiffre d'affaires). Les commandes annulées pesaient 17 551,32 € :

elles n'ont pas été vendues, et les compter gonflerait le chiffre d'affaires.

Intuition. Un export n'est pas un fait, c'est un **document** produit par un logiciel, avec ses conventions et ses accidents. Avant de calculer avec lui, on le **lit comme on lirait un rapport** : que contient-il, que signifie chaque ligne, qu'est-ce qui a changé ? Les deux accidents de cet export (les centimes, les doublons) se détectent par l'**ordre de grandeur** et par le **comptage**, pas par la lecture ligne à ligne.

2.2.5 Relier les ventes aux produits : la jointure qui multiplie

La caisse ne note ni numéro de produit ni coût d'achat, seulement le **nom de l'article** et son prix unitaire. Pour calculer une marge, il faut retrouver le produit dans la table des produits. La première idée est de joindre **sur le nom**, normalisé avec la fonction `cle_texte` de la section 2.1.6.

```
produits = pd.read_csv("donnees/produits.csv")
caisse["nom_cle"] = caisse["article"].map(cle_texte)
produits["nom_cle"] = produits["nom_produit"].map(cle_texte)
par_nom = caisse.merge(produits[["nom_cle", "id_produit", "cout_achat"]], on="nom_cle", how="left")
print("lignes avant :", len(caisse), "| après la jointure sur le nom :", len(par_nom))
try:
    caisse.merge(produits[["nom_cle", "id_produit"]], on="nom_cle", validate="m:1")
except Exception as e:
    print(type(e).__name__, ":", str(e).splitlines()[0])
```

```
lignes avant : 12678 | après la jointure sur le nom : 25356
MergeError : Merge keys are not unique in right dataset; not a many-to-one merge
```

Le nombre de lignes **double** : de 12 678 à 25 356. La cause, vue en 2.1.6 : **chaque nom de produit est porté par deux produits**, donc chaque ligne de caisse se retrouve associée aux deux. Le chiffre d'affaires, recalculé sur cette table, passerait de 547 896 € à 1 095 793 €. C'est le piège classique de la jointure **n-n** : aucune erreur, un résultat **doublement faux**. Le paramètre `validate="m:1"` l'attrape, avec un message clair (le dernier affichage ci-dessus) ; c'est pourquoi il faut **toujours** le déclarer.

Que faire ? Chercher un **second élément de clé**. Les deux produits qui partagent un nom ont des **prix différents** (sauf exception, voir ci-dessous) : le prix de la ligne de caisse doit égaler le prix du catalogue majoré de la hausse de 3 % du 1er janvier 2025, ce qui permet de départager. Joindre sur un **prix** pose un problème technique : des nombres décimaux ne s'égalent pas toujours exactement (arrondis). On joint donc sur le nom, puis l'on **filtre** les candidats dont le prix est « assez proche » (à un demi-centime près).

```
produits["prix_2025"] = (produits["prix_vente"] * 1.03).round(2)
c = caisse.reset_index().merge(produits[["nom_cle", "id_produit", "prix_2025", "cout_achat"]],
    ↪ on="nom_cle")
c = c[(c["prix_unitaire"] - c["prix_2025"]).abs() < 0.011]
n = c.groupby("index").size()
print("lignes appariées de façon unique :", int((n == 1).sum()), "| ambiguës :", int((n > 1).sum()),
    ↪ "| sans candidat :", len(caisse) - len(n))
```

```
lignes appariées de façon unique : 12452 | ambiguës : 226 | sans candidat : 0
```

Avec le prix, **12 452 lignes** trouvent un produit **unique**, aucune n'est sans candidat, et 226 restent **ambiguës**. Ces dernières correspondent à une seule paire de produits, les numéros 62 et 72 (le même nom), qui ont **exactement le même prix** (2,90 €) **et le même coût d'achat** (1,53 € et 1,53 €) : **aucun** élément de la ligne de caisse ne permet de dire de quel produit il s'agit. Deux conséquences :

- l'identité du produit reste **inconnue** pour ces 226 lignes : on laisse `id_produit` **vide** plutôt que de tirer au sort ;
- mais la **marge n'est pas affectée**, puisque les deux produits ont le même coût : on peut y mettre la moyenne des coûts candidats. C'est une **ambiguïté sans conséquence pour la question posée**, et c'est ce qu'il faut écrire (elle en aurait pour une question sur le stock par produit).

On range ce résultat dans la table de caisse : l'identifiant du produit quand il est unique, le coût d'achat dans tous les cas.

```
unique = n[n == 1].index
caisse["id_produit"] = np.nan
caisse.loc[unique, "id_produit"] = c[c["index"].isin(unique)].set_index("index")["id_produit"]
caisse["cout_achat"] = c.groupby("index")["cout_achat"].mean()
print(caisse[["article", "prix_unitaire", "id_produit",
↪ "cout_achat"]].head(3).to_string(index=False))
print("lignes sans id_produit :", int(caisse["id_produit"].isna().sum()), "| sans coût :",
↪ int(caisse["cout_achat"].isna().sum()))
```

	article	prix_unitaire	id_produit	cout_achat
	Poêle mat	40.07	2.0	21.47
	Tapis nordique	64.79	26.0	28.56
	Théière rustique	52.43	5.0	24.65

lignes sans id_produit : 226 | sans coût : 0

Pour le site, la question ne se pose pas : les lignes portent une **référence produit** (sku, comme P043) qui contient directement le numéro du produit. Une vraie clé vaut mieux que la meilleure reconstitution : si l'on peut obtenir de la source un identifiant plutôt qu'un libellé, **on le demande**.

Piège : les homonymes. Dans nos données, c'est un artefact de fabrication ; dans la vie réelle, c'est courant (deux clients nommés « Martin », deux articles « Coussin bleu » de tailles différentes). **Un nom n'est jamais une clé.** Quand on est forcé de joindre sur un libellé, on teste l'unicité de la clé (`validate=`), on lui adjoint un second critère, et l'on **compte** ce qui reste ambigu.

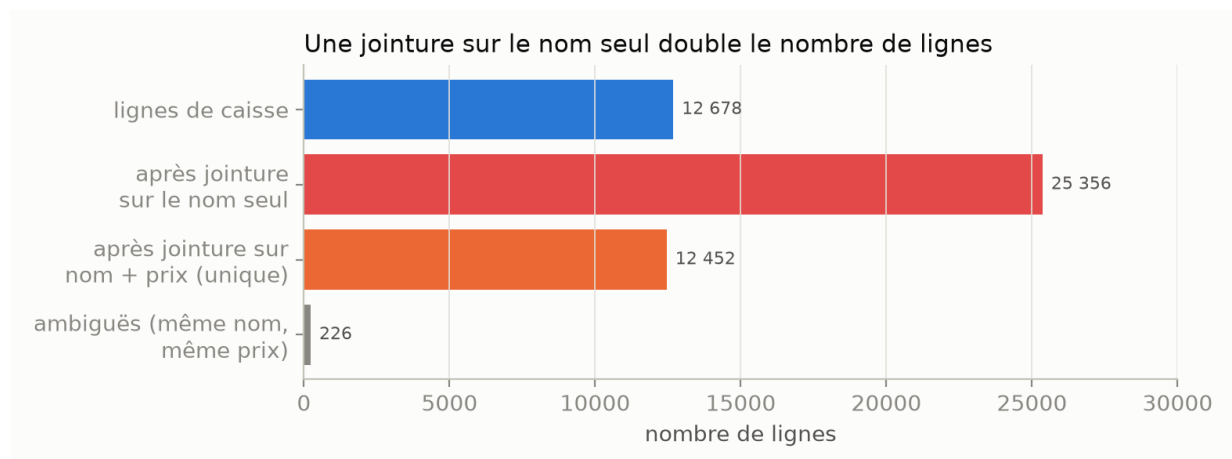


Figure 9: Nombre de lignes de la caisse avant et après jointure avec les produits : la jointure sur le nom seul double les lignes ; ajouter le prix à la clé rétablit un produit unique, sauf pour une paire de produits indiscernables.

2.2.6 Harmoniser et empiler : la table des ventes

Les deux sources sont maintenant propres. Pour les empiler, il faut qu'elles aient le **même schéma** : mêmes noms de colonnes, mêmes types, mêmes unités. On le **conçoit d'abord**, comme un contrat.

Colonne commune	Définition	Caisse	Site
source, canal	d'où vient la ligne	caisse, Boutique	site, Site
id_commande	numéro de commande	numéro du ticket sans le T	référence sans WEB-
date	jour de la vente	date du ticket	date de l'horodatage
id_produit, nom_produit, categorie	produit, en écriture normalisée	reconstitués (2.2.5) ; nom et catégorie pris dans le catalogue	sku converti ; nom et catégorie pris dans le catalogue
quantite, prix_unitaire, remise_pct	comme dans la source	remise connue seulement au dernier trimestre	qty, unit_price, discount_pct
montant	montant TTC payé, en euros	lu, ou reconstitué	quantité × prix × (1 – remise)
cout_achat	coût d'achat unitaire	du catalogue	du catalogue
montant_reconstitue, identique_precedente	indicateurs de qualité	oui / ligne identique à la précédente	non

Le schéma est le **contrat** entre les sources et l'analyse : si une colonne ne peut pas être alimentée honnêtement, on la laisse **vide** plutôt que de lui donner une valeur plausible. Construisons d'abord la partie caisse. Le nom et la catégorie viennent du **catalogue** (une seule écriture normalisée au lieu des cinq de la caisse).

```
ref_nom = produits.drop_duplicates("nom_cle").set_index("nom_cle")[["nom_produit", "categorie"]]
v_caisse = caisse.drop(columns=["categorie"]).join(ref_nom, on="nom_cle")
v_caisse = v_caisse.assign(source="caisse", canal="Boutique", montant=v_caisse["montant_corrige"],
↳ montant_reconstitue=v_caisse["montant_vide"])
print(v_caisse[["id_commande", "nom_produit", "categorie", "quantite", "montant",
↳ "montant_reconstitue"]].head(3).to_string(index=False))
```

id_commande	nom_produit	categorie	quantite	montant	montant_reconstitue
23468	Poêle mat	Cuisine	1	40.07	False
23468	Tapis nordique	Maison	1	64.79	False
23458	Théière rustique	Cuisine	1	52.43	False

Puis la partie site : on ne garde que les commandes valides, on relie les lignes à leur commande, on convertit la référence produit, on calcule le montant, et l'on ajoute le nom, la catégorie et le coût du catalogue par une jointure **m:1** (vérifiée).

```
ok = cmd_site[~cmd_site["est_test"] & ~cmd_site["est_double"] & (cmd_site["statut"] != "cancelled")]
ls = lignes_site.merge(ok[["order_ref",
↳ "date_heure"]].assign(id_commande=ok["order_ref"].str[4:].astype(int)), on="order_ref",
↳ validate="m:1")
ls = ls.assign(id_produit=ls["sku"].str[1:].astype(int), quantite=ls["qty"],
↳ prix_unitaire=ls["unit_price"], remise_pct=ls["discount_pct"])
v_site = ls.merge(produits[["id_produit", "nom_produit", "categorie", "cout_achat"]],
↳ on="id_produit", validate="m:1")
v_site = v_site.assign(source="site", canal="Site", date=v_site["date_heure"].dt.normalize(),
↳ montant_reconstitue=False, identique_precedente=False)
print("lignes de commandes valides :", len(ls), "| lignes après jointure produits :", len(v_site))
```

lignes de commandes valides : 13510 | lignes après jointure produits : 13510

Il ne reste qu'à **empiler** les deux parties, en ne gardant que les colonnes du contrat et dans le même ordre, puis à calculer la marge (2.1.2).

```
cols = ["source", "canal", "id_commande", "date", "id_produit", "nom_produit", "categorie",
        ↪ "quantite", "prix_unitaire", "remise_pct", "montant",
        ↪ "cout_achat", "montant_reconstitue", "identique_precedente"]
ventes = pd.concat([v_caisse[cols], v_site[cols]], ignore_index=True)
ventes["marge_ht"] = ventes["montant"] / 1.20 - ventes["quantite"] * ventes["cout_achat"]
resume = ventes.groupby("canal").agg(lignes=("montant", "size"), commandes=("id_commande",
        ↪ "nunique"), ca=("montant", "sum"), marge=("marge_ht", "sum"))
print(resume.round(0).to_string())
```

	lignes	commandes	ca	marge
canal				
Boutique	12678	5442	564473.0	178813.0
Site	13510	5897	600164.0	189431.0

La table demandée par la gérante existe : 26 188 lignes de vente pour l'année 2025, avec le produit, la catégorie, le montant, le coût d'achat et la marge (37,9 % du chiffre d'affaires hors taxe sur l'ensemble). La caisse a 5 442 commandes pour 564 473 € et le site 5 897 commandes pour 600 164 €. Mais **une table n'est pas fiable parce qu'elle a été construite sans erreur** : il faut maintenant la contrôler.

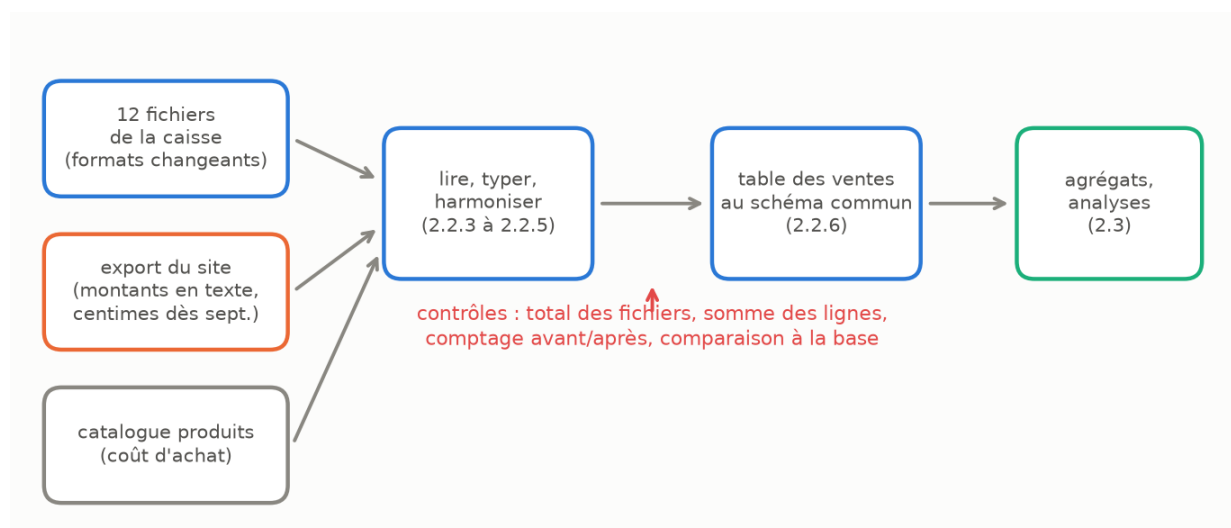


Figure 10: Chaîne de préparation des ventes de 2025.

2.2.7 Contrôler : comparer à la base, puis ouvrir la vérité

Pour contrôler la table des ventes, il faut un **point de comparaison indépendant**. Ici, nous en avons un : la base du volume I (commandes et lignes_commande), qui contient les mêmes ventes telles que le système de gestion les a enregistrées. On compare, **canal par canal**, le chiffre d'affaires de notre table avec celui de la base.

```
cmd_b = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
base = lig.merge(cmd_b[["id_commande", "date_commande", "canal"]], on="id_commande", validate="m:1")
base = base[base["date_commande"].dt.year == 2025]
comp = pd.DataFrame({"table_ventes": ventes.groupby("canal")["montant"].sum(), "base":
        ↪ base.groupby("canal")["montant"].sum()}).dropna()
comp["ecart"] = comp["table_ventes"] - comp["base"]
comp["ecart_pct"] = comp["ecart"] / comp["base"] * 100
print(comp.round(2).to_string())
```

	table_ventes	base	ecart	ecart_pct
canal				
Boutique	564472.59	560973.91	3498.68	0.62
Site	600164.13	617715.45	-17551.32	-2.84

Deux écarts, **deux explications** à produire. Pour la **Boutique**, la table dépasse la base de 3 498,68 € (+0,62 %) : on retrouve exactement l'écart du total de contrôle de 2.2.3, c'est-à-dire les lignes identiques **et** la reconstitution approximative des montants. Pour le **Site**, la table est **inférieure** à la base de 17 551,32 € (2,8 %) : c'est la somme des commandes annulées retirées en 2.2.4 (17 551,32 €), qui existent dans la base mais ne sont pas des ventes. Un écart n'est « bon » ou « mauvais » que **s'il est expliqué**.

Ouvrons maintenant la **vérité**. Le fichier `verite_caisse.csv` indique, pour chaque ligne de chaque fichier, le numéro de la ligne de commande d'origine et si c'est un vrai double scan. Cela permet de savoir **ce que valaient nos décisions**, une chose impossible dans la vie réelle.

```
verite = pd.read_csv("donnees/verite_caisse.csv")
caisse["rang"] = caisse.groupby("fichier").cumcount(); verite["rang"] =
↳ verite.groupby("fichier").cumcount()
cv = caisse.merge(verite[["fichier", "rang", "id_ligne", "est_doublon"]], on=["fichier", "rang"],
↳ validate="1:1")
cv["montant_vrai"] = cv["id_ligne"].map(lig.set_index("id_ligne")["montant"])
ident = cv[cv["identique_precedente"]]
print("lignes identiques :", len(ident), "| vrais doubles scans :", int(ident["est_doublon"].sum()),
↳ "| répétitions légitimes :", int((ident["est_doublon"] == 0).sum()))
vides = cv[cv["montant_vide"] & (cv["est_doublon"] == 0)]
print("montants vides : reconstitués", round(vides["montant_corrige"].sum(), 2), "€ | vrais",
↳ round(vides["montant_vrai"].sum(), 2), "€")
```

```
lignes identiques : 154 | vrais doubles scans : 67 | répétitions légitimes : 87
montants vides : reconstitués 16445.46 € | vrais 16203.78 €
```

La vérité éclaire notre choix. Sur les 154 lignes identiques, **67** seulement sont de vrais doubles scans ; **87** sont des répétitions légitimes. Supprimer toutes les lignes identiques aurait donc retiré 87 ventes réelles. En ne supprimant rien, nous avons conservé 67 lignes en trop, pour 3 257 € : c'est le gros de l'écart de 3 499 € de la Boutique. Le reste, 242 €, vient de la reconstitution des montants vides : nous avons supposé une remise nulle avant octobre, et le montant reconstitué (16 445,46 €) dépasse un peu le vrai (16 203,78 €), parce que des remises existaient. **Aucune de nos deux hypothèses n'était parfaite, et l'erreur totale reste à 0,62 %** : c'est ce que l'on écrit dans la note de méthode, et c'est suffisant pour la question de la gérante (la marge par catégorie), mais ce ne serait pas acceptable pour un rapprochement comptable au centime (chapitre 3).

2.2.8 Chercher ce qui manque : les anti-jointures

Une jointure ne dit pas seulement ce qui se **retrouve**, mais aussi ce qui **ne se retrouve pas**. L'**anti-jointure** (les lignes d'une table qui n'ont **aucune** correspondance dans l'autre) est l'outil de recherche des manques. On l'obtient avec `indicator=True` puis un filtre sur `left_only`. Quatre questions, quatre anti-jointures.

Les références du site existent-elles toutes dans la base ? On joint les références de l'export à celles de la base (qui s'écrivent WEB- suivi du numéro sur six chiffres).

```
refs_base = "WEB-" + cmd_b["id_commande"].astype(str).str.zfill(6)
anti = site[["order_ref"]].drop_duplicates().merge(refs_base.rename("order_ref"), on="order_ref",
↳ how="left", indicator=True)
print("références de l'export absentes de la base :", int((anti["_merge"] == "left_only").sum()), "|
↳ exemples :", anti.loc[anti["_merge"] == "left_only", "order_ref"].head(3).tolist())
sens_inverse = cmd_b[(cmd_b["canal"] == "Site") & (cmd_b["date_commande"].dt.year == 2025)]
```

```
print("commandes Site 2025 de la base absentes de l'export :", int((~("WEB-" +
↳ sens_inverse["id_commande"].astype(str).str.zfill(6)).isin(site["order_ref"])).sum()))
```

références de l'export absentes de la base : 60 | exemples : ['WEB-T0011', 'WEB-T0015', 'WEB-T0014']
commandes Site 2025 de la base absentes de l'export : 0

Les 60 références absentes de la base sont des **commandes de test** (leur numéro commence par WEB-T) : l'anti-jointure les aurait retrouvées même sans l'adresse électronique. Dans l'autre sens, **aucune** commande de la base n'est absente de l'export : rien n'a été perdu à l'extraction.

Quelle part de l'activité n'est dans aucun des deux fichiers ? La gérante a demandé « la caisse et le site ». Or la boutique vend aussi par un troisième canal, les réseaux : la base le montre.

```
ca_canaux = base.groupby("canal")["montant"].sum()
couvert = ca_canaux[["Boutique", "Site"]].sum()
print("part du chiffre d'affaires 2025 couverte par la caisse et le site :", round(couvert /
↳ ca_canaux.sum() * 100, 1), "%")
print("chiffre d'affaires du canal Réseaux, absent des deux sources :", round(ca_canaux["Réseaux"],
↳ 0), "€")
```

part du chiffre d'affaires 2025 couverte par la caisse et le site : 89.0 %
chiffre d'affaires du canal Réseaux, absent des deux sources : 146074.0 €

La table construite couvre **89 %** du chiffre d'affaires de 2025 : le canal Réseaux, soit 146 074 € (11 %), n'a **aucune source** dans les fichiers que l'on nous a remis. C'est une information à **rendre à la gérante** avant qu'elle ne présente la table comme « toutes les ventes ».

Les adresses électroniques du site retrouvent-elles les clients du CRM ? Joindre les commandes du site aux clients du CRM se fait par l'adresse électronique. Essayons d'abord sur l'écriture **brute**, puis sur une écriture **normalisée** (espaces retirés, minuscules).

```
crm = pd.read_csv("donnees/crm_clients.csv", dtype=str, keep_default_na=False)
valides = site[site["email_norm"] != "test@example.com"].drop_duplicates("order_ref")
brut_ok = valides["customer_email"].isin(set(crm["email"]))
norm_ok = valides["email_norm"].isin(set(crm["email"].str.strip().str.lower()))
print("e-mails retrouvés dans le CRM : brut", round(brut_ok.mean() * 100, 1), "% | normalisé",
↳ round(norm_ok.mean() * 100, 1), "%")
```

e-mails retrouvés dans le CRM : brut 87.7 % | normalisé 100.0 %

Sur l'écriture brute, **750 commandes** sur 6 078 ne retrouvent pas leur client (87,7 % de succès) : des majuscules, des espaces superflus. Après une normalisation de deux lignes, tout se retrouve (100 %). Cette différence illustre le principe de toute jointure sur du texte : **normaliser avant de joindre**, puis mesurer le taux de correspondance.

À retenir. Une **anti-jointure** n'est pas un accessoire : c'est le moyen de répondre à « *qu'est-ce qui manque ?* », celui que la gérante n'a pas pensé à poser (le canal Réseaux), et à « *qu'est-ce qui ne devrait pas être là ?* » (les commandes de test).

2.2.9 Le même résultat avec trois outils

Une table de ventes de cette importance se recoupe avec **un autre outil**, pas avec le même code relancé. On écrit la table dans un fichier, puis on calcule le même résumé (lignes et chiffre d'affaires par canal et par mois) avec **DuckDB** (SQL) et avec **R**, et l'on compare à pandas.

```
ventes["mois"] = ventes["date"].dt.strftime("%Y-%m")
ventes.to_csv(os.path.join(TMP2, "ventes.csv"), index=False)
```

```

os.environ["TMP2"] = TMP2
q = f"select canal, strftime(date, '%Y-%m') as mois, count(*) as lignes, round(sum(montant), 2) as ca
↳ from '{TMP2}/ventes.csv' group by 1, 2 order by 1, 2"
sql = duckdb.sql(q).df()
pdm = ventes.groupby(["canal", "mois"]).agg(lignes=("montant", "size"), ca=("montant",
↳ "sum")).round(2).reset_index()
print("DuckDB = pandas :", bool((sql["lignes"].values == pdm["lignes"].values).all() and
↳ np.allclose(sql["ca"].values, pdm["ca"].values)))

```

```
DuckDB = pandas : True
```

Et le même calcul en **R**, avec **dplyr** (la jointure ou le regroupement s'écrivent à peu près comme en pandas : volume I, section 4.2) ; R écrit son résultat dans un fichier, que Python relit pour le comparer.

```

library(dplyr, warn.conflicts = FALSE)
v <- read.csv(file.path(Sys.getenv("TMP2"), "ventes.csv"))
r <- v |> group_by(canal, mois) |> summarise(lignes = n(), ca = round(sum(montant), 2), .groups =
↳ "drop")
write.csv(r, file.path(Sys.getenv("TMP2"), "ventes_r.csv"), row.names = FALSE)
print(as.data.frame(r |> group_by(canal) |> summarise(lignes = sum(lignes), ca = round(sum(ca), 0))))

```

```

   canal lignes   ca
1 Boutique 12678 564473
2   Site 13510 600164

```

```

r = pd.read_csv(os.path.join(TMP2, "ventes_r.csv"))
print("R = pandas :", bool((r["lignes"].values == pdm["lignes"].values).all() and
↳ np.allclose(r["ca"].values, pdm["ca"].values)))

```

```
R = pandas : True
```

Trois outils, un même résultat à l'euro près. Si DuckDB ou R avaient donné un écart, la première hypothèse n'aurait pas été « l'outil se trompe », mais « *je n'ai pas demandé la même chose aux deux* » : un filtre de dates différent, une jointure qui duplique, un arrondi. C'est cette recherche de la **différence de question** qui fait la valeur de la vérification croisée.

À retenir.

- **Joindre** ajoute des colonnes, **empiler** ajoute des lignes ; le **type** de jointure est un choix d'analyse, et `indicator=True` en est le diagnostic.
- À chaque jointure, **quatre contrôles** : lignes avant et après, somme avant et après, lignes sans correspondance, unicité de la clé. `validate=` transforme l'hypothèse de cardinalité en **vérification**.
- Une jointure **n-n** multiplie les lignes **sans erreur** : un nom n'est jamais une clé ; on ajoute un critère et l'on **compte** les ambiguïtés restantes.
- Une source qui **dérive** (codage, séparateur, décimale, unité) se lit en **déTECTANT** son format, pas en le devinant ; la **ligne de total** d'un fichier est une somme de contrôle gratuite.
- On **ne supprime rien sans preuve** : on garde, on marque, on documente l'incertitude résiduelle (ici moins de 1 %).
- Un écart n'est acceptable que **expliqué** ; une **anti-jointure** cherche ce qui manque (un canal entier, des commandes de test).
- Deux outils valent mieux qu'un : DuckDB et R ont retrouvé les chiffres de pandas.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.3 à 2.6, exercices 2.5 à 2.8.

2.3 Agrégation et restructuration

La table des ventes compte plus de vingt-six mille lignes, et la gérante n'en lira aucune. Ce qu'elle veut, ce sont des chiffres **à un autre niveau de détail** : par mois, par canal, par catégorie, par client. **Agréger**, c'est passer d'un niveau fin à un niveau plus grossier en **résumant** : sommer, compter, moyenner. C'est le geste le plus fréquent de l'analyste, et celui qui cache le plus de pièges, parce que **chaque niveau de détail a sa propre définition du « nombre de... »**. Cette section apprend à raisonner sur le **grain** d'une table, à agréger sans compter deux fois, à calculer des parts, des rangs et des cumuls, à organiser les tables en **faits et dimensions**, et à vérifier par une **somme de contrôle** que rien n'a bougé.

2.3.1 Le grain d'une table et le changement de grain

Le **grain** d'une table est ce que représente **une ligne**. Dans notre table des ventes, une ligne est une **ligne de vente** (un article, dans une commande). Une commande compte plusieurs lignes ; un client passe plusieurs commandes. Trois niveaux, donc trois grains :

Grain	Une ligne représente...	Exemple de colonnes	Nombre de lignes en 2025 (sources du chapitre)
ligne	un article vendu	produit, quantité, montant	26 188 (caisse et site)
commande	un panier	date, canal, montant total, nombre d'articles	11 339
client	une personne	nombre de commandes, chiffre d'affaires, dernière date	3 875 clients acheteurs (base)

Passer d'un grain fin à un grain grossier est **l'agrégation** ; passer dans l'autre sens est impossible (on ne retrouve pas les lignes à partir des totaux). Il faut donc **garder la table la plus fine** et fabriquer les autres, jamais l'inverse. Et à chaque changement de grain, les mêmes mots ne désignent plus les mêmes choses : « nombre de commandes » est un **comptage de lignes distinctes** au grain ligne, et un simple **comptage de lignes** au grain commande.

Une remarque honnête : la table ventes (caisse et site) **ne contient pas le client**. La caisse n'enregistre pas qui achète, et le site fournit une adresse électronique que l'on peut relier au CRM, mais pas avec certitude (section 2.5). Pour le niveau « client », nous repartirons donc de la **base** du volume I, qui porte `id_client` sur chaque commande. C'est un exemple de la règle : **on ne peut agréger que selon ce que la table contient**.

2.3.2 groupby et agg : compter sans compter deux fois

L'outil est groupby suivi de agg, avec des **agrégats nommés** : `nom_de_la_colonne_résultat=("colonne_source", "fonction")`. Voici, par canal et par mois, le chiffre d'affaires, le nombre de lignes, le nombre de **commandes distinctes**, le panier moyen et le taux de marge.

```
par_mois = ventes.groupby(["canal", "mois"]).agg(ca=("montant", "sum"), lignes=("montant", "size"),
                                                commandes=("id_commande", "nunique"), marge=("marge_ht",
                                                ↪ "sum"))
par_mois["panier_moyen"] = par_mois["ca"] / par_mois["commandes"]
par_mois["taux_marge"] = par_mois["marge"] / (par_mois["ca"] / 1.20)
print(par_mois.loc["Site"].tail(3).round(2).to_string())
```

	ca	lignes	commandes	marge	panier_moyen	taux_marge
mois						
2025-10	54999.72	1214	514	17848.28	107.00	0.39
2025-11	62884.28	1552	693	19364.05	90.74	0.37
2025-12	87496.07	1995	882	28616.09	99.20	0.39

Trois définitions à ne pas confondre. Le **nombre de lignes** (size) est un comptage d'articles : 1 995 en décembre pour le site. Le **nombre de commandes** (nunique sur id_commande) est un comptage de paniers : 882. Le **panier moyen** est le **chiffre d'affaires divisé par le nombre de commandes** (87 496 € / 882 = 99,20 €), et **non** le montant moyen d'une ligne, qui serait environ deux fois plus petit. Le taux de marge de décembre (39,2 %) est, comme en 2.1.2, un rapport de sommes.

Il y a pourtant un piège plus subtil que le choix de la fonction : certains comptages **ne s'additionnent pas**. Une commande appartient à **un seul** mois, donc le nombre de commandes distinctes d'une année est bien la somme des nombres mensuels. Un client, lui, peut acheter plusieurs mois. Comparons, avec la base, la somme des clients distincts mois par mois et le nombre de clients distincts de l'année.

```
v25["mois"] = v25["date_commande"].dt.month
mensuel = v25.groupby("mois")["id_client"].nunique()
print("somme des clients distincts de chaque mois :", int(mensuel.sum()))
print("clients distincts de l'année :", v25["id_client"].nunique())
print("somme des commandes distinctes par mois :",
      ↪ int(v25.groupby("mois")["id_commande"].nunique().sum()), "| commandes distinctes de l'année :",
      ↪ v25["id_commande"].nunique())
```

```
somme des clients distincts de chaque mois : 10621
clients distincts de l'année : 3875
somme des commandes distinctes par mois : 12946 | commandes distinctes de l'année : 12946
```

La somme des clients mensuels (10 621) est **2,7 fois** le nombre réel de clients (3 875) : les clients qui reviennent sont comptés autant de fois qu'ils achètent de mois différents. **Un comptage distinct n'est additif que si les groupes ne se recouvrent pas**. Règle pratique : un total « tous mois » d'un nombre de clients se recalcule **sur les données**, il ne se déduit **jamais** d'une colonne de totaux mensuels.

Piège : additionner des moyennes, des taux, des comptages distincts. Les sommes s'additionnent ; les **moyennes**, les **ratios** et les **comptages distincts** non. Pour recomposer un niveau supérieur à partir d'un niveau inférieur, on garde les **composantes** (somme et effectif, numérateur et dénominateur) et l'on refait le ratio.

2.3.3 transform : un agrégat sans perdre les lignes

agg **réduit** : une ligne par groupe. transform calcule le même agrégat, mais le **recopie sur chaque ligne** du groupe : le nombre de lignes ne change pas. C'est l'outil de tout ce qui compare une ligne à son groupe : une **part du total**, un **écart à la moyenne**, un **rang**.

```
n_avant = len(ventes)
ventes["ca_mois"] = ventes.groupby("mois")["montant"].transform("sum")
ventes["part_du_mois"] = ventes["montant"] / ventes["ca_mois"]
ventes["rang_dans_categorie"] = ventes.groupby("categorie")["montant"].rank(method="dense",
↳ ascending=False)
t = ventes.groupby(["mois", "canal"])["part_du_mois"].sum().unstack()
print(t.tail(3).round(3).to_string())
print("lignes avant / après transform :", n_avant, len(ventes))
```

canal	Boutique	Site
mois		
2025-10	0.483	0.517
2025-11	0.492	0.508
2025-12	0.457	0.543

lignes avant / après transform : 26188 26188

Chaque ligne porte désormais la part qu'elle représente dans son mois ; en les resommant par canal, on retrouve la part de chacun : en décembre, la Boutique pèse 45,7 % et le Site 54,3 % du chiffre d'affaires des deux sources (la somme des parts fait 1, ce qui est **un contrôle**). La dernière ligne affichée montre, avant et après, que transform **conserve toutes les lignes**, contrairement à agg.

Le **rang** (rank) réclame une décision de présentation : method="dense" donne 1, 2, 2, 3 en cas d'égalité (pas de « trou »), alors que method="min" donnerait 1, 2, 2, 4. On choisit selon l'usage et l'on l'écrit.

2.3.4 Du grain ligne au grain commande, puis au grain client

Fabriquer la table des commandes, puis celle des clients, se fait par deux agrégations successives. Chaque ligne de la table des clients **résume** ce que le client a fait : combien de commandes, quel chiffre d'affaires, quand pour la dernière fois (la **récence**), dans quel canal il achète le plus souvent.

```
fin = pd.Timestamp("2025-12-31")
clients_2025 = v25.groupby("id_client").agg(nb_commandes=("id_commande", "nunique"), ca=("montant",
↳ "sum"),
                                             premiere=("date_commande", "min"),
↳ derniere=("date_commande", "max"))
clients_2025["recence_jours"] = (fin - clients_2025["derniere"]).dt.days
clients_2025["canal_principal"] = v25.groupby("id_client")["canal"].agg(lambda s: s.mode().iloc[0])
print(clients_2025.describe().loc[["mean", "50%", "max"], ["nb_commandes", "ca",
↳ "recence_jours"]].round(1).to_string())
```

	nb_commandes	ca	recence_jours
mean	3.3	341.9	91.5
50%	2.0	233.2	53.0
max	25.0	3382.3	364.0

La moyenne du nombre de commandes (3,3) est supérieure à la médiane (2) : quelques gros clients (jusqu'à 25 commandes) tirent la moyenne, et 32 % des clients n'ont acheté **qu'une fois** en 2025. Le client médian a dépensé 233 € et sa dernière commande date de 53 jours : tout ce qu'une **segmentation** de clientèle (volume III de cette série) saura exploiter.

Chaque changement de grain se **contrôle**. Les trois tables doivent porter le **même chiffre**

d'affaires et le même nombre de commandes.

```
v25_cmd = v25.groupby("id_commande", as_index=False).agg(ca=("montant", "sum"),
↳ id_client=("id_client", "first"))
print("CA des lignes :", round(v25["montant"].sum(), 2), "| des commandes :",
↳ round(v25_cmd["ca"].sum(), 2), "| des clients :", round(clients_2025["ca"].sum(), 2))
print("commandes :", v25["id_commande"].nunique(), "=", len(v25_cmd), "=",
↳ int(clients_2025["nb_commandes"].sum()))
```

```
CA des lignes : 1324763.72 | des commandes : 1324763.72 | des clients : 1324763.72
commandes : 12946 = 12946 = 12946
```

Un dernier point : la table des clients ne contient que les **acheteurs de 2025**. Si l'on veut parler **de tous** les clients (par exemple « quelle part est inactive ? »), il faut partir de la table complète des clients et faire une jointure **à gauche** : les clients sans achat auront des vides, qu'il faudra **remplacer par zéro pour les compteurs** (nombre de commandes, chiffre d'affaires), mais **pas** pour la récence (un client qui n'a jamais acheté n'a pas une récence de zéro jour). Cela rejoint le piège de 2.1.7 : le sens d'un vide dépend de la colonne.

2.3.5 Table de faits et dimensions

Une organisation des tables, très répandue en analyse, évite de tout recopier partout : le **schéma en étoile**. Au centre, une **table de faits** : ce qui se produit et se mesure (une ligne de vente, avec ses montants et ses quantités). Autour, des **dimensions** : les « axes » selon lesquels on regarde les faits (le produit, la date, le canal, le client). Chaque fait porte, pour chaque dimension, une **clé** qui renvoie à une ligne de la dimension.

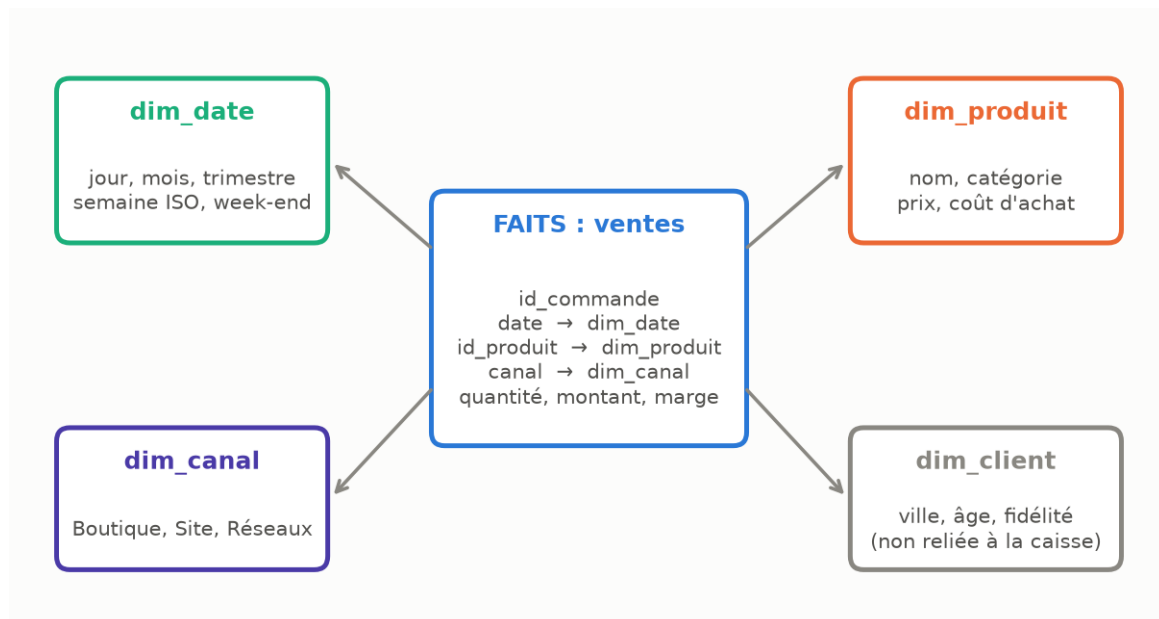


Figure 11: Schéma en étoile : une table de faits (les lignes de vente) entourée de dimensions (date, produit, canal, client), reliées par des clés.

Pourquoi cette organisation ? Parce qu'elle **sépare ce qui change souvent** (les ventes, qui s'ajoutent chaque jour) de **ce qui change rarement** (le catalogue, le calendrier), et parce qu'elle règle la question du grain : la table de faits a **un** grain, les dimensions ont **le leur**. Une jointure entre faits et dimension est par construction **n-1** : la clé de la dimension est unique, donc le nombre de lignes de faits ne change pas, et l'on peut le **vérifier**. Voici la dimension de date de 2025, un calendrier avec une ligne par jour, joint à la table de faits.

```

dim_date = pd.DataFrame({"date": pd.date_range("2025-01-01", "2025-12-31")})
dim_date["trimestre"] = dim_date["date"].dt.quarter
dim_date["semaine_iso"] = dim_date["date"].dt.isocalendar().week
dim_date["week_end"] = dim_date["date"].dt.dayofweek >= 5
f = ventes.merge(dim_date, on="date", how="left", validate="m:1")
print("lignes avant / après :", len(ventes), len(f), "| dates sans correspondance :",
      ↪ int(f["trimestre"].isna().sum()))
print(f.groupby("trimestre")["montant"].sum().round(0).to_dict())

```

```

lignes avant / après : 26188 26188 | dates sans correspondance : 0
{1: 225285.0, 2: 273107.0, 3: 274942.0, 4: 391302.0}

```

La jointure conserve les 26 188 lignes, sans date orpheline : le trimestre est ajouté à chaque ligne en un seul geste (le quatrième trimestre pèse 391 302 € contre 225 285 € pour le premier). Le **calendrier** offre autre chose : il liste **tous** les jours, y compris ceux où rien ne s'est vendu. Ici la Boutique a des ventes sur 365 jours sur 365, et le Site sur 365 : aucune journée creuse. Mais dans une série où un jour manquerait, c'est **la dimension de date qui le ferait voir**, alors qu'un groupby sur la table de faits ne produirait simplement **pas de ligne** pour ce jour (et un graphique le raccorderait sans rien dire).

Intuition. Une table de faits répond à « *combien ?* », une dimension à « *selon quoi ?* ». Quand une question commence par « par... » (par mois, par catégorie, par canal), elle désigne une dimension.

2.3.6 Cumuls, moyennes mobiles et comparaisons dans le temps

Une série quotidienne est bruitée (les samedis sont forts, les dimanches faibles) ; deux outils la lissent ou la **cumulent**. Le **cumul** (cumsum) donne le chiffre d'affaires depuis le début de l'année, utile pour comparer à un objectif. La **moyenne mobile** (rolling) remplace chaque jour par la moyenne des sept derniers jours : une fenêtre de **sept** jours contient exactement un exemplaire de chaque jour de la semaine, ce qui efface l'effet de la semaine.

```

jour = ventes.groupby(["canal", "date"])["montant"].sum().unstack("canal").fillna(0)
jour["total"] = jour.sum(axis=1)
jour["cumul"] = jour["total"].cumsum()
jour["moy7"] = jour["total"].rolling(7, min_periods=7).mean()
print(jour[["total", "cumul", "moy7"]].iloc[[0, 6, 7, -1]].round(0).to_string())

```

canal	total	cumul	moy7
date			
2025-01-01	2086.0	2086.0	NaN
2025-01-07	1969.0	17511.0	2502.0
2025-01-08	2552.0	20063.0	2568.0
2025-12-31	2838.0	1164637.0	4778.0

Le chiffre d'affaires cumulé atteint 1 164 637 € au 31 décembre (c'est exactement la somme de la table : un cumul **se termine par le total**, autre contrôle). La moyenne mobile ne commence qu'au septième jour : les six premières valeurs sont **vides** (6 jours), parce que `min_periods=7` refuse de calculer sur une fenêtre incomplète ; on préfère un vide à une valeur trompeuse. Elle vaut 3 208 € par jour à la mi-juin et 4 860 € à la mi-décembre. Le même cumul s'écrit en SQL par une **fonction fenêtre** (`sum(...)` over (order by date), volume I, section 3.3.5) ; calculé avec DuckDB sur le fichier écrit en 2.2.9, il donne la même série.

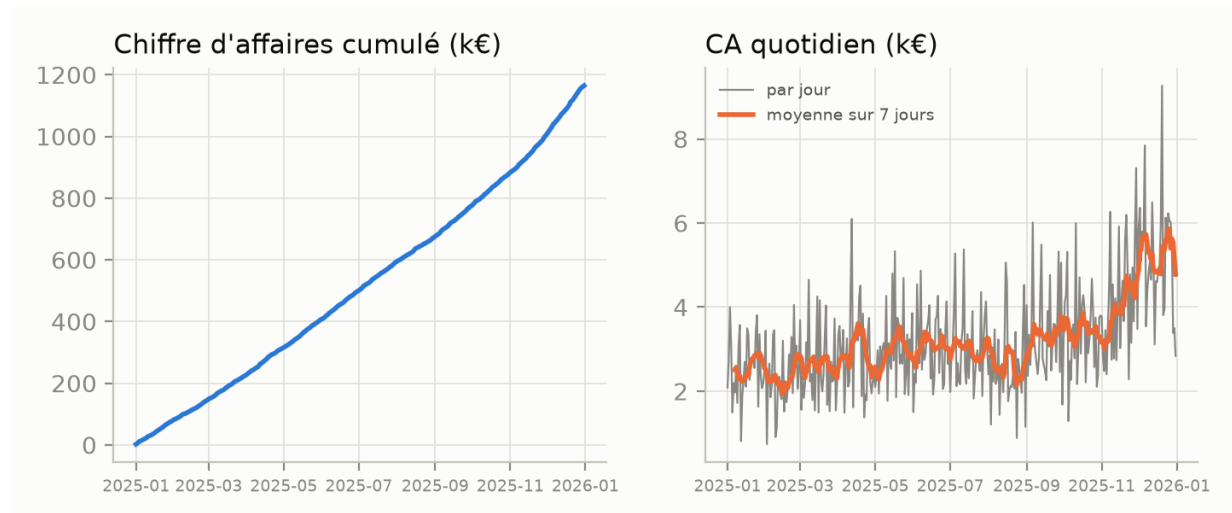


Figure 12: À gauche, chiffre d'affaires cumulé de 2025 (caisse et site) ; à droite, chiffre d'affaires quotidien et sa moyenne mobile sur sept jours, qui efface l'effet du jour de la semaine.

2.3.7 La somme de contrôle : rien n'a bougé

Agréger, c'est résumer, donc **perdre** du détail ; mais il ne faut perdre **ni argent ni lignes**. Un jeu de **contrôles d'invariants** vérifie que les quantités qui doivent se conserver se conservent : le chiffre d'affaires est le même à tous les niveaux (ligne, mois, canal, commande), le nombre de commandes aussi, et les cumuls finissent sur le total. On en fait une petite fonction, que l'on **relance à chaque modification** de la chaîne.

```
def controles(table):
    total = table["montant"].sum()
    return {"CA par canal = CA total": np.isclose(table.groupby("canal")["montant"].sum().sum(),
    ↪ total),
            "CA par mois = CA total": np.isclose(table.groupby("mois")["montant"].sum().sum(), total),
            "CA par commande = CA total": np.isclose(table.groupby(["canal",
    ↪ "id_commande"])[ "montant"].sum().sum(), total),
            "aucun montant vide": table["montant"].notna().all(),
            "aucune commande sans date": table["date"].notna().all()}
print(pd.Series(controles(ventes)).to_string())
```

```
CA par canal = CA total      True
CA par mois = CA total      True
CA par commande = CA total  True
aucun montant vide         True
aucune commande sans date  True
```

Tous les contrôles passent. Leur valeur apparaît quand quelque chose **casse**. Refaisons, volontairement, la faute de 2.2.5 : joindre les ventes aux produits **sur le nom seul**, puis relancer les contrôles sur la table obtenue.

```
fautive = ventes.merge(prod[["nom_produit", "cout_achat"]], on="nom_produit", suffixes=("", "_cat"))
print("lignes :", len(ventes), "->", len(fautive), "| CA :", round(ventes["montant"].sum()), "->",
    ↪ round(fautive["montant"].sum()))
print("contrôle « CA par canal = CA total » sur la table fautive : ", bool(controles(fautive)["CA par
    ↪ canal = CA total"]))
```

```
lignes : 26188 -> 52376 | CA : 1164637 -> 2329273
contrôle « CA par canal = CA total » sur la table fautive : True
```

Les contrôles de **cohérence interne** (ligne, mois, canal) passent même sur la table fautive : ils

comparent la table **à elle-même**, et une jointure qui duplique des lignes duplique aussi ses totaux. Ce que la faute change, c'est la **comparaison à l'extérieur** : le chiffre d'affaires est multiplié par 2,0 (2 329 273 € au lieu de 1 164 637 €), ce que révélerait la comparaison avec la base ou avec le total des fichiers de la caisse. **Il faut donc les deux** : des contrôles internes (les invariants entre niveaux de détail) et des contrôles **externes** (un total qui vient d'ailleurs). C'est ce que le chapitre 3 systématise.

À retenir.

- Le **grain** dit ce que représente une ligne ; on **garde la table la plus fine** et l'on en dérive les autres, jamais l'inverse.
- Un **ratio d'agrégat** se refait à partir des **sommes** ; un **comptage distinct** n'est additif que si les groupes ne se recouvrent pas (clients par mois).
- **agg** réduit, **transform** conserve les lignes : parts, écarts à la moyenne, rangs.
- Une jointure entre faits et dimension est **n-1** par construction : le nombre de lignes de faits ne doit pas changer ; un calendrier fait voir les **jours manquants**.
- Un cumul se termine par le **total** ; une moyenne mobile a des vides au début (`min_periods`).
- On contrôle par **invariants internes** (mêmes totaux à chaque niveau) **et** par une comparaison **externe** : les premiers ne voient pas une jointure qui duplique tout.

Pour s'entraîner. Cahier, chapitre 2 : application 2.7, exercices 2.9 à 2.10.

2.4 ⊕ Pour aller plus loin : restructuration, pivot et dépivot

Section complémentaire. Elle traite d'un geste que l'on rencontre dès qu'un tableur « fait main » entre dans l'analyse : passer d'une table **large** (un mois par colonne) à une table **longue** (un mois par ligne) et inversement. La gérante tient le stock de la boutique dans un tableur de ce genre ; c'est notre terrain d'essai. Rien de ce qui suit n'est nécessaire à la suite du volume.

Une même information peut s'écrire de deux façons. Le **format large** place chaque valeur d'une variable dans une colonne différente (une colonne par mois) : c'est celui que l'on aime lire, parce que l'œil compare les colonnes, et celui que produisent les tableurs. Le **format long** a **une ligne par observation** et une colonne par variable (une colonne mois, une colonne stock) : c'est celui que veulent les outils d'analyse, de tracé et de jointure (volume I, section 5.1.4 sur le « tableau propre »). Passer de l'un à l'autre est un **changement de grain**, avec le même contenu.

2.4.1 Trois fonctions à connaître

- **melt** (en R : `pivot_longer`) **dépivote** : il garde quelques colonnes d'identifiant et transforme **toutes les autres colonnes** en deux colonnes, variable et valeur. Il ne **calcule** rien.
- **pivot** (en R : `pivot_wider`) fait l'inverse, mais **ne sait pas agréger** : si deux lignes ont la même combinaison d'identifiants, il s'arrête.
- **pivot_table** pivote **et** agrège (`aggfunc`) : c'est l'équivalent d'un tableau croisé dynamique de tableur, avec des totaux optionnels (`margins=True`).

Reprenons la table des ventes. Un `pivot_table` donne le chiffre d'affaires par canal (en lignes) et par mois (en colonnes), avec les totaux.

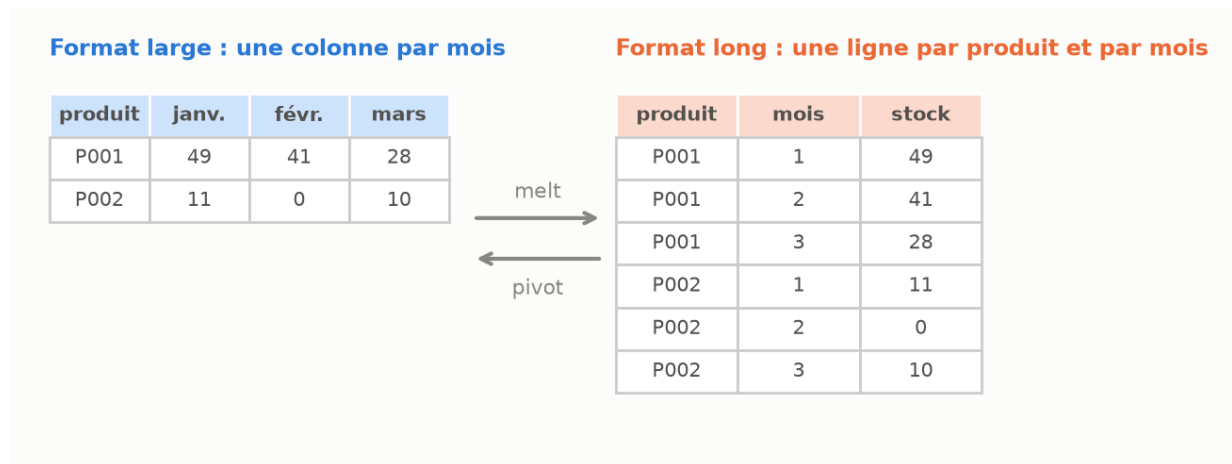


Figure 13: Le même contenu en format large (une colonne par mois) et en format long (une ligne par produit et par mois) ; melt passe du premier au second, pivot fait l'inverse.

```
large = ventes.pivot_table(index="canal", columns="mois", values="montant", aggfunc="sum",
    ↪ margins=True, margins_name="Total").round(0)
print(large.iloc[:, [0, 1, -2, -1]].to_string())
```

mois	2025-01	2025-02	2025-12	Total
canal				
Boutique	39234.0	33079.0	73688.0	564473.0
Site	40221.0	32746.0	87496.0	600164.0
Total	79455.0	65825.0	161184.0	1164637.0

On le **redresse** en format long avec melt (après avoir retiré la ligne et la colonne de totaux, qui sont des **résumés**, pas des observations), puis on retourne au format large avec pivot : si tout va bien, on retombe sur le tableau de départ.

```
corps = large.drop(columns="Total").drop(index="Total")
long = corps.reset_index().melt(id_vars="canal", var_name="mois", value_name="ca")
retour = long.pivot(index="canal", columns="mois", values="ca")
print(len(long), "lignes en format long | aller-retour identique :", bool(retour.equals(corps)))
print(long.head(3).to_string(index=False))
```

```
24 lignes en format long | aller-retour identique : True
canal  mois  ca
Boutique 2025-01 39234.0
        Site 2025-01 40221.0
Boutique 2025-02 33079.0
```

Deux canaux et douze mois donnent 24 lignes en format long : le **nombre de cellules** est conservé, c'est le contrôle de base. Le total du tableau large (1 164 637 €) est celui de la table des ventes, et le passage au format long n'a rien changé. Voyons maintenant ce qui se passe quand pivot rencontre deux lignes pour la même case.

```
try:
    ventes.pivot(index="canal", columns="mois", values="montant")
except ValueError as e:
    print(e)
```

```
Index contains duplicate entries, cannot reshape
```

Le message est clair : la table des ventes compte des milliers de lignes par couple (canal, mois) ; pivot ne sait pas **choisir** laquelle garder, donc il refuse. C'est une **protection** : pivot_table

accepte, mais à **condition de dire comment agréger** (`aggfunc="sum"`), et l'on est obligé d'y penser. Dans un tableur, un tableau croisé dynamique fait la même chose sans prévenir : il somme par défaut, ce qui est faux si la colonne contient des prix unitaires ou des moyennes.

Intuition. `melt` et `pivot` **déplacent** de l'information sans la changer (comme tourner un tableau d'un quart de tour) ; `pivot_table` **résume**. Un aller-retour qui ne retombe pas sur ses pieds trahit une clé en double ou une perte de lignes.

2.4.2 Le tableur de stocks : lire, nettoyer, dépivoter

Le fichier `stocks_tableur.xlsx` ressemble à ce que l'on trouve dans toutes les entreprises : saisi à la main, lisible pour un humain, **hostile à une machine**. Ouvrons-le sans rien présumer, en lisant les cellules telles quelles avec `openpyxl` (la lecture d'un tableur avec `pandas` est vue au volume I, chapitre 4).

```
import openpyxl
ws = openpyxl.load_workbook("donnees/stocks_tableur.xlsx")["Stock 2025"]
brut = pd.DataFrame(list(ws.iter_rows(min_row=5, values_only=True))).iloc[:, :14]
brut.columns = ["ref", "designation"] + list(range(1, 13))
print(brut.iloc[[0, 1, 2, 21, 22]].iloc[:, :6].to_string(index=False))
```

ref	designation	1	2	3	4
CUISINE	NaN	None	None	None	None
P001	Casserole nordique	49	41	28	26
P002	Poêle mat	11	rupture	10	rupture
NaN	Sous-total Cuisine	=SUM(C6:C25)	=SUM(D6:D25)	=SUM(E6:E25)	=SUM(F6:F25)
NaN	NaN	None	None	None	None

On y voit, en quelques lignes, les obstacles : une ligne **de catégorie** (CUISINE, cellule fusionnée, seule la première cellule porte la valeur), des lignes **de produits**, une ligne de **sous-total** (dont les cellules contiennent des **formules** ; le fichier n'ayant jamais été recalculé, `openpyxl` donne le texte de la formule et non son résultat), des lignes **vides**, et des **valeurs en texte**. Les lignes de produits se repèrent par un critère **sûr** : la référence suit le motif P suivi de trois chiffres. Les autres lignes (catégories, sous-totaux, vides) ne sont pas des observations et sont **écartées** (un sous-total lu comme une donnée ferait compter deux fois le stock). Regardons maintenant ce que contiennent les cellules textuelles.

```
est_produit = brut["ref"].astype(str).str.fullmatch(r"P\d{3}")
cellules = brut.loc[est_produit].iloc[:, 2:].stack()
textes = cellules[cellules.map(lambda v: isinstance(v, str))].str.strip().str.replace(r"^\d+$",
↳ "<nombre>", regex=True)
print(textes.value_counts().to_string())
```

```
<nombre>    131
rupture      120
ND           49
—            22
```

Sur 1 440 cellules de stock, quatre catégories de **texte** : **131** nombres écrits comme du texte avec un espace en fin ("28 "); **120** mentions « rupture » ; **49** « ND » (non disponible) ; **22** tirets « — ». Il faut une **décision par catégorie**, et elle est de métier :

- un nombre écrit en texte **est** un nombre : on retire l'espace et l'on convertit ;
- « rupture » signifie **zéro** en stock : c'est une information (on peut compter les ruptures), pas un manquant ;
- « ND » et « — » signifient **inconnu** : on met un **vide** (NaN), jamais zéro. Le zéro dirait « en rupture », ce qui est faux et ferait croire à des ruptures qui n'existent pas.

```
def vers_nombre(v):
    if isinstance(v, (int, float)):
        return v
    s = str(v).strip()
    if s == "rupture":
        return 0
    return int(s) if s.isdigit() else np.nan
produits_stock = brut[est_produit].copy()
mois_cols = list(range(1, 13))
produits_stock[mois_cols] = produits_stock[mois_cols].apply(lambda col: col.map(vers_nombre))
stock = produits_stock.melt(id_vars=["ref", "designation"], var_name="mois", value_name="stock")
stock["id_produit"] = stock["ref"].str[1:].astype(int)
print(len(stock), "lignes | vides :", int(stock["stock"].isna().sum()), "| ruptures :",
      ↵ int((stock["stock"] == 0).sum()))
```

```
1440 lignes | vides : 71 | ruptures : 120
```

Cent vingt produits et douze mois font **1 440 lignes** en format long, ce que l'on attendait : le nombre de cellules est conservé, aucune n'a été perdue. Comme pour la caisse, on peut maintenant **juger le travail** avec le fichier de vérité, ce qui n'est possible qu'ici.

```
verite_stock = pd.read_csv("donnees/verite_stocks.csv")
m = stock.merge(verite_stock, on=["id_produit", "mois"], suffixes=("", "_vrai"), validate="1:1")
egal = (m["stock"] == m["stock_vrai"]) | (m["stock"].isna() & m["stock_vrai"].isna())
print("cellules identiques à la vérité :", int(egal.sum()), "sur", len(m))
```

```
cellules identiques à la vérité : 1440 sur 1440
```

Toutes les 1 440 cellules coïncident avec la vérité, valeurs manquantes comprises. Ce n'est pas un exploit : les règles de lecture étaient simples et **chaque catégorie de cellule avait été examinée avant de convertir**. Le contrôle de fond reste le même que pour la caisse : **compter** (1 440 cellules avant et après), **classer** ce qui est inhabituel, **décider** par catégorie et **documenter**.

Piège : read_excel qui « devine ». Avec `pandas.read_excel`, la colonne qui contient des nombres et des mentions comme « rupture » serait lue en texte, ou convertie avec des surprises. Lire les cellules **brutes**, puis décider, est plus long mais ne laisse rien au hasard.

2.4.3 Joindre deux tables du même grain

Les stocks sont mensuels, par produit. Pour savoir **combien de mois de ventes** couvre un stock, il faut les rapprocher des ventes, qui sont au grain de la ligne de vente. Si l'on joignait telles quelles, chaque ligne de stock serait associée à **toutes** les lignes de vente du produit et du mois : une jointure 1-n qui multiplierait le stock. La bonne méthode est de **ramener d'abord les ventes au grain du stock** (produit × mois), puis de joindre deux tables dont la clé est unique des deux côtés.

```
q = ventes.dropna(subset=["id_produit"]).assign(mois=lambda d: d["date"].dt.month, id_produit=lambda
↳ d: d["id_produit"].astype(int))
unites = q.groupby(["id_produit", "mois"],
↳ as_index=False)["quantite"].sum().rename(columns={"quantite": "vendu"})
couv = stock.dropna(subset=["stock"]).merge(unites, on=["id_produit", "mois"], how="left",
↳ validate="1:1")
couv["vendu"] = couv["vendu"].fillna(0)
couv["stock_inferieur_aux_ventes"] = couv["stock"] < couv["vendu"]
print(len(couv), "lignes produit-mois | stock < ventes du mois :",
↳ int(couv["stock_inferieur_aux_ventes"].sum()))
```

1369 lignes produit-mois | stock < ventes du mois : 392

Sur 1 369 couples (produit, mois) dont le stock est connu, **29 %** (392) ont un stock de fin de mois **inférieur** aux ventes du mois : c'est la couverture inférieure à un mois, un signal de réassort à surveiller. Remarquez aussi que, parmi les 120 couples en rupture, 100 % ont **des ventes dans le mois** : une rupture de **fin** de mois n'interdit pas d'avoir vendu avant, et il ne faut pas confondre les deux.

Ce que ce calcul ne prouve pas. Dans nos données, les stocks et les ventes ont été **simulés indépendamment l'un de l'autre** : la couverture calculée ici montre la **mécanique** de la jointure (même grain, clé unique, vides conservés), pas une réalité de la boutique. Dans de vraies données, on s'attendrait à ce qu'un produit en rupture vende moins le mois suivant : on le **vérifierait**, on ne le supposerait pas.

Deux précautions de méthode s'imposent : les ventes de la caisse dont le produit est ambigu (la paire de produits indiscernables de 2.2.5) ont été **écartées** de ce calcul, donc les ventes de ces deux produits sont **sous-estimées** ; et les stocks « inconnus » sont **exclus** plutôt que traités comme zéro.

2.4.4 Le même geste dans les autres outils

Dans **DuckDB**, **UNPIVOT** fait ce que fait **melt** ; on vérifie qu'il donne le même nombre de lignes.

```
con2 = duckdb.connect(); con2.register("stock_large", produits_stock.rename(columns=str))
n_sql = con2.sql("select count(*) from (unpivot stock_large on columns(* exclude (ref, designation))
↳ into name mois value stock)").fetchone()[0]
print("lignes après UNPIVOT en SQL :", n_sql)
```

lignes après UNPIVOT en SQL : 1369

Dans **Power Query** (Excel), la même opération s'appelle « dépivoter les autres colonnes » ; on la lit dans le langage M de l'éditeur avancé. Ce qui suit n'est **pas exécuté** ici (Power Query n'est pas disponible sur la machine qui a produit ce livre) ; le résultat attendu est celui que pandas vient de donner : 1 440 lignes. Les noms des étapes et des fonctions sont à vérifier dans la documentation de votre version.

```
// Power Query (langage M), non exécuté
let
    Source = Excel.Workbook(File.Contents("stocks_tableur.xlsx"), null, true),
    Feuille = Source[[Item = "Stock 2025", Kind = "Sheet"]][Data],
    Entetes = Table.PromoteHeaders(Table.Skip(Feuille, 3)),
    Produits = Table.SelectRows(Entetes, each Text.StartsWith([#"Réf."], "P")),
    Long = Table.UnpivotOtherColumns(Produits, {"Réf.", "Désignation"}, "Mois", "Stock")
in
    Long
```

En R, ce serait `pivot_longer(cols = -c(ref, designation), names_to = "mois", values_to = "stock")`, après `filter(grepl("^P[0-9]{3}$", ref))`.

À retenir.

- **Large** pour lire, **long** pour calculer, tracer et joindre ; `melt` et `pivot` déplacent sans modifier, `pivot_table` **résume**.
- Un **aller-retour** large → long → large qui ne retombe pas sur ses pieds trahit un doublon ou une perte ; le **nombre de cellules** est conservé.
- Un tableur « fait main » se lit en **cellules brutes** : repérer les lignes utiles par un motif sûr, écarter titres et sous-totaux, **classer** les valeurs textuelles et décider **par catégorie** (nombre, zéro, inconnu).
- « **Inconnu** » **n'est pas zéro** : un NaN dit « on ne sait pas », un zéro dit « rupture ».
- Avant de joindre, on ramène les deux tables au **même grain** (produit × mois), clé unique des deux côtés.

Pour s'entraîner. Cahier, chapitre 2 : application 2.8, exercices 2.11 à 2.12.

2.5 ⊕ Pour aller plus loin : appariement approximatif et rapprochement d'enregistrements

Section complémentaire. Elle traite d'un problème qui fait suite à tout ce qui précède : **relier deux enregistrements qui désignent la même chose sans être écrits de la même façon**. La gérante a un fichier clients (le CRM) où **les mêmes personnes apparaissent plusieurs fois**, et un catalogue de fournisseur qui nomme ses produits à sa manière. Rien de ce qui suit n'est nécessaire à la suite du volume ; le chapitre 3 reprend la **réconciliation** de sources avec ses seuils et ses rapports d'exceptions.

Jusqu'ici, nous avons joint des tables par des clés **exactes** : un numéro de produit, une référence de commande. Mais beaucoup de données n'ont **pas de clé commune**. Le CRM a enregistré la même cliente trois fois, une fois en majuscules, une fois sans accent, une fois avec une faute de frappe ; le fournisseur écrit « CASSEROLE NORDIQUE » là où la boutique écrit « Casserole nordique ». On parle de **rapprochement d'enregistrements** (*record linkage*) ou de **dédoublonnage** (*deduplication*) quand les deux fichiers sont le même. La méthode combine quatre idées : **normaliser**, **mesurer une ressemblance**, **limiter les comparaisons** (le *blocage*) et **décider** avec un seuil.

2.5.1 Pourquoi les clés exactes échouent

Le CRM contient 7 000 lignes une fois retirées les 140 lignes de test, pour 6 000 clients distincts : près de **mille lignes sont des doublons**. Pour **juger** les méthodes de cette section, nous avons besoin de savoir quelles lignes désignent la même personne. Dans la vie réelle, on étiquette à la main un **échantillon** et l'on évalue sur lui ; ici, le fichier de vérité nous donne **toutes** les paires, ce qui permet de mesurer exactement. Une **paire vraie** est un couple de lignes du CRM qui désignent le même client.

```
import itertools
crm = pd.read_csv("donnees/crm_clients.csv", dtype=str, keep_default_na=False)
n_total = len(crm)
crm = crm[crm["email"] != "test@example.com"].copy()
crm["id_crm"] = crm["id_crm"].astype(int)
verite_crm = pd.read_csv("donnees/verite_crm.csv")
groupes = crm.merge(verite_crm[["id_crm", "id_client"]],
    ↪ on="id_crm").groupby("id_client")["id_crm"].apply(sorted)
vraies = {p for g in groupes for p in itertools.combinations(g, 2)}
print(len(crm), "lignes |", len(vraies), "paires de lignes qui désignent le même client")
```

7000 lignes | 1050 paires de lignes qui désignent le même client

Il y a donc 1 050 paires vraies à retrouver. Les deux fonctions suivantes sont l'outil de mesure de toute la section : `paires` construit toutes les paires de lignes qui partagent une valeur de clé, et `juger` compare cet ensemble aux paires vraies. Deux mesures s'y lisent : la **précision** (parmi les paires proposées, quelle part est vraie ?) et le **rappel** (parmi les paires vraies, quelle part a été trouvée ?).

```
def paires(df, cles):
    sortie = set()
    for _, g in df.dropna(subset=cles).groupby(cles)["id_crm"]:
        ids = sorted(g)
        if 1 < len(ids) < 100:
            sortie.update(itertools.combinations(ids, 2))
    return sortie
def juger(trouvees):
    ok = len(trouvees & vraies)
    return {"paires": len(trouvees), "précision": round(ok / max(1, len(trouvees)), 3), "rappel":
    ↪ round(ok / len(vraies), 3)}
```

Essayons trois clés **exactes**, sur le texte tel quel : l'adresse électronique, le téléphone, le couple prénom et nom.

```
crm["mail_brut"] = crm["email"].replace("", np.nan)
crm["nom_brut"] = crm["prenom"] + "|" + crm["nom"]
for cle in ["mail_brut", "telephone", "nom_brut"]:
    print(f"{cle:10s}", juger(paires(crm, [cle])))
```

```
mail_brut  {'paires': 425, 'précision': 0.993, 'rappel': 0.402}
telephone  {'paires': 211, 'précision': 1.0, 'rappel': 0.201}
nom_brut   {'paires': 192, 'précision': 0.979, 'rappel': 0.179}
```

Les clés exactes sont **très précises** (presque toutes les paires proposées sont vraies : 99,3 % pour l'e-mail) mais leur **rappel est mauvais** : 40 % des paires vraies pour l'e-mail, 20 % pour le téléphone, 18 % pour le nom. L'égalité stricte rate tout ce qui est écrit **un peu** différemment. C'est la première leçon : une clé exacte donne peu de **faux positifs**, beaucoup de **faux négatifs**.

2.5.2 Normaliser d'abord

La première amélioration est la moins glorieuse et la plus rentable : **ramener chaque champ à une forme canonique** avant de comparer. Pour le texte, nous avons déjà `cle_texte` (2.1.6). Il faut y ajouter quelques particularités du fichier : le **mojibake** (un texte UTF-8 lu avec le mauvais codage, SorbertÃ© au lieu de Sorberté), que la bibliothèque `ftfy` répare ; les **villes** écrites de six façons dont l'une **en arabe** (المدينة أ signifie « la ville A ») ; le **téléphone**, que l'on réduit aux neuf derniers chiffres (les préfixes et séparateurs varient) ; l'**année de naissance**, extraite d'une date écrite en quatre formats (et ignorée si elle est impossible).

```
import ftfy
LETTRES = "أبتجخدذرزسشصضطظعغف"
def ville_cle(s):
    s = s.strip()
    if s.startswith("المدينة"):
        return "ville " + chr(97 + LETTRES.index(s.split()[1]))
    return re.sub(r"^\D", "", ville, cle_texte(s))
crm["mail_norm"] = crm["email"].str.strip().str.lower().replace("", np.nan)
crm["tel_norm"] = crm["telephone"].str.replace(r"\D", "", regex=True).str[-9:]
crm["nom_complet"] = (crm["prenom"] + " " + crm["nom"]).map(lambda s: cle_texte(ftfy.fix_text(s)))
crm["nom_tri"] = crm["nom_complet"].str.split().map(lambda t: " ".join(sorted(t)))
crm["ville_norm"] = crm["ville"].map(ville_cle)
crm["annee_naiss"] = pd.to_numeric(crm["date_naissance"].str.extract(r"(\d{4})")[0]).where(lambda a:
    a.between(1920, 2010))
print("villes distinctes :", crm["ville"].nunique(), "->", crm["ville_norm"].nunique(), "| années de
    naissance utilisables :", int(crm["annee_naiss"].notna().sum()))
```

```
villes distinctes : 119 -> 20 | années de naissance utilisables : 6960
```

Les 119 écritures de villes se ramènent à 20 villes, celles du fichier ; 99,4 % des lignes ont une année de naissance utilisable. Voyons l'effet sur les clés exactes.

```
for cle in ["mail_norm", "tel_norm"]:
    print(f"{cle:10s}", juger(paires(crm, [cle])))
```

```
mail_norm  {'paires': 698, 'précision': 0.994, 'rappel': 0.661}
tel_norm   {'paires': 1050, 'précision': 1.0, 'rappel': 1.0}
```

Le gain est spectaculaire pour le téléphone : une fois réduit à neuf chiffres, il retrouve 100 % des paires vraies avec une précision de 100 %, alors que l'e-mail normalisé n'en retrouve que 66 % (les lignes dont l'e-mail est absent, ou mal écrit, lui échappent). Voilà un cas où la **normalisation suffit** et où l'appariement approximatif serait superflu : c'est même la première chose à essayer. Notre téléphone a été fabriqué intact (il n'a subi que des changements de **forme**) ; **dans la vie réelle, un numéro change, manque ou est partagé par un foyer**, et l'on ne s'y fie pas à lui seul.

Pour apprendre la méthode générale, plaçons-nous donc dans un cas **fréquent** : le téléphone n'est **pas disponible** (le fichier que l'on rapproche n'en contient pas, ou la minimisation des données l'interdit : chapitre 5). Il ne reste que le nom, l'e-mail, la ville et l'année de naissance, et il faut une **mesure de ressemblance**. Le numéro de téléphone nous servira de **second avis indépendant** pour contrôler le résultat (2.5.5).

2.5.3 Mesurer une ressemblance

Deux textes **se ressemblent** s'il faut peu de modifications pour passer de l'un à l'autre. La mesure de base est la **distance de Levenshtein** : le nombre minimal d'opérations élémentaires (insérer, supprimer ou remplacer **une lettre**) pour transformer un mot en l'autre. On la calcule par une petite table : la case (i, j) contient la distance entre les i premières lettres du premier mot et les j premières du second, avec la règle

$$d(i, j) = \min(d(i-1, j) + 1, d(i, j-1) + 1, d(i-1, j-1) + c), \quad c = \begin{cases} 0 & \text{si les lettres sont égales} \\ 1 & \text{sinon.} \end{cases}$$

Prenons deux écritures d'un nom inventé, tavel et tavle (deux lettres permutées).

	ø	t	a	v	l	e
ø	0	1	2	3	4	5
t	1	0	1	2	3	4
a	2	1	0	1	2	3
v	3	2	1	0	1	2
e	4	3	2	1	1	1
l	5	4	3	2	1	2

La dernière case donne la distance : 2. Permuter deux lettres coûte **deux** opérations (deux remplacements), alors qu'un humain y voit **une** seule faute de frappe : certaines variantes de la distance (Damerau-Levenshtein) comptent la permutation pour un. On transforme la distance en **similarité** entre 0 et 100 en la rapportant à la longueur : similarité = $100 \times (1 - \text{distance} / \text{longueur maximale})$.

Trois autres mesures complètent la boîte à outils, car aucune ne convient à tout :

- la similarité de **Jaro-Winkler** : pense aux **fautes de frappe dans un nom** ; elle récompense les **lettres communes à peu près au même endroit** et donne un **bonus aux débuts identiques** (une faute au milieu d'un nom coûte moins qu'une au début) ;
- le **token_set_ratio** de rapidfuzz : compare des **ensembles de mots, sans tenir compte de l'ordre** ni des mots en plus : adapté aux noms inversés et aux désignations qui ajoutent un mot ;
- la comparaison d'**initiales** : « M. Dormar » désigne probablement « Mirelo Dormar ».

```
from rapidfuzz import fuzz
import jellyfish
cas = [("mirelo dormar", "mirelo dormra"), ("mirelo dormar", "dormar mirelo"), ("mirelo dormar", "m
↳ dormar"), ("mirelo dormar", "talina kelmar")]
tab = pd.DataFrame([(x, y, round(fuzz.ratio(x, y)), round(jellyfish.jaro_winkler_similarity(x, y) *
↳ 100), round(fuzz.token_set_ratio(x, y))) for x, y in cas],
                    columns=["texte 1", "texte 2", "ratio", "jaro-winkler", "token_set"])
print(tab.to_string(index=False))
```

texte 1	texte 2	ratio	jaro-winkler	token_set
mirelo dormar	mirelo dormra	92	98	92
mirelo dormar	dormar mirelo	46	62	100
mirelo dormar	m dormar	76	77	86
mirelo dormar	talina kelmar	46	55	38

Le tableau montre pourquoi on choisit la mesure **selon le défaut que l'on attend** : une faute de frappe garde une similarité élevée avec toutes les mesures (92 pour le simple ratio) ; l'**inversion** du nom et du prénom est invisible pour le ratio (46) mais **parfaite** pour token_set_ratio (100) ; l'initiale donne 86, un peu moins ; deux personnes différentes sont à 38. Pour nos données, où les défauts sont de ces quatre sortes, nous prendrons token_set_ratio sur les mots **triés** du nom.

2.5.4 Limiter les comparaisons : le blocage

Comparer toutes les lignes du CRM deux à deux est impossible à grande échelle : pour 7000 lignes, il y a **24 496 500 paires**. Même à un millier de comparaisons par seconde, c'est une journée entière. Le **blocage** consiste à ne comparer que des paires **plausibles** : celles qui partagent une valeur **grossière** et **fiable** (une clé de blocage). Ici, la **ville normalisée** et l'**année de naissance** : deux lignes qui désignent la même personne ont presque toujours ces deux valeurs identiques.

```
bloc = paires(crm, ["ville_norm", "annee_naiss"])
n_paires = len(crm) * (len(crm) - 1) // 2
print("paires à comparer :", len(bloc), "sur", n_paires, "| part gardée :", round(len(bloc) / n_paires
↳ * 100, 2), "%")
print("blocage :", juger(bloc))
```

```
paires à comparer : 42538 sur 24496500 | part gardée : 0.17 %
blocage : {'paires': 42538, 'précision': 0.024, 'rappel': 0.984}
```

Le blocage réduit le nombre de comparaisons de **576 fois** : de 24 496 500 à 42 538 paires (0,17 % du total). Il y a un prix : le **rappel** du blocage, 98,4 %, est une **limite supérieure** de tout ce qui suivra ; une paire vraie qui ne partage ni ville ni année de naissance n'est jamais comparée (par exemple si l'année est absente ou impossible). Le choix de la clé est un arbitrage entre vitesse et rappel ; on le **mesure**, on ne le devine pas. Dans la pratique, on fait souvent **plusieurs passes** avec des clés de blocage différentes (ville et année ; puis e-mail ; puis premières lettres du nom) et l'on réunit les paires trouvées.

2.5.5 Un score, trois zones

Pour chaque paire du blocage, on calcule un **score** qui combine les indices disponibles : la ressemblance des noms (pondérée 60 %) et celle de la partie locale de l'e-mail, avant le @ (40 %), quand les deux e-mails sont présents. Si l'un des deux manque, on ne peut s'appuyer que sur le nom, et l'on **pénalise** légèrement le score (90 % de la similarité du nom) : moins d'indices, moins de certitude. Ces poids sont des **choix**, que l'on réglera sur les résultats.

```
rec = crm.set_index("id_crm")[["nom_tri", "mail_norm"]].to_dict("index")
def local(m): return m.split("@")[0] if isinstance(m, str) else None
def score_paire(a, b):
    nom = fuzz.token_set_ratio(rec[a]["nom_tri"], rec[b]["nom_tri"])
    lx, ly = local(rec[a]["mail_norm"], local(rec[b]["mail_norm"]))
    if lx is None or ly is None:
        return nom, np.nan, 0.9 * nom
    mail = fuzz.ratio(lx, ly)
    return nom, mail, 0.6 * nom + 0.4 * mail
S = pd.DataFrame([(a, b, *score_paire(a, b)) for a, b in sorted(bloc)], columns=["a", "b", "nom",
↳ "mail", "score"])
S["vrai"] = [(a, b) in vraies for a, b in zip(S["a"], S["b"])]
print(S.groupby("vrai")["score"].describe()[["count", "mean", "min", "50%",
↳ "max"]].round(1).to_string())
```

	count	mean	min	50%	max
vrai					
False	41505.0	36.5	6.9	35.7	92.9
True	1033.0	95.3	60.0	97.9	100.0

Les deux populations sont **bien séparées** : les paires vraies ont un score médian de 98 et une moyenne de 95 ; les fausses paires (deux personnes différentes de la même ville et de la même année de naissance) ont une moyenne de 36. Elles se **chevauchent** pourtant dans une zone intermédiaire : la pire fausse paire atteint 93, la moins bonne paire vraie 60. C'est ce chevauchement qui rend inévitable une décision à **trois zones** plutôt qu'à deux : **accepter** automatiquement au-

dessus d'un seuil haut, **rejeter** en dessous d'un seuil bas, et envoyer la **zone grise** à une **revue manuelle**.

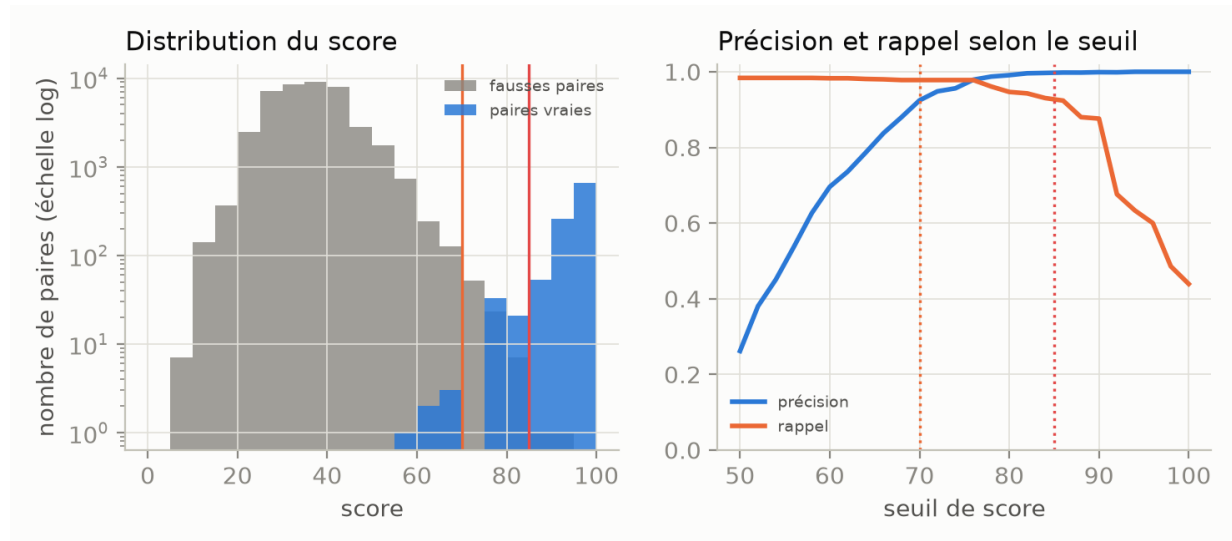


Figure 14: À gauche, distribution du score des paires comparées après blocage (échelle logarithmique) : les fausses paires sont des dizaines de milliers, avec des scores faibles ; les paires vraies forment un petit groupe à score élevé. À droite, précision et rappel en fonction du seuil. Les traits marquent les seuils 70 et 85.

Choisissons deux seuils : **85** pour l'acceptation automatique et **70** pour la limite basse de la revue manuelle. On compte ce que produit chaque zone, et l'on juge par rapport aux paires vraies.

```
auto = S[S["score"] >= 85]
grise = S[(S["score"] >= 70) & (S["score"] < 85)]
print("acceptées :", len(auto), "| précision :", round(auto["vrai"].mean(), 3), "| rappel global :",
      round(auto["vrai"].sum() / len(vraies), 3))
print("à revoir :", len(grise), "| part de vraies paires :", round(grise["vrai"].mean(), 3))
print("rappel si la revue manuelle tranche juste :", round((auto["vrai"].sum() + grise["vrai"].sum())
      / len(vraies), 3))
```

```
acceptées : 975 | précision : 0.998 | rappel global : 0.927
à revoir : 136 | part de vraies paires : 0.397
rappel si la revue manuelle tranche juste : 0.978
```

La zone **automatique** ne propose que 975 paires, dont **99,8 %** sont vraies (seulement 2 fausses) : on peut fusionner sans relire. Elle retrouve **93 %** des paires vraies. La **zone grise** compte 136 paires dont 54 sont vraies et 82 fausses : à la limite du jugement humain, et c'est justement pour cela qu'on les **soumet à un humain** plutôt qu'à un seuil. Si la revue manuelle tranche juste, le rappel monte à **98 %** : le reste est hors de portée du blocage.

Dans la vie réelle, on ne dispose pas des paires vraies. Mais on peut **auditer** avec un **second indice indépendant**, ici le téléphone que nous avons gardé de côté : parmi les paires acceptées, combien partagent aussi le même numéro ?

```
tel = crm.set_index("id_crm")["tel_norm"]
S["meme_tel"] = [tel[a] == tel[b] for a, b in zip(S["a"], S["b"])]
print(pd.crosstab(S["score"] >= 80, S["meme_tel"], rownames=["score ≥ 80"], colnames=["même
      téléphone"]).to_string())
```

```
même téléphone  False  True
score ≥ 80
False           41496    39
```

True 9 994

Parmi les paires au score d'au moins 80, **994 sur 1003** partagent le même numéro (99,1 %), alors que seulement 39 paires de **plus bas** score le partagent aussi. Les deux avis **concordent** presque toujours, ce qui renforce la confiance sans jamais la prouver : c'est ce que l'on appelle une **validation croisée par un indice indépendant**.

Intuition. Un score d'appariement n'est **pas une probabilité** : un 85 ne veut pas dire « 85 % de chances d'être le même client ». C'est un **classement** des paires, de la plus probable à la moins probable. Le **seuil** transforme ce classement en décision, et c'est **la décision, pas le score, qu'il faut mesurer** (précision, rappel, charge de revue).

2.5.6 Le coût des erreurs décide du seuil

Un seuil n'est ni bon ni mauvais : il dépend de **ce que coûte chaque erreur**. Deux erreurs sont possibles : fusionner deux personnes **différentes** (un **faux positif** : on mélange deux historiques d'achats, on écrit à quelqu'un sous le nom d'un autre) et **rater** un doublon (un **faux négatif** : la même personne est comptée deux fois, reçoit deux courriers). Leur gravité n'est pas la même, et elle dépend de l'usage. Calculons le **coût total** pour trois hypothèses de coût, en balayant le seuil (le calcul, une simple somme de faux positifs et de faux négatifs pondérés, est refait dans l'exercice 2.14 du cahier).

coûts égaux	seuil optimal : 75 (coût 55)
fusion erronée 10 fois plus grave	seuil optimal : 85 (coût 97)
doublon raté 10 fois plus grave	seuil optimal : 75 (coût 262)

Avec des coûts égaux, le seuil optimal est **75** ; quand une **fusion erronée est dix fois plus grave** qu'un doublon raté (par exemple parce que la fusion supprime l'historique), il monte à **85** : on accepte moins, on laisse plus de doublons ; quand c'est un **doublon raté qui est dix fois plus grave** (envoi en double d'un courrier coûteux), il est de **75** (sur une grille de seuils de cinq en cinq : un balayage plus fin déplacerait un peu ces optimums). La conclusion pratique : **on ne choisit pas un seuil « en soi »**, on le choisit **avec la personne qui supporte les conséquences**, et l'on écrit pourquoi.

Dernière règle, la plus importante : **ne jamais écraser les données d'origine**. On ne fusionne pas en supprimant des lignes : on construit une **table de correspondance** (`id_crm` → `id_client_unique`) avec le score et la décision, que l'on peut relire, corriger et **annuler**. Une fusion erronée sans trace est irréparable ; une fusion tracée se défait en une ligne.

2.5.7 Rapprocher le catalogue du fournisseur

Le même raisonnement s'applique au catalogue du fournisseur : relier chacune de ses lignes au produit correspondant de la boutique. Ici l'information la plus fiable n'est pas dans le texte : c'est le **prix d'achat** (à quelques pour cent près). Pour chaque ligne du catalogue, on cherche, **dans la même famille de produits**, le produit dont le nom ressemble le plus et dont le coût d'achat est proche, en combinant les deux indices : on retranche à la similarité de nom une **pénalité proportionnelle à l'écart de prix**. Le code (une boucle sur les lignes du catalogue, une similarité de nom par candidat, un écart de prix par candidat) est rangé dans `build/outils_ch02.py` et refait pas à pas dans le cahier (application 2.10) ; seul son résultat nous intéresse ici.

lignes du catalogue : 118 | acceptées : 108 | rejetées : 10

On accepte un appariement si la similarité de nom est d'**au moins 70** et si l'écart de prix ne dépasse

pas 3,5 % (le catalogue a été fabriqué avec ± 3 % d'écart sur le coût d'achat : le seuil a été choisi avec cette information, comme on choisit une tolérance avec le fournisseur). Jugeons le résultat avec la vérité.

```
vp = pd.read_csv("donnees/verite_produits.csv").rename(columns={"id_produit": "id_vrai"})
J = R.merge(vp, on="code_fournisseur", validate="1:1")
vrais_produits = J[J["id_vrai"] != -1]
print("produits du catalogue qui existent à la boutique :", len(vrais_produits), "| nouveautés :",
      ↪ int((J["id_vrai"] == -1).sum()))
print("nouveautés correctement rejetées :", int((~J.loc[J["id_vrai"] == -1, "accepte"]).sum()))
print("bons appariements avec le nom et le prix :", int((vrais_produits["id_produit"] ==
      ↪ vrais_produits["id_vrai"]).sum()), "| avec le nom seul :", int((vrais_produits["id_nom_seul"] ==
      ↪ vrais_produits["id_vrai"]).sum()))
```

```
produits du catalogue qui existent à la boutique : 108 | nouveautés : 10
nouveautés correctement rejetées : 10
bons appariements avec le nom et le prix : 107 | avec le nom seul : 58
```

Les 108 lignes acceptées sont exactement les 108 produits qui existent à la boutique ; les 10 **nouveautés** du fournisseur (qui n'ont aucun équivalent) sont toutes **rejetées** (10 sur 10) : aucun faux appariement. Le prix a fait la différence : avec le **nom seul**, seuls 58 appariements sur 108 (54 %) sont bons, parce que chaque nom est porté par deux produits ; avec le **nom et le prix**, 107 sur 108 (99 %). Le seul échec (1 ligne) est le produit 62 apparié à la place du 72 : **les deux produits sont indiscernables** (même nom, même coût d'achat), comme nous l'avons vu en 2.2.5. Aucune méthode, humaine ou automatique, ne pourrait trancher sans une information de plus (un numéro d'article, une date d'entrée au catalogue). **Savoir qu'on ne peut pas décider est un résultat.**

À retenir.

- Une clé **exacte** a une bonne précision et un mauvais rappel ; **normaliser** (casse, accents, espaces, mojibake, chiffres seuls) est le premier gain, souvent suffisant.
- Une **ressemblance** se mesure par une distance (Levenshtein), une similarité (Jaro-Winkler) ou une comparaison d'**ensembles de mots** (token_set_ratio) : on choisit selon le **défaut attendu** (faute, inversion, initiale).
- Le **blocage** réduit les comparaisons de plusieurs ordres de grandeur ; son rappel borne celui de tout le reste.
- On décide à **trois zones** : accepter, **revue manuelle**, rejeter ; le seuil se choisit selon le **coût des deux erreurs**, avec ceux qui en supportent les conséquences.
- Un score n'est pas une probabilité ; on mesure la **décision** (précision, rappel) et l'on **audite** avec un indice indépendant.
- On **ne supprime pas** : on garde une table de correspondance tracée. Et quand deux objets sont **indiscernables**, on l'écrit.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.9 et 2.10, exercices 2.13 et 2.14.

Bilan du chapitre 2

Vous savez maintenant :

- **créer des variables dérivées** en connaissant les décisions qu'elles cachent : une **marge** (hors taxe, après remise, rapport de sommes), des **morceaux de date** (en gardant l'année ISO avec la semaine ISO), des **classes** (cut, qcut, bornes écrites), des **indicateurs** et des **délais** entre lignes (trier, regrouper, décaler), des **clés de texte** normalisées ; et les **documenter** par une fiche ;
- repérer les trois pièges des variables dérivées : la **division par zéro**, le **NaN qui se propage** (sum ignore, + propage) et la **fuite d'information** (un calcul qui utilise le futur) ;
- **joindre** des tables en choisissant le **type** de jointure, en déclarant la **cardinalité** (validate=), en appliquant les **quatre contrôles** (lignes, somme, lignes sans correspondance, unicité de la clé) et en reconnaissant la jointure **n-n** qui multiplie les lignes (un nom n'est jamais une clé) ;
- **empiler** douze fichiers dont le **format dérive** (codage, séparateur, décimale, noms de colonnes, date), en **détectant** le format de chacun et en utilisant la **ligne de total** comme somme de contrôle ;
- lire un **export** comme un document : statuts et devises en plusieurs écritures, **changement d'unité** (centimes) repéré par l'ordre de grandeur et confirmé par une autre source, commandes de **test**, **doublons** et **annulations** retirés en comptant ;
- **harmoniser** deux sources en une **table de ventes** au schéma commun, **la contrôler** contre une référence indépendante et **expliquer chaque écart** ; **chercher ce qui manque** par des anti-jointures (un canal entier absent des sources) ;
- **agréger** en raisonnant sur le **grain** : agrégats nommés, transform pour les parts et les rangs, comptages distincts **non additifs**, cumuls et moyennes mobiles, schéma **en étoile** (faits et dimensions), et **contrôler par invariants** internes et par une comparaison externe ;
- (en option) **passer du format large au format long** (melt, pivot, pivot_table), lire un **tableur saisi à la main** et décider par catégorie de cellule (nombre, zéro, inconnu), joindre deux tables **du même grain** ;
- (en option) **rapprocher des enregistrements sans clé commune** : normaliser, mesurer une ressemblance, **bloquer**, noter, décider à **trois zones** selon le **coût des erreurs**, auditer par un indice indépendant, et ne jamais écraser les données d'origine.

Le chapitre a mis des chiffres sur des idées que l'on retient souvent comme des conseils :

Ce que l'on a vu	Ce que l'on a mesuré
Fichiers de la caisse lus sans reconstitution	13 077 € de moins que le total affiché en pied de fichier
Après reconstitution des montants vides	3 499 € de plus (0,62 %) : les lignes identiques
Jointure sur le nom seul	de 12 678 à 25 356 lignes, chiffre d'affaires doublé
Montants du site	médiane de 87 € en août, 8 426 € en octobre : changement d'unité
Canal absent des deux sources	11 % du chiffre d'affaires de l'année
Comptage de clients mois par mois, sommé	2,7 fois le nombre réel de clients
Clés exactes sur le CRM, e-mail brut	40 % des doublons retrouvés ; téléphone normalisé : 100 %
Blocage (ville, année de naissance)	576 fois moins de comparaisons pour 98 % des paires vraies conservées
Catalogue : nom seul contre nom et prix	54 % contre 99 % de bons appariements

Trois leçons dépassent ce chapitre. **D'abord, aucune erreur ne s'affiche** : une jointure qui double

les lignes, une décimale mal lue, une unité qui change, un comptage non additif donnent des chiffres plausibles ; seules la comparaison **avant et après**, la **somme de contrôle** et la **référence extérieure** les révèlent. **Ensuite, ne rien supprimer sans preuve** : on garde, on marque, on documente l'incertitude résiduelle, car deux lignes identiques peuvent être légitimes et deux noms identiques peuvent désigner deux choses. **Enfin, chaque correction est une décision de métier** (que faire d'un montant vide, d'un « ND », d'une fusion douteuse) : on l'écrit, on la mesure et on la fait valider par celui qui en supporte les conséquences.

En pratique : la liste de contrôle d'une préparation.

1. Compter les lignes **avant et après** chaque opération (jointure, empilement, filtre) et expliquer l'écart.
2. Comparer un **total** avec une source **indépendante** (ligne de total, base, autre outil).
3. Déclarer la **cardinalité** de chaque jointure (validate=) et vérifier l'**unicité** des clés.
4. **Détecter** le format de chaque source (codage, séparateur, décimale, unité) plutôt que le deviner.
5. Ne jamais traiter « inconnu » comme « zéro » ; marquer les valeurs reconstituées.
6. Garder la table **la plus fine** et en dériver les agrégats ; rappeler le **grain** de chaque table.
7. **Documenter** chaque variable créée et chaque décision (chapitre 4).

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 à 2.10 (variables dérivées, délais et fuite d'information, contrôle d'une jointure, lecture de la caisse, nettoyage du site, table des ventes, agrégation et contrôles, stocks, dédoublonnage du CRM, catalogue du fournisseur) et exercices 2.1 à 2.14.

Le chapitre 3 fait de ces contrôles une démarche : les **dimensions de la qualité** (exactitude, complétude, cohérence, actualité), les **contrôles de validation** que l'on écrit une fois pour toutes, et la **réconciliation** de sources, avec ses seuils de tolérance et ses rapports d'exceptions.

Chapitre 3 : Qualité des données et réconciliation

« Un chiffre faux et précis fait plus de dégâts qu'un chiffre vague et honnête. »

Un mardi de janvier, la gérante pose trois fichiers sur votre bureau : le **CRM** (la liste de ses clients), l'**export du site** et les **douze fichiers de la caisse** de la boutique. Elle voudrait un tableau de bord commun, et elle vous demande : « **Puis-je faire confiance à ces fichiers ? Et pourquoi leurs totaux ne tombent-ils jamais juste ?** » Elle ajoute, un peu gênée : « J'ai déjà envoyé deux fois la même lettre à certaines clientes, et le tableau du site affichait des chiffres d'affaires impossibles depuis la mi-septembre. »

Vous avez les moyens de répondre. Les chapitres 1 et 2 de ce volume vous ont appris à **corriger** (valeurs manquantes, doublons, formats) et à **transformer** (fusionner, restructurer). Ce chapitre apprend à faire ce qui vient avant et après chaque correction : **mesurer** la qualité, **contrôler** que les règles tiennent, et **réconcilier** deux sources qui prétendent décrire la même réalité. Sans cela, on nettoie à l'aveugle : on ne sait ni ce qui était cassé, ni si l'on a réparé, ni si l'on n'a rien cassé de plus.

Pourquoi un chapitre sur la qualité ?

Une erreur de données coûte rarement de l'argent le jour où elle se produit. Elle en coûte **plus tard**, quand quelqu'un prend une décision sur un chiffre faux : commander trop de stock, relancer deux fois la même cliente, croire qu'une campagne a doublé les ventes alors qu'une plateforme a changé d'unité. Trois idées guident le chapitre.

- **La qualité se mesure.** « Les données sont sales » n'est pas un diagnostic. « 3,2 % des e-mails sont absents, 1,4 % de ceux qui sont renseignés sont mal formés et 14,3 % des lignes sont des copies d'une autre ligne » en est un, que l'on peut suivre dans le temps et comparer à un seuil.
- **La qualité est relative à un usage.** Un fichier de clients peut être excellent pour calculer le chiffre d'affaires par ville et inutilisable pour envoyer des e-mails. On ne dit pas « propre » ou « sale » : on dit « assez bon pour *quoi* ».
- **Un chiffre ne vaut que par son recoupement.** Une source seule ne prouve rien ; deux sources qui s'accordent, ou dont on **explique** le désaccord à l'euro près, valent beaucoup. C'est la réconciliation.

Intuition. Le comptable et le pilote ont le même réflexe : ils **recoupent**. Le comptable rapproche le relevé de la banque et le journal des écritures ; le pilote vérifie l'altimètre contre le vario et la carte. Aucun des deux ne se fie à un instrument unique, et aucun n'accepte un

écart qu'il ne sait pas expliquer.

Le chemin de ce chapitre

Le parcours essentiel suit le chemin d'une analyste qui reçoit des fichiers inconnus.

- **3.1 Dimensions de la qualité** : exactitude, complétude, cohérence, actualité (et validité, unicité) ; pour chacune, une définition, un indicateur chiffré et un exemple sur les fichiers de la boutique ; un tableau de bord de qualité par source, avec des seuils.
- **3.2 Contrôles de validation** : des règles écrites comme de petites fonctions qui renvoient **les lignes en échec** ; contrôles de forme, de plage, de dépendance, d'unicité, de total, de temps ; un rapport de résultats ; le même contrôle en SQL ; et la décision à prendre quand un contrôle échoue.
- **3.3 Réconciliation de sources** : comparer deux sources en trois temps (compter, sommer, expliquer) ; une cascade qui explique **100 %** d'un écart ; trois cas réels (le site contre la base, la caisse contre la base, le catalogue du fournisseur contre les produits).

Deux sections facultatives prolongent ce parcours : ➤ **3.4 Règles de réconciliation, seuils de tolérance, rapports d'exceptions** (industrialiser le rapprochement) et ➤ **3.5 Cadres de validation : Great Expectations et pandera** (laisser une bibliothèque exécuter les contrôles).

Les données du chapitre

Cinq fichiers désordonnés, une base de référence. Les fichiers de la boutique ont été **fabriqués** à partir d'une base propre, avec des défauts connus ; la « vérité » est conservée à part, ce qui permet de **juger** un contrôle ou un rapprochement. Les fichiers `verite_*` ne sont jamais des entrées d'un traitement : nous ne les ouvrirons qu'à la fin d'une étude, pour savoir si nous avons raison.

- **crm_clients.csv** : le CRM, 7 140 lignes pour 6 000 clients (des clients en double, des lignes de test, des formats mêlés) ;
- **site_commandes.csv** et **site_lignes.csv** : l'export de la plateforme web pour le canal Site en 2025, 6 259 lignes d'en-tête ;
- **caisse/caisse_2025-01.csv ... caisse_2025-12.csv** : 12 fichiers de la caisse de la boutique, 12 678 lignes en tout, dont le format a changé deux fois dans l'année ;
- **catalogue_fournisseur.csv** : le catalogue d'un fournisseur, 118 lignes, avec ses propres codes et ses propres désignations ;
- **montants_saisis.csv** : 6 000 lignes de commande saisies à la main, avec quelques erreurs de saisie ;
- la **base de référence** (`commandes.csv`, `lignes_commande.csv`, `produits.csv`), propre : c'est avec elle que l'on réconcilie.

Tous ces fichiers sont **simulés**. Nous fixons au 5 janvier 2026 la date du rapport : c'est le « maintenant » de ce chapitre, utile pour mesurer la fraîcheur des données. Les lignes de code de ce chapitre sont réellement exécutées au moment de la fabrication du livre ; les chiffres cités viennent de ces exécutions.

En pratique : trois questions avant de toucher un fichier. (1) *D'où vient-il, et qui le produit ?* (2) *Que représente une ligne ?* (le grain, voir le volume I, section 5.1.4) (3) *À quelle date a-t-il été extrait, et couvre-t-il la période que je crois ?* Ces trois questions évitent la moitié des erreurs de ce chapitre.

Les applications guidées et les exercices de ce chapitre sont dans le cahier : vous y mesurerez la qualité du CRM, vous écrirez vos propres contrôles et vous réconcilierez les fichiers de la boutique pas à pas.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.9 et exercices 3.1 à 3.12 (chacun renvoie à la section du livre qu'il met en pratique).

3.1 Dimensions de la qualité : exactitude, complétude, cohérence, actualité

Dire qu'une donnée est « de bonne qualité » ne veut rien dire tant qu'on n'a pas précisé **sur quel plan**. Une adresse peut être renseignée mais fausse, correcte mais périmée, juste mais écrite de trois façons. Les praticiens de la qualité ont donc découpé la notion en **dimensions**, chacune avec sa question et son indicateur. Cette section présente les six que l'on rencontre le plus (les quatre du titre, plus la validité et l'unicité), les mesure sur les fichiers de la boutique, puis les assemble en un tableau de bord.

3.1.1 Une donnée est de qualité « pour un usage »

La définition la plus utile est celle des normes de qualité : une donnée est de qualité quand elle est **adaptée à l'usage que l'on veut en faire**. Elle n'est ni propre ni sale en soi. Prenons le CRM de la boutique et deux usages :

- **envoyer une lettre d'information** : il faut une adresse e-mail bien formée, un consentement, et une seule ligne par personne ;
- **répartir les clients par tranche d'âge** : il faut une date de naissance lisible et plausible ; l'e-mail n'a aucune importance.

Le même fichier peut être excellent pour le second usage et mauvais pour le premier. C'est pourquoi toute mesure de qualité commence par la question : *qualité pour quoi faire ?* Nous reviendrons sur ces deux usages avec des chiffres en 3.1.7.

Intuition. La qualité des données ressemble à la qualité d'un vêtement : un manteau parfait pour l'hiver est inutilisable à la plage. On juge l'**adéquation**, pas la perfection.

Voici les six dimensions, avec la question que chacune pose.

Dimension	Question	Exemple dans la boutique
Complétude	Les valeurs attendues sont-elles là ?	l'e-mail du client est-il renseigné ?
Validité	Les valeurs respectent-elles la forme et les valeurs permises ?	l'e-mail a-t-il la forme d'un e-mail ?
Unicité	Chaque réalité est-elle représentée une seule fois ?	la cliente apparaît-elle deux fois ?

Dimension	Question	Exemple dans la boutique
Cohérence	Les données ne se contredisent-elles pas, entre colonnes et entre sources ?	le total de la caisse égale-t-il la somme des lignes ?
Exactitude	Les valeurs sont-elles proches de la réalité ?	le montant de la commande est-il le bon ?
Actualité	Les données sont-elles assez récentes pour l'usage ?	l'export contient-il les ventes d'hier ?

3.1.2 La complétude : ce qui manque

La **complétude** est la part des valeurs attendues qui sont effectivement présentes. Elle se mesure à trois échelles : la **cellule** (quelle part des cellules d'une colonne est renseignée ?), la **ligne** (quelle part des lignes a tous ses champs obligatoires ?) et la **table** (a-t-on toutes les lignes attendues, par exemple tous les jours de l'année ?).

Un premier regard sur le CRM, colonne par colonne, se fait en une ligne :

```
completude = (crm.notna().mean() * 100).round(1)
print(completude.sort_values().head(4).to_string())
```

```
consentement_marketing    69.7
code_postal               94.1
email                    96.8
prenom                   100.0
```

Trois colonnes manquent de valeurs : le consentement est absent dans 30,3 % des lignes, le code postal dans 5,9 % et l'e-mail dans 3,2 %. Mais attention au **sens** de l'absence. Une cellule vide peut signifier « non demandé », « refusé », « oublié », ou « n'existe pas ». Pour un consentement, l'absence ne veut **pas** dire « oui » : elle veut dire « on ne sait pas », et la règle prudente est de ne pas écrire à la personne. À l'échelle de la ligne, seules 63,8 % des lignes (4 557) ont à la fois un e-mail, un code postal et un consentement.

Piège : l'absence déguisée. Une valeur « manquante » n'est pas toujours vide. Dans un autre fichier de la boutique (le tableur des stocks), l'absence s'écrit « ND », « — » ou « rupture » ; dans un formulaire, on trouve « 0000000000 » ou « inconnu ». Ces valeurs comptent comme **présentes** pour un indicateur naïf, alors qu'elles sont absentes pour l'usage. Avant de mesurer la complétude, il faut lister les valeurs « bidon » de chaque colonne (le chapitre 1 de ce volume en traite le nettoyage).

Les manquants ne sont pas tous équivalents pour l'analyse : tout dépend de **pourquoi** ils manquent (au hasard, selon une autre variable, ou à cause de la valeur elle-même) ; la section 1.1 de ce volume en donne la typologie. Ici, nous mesurons seulement leur **ampleur**.

Pour s'entraîner. Cahier, chapitre 3 : application 3.1, exercice 3.1.

3.1.3 La validité et l'unicité : la forme et les copies

La **validité** demande si une valeur respecte les règles de forme : un e-mail contient un @, un code postal a cinq chiffres, un consentement vaut « oui » ou « non ». Elle se vérifie sans rien connaître d'autre que la règle, souvent avec une **expression régulière** (un motif de texte). Voici le test d'un e-mail, écrit pour accepter toutes les formes ordinaires et refuser les @ doublés et les points consécutifs :

```
RE_EMAIL = r"^@\s.]+(\. [^@\s.]+)*@[^\s.]+(\. [^@\s.]+)+"
renseigne = crm["email"].notna()
mal_forme = renseigne & ~crm["email"].str.fullmatch(RE_EMAIL, na=False)
print("e-mails mal formés :", int(mal_forme.sum()), "sur", int(renseigne.sum()))
```

```
e-mails mal formés : 95 sur 6909
```

95 adresses, soit 1,4 % des e-mails renseignés, sont mal formées. De même, 199 codes postaux n'ont que quatre chiffres : le zéro initial a été perdu quand le code postal a été lu comme un nombre (on l'a vu au volume I, section 5.1.3 : un identifiant se lit en texte). Le champ « ville » prend 119 écritures différentes pour 20 villes, et le champ « consentement » 7. Ces écarts de forme ne sont pas des erreurs de fond, mais ils **empêchent de compter** : on ne peut pas regrouper « Ville A » et « VILLE A » sans les normaliser.

La **validité** a deux limites. D'abord elle ne dit rien du **sens** : 0000000000 est un numéro de téléphone parfaitement valide, et il appartient à une ligne de test. Ensuite une valeur peut respecter la forme et être fausse (une date de naissance 03/04/1980 est valide, qu'elle soit le 3 avril ou le 4 mars). L'exactitude, qui répond à cette seconde question, demande une référence (3.1.5).

L'**unicité** demande qu'une même réalité ne soit pas représentée plusieurs fois. Pour une clé comme le numéro de commande, le test est simple : aucun doublon. Pour une personne, il n'y a pas de clé fiable, et l'on recourt à un indice, par exemple l'e-mail normalisé :

```
cle = crm["email"].str.strip().str.lower()
doublon = cle.notna() & cle.ne("test@example.com") & cle.duplicated(keep="first")
print("lignes redondantes par e-mail :", int(doublon.sum()), "sur", len(crm))
```

```
lignes redondantes par e-mail : 677 sur 7140
```

Le test trouve 677 lignes redondantes, soit 9,7 % du fichier hors tests. Garde-fou : ce chiffre est une **mesure**, pas la vérité. Nous verrons en 3.1.8 combien de doublons il laisse passer.

Pour s'entraîner. Cahier, chapitre 3 : application 3.1, exercices 3.2 et 3.3.

3.1.4 La cohérence : ne pas se contredire

La **cohérence** demande que les données ne se contredisent pas. On la regarde à trois niveaux.

- **Entre colonnes d'une même ligne.** Un client ne peut pas s'être inscrit avant sa naissance, ni à six ans ; un code postal appartient à une ville ; la quantité multipliée par le prix donne (au plus) le montant.
- **Entre lignes d'une même table.** Deux lignes qui décrivent le même client ne doivent pas lui donner deux dates de naissance.
- **Entre sources.** Le total que la caisse imprime en bas de son fichier doit égaler la somme de ses lignes ; l'export du site doit retrouver, commande par commande, ce que la base enregistre. Cette cohérence-là est celle de la **réconciliation** (3.3).

Voici un contrôle entre colonnes : l'âge à l'inscription.

```
naiss = 0.parse_naissance(crm["date_naissance"])
insc = pd.to_datetime(crm["date_inscription"], format="%d/%m/%Y")
age_insc = (insc - naiss).dt.days / 365.25
print("inscrits avant 16 ans :", int((age_insc < 16).sum()), "| dates de naissance illisibles :",
      ↪ int(naiss.isna().sum()))
```

```
inscrits avant 16 ans : 277 | dates de naissance illisibles : 666
```

Deux observations. Premièrement, 666 dates de naissance (9,3 %) ne se lisent pas du tout : 68 sont impossibles (le 31 février, l'année 2030...), les autres viennent d'une saisie à l'américaine (mois/jour/année) dont le « mois » dépasse 12 quand on la lit à la française. Deuxièmement, et c'est plus inquiétant, ces mêmes saisies à l'américaine (1051 lignes) donnent dans 429 cas une date **parfaitement lisible mais fausse** (le 3 avril devenu le 4 mars) : le contrôle de cohérence ne les voit pas. Un contrôle n'attrape que ce qu'il sait voir. Le contrôle d'âge signale aussi 277 clients « inscrits avant 16 ans » : sans enquête, on ne sait pas si c'est la date de naissance ou la date d'inscription qui est fausse, et l'on **signale** la ligne au lieu de la corriger.

Piège : lire sans deviner. Quand le format d'une date est ambigu, ne choisissez pas silencieusement : comptez les lignes ambiguës, signalez-les, et demandez à la source quel format elle emploie. Convertir 1 051 dates « au pif » vous donne un fichier qui a l'air propre et qui ment.

3.1.5 L'exactitude : proche de la réalité

L'**exactitude** demande si la valeur est proche de la réalité qu'elle décrit. C'est la dimension la plus importante et la plus difficile à mesurer, parce qu'on ne sait pas, en général, ce que la réalité vaut : il faut une **référence**, c'est-à-dire une autre source jugée plus fiable (la base, un relevé, un inventaire physique, une vérité programmée dans le cas de nos données simulées).

Pour la boutique, la base de référence nous permet de mesurer l'exactitude de l'export du site : commande par commande, le total lu dans l'export est-il égal à celui de la base ?

```
t_site = site["total"].map(0.montant_site_en_nombre)
tot_base = lig.groupby("id_commande")["montant"].sum()
base_site = site["order_ref"].str.extract(r"WEB-(\d{6})")[0].astype(float).map(tot_base)
exact = (t_site - base_site).abs().dropna() <= 0.01
print("commandes dont le total lu est exact :", round(100 * exact.mean(), 1), "%")
```

```
commandes dont le total lu est exact : 59.8 %
```

Seules 59,8 % des commandes sont exactes. En séparant l'export en deux périodes, la réponse devient limpide : 100,0 % d'exactitude **avant** le 15 septembre, 0,0 % **après**. Ce n'est pas une panne progressive : c'est un événement ponctuel (la plateforme a changé l'unité de ses montants, qui sont devenus des centimes), que 3.2.5 et 3.3.3 sauront détecter et expliquer.

L'exactitude se mesure aussi sans référence externe, par des **bornes de plausibilité**. Pour les 6 000 lignes de commande saisies à la main, un montant doit être strictement positif et ne pas dépasser la quantité multipliée par le prix : 98,6 % des lignes respectent cette règle, et les 85 autres sont les erreurs de saisie injectées dans le fichier (décimale décalée, signe inversé, zéro, valeur 9999). Cette règle de **plausibilité** ne prouve pas l'exactitude d'une valeur, mais elle prouve l'**inexactitude** de celles qui l'enfreignent.

3.1.6 L'actualité : à quel point est-ce récent ?

L'**actualité** (on dit aussi la fraîcheur) mesure l'écart entre l'état du monde et l'état de la donnée. Elle se lit à deux endroits : la **date du dernier enregistrement** (est-elle récente ?) et la **cadence** d'actualisation (la source est-elle mise à jour assez souvent pour l'usage ?). Un tableau de bord quotidien qui se nourrit d'un export mensuel n'est jamais à jour, quelle que soit la qualité de chaque ligne.

```

dernier = {"CRM (dernière inscription)": insc.max(), "Site (dernière commande)":
    pd.to_datetime(site["created_at"].str.replace("Z", ""), format="ISO8601").max(),
    "Caisse (dernière vente)": caisse["date"].max()}
for nom, d in dernier.items():
    print(f"{nom:28s} {d:%d/%m/%Y}    retard : {(REF - d.normalize()).days} jours")

```

CRM (dernière inscription)	30/12/2025	retard : 6 jours
Site (dernière commande)	31/12/2025	retard : 5 jours
Caisse (dernière vente)	31/12/2025	retard : 5 jours

Le rapport est daté du 5 janvier 2026, et les trois sources se sont arrêtées entre le 30 et le 31 décembre : le retard est de cinq à six jours, ce qui est **normal** pour un bilan annuel et **trop long** pour un suivi hebdomadaire. Là encore, l'adéquation à l'usage décide.

L'actualité a un autre sens, plus insidieux : la **stabilité de la définition dans le temps**. Une donnée peut être à jour et changer de signification en cours de route, comme le montant du site passé en centimes le 15 septembre. Une série longue n'est cohérente que si chaque colonne signifie la même chose du début à la fin ; c'est ce que les contrôles temporels de 3.2.5 vérifient.

3.1.7 Un tableau de bord de qualité

Pour piloter, on regroupe ces mesures en un **tableau de bord** : une ligne par source, une colonne par dimension, une couleur par niveau. Chaque case est la moyenne de quelques **indicateurs** simples (des pourcentages de lignes conformes). La fonction `indicateurs_qualite` (dans `build/outils_ch03.py`) calcule vingt-six indicateurs sur nos trois sources ; voici comment on les assemble :

```

ind = 0.indicateurs_qualite(crm, site, site_l, caisse, fichiers, cmd, lig, prod, REF)
sc = ind[ind["dimension"] != "Actualité"].groupby(["source", "dimension"])["valeur"].mean().round(1)
print(sc.loc["CRM"].to_string())

```

dimension	
Cohérence	95.2
Complétude	86.9
Unicité	90.3
Validité	95.0

dimension	Complétude	Validité	Unicité	Cohérence	Exactitude
source					
CRM	86.9	95.0	90.3	95.2	n.d.
Caisse	96.9	n.d.	98.8	25.0	99.5
Site	100.0	79.9	98.1	59.8	59.8

La première ligne de code calcule les indicateurs, la deuxième moyenne ceux qui sont des pourcentages (l'actualité est un retard en jours, qu'on lit à part) ; le tableau suivant donne le score de chaque dimension pour chaque source.

Les seuils (98 % et 90 %) sont des **conventions d'exemple** : ils doivent être fixés avec ceux qui utilisent les données, pour chaque usage. Le tableau compte 5 cases rouges, 4 orange et 4 vertes. Trois lectures :

	Complétude	Validité	Unicité	Cohérence	Exactitude	Actualité
CRM	86,9 %	95,0 %	90,3 %	95,2 %	n.d.	6 j
Site	100,0 %	79,9 %	98,1 %	59,8 %	59,8 %	5 j
Caisse	96,9 %	n.d.	98,8 %	25,0 %	99,5 %	5 j

Figure 15: Tableau de bord de qualité des trois sources de la boutique : score par dimension (pourcentage de conformité, retard en jours pour l'actualité), coloré selon des seuils d'exemple (vert à partir de 98 %, orange de 90 à 98 %, rouge en dessous). « n.d. » : dimension non mesurable avec les données dont nous disposons.

- le **CRM** est rouge ou orange sur l'unicité (90,3 %) et sur la validité (95,0 %), et la complétude moyenne (86,9 %) est tirée vers le bas par le consentement ;
- le **site** a une exactitude de 59,8 % : c'est le changement d'unité du 15 septembre ;
- la **caisse** est exacte ligne à ligne (99,5 % des lignes se retrouvent dans la base) mais peu **cohérente** (25,0 % en moyenne) : ses fichiers n'ont pas tous le même format et aucun de ses totaux de contrôle n'est respecté.

Revenons à l'usage. Le CRM est-il utilisable **pour envoyer la lettre d'information** ? Il faut un e-mail bien formé, un consentement « oui », et que la ligne ne soit ni un test ni une copie d'une autre : cela ne laisse que 58,1 % des lignes (4 149). Est-il utilisable **pour répartir les clients par âge** ? Il faut une date de naissance plausible : 88,4 % des lignes (6 310) conviennent. Le même fichier, deux verdicts : c'est ce que veut dire « qualité pour un usage ».

À retenir. Un tableau de bord de qualité se lit **par source, par dimension et par usage**. Il sert à décider *quoi corriger en premier*, pas à décerner une note.

Pour s'entraîner. Cahier, chapitre 3 : application 3.1, exercices 3.4 et 3.5.

3.1.8 Quatre pièges des indicateurs de qualité

Un indicateur est un **instrument**, avec ses limites. Quatre erreurs fréquentes.

Premier piège : l'indicateur n'est pas la chose. Le test d'unicité par e-mail a trouvé 677 lignes redondantes (9,7 %). La vérité programmée, que nous pouvons ouvrir pour une fois, en contient 1 000 (14,3 %) : le test en laisse passer près d'un tiers, parce que les copies ne partagent pas toujours l'e-mail (il est absent ou abîmé, par exemple avec un @ ou un point doublé). Un indicateur de doublons ne mesure que **les doublons qu'il sait reconnaître**.

Deuxième piège : un indicateur trop zélé se trompe aussi. Sur la caisse, chercher les lignes **identiques** en signale 154, soit 1,2 % des lignes, alors que le double scan n'en explique que 67 (0,5 %) : les autres sont des achats légitimes d'un même article en deux lignes d'un même ticket. Une règle doit être jugée sur ses **faux positifs** autant que sur ses oublis.

Troisième piège : cent pour cent ne prouve rien. Une colonne « téléphone » remplie à 100 %

de 0000000000 est complète et valide. Une moyenne de scores peut aussi cacher un désastre : un tableau de bord à 97 % de conformité moyenne peut contenir une colonne vitale à 60 %. On regarde les **indicateurs**, pas seulement leur moyenne.

Quatrième piège : mesurer ce qui est facile. On mesure volontiers la complétude et la validité, qui ne demandent aucune référence, et l'on néglige l'exactitude, qui en demande une. Or les erreurs les plus coûteuses sont des erreurs d'exactitude (une unité qui change, un montant décalé) que ni la complétude ni la validité ne voient.

À retenir de la section 3.1.

- Une donnée est de qualité **pour un usage** ; on mesure des **dimensions** : complétude, validité, unicité, cohérence, exactitude, actualité.
- Chaque dimension se chiffre par un ou plusieurs **pourcentages de lignes conformes** (ou un retard en jours) ; un tableau de bord les assemble par source.
- L'exactitude exige une **référence** ; sans elle, on ne mesure que la plausibilité.
- Un indicateur est un instrument : il oublie des cas (doublons non reconnus) et en invente d'autres (faux positifs).

Pour s'entraîner. Cahier, chapitre 3 : application 3.1, exercices 3.1 à 3.5.

3.2 Contrôles de validation

La section précédente a **mesuré** la qualité. Celle-ci **l'impose** : un contrôle de validation est une règle que chaque ligne (ou chaque fichier) doit respecter, vérifiée automatiquement, avec un résultat qui dit **combien de lignes échouent, lesquelles, et ce qu'on en fait**. C'est l'outil de base de toute chaîne de traitement fiable : sans contrôle, un défaut ne se révèle que le jour où quelqu'un s'en étonne.

3.2.1 Un contrôle est une règle qui renvoie ses échecs

Un bon contrôle a quatre qualités.

- **Il est explicite** : il porte un nom lisible (« e-mail mal formé »), pas un numéro.
- **Il renvoie les lignes en échec**, pas seulement un « oui » ou un « non ». Savoir qu'il y a 95 e-mails mal formés est utile ; avoir la liste des 95 lignes est ce qui permet d'agir.
- **Il a une gravité** : une erreur qui bloque (un total qui ne tombe pas juste) n'est pas un avertissement (une ville écrite en majuscules).
- **Il est rejouable** : on le relance à chaque nouvelle livraison de données, sans rien changer.

En pandas, la forme la plus simple d'un contrôle est une fonction qui reçoit une colonne et renvoie un **masque booléen** des lignes en échec : True là où la règle est violée. On peut alors compter (`masque.sum()`), regarder (`df[masque]`) et, plus tard, corriger ou exclure.

Intuition. Un contrôle est un **détecteur de fumée** : il ne combat pas l'incendie, il le signale tôt, à l'endroit exact. La décision (éteindre, évacuer, ignorer) vient après (3.2.8).

3.2.2 Les contrôles de forme : type, format, liste de valeurs

Les contrôles les plus courants regardent la **forme** d'une valeur. Trois familles couvrent presque tous les besoins.

- **Le type** : la valeur est-elle un nombre, une date, un texte ? On le teste en essayant de convertir : `pd.to_numeric(colonne, errors="coerce")` renvoie une absence là où la conversion échoue, et c'est cette absence que l'on compte.
- **Le format** : la valeur respecte-t-elle un motif ? On utilise une **expression régulière** (un petit langage de motifs : `\d{5}` veut dire « cinq chiffres », `[@\s]+` « un ou plusieurs caractères qui ne sont ni @ ni un espace »).
- **La liste de valeurs** : la valeur appartient-elle à un ensemble autorisé (oui, non) ?

On écrit une fois la mécanique du format, et on la réutilise pour chaque colonne :

```
def echecs_format(serie, motif):
    """lignes renseignées dont la valeur ne respecte pas le motif (expression régulière)"""
    return serie.notna() & ~serie.str.fullmatch(motif, na=False)

def echecs_liste(serie, permises):
    """lignes renseignées dont la valeur n'est pas dans la liste"""
    return serie.notna() & ~serie.isin(permises)
```

Les deux fonctions ignorent volontairement les valeurs **absentes** : une absence relève de la complétude (3.1.2), pas du format. Mélanger les deux produirait un rapport illisible, où chaque cellule vide compterait comme une « erreur de format ».

Appliquons-les au CRM, avec les contrôles de lecture du chapitre précédent :

```
regles_crm = {
    "e-mail mal formé": echecs_format(crm["email"], 0.RE_EMAIL),
    "code postal ≠ 5 chiffres": echecs_format(crm["code_postal"], 0.RE_CP),
    "consentement hors {oui, non}": echecs_liste(crm["consentement_marketing"], ["oui", "non"]),
    "ville non reconnue": 0.canonique_ville(crm["ville"]).isna(),
    "date de naissance illisible": naiss.isna(),
    "ligne de test": crm["email"].eq("test@example.com"),
}
print(0.resume_controles(regles_crm, len(crm)).to_string(index=False))
```

	controle	echecs	taux_pct
e-mail mal formé		95	1.3
code postal ≠ 5 chiffres		199	2.8
consentement hors {oui, non}		2428	34.0
ville non reconnue		634	8.9
date de naissance illisible		666	9.3
ligne de test		140	2.0

Le contrôle de consentement est le plus bruyant : 2428 lignes échouent, parce que « Oui », « OUI », « O », « 1 » et « TRUE » ne sont pas des « oui » au sens strict. C'est un excellent exemple de **règle trop stricte** : le contrôle est juste (la forme n'est pas normée), mais la bonne réponse est de **normaliser** (les quatre écritures veulent dire « oui »), pas de rejeter. À l'inverse, les 95 e-mails mal formés et les 199 codes postaux à quatre chiffres appellent une correction ou une enquête.

Pour s'entraîner. Cahier, chapitre 3 : application 3.2, exercice 3.6.

3.2.3 Les contrôles de plage et de dépendance entre colonnes

Un **contrôle de plage** vérifie qu'une valeur reste dans des bornes (un montant positif, un âge entre 16 et 100). Une borne fixe se choisit avec le métier ; une borne statistique se déduit des données, comme la règle des 1,5 écart interquartile (voir le volume I, section 1.1.3).

Un **contrôle de dépendance** compare plusieurs colonnes d'une même ligne : le montant d'une ligne de commande ne peut pas dépasser la quantité multipliée par le prix, et ne peut pas être négatif ni nul. Ce second type de règle utilise de l'information que la première ignore. Comparons-les sur les 6 000 lignes de commande saisies à la main, dont 85 contiennent une erreur de saisie connue (nous les révélerons à la fin) :

```
q1, q3 = mont["montant"].quantile([0.25, 0.75])
plage = mont["montant"] > q3 + 1.5 * (q3 - q1) # règle statistique, une
↳ seule colonne
croisee = (mont["montant"] <= 0) | (mont["montant"] > mont["quantite"] * mont["prix_unitaire"] + 0.01)
vrai = mont["anomalie"].notna() # la vérité (réservée à
↳ l'évaluation)
for nom, r in {"plage (1,5 écart interquartile)": plage, "dépendance (0 < montant ≤ qté × prix)":
↳ croisee}.items():
    print(f"{nom:38s} signalées {int(r.sum()):4d} | vraies {int((r & vrai).sum()):3d} | fausses
↳ alertes {int((r & ~vrai).sum()):3d}")
```

```
plage (1,5 écart interquartile)    signalées 403 | vraies 52 | fausses alertes 351
dépendance (0 < montant ≤ qté × prix) signalées 85 | vraies 85 | fausses alertes 0
```

La règle de plage, qui ne regarde que le montant, signale 403 lignes au-dessus de 109,32 € mais n'en trouve que 52 vraies ; les 351 autres sont de grosses commandes parfaitement légitimes. La règle de dépendance en signale 85, **toutes** vraies (85), avec 0 fausse alerte. Elle repère même les montants **négatifs** et **nuls**, que la règle statistique, tournée vers le haut, ne voit pas.

Intuition. Une valeur n'est pas aberrante seule : elle l'est **par rapport à autre chose**. Un montant de 400 € est banal pour dix chaises et absurde pour un stylo à 2 €. Les contrôles les plus précis comparent **des colonnes entre elles** (montant et prix, date de commande et date de livraison, ville et code postal).

3.2.4 Les contrôles d'unicité et de comptage : totaux de contrôle

Trois contrôles très rentables n'ont rien de sophistiqué.

- **L'unicité d'une clé** : un numéro de commande ne doit apparaître qu'une fois (`serie.duplicated()`). Sur l'export du site, le test trouve 121 numéros répétés.
- **Les totaux de contrôle** : quand une source donne un total, on le **recalcule** à partir du détail. La caisse en imprime un au bas de chaque fichier ; la comparaison avec la somme des lignes est un contrôle de bout en bout.
- **Les comptages entre tables** : chaque en-tête doit avoir ses lignes, chaque ligne son en-tête.

```
lu = caisse.groupby("fichier")["montant"].sum().round(2)
affiche = fichiers.set_index("fichier")["total_affiche"]
ctrl = pd.DataFrame({"lu": lu, "affiche": affiche, "ecart": (lu - affiche).round(2)})
ctrl["ecart_pct"] = (100 * ctrl["ecart"] / ctrl["affiche"]).round(2)
print(ctrl.head(6).to_string())
```

	lu	affiche	ecart	ecart_pct
fichier				
caisse_2025-01.csv	37826.31	38882.41	-1056.10	-2.72
caisse_2025-02.csv	32273.46	33079.41	-805.95	-2.44
caisse_2025-03.csv	37863.67	39038.90	-1175.23	-3.01

caisse_2025-04.csv	45426.14	45832.57	-406.43	-0.89
caisse_2025-05.csv	43381.31	44905.92	-1524.61	-3.40
caisse_2025-06.csv	42277.42	43118.16	-840.74	-1.95

Aucun des douze fichiers ne respecte son total : l'écart va de 0,9 % à 3,4 % en valeur absolue, et 12 fichiers dépassent la tolérance de 0,5 %. Ce n'est pas une erreur du total (nous verrons en 3.3.4 qu'il est exact), mais le signe que la somme des lignes lues est **incomplète** : il manque les montants vides. Le contrôle ne dit pas *pourquoi* ; il dit *qu'il y a quelque chose à expliquer*, et c'est son rôle.

Le comptage entre tables est tout aussi parlant :

```
sans_ligne = ~site["order_ref"].isin(site_l["order_ref"])
print("en-têtes sans aucune ligne :", int(sans_ligne.sum()), "| lignes sans en-tête :",
      ↪ int(~site_l["order_ref"].isin(site["order_ref"])).sum())
print(site.loc[sans_ligne, "customer_email"].value_counts().to_string())
```

```
en-têtes sans aucune ligne : 60 | lignes sans en-tête : 0
customer_email
test@example.com      60
```

Les 60 commandes sans ligne ont toutes la même adresse, `test@example.com` : ce sont les commandes de test de l'équipe web. Un simple contrôle d'intégrité référentielle (« toute commande a au moins une ligne ») les a trouvées sans connaître leur existence. On parle d'**orphelins** ; ils sont presque toujours le symptôme d'un autre défaut.

3.2.5 Les contrôles temporels : trous, futur et ruptures

Les séries de données ont une dimension de plus, le **temps**, et trois contrôles s'y attachent.

- **Les dates impossibles** : dans le futur (une commande datée de demain) ou avant le lancement de l'activité.
- **Les trous** : un jour, une semaine sans ligne alors que l'activité n'est jamais nulle. Sur l'export du site, il y a 0 jour sans commande en 2025 ; aucune date n'est dans le futur (0).
- **Les ruptures** : un niveau, une unité ou un format qui change brutalement. C'est le plus dangereux, parce que chaque ligne reste valide.

La rupture du 15 septembre se détecte en comparant chaque jour à la semaine qui précède :

```
site["date"] = pd.to_datetime(site["created_at"].str.replace("Z", ""),
                              ↪ format="ISO8601").dt.normalize()
site["t"] = site["total"].map(0.montant_site_en_nombre)
jour = site.groupby("date")["t"].median()
rapport_jour = jour / jour.rolling(7).median().shift(1)
premier = rapport_jour[rapport_jour > 10]
print("premier jour dont le montant médian est plus de 10 fois celui de la semaine d'avant :",
      ↪ premier.index[0].date(), "| rapport :", round(premier.iloc[0]))
```

```
premier jour dont le montant médian est plus de 10 fois celui de la semaine d'avant : 2025-09-15 |
↪ rapport : 147
```

Le 15/09/2025, le rapport est de 147 : le montant médian des commandes passe d'une centaine d'euros à plus d'une dizaine de milliers. Aucune ligne, prise seule, n'est invalide ; c'est la **série** qui casse.

La médiane mensuelle vaut 85,7 € en août et 8396,0 € en octobre, soit 98 fois plus. Une analyste qui n'aurait fait que sommer les montants du site aurait annoncé un chiffre d'affaires de quarante fois la réalité (3.3.3). Le contrôle temporel coûte cinq lignes et épargne cette erreur.

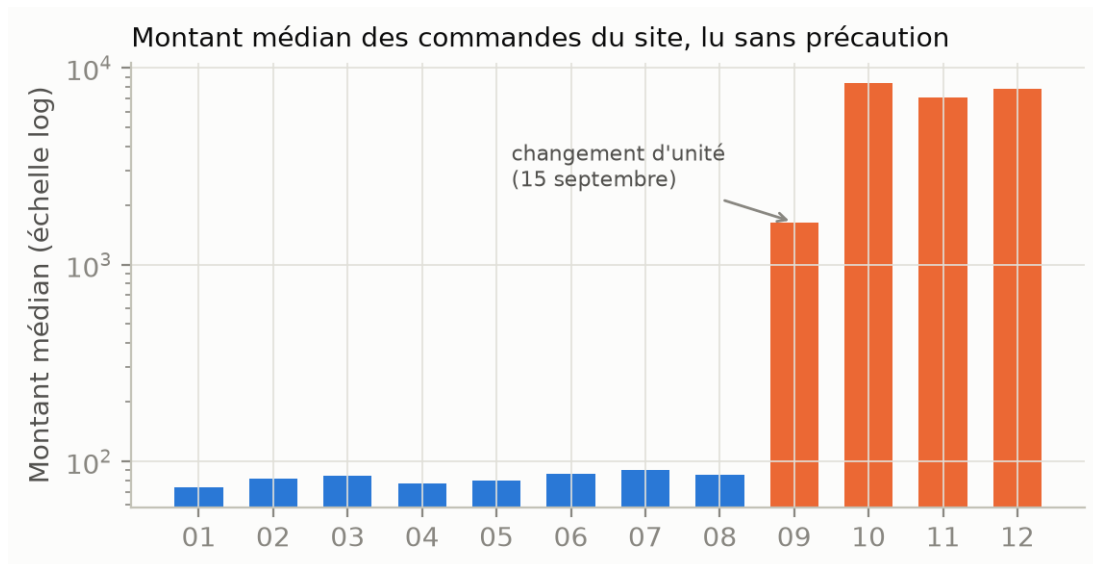


Figure 16: Montant médian des commandes du site par mois, lu sans précaution (échelle logarithmique). Il vaut environ 80 € de janvier à août, puis des milliers à partir de septembre : la plateforme exporte ses montants en centimes depuis le 15 septembre.

Piège : le contrôle ligne à ligne est aveugle aux ruptures. Les contrôles de forme, de plage et d'unicité jugent chaque ligne isolément. Pour voir une rupture, il faut comparer des **périodes** : un niveau, une moyenne, une répartition de valeurs. La règle générale est de vérifier chaque colonne quantitative **dans le temps**, avant de l'agréger.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.3 et 3.4, exercices 3.7 et 3.8.

3.2.6 Un rapport de contrôles

Quand les contrôles se multiplient, on les range dans un **rapport** : une ligne par règle, avec la source, la gravité, le nombre d'échecs, le taux, et un exemple. Voici le principe, sur les contrôles du CRM ; le rapport complet rassemble les trois sources :

```
rapport = 0.resume_controles({f"CRM : {k}": v for k, v in regles_crm.items()}), len(crm))
rapport["gravite"] = ["avertissement", "erreur", "avertissement", "avertissement", "erreur", "erreur"]
print(rapport.sort_values("taux_pct", ascending=False).head(5).to_string(index=False))
```

	controle	echecs	taux_pct	gravite
CRM : consentement hors {oui, non}		2428	34.0	avertissement
CRM : date de naissance illisible		666	9.3	erreur
CRM : ville non reconnue		634	8.9	avertissement
CRM : code postal ≠ 5 chiffres		199	2.8	erreur
CRM : ligne de test		140	2.0	erreur

Le principe tient en trois gestes : on **rassemble** les masques, on en tire un tableau, on **trie** par importance. La figure donne la vue complète des trois sources : 14 règles, dont 9 classées « erreur ».

Un bon rapport donne, pour chaque règle, **quelques exemples de lignes en échec** : c'est ce qui rend le rapport actionnable. Voici trois e-mails mal formés du CRM :

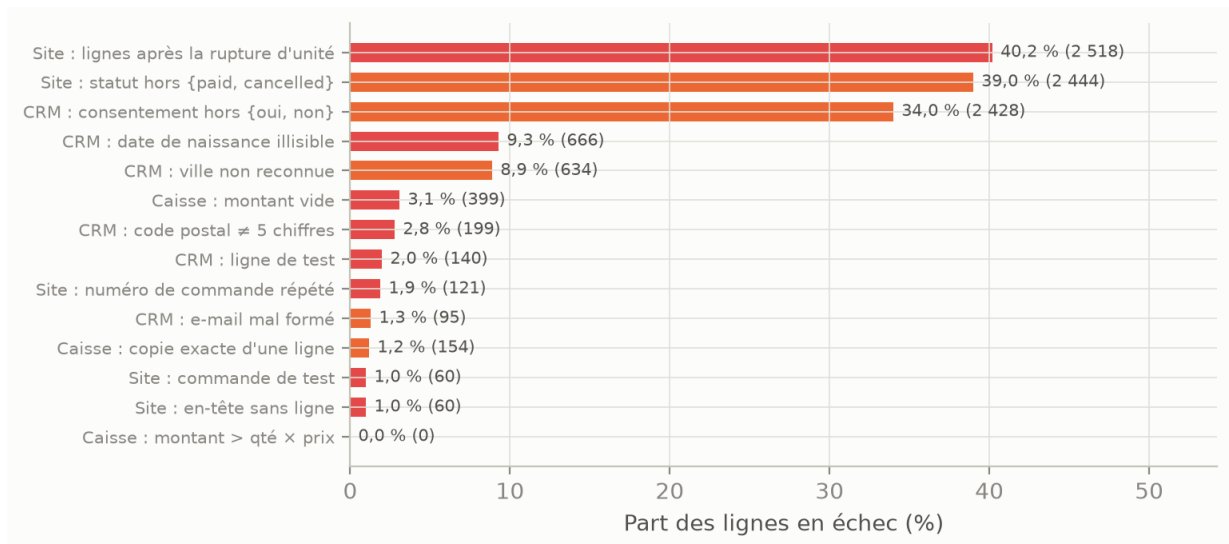


Figure 17: Rapport de contrôles : part des lignes en échec pour chaque règle des trois sources. Rouge : erreur (à corriger ou à bloquer) ; orange : avertissement.

```
print(crm.loc[regles_crm["e-mail mal formé"], ["id_crm", "email"]].head(3).to_string(index=False))
```

```
id_crm      email
6450  vemonea..ostlin@mail.example
2108  fevinia..valvane@mail.example
2106  orlonel..kelkan@exemple.org
```

On y reconnaît à l'œil un @ doublé ou un point doublé : l'exemple dit la cause plus vite que le compteur.

3.2.7 Les mêmes contrôles en SQL

Quand les données vivent dans une base, les contrôles s'écrivent en SQL, de deux façons.

La première est **préventive** : la base **refuse** une ligne invalide à l'écriture, grâce à des **contraintes** déclarées sur la table (PRIMARY KEY, NOT NULL, UNIQUE, CHECK). C'est le contrôle le plus fort, puisqu'une donnée invalide n'entre jamais :

```
con.execute("""CREATE TABLE clients_propres (id_crm INTEGER PRIMARY KEY, code_postal TEXT CHECK
↳ (code_postal IS NULL OR length(code_postal) = 5),
                                consentement TEXT CHECK (consentement IN ('oui', 'non'))""")
try:
    con.execute("INSERT INTO clients_propres VALUES (1, '1234', 'oui')")
except sqlite3.IntegrityError as e:
    print("ligne refusée :", e)
```

```
ligne refusée : CHECK constraint failed: code_postal IS NULL OR length(code_postal) = 5
```

La seconde est **détective** : on interroge des données déjà là, avec des requêtes de contrôle qui renvoient les lignes en échec. Le contrôle de format du code postal :

```
SELECT COUNT(*) AS echecs FROM crm WHERE code_postal IS NOT NULL AND length(code_postal) <> 5;
```

```
echecs
199
```

et le contrôle d'unicité de l'e-mail normalisé, en regroupant :

```
SELECT lower(trim(email)) AS email_normalise, COUNT(*) AS lignes FROM crm
WHERE email IS NOT NULL AND email <> 'test@example.com'
GROUP BY lower(trim(email)) HAVING COUNT(*) > 1 ORDER BY lignes DESC LIMIT 3;
```

email_normalise	lignes
zomonin.valgren@example.org	3
yulono.wenmer@mail.example	3
tamonia.nevtier21@courrier.test	3

et le contrôle d'orphelins avec une anti-jointure (volume I, section 3.2.3) :

```
SELECT COUNT(*) AS en_tetes_sans_ligne FROM site s LEFT JOIN site_lignes l ON l.order_ref =
s.order_ref WHERE l.order_ref IS NULL;
```

en_tetes_sans_ligne
60

Les trois requêtes retrouvent les chiffres de pandas. Les deux approches se complètent : la contrainte protège la porte, la requête de contrôle inspecte la maison. Une base bien conçue déclare ses contraintes, mais les fichiers (CSV, exports, tableurs) n'ont **aucune** contrainte : c'est là que les contrôles en code sont indispensables.

Pour s'entraîner. Cahier, chapitre 3 : application 3.2, exercice 3.9.

3.2.8 Que faire quand un contrôle échoue ?

Un contrôle qui échoue appelle une **décision**, et elle dépend de la gravité. Quatre réponses possibles.

Réponse	Quand	Exemple dans la boutique
Bloquer	l'erreur invalide tout ce qui suit	le total de la caisse ne tombe pas : on ne publie pas le chiffre d'affaires du mois
Mettre en quarantaine	les lignes fautives sont isolées, le reste passe	les commandes de test sont écartées vers une table à part
Corriger	la correction est certaine et réversible	normaliser « Oui », « O », « 1 » en « oui »
Avertir	l'écart est connu et tolérable	une ville écrite en majuscules

Trois règles de bon sens.

1. **Ne corrigez jamais en silence.** Toute correction automatique doit être comptée, journalisée et reproductible : « 2 428 consentements normalisés en « oui », 140 « non » conservés ». Un fichier corrigé sans trace est un fichier dont personne ne sait plus ce qu'il contient.
2. **Conservez l'original.** On ne modifie pas la source : on produit une version nettoyée à côté, et l'on garde la brute, qui est la seule pièce à conviction en cas de litige.
3. **Décidez de la gravité avant de lancer les contrôles,** pas après avoir vu les résultats. Une gravité fixée après coup finit toujours par minimiser ce qui gêne.

À retenir de la section 3.2.

- Un contrôle est une règle nommée qui **renvoie les lignes en échec**, avec une gravité, rejouable à chaque livraison.
- Les contrôles **entre colonnes** sont plus précis que les contrôles de plage sur une co-

bonne ; les **totaux de contrôle** et les **comptages entre tables** attrapent des défauts qu'aucun contrôle de ligne ne voit.

- Un contrôle ligne à ligne est aveugle aux **ruptures dans le temps** : on compare des périodes.
- En SQL, on **prévient** (contraintes) et on **détecte** (requêtes de contrôle) ; sur des fichiers, tout est à écrire.
- Un contrôle qui échoue appelle une décision : **bloquer, mettre en quarantaine, corriger ou avertir** ; jamais corriger en silence.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.2 à 3.4, exercices 3.6 à 3.9.

3.3 Réconciliation de sources

Les contrôles de la section précédente jugent **une** source contre ses propres règles. La **réconciliation** compare **deux** sources qui prétendent décrire la même réalité (les ventes de décembre, la liste des produits) et cherche à **expliquer chaque écart**. C'est le geste du comptable qui rapproche le relevé de banque du journal : on ne s'arrête pas à « les deux totaux sont proches », on s'arrête à « l'écart de 17 551,32 € est exactement ces 181 commandes annulées ».

3.3.1 Deux sources, une mesure

Réconcilier suppose de se mettre d'accord sur trois choses.

- **La mesure** : de quel chiffre parle-t-on ? (le chiffre d'affaires **toutes commandes** ou **hors annulations** ? **toutes taxes** comprises ? à quelle date : celle de la commande ou celle de l'encaissement ?)
- **Le périmètre** : quelles lignes sont concernées ? (le canal Site seulement ? l'année 2025 ? les commandes de test ?)
- **La source de référence** : si les deux disent des choses différentes, laquelle croit-on en premier ? On choisit la plus **proche de l'événement** (la caisse enregistre la vente au moment où elle a lieu) ou la plus **contrôlée** (la base alimente la comptabilité). Ce choix est une convention, à écrire.

Nous réconcilierons trois paires de sources de la boutique, de plus en plus difficiles.

Paire	Mesure	Clé de rapprochement	Difficulté
Site contre base	chiffre d'affaires du canal Site en 2025	numéro de commande	l'export est désordonné (unité, doublons, tests)
Caisse contre base	chiffre d'affaires du canal Boutique en 2025	ticket + article + quantité + prix	douze fichiers, trois formats, des montants vides
Catalogue contre produits	liste des produits et de leur coût	aucune : un code étranger, une désignation réécrite	il faut fabriquer la clé

3.3.2 La méthode en trois temps

Une réconciliation sérieuse suit toujours le même ordre, du plus grossier au plus fin.

1. **Compter.** Les deux sources ont-elles le même nombre de lignes (de commandes, de clients, de produits) ? Un écart d'effectif dit déjà si l'on perd, duplique ou invente des lignes.
2. **Sommer.** Les deux sources ont-elles le même total ? On compare les totaux de la mesure convenue, et l'on note l'écart (absolu et relatif).
3. **Expliquer.** On ventile l'écart en **causes** chiffrées, jusqu'à ce que la somme des causes égale l'écart. Seule cette étape transforme « c'est à peu près bon » en « c'est juste, et voici pourquoi ».

Intuition. C'est un jeu de piste : la première étape dit *s'il y a* un problème, la deuxième *de quelle taille*, la troisième *où il se cache*. On n'a fini que lorsque l'écart restant vaut zéro (ou est inférieur à une tolérance décidée à l'avance, voir 3.4).

La cascade (en anglais *waterfall*) est l'outil qui rend la troisième étape lisible : une barre pour le total de départ, une barre par cause (vers le haut ou vers le bas), et une barre pour le total d'arrivée, qui doit tomber exactement sur la référence.

3.3.3 Le site contre la base : un écart de quarante fois

Commençons par les deux premiers temps sur l'export du site, pour le canal Site en 2025. On compte les lignes, puis on somme les montants **sans précaution** (le texte devient un nombre, c'est tout) :

```
t_export = site["total"].map(0.montant_site_en_nombre)
base_site = cmd[(cmd["canal"] == "Site") & (cmd["date_commande"] >= "2025-01-01")]
ca_base = base_site["id_commande"].map(lig.groupby("id_commande")["montant"].sum()).sum()
print("compter :", len(site), "lignes d'export contre", len(base_site), "commandes en base")
print("sommer :", f"{t_export.sum():.2f}".replace(",", " "), "€ contre",
      f"{ca_base:.2f}".replace(",", " "), "€")
```

```
compter : 6259 lignes d'export contre 6078 commandes en base
sommer : 25 012 599.35 € contre 617 715.45 €
```

L'export compte 181 lignes de plus que la base (6 259 contre 6 078), et son total vaut **40,5 fois** celui de la base. Cet écart n'est pas un arrondi : il se cache plusieurs causes, que la section 3.2 nous a appris à reconnaître. Allons-y dans l'ordre de leur poids.

Première cause : l'unité. À partir du 15 septembre, la date change de format (elle se termine par Z) et, en même temps, les montants sont exprimés en **centimes**. On le voit en comparant les montants moyens avant et après :

```
apres = site["created_at"].str.endswith("Z") # format de date de la nouvelle
↳ plateforme
print(t_export.groupby(apres).mean().round(1).to_string())
```

```
created_at
False    102.5
True     9781.2
```

Le montant moyen vaut 102,5 € avant (la ligne False) et 9781,2 après (la ligne True) : cent fois plus, pour des commandes qui n'ont pas changé. On divise par cent les montants des 2 518 lignes du nouveau format. Notez que nous **déduisons** le changement d'unité de l'observation, nous ne l'avons lu nulle part : il faudra le **faire confirmer** par l'équipe du site.

Deuxième et troisième causes : les tests et les copies. Les commandes de test se reconnaissent

à leur adresse, les copies à leur numéro répété (la plateforme a réexporté certaines commandes). On les met de côté et l'on construit la cascade :

```
test = site["customer_email"].eq("test@example.com")
copie = site["order_ref"].duplicated(keep="first") & ~test
etapes = [("Somme brute de l'export", t_export.sum()), ("montants en centimes (÷ 100)", t_eur.sum() -
↳ t_export.sum()),
          ("commandes de test", -t_eur[test].sum()), ("copies d'export", -t_eur[copie].sum())]
tab = pd.DataFrame(etapes, columns=["étape", "montant"]); tab["cumul"] = tab["montant"].cumsum()
print(tab.round(2).to_string(index=False))
print("base :", round(ca_base, 2), "| écart restant :", abs(round(tab["cumul"].iloc[-1] - ca_base,
↳ 2)))
```

	étape	montant	cumul
	Somme brute de l'export	25012599.35	25012599.35
	montants en centimes (÷ 100)	-24382796.13	629803.22
	commandes de test	-36.24	629766.98
	copies d'export	-12051.53	617715.45
base : 617715.45 écart restant : 0.0			

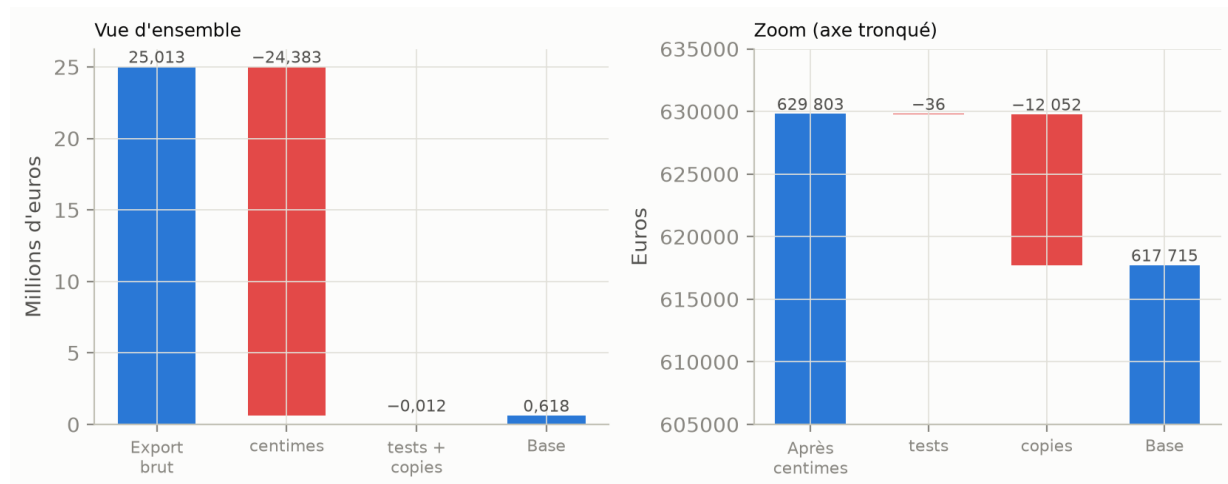


Figure 18: Cascade de réconciliation du canal Site : de la somme brute de l'export (25 millions d'euros) au chiffre d'affaires de la base (618 000 euros). À gauche, la vue d'ensemble ; à droite, un zoom sur les deux petites causes (axe tronqué : les barres de départ et d'arrivée sont coupées).

La cascade est exacte : **l'écart restant est de 0,0 €**. Le changement d'unité explique à lui seul 24 382 796 €, les 121 copies d'export 12 051,53 € et les 60 commandes de test 36,24 €. Ce qui reste, 6 078 commandes pour 617 715,45 €, **égale** la base, centime pour centime. Notez deux détails : les commandes de test pèsent presque rien en euros (1 € l'une) mais elles ne sont pas anodines dans un **comptage** (60 commandes qui n'existent pas), et si l'on n'avait corrigé que l'unité, on aurait gardé 12 000 € de copies.

Piège : corriger dans le mauvais ordre. Si l'on retire les copies **avant** d'avoir converti l'unité, on retire des montants dont la taille dépend de la date : l'écart calculé pour chaque étape change, même si le total final reste juste. On décide d'un ordre, et l'on s'y tient : ici, d'abord les **formats** (unité), puis les **lignes parasites** (tests, copies).

Il reste à traiter trois points qui ne sont pas des écarts, mais des **définitions**.

Les annulations. L'export contient des commandes annulées. La base les contient aussi (elle ne porte pas de statut) : elles sont donc **dans les deux totaux**, et ne créent aucun écart. Mais elles comptent pour le chiffre d'affaires **encaissé** :

```
annule = propre["status"].str.lower().eq("cancelled")
print("commandes annulées :", int(annule.sum()), "| montant :", round(propre.loc[annule,
↪ "eur"].sum(), 2), "€")
```

```
commandes annulées : 181 | montant : 17551.32 €
```

181 commandes sont annulées, pour 17551,32 € (2,8 % du chiffre d'affaires). Ce montant n'explique **aucun écart entre les sources**, mais il explique l'écart entre deux **définitions** du chiffre d'affaires : 617715,45 € toutes commandes, 600 164,13 € hors annulations. Quand une direction dit « le site m'annonce un chiffre, la comptabilité un autre », la première chose à demander est : *avec ou sans annulations ?*

Les arrondis. Le total d'en-tête d'une commande est-il la somme de ses lignes ? Chaque ligne est arrondie au centime, et le total est arrondi lui aussi : de petits écarts apparaissent.

```
somme_l = site_l.assign(m=site_l["qty"] * site_l["unit_price"] * (1 - site_l["discount_pct"] /
↪ 100)).groupby("order_ref")["m"].sum()
diff = propre["eur"].values - propre["order_ref"].map(somme_l).values
print("somme des écarts :", round(float(diff.sum()), 3), "€ | écart maximal :",
↪ round(float(np.abs(diff).max()), 3), "€ | commandes à plus d'un demi-centime :",
↪ int((np.abs(diff) > 0.005).sum()))
```

```
somme des écarts : 0.078 € | écart maximal : 0.016 € | commandes à plus d'un demi-centime : 172
```

Les écarts d'arrondi sont bien réels (172 commandes), mais leur somme est de 0,078 € sur plus de six mille commandes : ils se compensent. On ne les **explique** pas un par un ; on fixe une **tolérance** (par exemple un centime par ligne) et l'on vérifie qu'ils restent en dessous (3.4.2).

Le fuseau horaire. Le nouveau format se termine par Z, qui veut dire « heure UTC ». Si c'était vrai, une commande passée à 23 h 30 à l'heure locale serait datée du lendemain, et les totaux **journaliers** du site ne tomberaient plus sur ceux de la base. Vérifions, commande par commande, ce que dit la base :

```
cl = propre.assign(id_commande=propre["order_ref"].str.extract(r"WEB-(\d{6})")[0].astype(int,
↪ ).merge(cmd[["id_commande", "date_commande", "heure"]], on="id_commande")
meme_jour = (pd.to_datetime(cl["created_at"].str.replace("Z", ""),
↪ format="ISO8601").dt.strftime("%Y-%m-%d") == cl["date_commande"])
meme_heure = (cl["created_at"].str[11:16] == cl["heure"])
print("même jour que la base :", round(100 * meme_jour.mean(), 1), "% | même heure que la base :",
↪ round(100 * meme_heure.mean(), 1), "%")
```

```
même jour que la base : 100.0 % | même heure que la base : 100.0 %
```

Les dates et les heures sont **identiques** à celles de la base, y compris après le 15 septembre : le Z est une étiquette trompeuse (l'heure écrite est l'heure locale), ou la base est elle aussi en UTC ; dans les deux cas, **aucune commande ne change de jour**. La dernière commande de la base est à 21:59 : même avec un décalage de deux heures, personne ne passerait minuit. Le fuseau n'a donc rien changé à nos totaux journaliers ; il changerait une analyse **par heure**. Ce qui compte est de l'avoir **vérifié** au lieu de le supposer.

Pour s'entraîner. Cahier, chapitre 3 : application 3.5, exercice 3.11.

3.3.4 La caisse contre la base : douze fichiers, trois formats

La caisse de la boutique envoie un fichier par mois. Au cours de l'année, le logiciel a changé deux fois de format. On le voit en regroupant les fichiers par leurs caractéristiques :

```
print(fichiers.groupby(["encodage", "separateur", "decimale", "colonnes"]).agg(fichiers=("fichier",
↪ "count"), premier=("fichier", "min"), dernier=("fichier", "max")).to_string())
```

				fichiers	premier	dernier
encodage	separateur	decimale	colonnes			
cp1252	;	,	8	6	caisse_2025-01.csv	caisse_2025-06.csv
utf-8-sig	,	.	9	3	caisse_2025-10.csv	caisse_2025-12.csv
	;	,	8	3	caisse_2025-07.csv	caisse_2025-09.csv

Trois formats : de janvier à juin, un fichier en cp1252 avec ; et virgule décimale ; de juillet à septembre, le même mais en UTF-8 (et l'en-tête de la colonne des quantités devient « Quantité », la date prend deux chiffres d'année) ; d'octobre à décembre, une virgule comme séparateur, un point décimal, des champs entre guillemets et une **neuvième colonne** (la remise). La fonction `lire_caisse` de `build/outils_ch03.py` absorbe ces différences ; l'écrire est un exercice de nettoyage du chapitre 1, ce qui nous intéresse ici est ce qu'elle rend : une table unique de 12 678 lignes.

Premier et deuxième temps :

```
base_b = lig.merge(cmd[["id_commande", "date_commande", "canal"]], on="id_commande")
base_b = base_b[(base_b["canal"] == "Boutique") & (base_b["date_commande"] >= "2025-01-01")]
print("compter :", len(caisse), "lignes en caisse contre", len(base_b), "en base")
print("sommer : lu", round(caisse["montant"].sum(), 2), "| total affiché",
↪ round(fichiers["total_affiche"].sum(), 2), "| base", round(base_b["montant"].sum(), 2))
```

```
compter : 12678 lignes en caisse contre 12611 en base
sommer : lu 547896.42 | total affiché 560973.91 | base 560973.91
```

Deux enseignements. La caisse a 67 lignes **de plus** que la base, et la somme des montants lus est inférieure de 13077,49 € : deux défauts de sens opposé (des lignes en trop, des montants manquants). Et le **total affiché** par la caisse (560 973,91 €) est **égal** à celui de la base (560 973,91 €) : ce total est juste, ce sont les lignes que nous avons lues qui sont fautives.

Pour le troisième temps, il faut descendre à la **ligne**. Il n'y a pas d'identifiant de ligne dans la caisse ; on fabrique une clé avec le numéro de ticket, l'article (en minuscules), la quantité, le prix et un **rang** qui départage deux lignes identiques d'un même ticket (le rang vaut 0 pour la première, 1 pour la deuxième, etc.). Le rapprochement se fait par une **fusion externe**, qui garde les lignes des deux côtés et indique d'où elles viennent :

```
r = 0.rapprocher_caisse_base(caisse, cmd, lig, prod)
print(r["_merge"].value_counts().to_string())
```

_merge	
both	12611
left_only	67
right_only	0

12611 lignes se retrouvent des deux côtés, **67** n'existent qu'en caisse et **0** qu'en base. Aucune ligne de la base ne manque à la caisse (rien n'est perdu), mais 67 lignes de la caisse n'ont pas d'équivalent : ce sont des **copies**, c'est-à-dire des lignes scannées deux fois. Le rang le montre : la copie a le rang 1 et la base n'a pas de ligne de rang 1 pour ce ticket.

Intuition. Sans le rang, la fusion aurait associé les deux copies à la **même** ligne de la base, et nous n'aurions rien vu : les doublons auraient simplement doublé les lignes après jointure (le piège du volume I, section 3.2.6). La clé doit être **assez fine pour que chaque ligne n'ait qu'une correspondance possible**.

On peut maintenant expliquer l'écart de somme. On retire les copies, on complète les montants vides (une ligne de la caisse sans montant, mais qui existe en base), puis on tient compte des remises :

```
copies = r[r["_merge"] == "left_only"]
vides = r[(r["_merge"] == "both") & r["montant"].isna()]
qp = vides["qte"] * vides["prix_unitaire"]
etapes = [("Somme des montants lus", caisse["montant"].sum()), ("copies de scan",
↳ -copies["montant"].sum()),
        ("montants vides complétés (qté × prix)", qp.sum()), ("remises des lignes vides", -(qp -
↳ vides["montant_base"]).sum()))
tab_c = pd.DataFrame(etapes, columns=["étape", "montant"]); tab_c["cumul"] = tab_c["montant"].cumsum()
print(tab_c.round(2).to_string(index=False))
```

	étape	montant	cumul
	Somme des montants lus	547896.42	547896.42
	copies de scan	-3126.29	544770.13
	montants vides complétés (qté × prix)	16518.59	561288.72
	remises des lignes vides	-314.81	560973.91

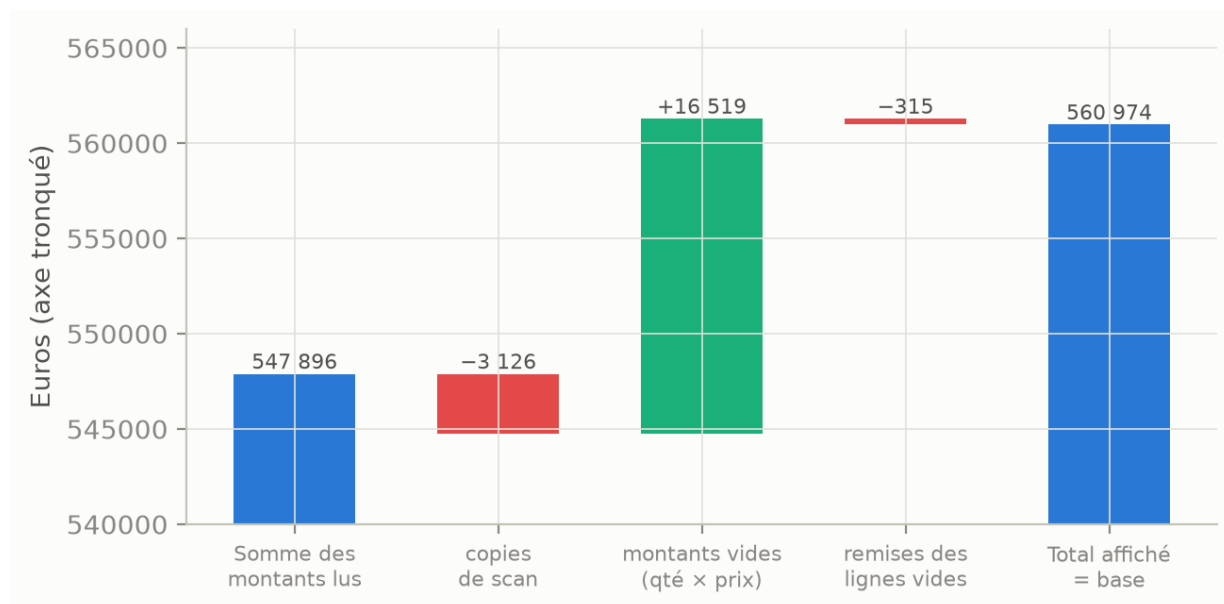


Figure 19: Cascade de réconciliation de la caisse : de la somme des montants lus dans les douze fichiers au total affiché par la caisse, égal à celui de la base. Axe vertical tronqué.

L'écart restant est de 0,0 € : l'explication est **complète**. Les 398 montants vides retrouvés dans la base valent 16 518,59 € au prix catalogue ; les remises que la caisse n'imprime pas avant octobre les ramènent à leur valeur exacte, soit 314,81 € de moins. Retenez la mécanique : **chaque étape de la cascade est un nombre que l'on a calculé, pas un reste que l'on a « mis dans une case »**. Un écart qu'on ne sait expliquer que par un « divers » n'est pas expliqué.

Un dernier contrôle de la réconciliation : les copies que nous avons trouvées sont-elles bien les doublons du double scan ? La vérité programmée donne, pour chaque fichier, le nombre de lignes doublées : les douze comptes **coïncident** avec ceux de notre rapprochement (vérifié : oui). Dans une étude réelle, on ne dispose pas de cette vérité : on l'aurait remplacée par une question à

l'équipe de la caisse (« avez-vous un double scan ? ») et par le fait que le total affiché retombe.

Pour s'entraîner. Cahier, chapitre 3 : application 3.6, exercice 3.10.

3.3.5 Le catalogue contre les produits : fabriquer la clé

Le troisième cas est le plus fréquent en pratique et le plus délicat : deux listes **sans clé commune**. Le catalogue du fournisseur a ses propres codes (F-1497) et ses propres désignations ; la boutique a ses numéros de produit. Seule la **désignation** les rapproche, et elle est écrite autrement : majuscules, accents retirés, ordre des mots, abréviations, espaces parasites.

On commence par une **normalisation** qui supprime les écarts de pure forme (accents, casse, espaces), puis on joint sur le nom :

```
def cle_nom(s):
    return s.map(lambda x: 0.sans_accents(str(x)).lower().strip())
cat["cle"], prod["cle"] = cle_nom(cat["designation"]), cle_nom(prod["nom_produit"])
m = cat.merge(prod[["id_produit", "cle", "cout_achat"]], on="cle", how="left")
print("lignes après jointure sur le nom :", len(m), "pour", len(cat), "lignes de catalogue | sans
↳ correspondance :", int(m["id_produit"].isna().sum()))
```

```
lignes après jointure sur le nom : 196 pour 118 lignes de catalogue | sans correspondance : 40
```

Deux surprises. D'abord la jointure a produit 196 lignes pour 118 lignes de catalogue : les produits de la boutique n'ont que 60 noms distincts pour 120 produits (deux produits portent chacun le même nom, à des prix différents), donc chaque ligne de catalogue se multiplie : c'est le **piège du volume I** (section 3.2.6). Ensuite 40 lignes de catalogue n'ont aucun correspondant.

Pour départager les homonymes, on ajoute une **seconde clé** : le **prix**. Le prix d'achat du catalogue doit être proche du coût d'achat de la boutique, à quelques pour cent près ; on retient les paires dont l'écart relatif est inférieur à 3,5 % :

```
m["ecart_prix"] = (m["prix_achat_ht"] / m["cout_achat"] - 1).abs()
ok = m[m["ecart_prix"] <= 0.035]
print("codes appariés par nom + prix :", ok["code_fournisseur"].nunique(), "| codes qui désignent
↳ encore deux produits :", int(ok["code_fournisseur"].duplicated().sum()))
reste = cat[~cat["code_fournisseur"].isin(ok["code_fournisseur"])]
print(reste[["code_fournisseur", "designation", "prix_achat_ht"]].head(5).to_string(index=False))
```

```
codes appariés par nom + prix : 78 | codes qui désignent encore deux produits : 2
```

code_fournisseur	designation	prix_achat_ht
F-1707	Diffu. design	4.86
F-1798	Nouveauté 6 Brillant	31.38
F-1553	Gants. design	9.77
F-1147	compact Étagère	14.76
F-1280	compact Guirlande	13.98

La clé « nom + prix » rapproche 78 codes du catalogue, dont 2 désignent encore **deux** produits (deux produits de même nom dont les coûts sont trop proches pour être départagés) ; ces cas sont des **exceptions**, à confier à quelqu'un, pas à deviner. Il reste 40 lignes sans correspondant : la liste de tête montre pourquoi (« Diffu. design » est une abréviation, « compact Étagère » a l'ordre des mots inversé).

Que dit la vérité programmée ? Parmi les codes appariés sans ambiguïté, **76** sont exacts et 0 faux : la clé est fiable quand elle tranche. Parmi les 40 lignes restantes, 10 sont de vrais produits nouveaux du fournisseur (qui n'existent pas à la boutique : ils ne **doivent pas** s'apparier), et 30 sont des produits que nous **avons** et que la normalisation simple n'a pas su reconnaître. Côté boutique, 41 produits n'ont pas de ligne de catalogue : 12 sont absents du catalogue du fournisseur,

les autres font partie des lignes non reconnues. Les 30 cas difficiles relèvent de l'**appariement approximatif** (*fuzzy matching*, section 2.5 de ce volume) : des mesures de ressemblance entre textes plutôt qu'une égalité stricte.

Piège : une clé fabriquée n'est pas une clé. Une clé fabriquée (nom normalisé + prix) apparie des choses qui se ressemblent, pas des choses identiques. Elle peut se tromper, et l'erreur est silencieuse : deux produits de même nom et de même coût seraient confondus sans alerte. On mesure donc **trois nombres** : combien d'appariements certains, combien d'ambigus, combien de non-appariés, et l'on traite chaque catégorie séparément.

Pour s'entraîner. Cahier, chapitre 3 : application 3.7.

3.3.6 Un catalogue des causes d'écart

Les trois exemples ont montré les causes d'écart les plus fréquentes. Les connaître d'avance fait gagner du temps : on les passe en revue **dans cet ordre**, du plus fréquent au plus rare.

Cause	Symptôme	Où nous l'avons vue
Unité ou échelle	rapport constant ($\times 100$, $\times 1\,000$) entre les deux totaux	export du site, centimes à partir du 15 septembre
Doublons	plus de lignes dans une source que dans l'autre	copies d'export du site ; double scan de la caisse
Lignes parasites	lignes sans équivalent : tests, orphelins	commandes de test du site
Valeurs manquantes	total inférieur au total affiché	montants vides de la caisse
Périmètre et définition	écart égal à un sous-ensemble connu	commandes annulées (avec ou sans)
Calendrier et fuseau	écarts qui se déplacent d'un jour à l'autre	vérifié : nul ici
Arrondis	écarts de quelques centimes, qui se compensent	total d'en-tête contre somme des lignes
Remises, taxes, frais	écart proportionnel aux lignes concernées	remises non imprimées avant octobre
Clé mal fabriquée	appariements faux ou multiples	homonymes du catalogue

3.3.7 Quand s'arrêter ?

Une réconciliation s'arrête quand l'écart restant est **expliqué ou tolérable**, et pas avant. Trois critères de sortie.

- **Écart nul** : comme pour le site et la caisse, la cascade tombe exactement sur la référence. C'est le cas idéal.
- **Écart expliqué mais non nul** : une cause connue ne peut pas être chiffrée précisément (les remises avant octobre, si l'on n'avait pas eu la base). On l'écrit, avec une **fourchette**.
- **Écart tolérable** : sous un seuil fixé d'avance (3.4.2), et dont les causes sont de même nature que les arrondis.

Il y a aussi des raisons de **ne pas** s'arrêter : un écart qui change de signe d'un mois à l'autre, une cause dont le poids grandit, un « divers » qui dépasse le seuil. Dans tous ces cas, la réconciliation n'est pas finie.

Terminez toujours par une **note de réconciliation** d'une demi-page : les sources et leurs dates d'extraction, la mesure et le périmètre, les trois temps avec leurs chiffres, la cascade, les décisions prises (« les montants du site sont divisés par cent à partir du 15 septembre, à faire confirmer »), les exceptions restantes avec un responsable. C'est cette note, bien plus que le code, qui permet à quelqu'un d'autre de **refaire et de croire** votre résultat (le chapitre 4 de ce volume en fait un document type).

À retenir de la section 3.3.

- Réconcilier, c'est comparer deux sources sur **la même mesure, le même périmètre**, et **expliquer chaque écart** par des causes chiffrées.
- La méthode : **compter, sommer, expliquer** ; la cascade rend la troisième étape lisible, et doit tomber **exactement** sur la référence.
- Les causes fréquentes : unité, doublons, lignes parasites, valeurs manquantes, définitions, calendrier, arrondis, remises, clés mal fabriquées.
- Sans clé commune, on **fabrique** une clé (normalisation, prix, rang) et l'on distingue appariements certains, ambigus et non appariés.
- Une réconciliation se termine par une **note** : les chiffres, les décisions, les exceptions et leur responsable.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.5 à 3.7, exercices 3.10 et 3.11.

3.4 ⊕ Pour aller plus loin : règles de réconciliation, seuils de tolérance, rapports d'exceptions

Section complémentaire. Elle ne change pas la méthode de 3.3 ; elle l'**industrialise**. Quand une réconciliation est faite une fois, on se contente d'un cahier d'analyse. Quand elle se répète chaque mois, il faut des règles écrites, des seuils décidés d'avance et un rapport d'exceptions que d'autres personnes peuvent traiter sans vous. La suite du volume n'en dépend pas.

3.4.1 Écrire une règle de réconciliation

Une règle de réconciliation est un **contrat** entre deux sources. Elle se décrit en huit champs.

Champ	Question	Exemple (caisse contre base)
Identifiant	comment l'appeler ?	R3
Mesure	que compare-t-on ?	chiffre d'affaires toutes taxes comprises, après remises
Source A, source B	qui est la référence ?	A : fichiers de la caisse ; B : base (référence)
Périmètre	quelles lignes ?	canal Boutique, année 2025
Clé	comment appairer ?	ticket + article + quantité + prix + rang
Tolérance	quel écart accepte-t-on ?	1 € sur le total, 0,01 € par ligne
Gravité	que se passe-t-il en cas d'écart ?	erreur : on ne publie pas le chiffre du mois
Propriétaire	qui traite l'écart ?	équipe caisse

Écrire ces huit champs avant d'exécuter la règle oblige à trancher des questions qui, sinon, se règlent à la louche (« le site et la caisse sont proches, c'est bon »). Deux règles du même tableau ne se contredisent pas, parce que chacune a son périmètre et sa référence.

3.4.2 Les seuils de tolérance

Un écart nul est rare dès que les sources n'appliquent pas les mêmes arrondis ; il faut donc décider **à partir de quel écart on s'inquiète**. Trois façons de fixer un seuil.

- **Absolu** : un écart de plus de 1 € sur le total, de plus d'un centime sur une ligne. Adapté aux montants en euros et aux totaux d'un ordre de grandeur connu.
- **Relatif** : un écart de plus de 0,5 % du total de référence. Adapté quand l'ordre de grandeur varie (un mois de décembre pèse deux fois un mois de février).
- **Combiné** : on accepte l'écart s'il est inférieur au **plus grand** des deux seuils (un euro **ou** 0,01 %), ce qui évite qu'un seuil relatif devienne ridiculement petit pour un tout petit total.

On distingue aussi la tolérance **par ligne** (chaque montant concorde au centime) et la tolérance **sur le total** (la somme concorde). Les deux ne disent pas la même chose : un total peut concorder parce que les erreurs de signes opposés **se compensent**. La règle d'or est de vérifier les deux, et de regarder aussi la somme des écarts **en valeur absolue** : si elle est bien plus grande que la somme algébrique, des erreurs se compensent et méritent un regard.

```
def dans_la_tolerance(a, b, abs_tol=0.01, rel_tol=0.0):
    """vrai si |a - b| ne dépasse pas la plus grande des deux tolérances"""
    return abs(a - b) <= max(abs_tol, rel_tol * abs(b))

lu_t, ref_t = caisse["montant"].sum(), base_b["montant"].sum()
print("somme brute :", dans_la_tolerance(lu_t, ref_t, abs_tol=1.0), "| à 0,5 % près :",
      ↪ dans_la_tolerance(lu_t, ref_t, abs_tol=1.0, rel_tol=0.005))
```

```
somme brute : False | à 0,5 % près : False
```

La somme brute de la caisse ne passe **aucun** des deux seuils : l'écart est de 2,33 % du total, plus de quatre fois la tolérance de 0,5 %. Reprenons l'exemple du site (3.3.3) : les écarts d'arrondi entre le total d'en-tête et la somme des lignes valent 0,078 € en somme algébrique et 3,09 € en valeur absolue. Cet écart entre les deux sommes (la somme algébrique est bien plus petite que la somme des valeurs absolues) dit que les erreurs sont de signes opposés et se compensent, comme pour des arrondis : c'est le comportement d'un **bruit**, pas d'un **biais**. Si les deux sommes avaient été égales, tous les écarts auraient eu le même signe, et il aurait fallu chercher une cause **systématique** (une règle d'arrondi différente d'une source à l'autre, par exemple).

Piège : une tolérance qui avale un biais. Un seuil large rend la réconciliation facile et inutile. Fixez les seuils **avant** de voir les écarts, **justifiez-les** (par exemple : un demi-centime d'arrondi par ligne, multiplié par la racine du nombre de lignes) et gardez un œil sur la **tendance** : un écart de 0,3 % chaque mois, toujours dans le même sens, est un biais même s'il est sous le seuil.

3.4.3 Appliquer les règles : avant et après nettoyage

Mettons les règles au travail sur les trois paires de la section précédente. Chaque règle est évaluée **avant** puis **après** les corrections de 3.3, ce qui donne un tableau de bord de réconciliation :

```
regles = [("R1", "CA du site (toutes commandes)", t_export.sum(), ca_base, 1.0, 0.0,
          ↪ propre["eur"].sum()),
          ("R2", "Commandes du site", len(site), len(base_site), 0, 0.0, len(propre)),
```

```

    ("R3", "CA de la caisse", lu_t, ref_t, 1.0, 0.0, tab_c["cumul"].iloc[-1]),
    ("R4", "Lignes de la caisse", len(caisse), len(base_b), 0, 0.0, int((r["_merge"] ==
↳ "both").sum()))),
    ("R6", "Arrondis du site (somme algébrique)", float(diff.sum()), 0.0, 1.0, 0.0,
↳ float(diff.sum()))]
res = pd.DataFrame([{"règle": i, "libellé": n, "A": round(a, 2), "B": round(b, 2), "avant": "OK" if
↳ dans_la_tolerance(a, b, at, rt) else "ÉCART",
    "après": "OK" if dans_la_tolerance(ap, b, at, rt) else "ÉCART"} for i, n, a, b,
↳ at, rt, ap in regles])
print(res.to_string(index=False))

```

règle	libellé	A	B	avant	après
R1	CA du site (toutes commandes)	25012599.35	617715.45	ÉCART	OK
R2	Commandes du site	6259.00	6078.00	ÉCART	OK
R3	CA de la caisse	547896.42	560973.91	ÉCART	OK
R4	Lignes de la caisse	12678.00	12611.00	ÉCART	OK
R6	Arrondis du site (somme algébrique)	0.08	0.00	OK	OK

Avant nettoyage, 1 règle sur 5 est respectée (l'arrondi, qui n'a rien à nettoyer) ; après, les 5 le sont. C'est le résultat attendu : le nettoyage de 3.3 a **résolu** les écarts de comptage et de total. La règle R5 du catalogue (taux d'appariement) n'est pas dans ce tableau parce qu'elle n'a pas d'écart numérique : elle mesure un **taux** que la normalisation simple ne suffit pas à amener au seuil, et reste ouverte jusqu'à l'appariement approximatif (section 2.5).

Pour la tolérance par ligne, on contrôle que chaque montant lu en caisse coïncide, au centime, avec la base :

```

both = r[r["_merge"] == "both"].dropna(subset=["montant"])
ecart_ligne = (both["montant"] - both["montant_base"]).abs()
print("lignes comparées :", len(both), "| au-delà d'un centime :", int((ecart_ligne > 0.01).sum()),
↳ "| écart maximal :", round(float(ecart_ligne.max()), 4))

```

```
lignes comparées : 12213 | au-delà d'un centime : 0 | écart maximal : 0.0
```

Sur 12213 lignes comparées, aucune ne sort de la tolérance d'un centime (0) : quand la caisse a un montant, il est **juste** ; ce sont les montants **absents** qui posaient problème.

3.4.4 Le rapport d'exceptions

Une réconciliation industrielle produit deux livrables : le **tableau de bord** des règles (ci-dessus) et la **liste des exceptions**, c'est-à-dire des lignes ou des lots de lignes qui violent une règle et demandent une action. Une exception comporte au minimum :

- **où** : la source et la référence (fichier et ticket, numéro de commande, code fournisseur) ;
- **quoi** : la nature de l'écart (copie, montant vide, test, désignation non reconnue) ;
- **combien** : le montant en jeu, quand il existe ;
- **pourquoi** : la cause probable ;
- **qui** : le propriétaire, la personne qui peut corriger à la source ;
- **où en est-on** : le statut (à traiter, en cours, corrigée à la source, acceptée).

On assemble les exceptions de nos trois paires dans une table commune :

```

def exceptions(source, nature, ref, montant, proprietaire):
    return pd.DataFrame({"source": source, "nature": nature, "reference": ref, "montant": montant,
↳ "proprietaire": proprietaire, "statut": "à traiter"})

rapport_exc = pd.concat([
    exceptions("Caisse", "copie de scan", copies["fichier"] + " / " + copies["ticket"],
↳ copies["montant"], "équipe caisse"),

```

```

exceptions("Caisse", "montant vide", vides["fichier"] + " / " + vides["ticket"],
↳ vides["montant_base"], "équipe caisse"),
exceptions("Site", "copie d'export", site.loc[copie, "order_ref"], t_eur[copie], "équipe web"),
exceptions("Site", "commande de test", site.loc[test, "order_ref"], t_eur[test], "équipe web"),
exceptions("Catalogue", "désignation non reconnue", reste["code_fournisseur"], np.nan,
↳ "achats"]], ignore_index=True)
print(rapport_exc.sort_values("montant", ascending=False).head(5).to_string(index=False))

```

source	nature	reference	montant	proprietaire	statut
Caisse	copie de scan	caisse_2025-04.csv / T26399	485.76	équipe caisse	à traiter
Caisse	montant vide	caisse_2025-05.csv / T27879	364.32	équipe caisse	à traiter
Site	copie d'export	WEB-025612	287.60	équipe web	à traiter
Site	copie d'export	WEB-028792	279.38	équipe web	à traiter
Site	copie d'export	WEB-031582	274.31	équipe web	à traiter

Le rapport compte 686 exceptions, dont 645 ont un montant. Chacune est **actionnable** : un propriétaire et une référence suffisent pour que quelqu'un d'autre que vous la traite. Notez ce que le rapport **ne contient pas** : les montants en centimes du site. Ce n'est pas une exception ligne à ligne mais un **défaut de lot** (2518 lignes d'un coup), qui se traite en une fois, à la source, par une seule décision (« exporter en euros »). Un bon rapport sépare les **défauts systémiques**, qui se corrigent en amont, des **exceptions individuelles**, qui se traitent une à une.

3.4.5 Trier : où est l'argent ?

Quand les exceptions sont nombreuses, on ne les traite pas dans l'ordre d'arrivée mais dans l'ordre de leur **poids**. Une exception a deux poids : son **montant** et son **effectif**. On les résume par nature :

```
print(par_nature.sort_values("montant", ascending=False).round(2).to_string(index=False))
```

source	nature	lignes	montant
Caisse	montant vide	398	16203.78
Site	copie d'export	121	12051.53
Caisse	copie de scan	67	3126.29
Site	commande de test	60	36.24
Catalogue	désignation non reconnue	40	NaN

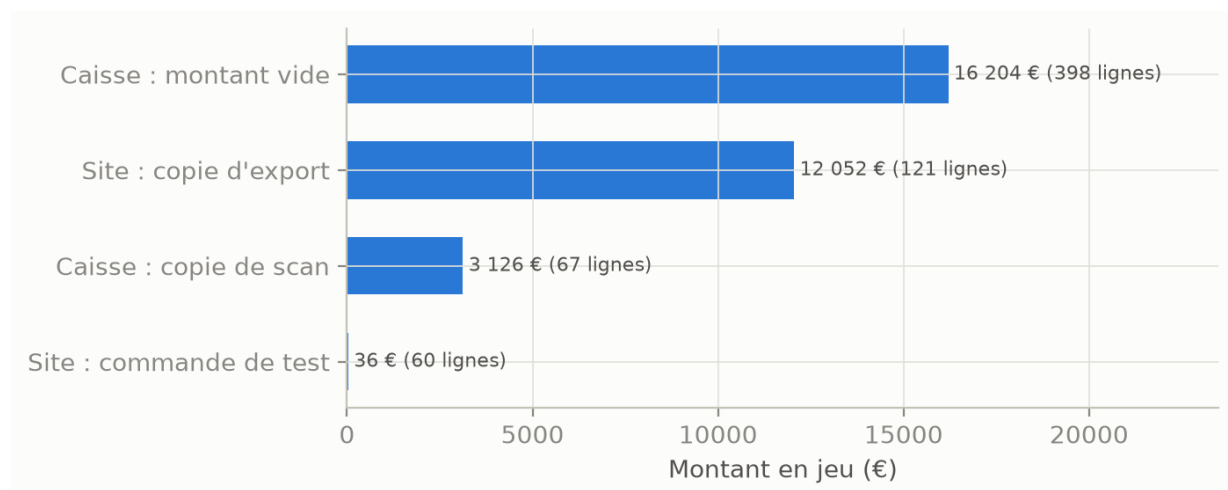


Figure 20: Exceptions de réconciliation par nature : montant en jeu et nombre de lignes. Les exceptions du catalogue n'ont pas de montant et ne figurent pas.

Les montants vides représentent 52 % du montant des exceptions chiffrées, les copies d'export du site 38 % : deux natures pèsent presque tout, et traiter ces deux-là épuise l'essentiel de l'enjeu

financier. Les 40 désignations du catalogue sans correspondant n'ont **pas** de montant, mais elles bloquent un autre usage (le suivi des coûts d'achat) : le critère de priorité ne se réduit pas aux euros.

Intuition. On retrouve la règle de Pareto : une ou deux natures d'exception portent la plus grande part de l'enjeu. Cherchez-les d'abord, mais n'oubliez pas qu'une exception peu coûteuse **aujourd'hui** peut être le signal avant-coureur d'un défaut plus large (les commandes de test, anodines en euros, révélaient des orphelins en 3.2.4).

3.4.6 Résoudre et suivre

Une exception n'est pas résolue quand on l'**exclut** de l'analyse, mais quand sa **cause** est traitée. Un cycle de vie sobre suffit :

Statut	Signification	Qui décide
À traiter	détectée, personne ne s'en est encore occupé	l'analyste
En cours	un propriétaire a pris le dossier	le propriétaire
Corrigée à la source	la donnée est corrigée là où elle est produite	le propriétaire
Acceptée	l'écart est connu et tolérable, avec justification écrite	la gérante, avec l'analyste

Trois habitudes rendent le suivi durable.

1. **Remonter à la cause racine.** Corriger 121 copies d'export ne sert à rien si l'export en produit 121 nouvelles le mois suivant. Pour chaque nature, demandez : *que faut-il changer à la source pour que cette exception ne revienne pas ?* (une clé unique à l'export, un contrôle de rupture d'unité, un formulaire qui force le format de date).
2. **Mesurer le stock et l'âge des exceptions.** Le nombre d'exceptions ouvertes et leur ancienneté sont des indicateurs de santé : un stock qui grossit est un symptôme.
3. **Rejouer à chaque livraison.** Les règles tournent à chaque nouvel export, et les résultats s'archivent : on voit si un écart revient, et l'on peut prouver, trois mois plus tard, qu'il était connu.

À retenir de la section 3.4.

- Une règle de réconciliation s'écrit en **huit champs** (mesure, sources, périmètre, clé, tolérance, gravité, propriétaire) **avant** d'être exécutée.
- Une tolérance se fixe **à l'avance** : absolue, relative ou combinée, par ligne **et** sur le total ; comparer la somme des écarts et la somme de leurs valeurs absolues distingue un bruit d'un biais.
- Un rapport d'exceptions donne pour chaque exception **où, quoi, combien, pourquoi, qui, où en est-on** ; on sépare les **défauts de lot**, traités à la source, des exceptions individuelles.
- On trie par poids (montant et effectif) et l'on remonte à la **cause racine** pour que l'exception ne revienne pas.

Pour s'entraîner. Cahier, chapitre 3 : application 3.8, exercice 3.12.

3.5 ⊕ Pour aller plus loin : cadres de validation, pandera et Great Expectations

Section complémentaire. Jusqu'ici nous avons écrit nos contrôles à la main, avec des fonctions pandas. C'est la bonne façon de **comprendre** ; ce n'est pas toujours la bonne façon de **tenir** un pipeline qui tourne chaque semaine. Deux bibliothèques libres, **pandera** et **Great Expectations**, permettent de **déclarer** les règles plutôt que de les programmer. La suite du volume n'en dépend pas.

3.5.1 Pourquoi un cadre de validation ?

Des fonctions maison présentent trois faiblesses à mesure que les contrôles se multiplient. Elles **se dispersent** (chaque analyste écrit les siennes, avec ses conventions) ; elles **se lisent mal** (une règle est noyée dans le code qui l'applique) ; et elles **produisent des rapports hétérogènes** (ici un tableau, là un message). Un cadre de validation apporte :

- un **langage déclaratif** : on décrit ce que les données doivent être (« le code postal a cinq chiffres »), pas comment le vérifier ;
- un **schéma réutilisable** : la même description sert pour chaque livraison, chaque mois, chaque fichier ;
- un **rapport standard** : combien de règles tenues, lesquelles ont échoué, avec quels exemples ;
- une **intégration** dans les chaînes de traitement : un contrôle qui échoue arrête le traitement.

Intuition. Passer de fonctions maison à un cadre, c'est passer d'une **liste de courses griffonnée** à un **cahier des charges**. La liste suffit tant qu'on est seul et que les courses sont rares ; le cahier des charges permet de déléguer et de répéter.

3.5.2 pandera : un schéma de DataFrame

pandera est la plus légère des deux : une bibliothèque Python qui décrit le **schéma** attendu d'un DataFrame (colonnes, types, contrôles) et le valide. Voici le schéma des quatre colonnes du CRM que nous avons contrôlées à la main :

```
import pandera.pandas as pa
schema_crm = pa.DataFrameSchema({
    "id_crm": pa.Column(str, unique=True),
    "email": pa.Column(str, pa.Check.str_matches(r"^" + 0.RE_EMAIL + r"$"), nullable=True),
    "code_postal": pa.Column(str, pa.Check.str_matches(r"^\d{5}$"), nullable=True),
    "consentement_marketing": pa.Column(str, pa.Check.isin(["oui", "non"]), nullable=True),
})
try:
    schema_crm.validate(crm, lazy=True)
except pa.errors.SchemaErrors as e:
    echecs_pa = e.failure_cases
print(echecs_pa.groupby("column").size().to_string())
```

column	
code_postal	199
consentement_marketing	2428
email	95

Trois choses à noter. Le schéma tient en quelques lignes lisibles : **il se lit comme une spécification**.

L'option `nullable=True` dit qu'une valeur absente est permise : on retrouve la séparation entre complétude et validité de 3.2.2. Et l'option `lazy=True` demande à pandera de **tout vérifier** avant de signaler les échecs, au lieu de s'arrêter au premier : sans elle, on ne verrait qu'un seul défaut à la fois. Les comptes (95 e-mails, 199 codes postaux, 2428 consentements) sont exactement ceux de nos fonctions maison (3.2.2) : le cadre n'invente rien, il **range**.

Piège : les motifs sont « ancrés » ou non. La vérification `str_matches` de pandera cherche le motif **au début** de la valeur (elle ne vérifie pas la fin) : `\d{5}` accepterait « 12345678 ». On ancre le motif avec `^` et `$`, comme ci-dessus. Les cadres ne vous dispensent pas de comprendre ce que fait une règle.

Le tableau `failure_cases` (ici `echecs_pa`) liste chaque échec avec sa colonne, la règle, la valeur fautive et l'**index** de la ligne : c'est le rapport d'échecs de 3.2.6, produit sans effort. Les règles **entre colonnes** se déclarent au niveau du `DataFrame` :

```
schema_mont = pa.DataFrameSchema(
    {"quantite": pa.Column(int, pa.Check.ge(1)), "prix_unitaire": pa.Column(float, pa.Check.gt(0)),
    ↪ "montant": pa.Column(float, pa.Check.gt(0))},
    checks=pa.Check(lambda d: d["montant"] <= d["quantite"] * d["prix_unitaire"] + 0.01, name="montant
    ↪ ≤ qté × prix"), strict=False)
try:
    schema_mont.validate(mont, lazy=True)
except pa.errors.SchemaErrors as e:
    echecs_m = e.failure_cases
print(echecs_m.groupby("check")["index"].nunique().to_string())
```

```
check
greater_than(0)          32
montant ≤ qté × prix     53
```

La règle de dépendance (`montant ≤ qté × prix`) et la règle de plage (`montant > 0`) trouvent 53 et 32 lignes en échec : à elles deux, 85 lignes, soit les 85 erreurs de saisie connues, ni plus ni moins. Le cadre retrouve exactement ce que nos fonctions avaient trouvé, avec une syntaxe qui se lit comme un cahier des charges.

Le vrai gain de pandera apparaît quand **le même schéma sert plusieurs fois**. Les douze fichiers de la caisse ont le même contenu logique : on écrit le schéma une fois, on le passe sur chaque fichier.

```
schema_caisse = pa.DataFrameSchema({"qte": pa.Column(int, pa.Check.in_range(1, 20)), "prix_unitaire":
    ↪ pa.Column(float, pa.Check.gt(0)),
    "montant": pa.Column(float, pa.Check.gt(0), nullable=False)},
    ↪ strict=False)
manquants = {}
for nom, bloc in caisse.groupby("fichier"):
    try:
        schema_caisse.validate(bloc, lazy=True)
    except pa.errors.SchemaErrors as e:
        manquants[nom[-6:-4]] = int(e.failure_cases["index"].nunique())
print(manquants)
```

```
{'01': 36, '02': 22, '03': 28, '04': 31, '05': 31, '06': 30, '07': 21, '08': 20, '09': 31, '10': 38,
    ↪ '11': 48, '12': 63}
```

Ici nous avons déclaré le montant **non nullable** : chaque montant vide devient un échec, et la boucle compte 399 lignes en échec sur 12 fichiers : ce sont les 398 montants vides de la section 3.3.4, plus un montant vide dans une copie de scan. Un seul schéma, douze fichiers, un compte par fichier : c'est l'esprit d'un contrôle de livraison.

Pour s'entraîner. Cahier, chapitre 3 : application 3.9.

3.5.3 Great Expectations : des attentes, des suites, des rapports

Great Expectations (souvent abrégé GX) est plus riche et plus lourd. Il part de la même idée, décrire les **attentes** que l'on a sur les données, mais organise tout autour de quelques objets : une **source de données** (un DataFrame, un fichier, une base), une **attente** (*expectation* : « les valeurs de cette colonne respectent ce motif »), une **suite d'attentes** qui regroupe plusieurs règles, une **définition de validation** qui relie une suite à des données, et, enfin, des **rapports** (les *Data Docs*) lisibles par des non-spécialistes.

Le plus petit exemple valide **une** attente sur un lot de données :

```
import great_expectations as gx
with 0.silencieux():
    ctx = gx.get_context(mode="ephemeral")
    lot_def = ctx.data_sources.add_pandas("boutique").add_dataframe_asset("crm")
    ↪ .add_batch_definition_whole_dataframe("tout")
    lot = lot_def.get_batch(batch_parameters={"dataframe": "crm"})
    res = lot.validate(gx.expectations.ExpectColumnValuesToMatchRegex(column="code_postal",
    ↪ regex=r"^\d{5}$"))
print("succès :", res.success, "| valeurs anormales :", res.result["unexpected_count"], "sur",
    ↪ res.result["element_count"], "lignes")
```

```
succès : False | valeurs anormales : 199 sur 7140 lignes
```

Le contexte `ephemeral` vit en mémoire et ne laisse aucun fichier ; c'est le mode des expérimentations. L'attente échoue (199 valeurs anormales : les codes à quatre chiffres), et le résultat dit combien.

Piège : un compte qui n'a pas le même dénominateur. Great Expectations rapporte le pourcentage d'anormales **parmi les valeurs renseignées**, pas parmi toutes les lignes : 199 anormales donnent 3,0 % des codes postaux renseignés, et 2,8 % de l'ensemble des lignes. Quand on compare avec nos propres indicateurs, il faut s'assurer qu'on divise par la même chose.

Une **suite** regroupe les attentes que l'on veut vérifier ensemble ; l'option `mostly` fixe la **proportion minimale** de valeurs conformes pour que l'attente soit tenue (par exemple 99 %), ce qui traduit les seuils de tolérance de 3.4.2 :

```
E = gx.expectations
with 0.silencieux():
    suite = ctx.suites.add(gx.ExpectationSuite(name="crm"))
    for e in [E.ExpectColumnValuesToNotBeNull(column="email", mostly=0.95),
    ↪ E.ExpectColumnValuesToMatchRegex(column="code_postal", regex=r"^\d{5}$", mostly=0.99),
        E.ExpectColumnValuesToBeInSet(column="consentement_marketing", value_set=["oui", "non"],
    ↪ mostly=0.9), E.ExpectColumnValuesToBeUnique(column="id_crm")]:
        suite.add_expectation(e)
    definition = ctx.validation_definitions.add(gx.ValidationDefinition(name="crm_def", data=lot_def,
    ↪ suite=suite))
    resultat = definition.run(batch_parameters={"dataframe": "crm"})
print("attentes tenues :", resultat.statistics["successful_expectations"], "sur",
    ↪ resultat.statistics["evaluated_expectations"])
for x in resultat.results:
    print(f"{x.expectation_config.type:42s} {'OK' if x.success else 'KO'}
    ↪ {x.result['unexpected_percent']:.1f} % d'anormales")
```

```
attentes tenues : 2 sur 4
expect_column_values_to_not_be_null      OK  3.2 % d'anormales
```

expect_column_values_to_match_regex	KO	3.0 % d'anormales
expect_column_values_to_be_in_set	KO	48.8 % d'anormales
expect_column_values_to_be_unique	OK	0.0 % d'anormales

Sur quatre attentes, 2 sont tenues : l'e-mail est suffisamment renseigné (moins de 5 % d'absents), les identifiants sont uniques, mais le code postal (trop de codes à quatre chiffres) et le consentement (trop d'écritures hors liste) échouent. Les valeurs de `mostly` (95 %, 99 %, 90 %) sont des choix d'usage, comme les seuils du tableau de bord de 3.1.7.

3.5.4 Les Data Docs

L'intérêt principal de Great Expectations est de produire un **rapport HTML** que l'on peut ouvrir et partager sans lire une ligne de code : les *Data Docs*. Voici ce que donne la même suite, dans un projet GX complet. Il s'agit d'une **vraie capture d'écran** (faite avec un navigateur sans interface) de la page de résultat produite par la bibliothèque sur nos données.

Home / Validations / crm / __none__ / 2026-10-07T16:38:08Z

Expectation Validation Result

Evaluates whether a batch of data matches expectations.

Actions

Validation Filter:

Show All Failed Only

Table of Contents

- Overview
- code_postal
- consentement_marketing
- email
- id_crm

Overview

Expectation Suite: [crm](#)

Data asset: None

Status: ✗ Failed

Statistics

Evaluated Expectations	4
Successful Expectations	2
Unsuccessful Expectations	2
Success Percent	50%

[Show more info...](#)

code_postal

Search

Status	Expectation	Observed Value
✗	values must match this regular expression: <code>^\d{5}\$</code>, at least 99 % of the time. 199 unexpected values found. ≈2.963% of 7140 total rows. Sampled Unexpected Values 5723	≈2.9631% unexpected

Figure 21: Page de résultat de validation de Great Expectations (Data Docs) pour le CRM de la boutique : quatre attentes évaluées, deux tenues, deux échouées ; pour la première attente en échec (le code postal), le nombre de valeurs anormales et un échantillon. Capture réelle d'un logiciel libre (licence Apache 2.0) exécuté pour ce livre ; le logo, qui se charge depuis Internet, a été masqué.

On y lit en haut le **statut** global et les statistiques de la suite ; en dessous, **chaque attente**, avec sa description en langage naturel, le nombre et le pourcentage de valeurs anormales, et un **échantillon** de ces valeurs. À gauche, un sommaire par colonne. C'est le format que l'on peut déposer sur un espace partagé pour que la gérante, ou l'équipe du site, consulte l'état des données **sans vous**.

3.5.5 Choisir : fonctions maison, pandera ou Great Expectations ?

	Fonctions maison	pandera	Great Expectations
Mise en route	immédiate	une dépendance, quelques minutes	plusieurs objets à configurer, plus lourd
Lisibilité des règles	dépend de l'auteur	schéma déclaratif , très lisible	attentes nommées, très lisibles
Rapport	à écrire	tableau d'échecs détaillé	rapport HTML partageable
Seuils (« au moins 99 % »)	à écrire	possible	natif (mostly)
Intégration à un pipeline	libre	tests dans le code	jalons, actions, orchestrateurs
Quand l'adopter	explorations, un seul fichier	chaîne récurrente en Python , validation de schéma en entrée	plusieurs équipes, rapports à partager, nombreuses sources

Quelques repères de bon sens. Pour **explorer** un fichier inconnu, les fonctions maison suffisent et se lisent mieux. Pour **fiabiliser un traitement récurrent écrit en Python**, pandera donne le meilleur rapport simplicité/valeur. Pour **partager l'état des données avec des non-programmeurs**, ou gérer beaucoup de sources, Great Expectations rentabilise sa complexité. Dans tous les cas, **l'outil ne remplace pas la réflexion** : ce que l'on contrôle (les règles), les seuils, la gravité et ce qu'on fait d'un échec restent des décisions humaines, écrites dans la documentation des données (chapitre 4).

À retenir de la section 3.5.

- Un cadre de validation **déclare** les règles (schéma, attentes) au lieu de les programmer, les rend **réutilisables** et produit un **rapport** standard.
- **pandera** valide un DataFrame contre un schéma ; `lazy=True` rapporte tous les échecs ; un même schéma sert pour chaque livraison.
- **Great Expectations** organise sources, attentes, suites et rapports (Data Docs) ; `mostly` traduit un seuil de tolérance.
- Attention aux **pièges de lecture** : motifs non ancrés, dénominateur des pourcentages.
- On choisit selon l'usage : fonctions maison pour explorer, pandera pour un traitement récurrent, Great Expectations pour partager et passer à l'échelle.

Pour s'entraîner. Cahier, chapitre 3 : application 3.9.

Bilan du chapitre 3

Vous savez maintenant :

- **mesurer la qualité d'un fichier** selon six dimensions (complétude, validité, unicité, cohérence, exactitude, actualité), chacune par un **pourcentage de lignes conformes** (ou un retard en jours), et les assembler en un **tableau de bord** par source et par usage ;
- **juger un indicateur** : il ne mesure que ce qu'il sait voir (les doublons reconnus par l'e-mail ne sont que 677 sur 1 000), et il peut s'exciter à tort (les copies exactes en caisse : 154 signalées pour 67 vraies) ;

- **écrire des contrôles de validation** comme de petites fonctions qui renvoient les lignes en échec : forme (type, format, liste), plage, **dépendance entre colonnes**, unicité, **totaux de contrôle**, comptages entre tables, **contrôles dans le temps** ; en rassembler les résultats en un rapport, les écrire aussi en **SQL** (contraintes et requêtes de contrôle) ;
- **décider quoi faire d'un échec** : bloquer, mettre en quarantaine, corriger ou avertir, sans jamais corriger en silence ;
- **réconcilier deux sources** en trois temps (compter, sommer, expliquer), construire une **cascade** qui tombe exactement sur la référence, fabriquer une clé quand il n'y en a pas, et distinguer appariements certains, ambigus et non appariés ;
- (en option) **écrire des règles de réconciliation** avec leurs seuils de tolérance, tenir un **rapport d'exceptions** (où, quoi, combien, pourquoi, qui, où en est-on) et le **prioriser** ;
- (en option) **déclarer** ses contrôles avec **pandera** ou **Great Expectations**, et lire un rapport *Data Docs*.

Le tableau suivant résume **ce que nous avons mesuré** sur les fichiers de la boutique.

Question	Résultat mesuré
Fiches du CRM avec e-mail, code postal et consentement	63,8 %
CRM : lignes utilisables pour une lettre d'information, pour une répartition par âge	58,1 % ; 88,4 %
Montants saisis : règle de plage (1,5 écart interquartile) contre règle de dépendance	52 vraies et 351 fausses alertes ; 85 vraies et 0 fausse alerte
Site : rupture d'unité détectée	le 15/09/2025, montant médian multiplié par 147
Site : somme brute de l'export contre base, écart expliqué par une cascade	25 012 599 € contre 617 715,45 € ; écart restant 0,0 €
Caisse : somme lue contre total affiché, écart expliqué par une cascade	547 896,42 € contre 560 973,91 € ; écart restant 0,0 €
Caisse : copies de scan trouvées par rapprochement ligne à ligne	67 lignes, retrouvées fichier par fichier
Catalogue : codes appariés par nom + prix, ambigus, non appariés	78 ; 2 ; 40

Le fil conducteur du chapitre tient en une phrase : **la qualité se mesure, se contrôle et se réconcilie, et chaque écart doit pouvoir s'expliquer à l'euro près**. Trois habitudes à retenir :

En pratique : trois habitudes.

1. **Mesurez avant de corriger**. Un tableau de bord par source et par dimension dit quoi réparer en premier ; un indicateur est un instrument, pas la vérité.
2. **Écrivez le contrôle, pas seulement la correction**. Un contrôle qui renvoie ses lignes en échec, rejoué à chaque livraison, détecte la rupture du 15 septembre ; une correction faite à la main, non.
3. **Ne vous arrêtez pas à « à peu près »**. Une réconciliation est finie quand l'écart est expliqué par des causes chiffrées ou inférieur à une tolérance fixée d'avance, et qu'une note le consigne.

Rappel d'honnêteté. Les fichiers sont **simulés** et leur « vérité » est connue : c'est ce qui nous a permis de dire que les doublons étaient au nombre de 1 000 et que les copies de la caisse étaient bien celles-là. Dans une étude réelle, vous n'aurez pas cette vérité : vous aurez des questions à poser aux équipes qui produisent les données, et des écarts que l'on explique ou que l'on **déclare inexpliqués**. Les seuils (98 %, 0,5 %, 1 €) sont des **exemples** à fixer avec les

utilisateurs.

Le chapitre 4 prolonge cette réflexion : tout ce que nous avons décidé (les règles, les seuils, les corrections, les exceptions) doit être **écrit** pour que quelqu'un d'autre puisse le comprendre et le refaire. C'est l'objet de la **documentation des données** et des **dictionnaires de données**.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.9 (mesurer la qualité du CRM, écrire des contrôles, contrôles de plage et de dépendance, détecter une rupture, réconcilier le site, la caisse et le catalogue, tolérances et exceptions, pandera et Great Expectations) et exercices 3.1 à 3.12.

Chapitre 4 : Documentation et dictionnaires de données

« Un chiffre que personne ne peut refaire est un chiffre que personne ne peut défendre.
»

La gérante vous arrête devant la machine à café, un dossier à la main. « Ma collègue de la comptabilité a repris ton analyse du quatrième trimestre pour le bilan. **Elle ne retrouve pas ton chiffre.** Tu m'avais dit 211 434 € pour le Site, elle m'en annonce un autre. Qu'est-ce qui manque ? »

Vous reprenez la question avec calme, parce que vous savez qu'il y a deux réponses possibles. Ou bien l'un de vous deux s'est trompé, ou bien — c'est le cas le plus fréquent — **vous avez tous deux raison**, mais vous ne parlez pas du même chiffre. Faisons l'expérience sur les données de la boutique : quatre calculs honnêtes du « chiffre d'affaires du quatrième trimestre pour le canal Site », qui ne diffèrent que par un choix que personne n'a écrit.

```
site_t4 = x[(x["canal"] == "Site") & (x["date_commande"] >= "2025-10-01")]
variantes = {
    "TTC, remises déduites, jusqu'au 31/12": site_t4["montant"].sum(),
    "hors taxe (TVA fictive de 20 %)": site_t4["montant"].sum() / 1.2,
    "avant remises": (site_t4["quantite"] * site_t4["prix_unitaire"]).sum(),
    "extraction arrêtée au 15/12": site_t4[site_t4["date_commande"] <= "2025-12-15"]["montant"].sum(),
}
for nom, v in variantes.items():
    print(f"{0.eur(v):>14} {nom}")
```

```
211 433,79 € TTC, remises déduites, jusqu'au 31/12
176 194,83 € hors taxe (TVA fictive de 20 %)
214 992,95 € avant remises
166 108,12 € extraction arrêtée au 15/12
```

Quatre chiffres, tous **exacts**, séparés de près de 49 000 € entre le plus bas et le plus haut : la **taxe**, les **remises**, la **date** de l'extraction. Aucune erreur de calcul là-dedans ; ce qui manque, c'est la **phrase** qui dit lequel on a calculé. Voici ce que votre collègue aurait voulu trouver à côté du chiffre.

Ce chapitre vous apprend à répondre à ces six questions **avant** qu'on vous les pose : en **documentant** ce que l'on a reçu (les jeux de données), ce que l'on en a fait (les transformations), ce que veulent dire les colonnes (le dictionnaire) et, pour aller plus loin, **d'où vient chaque chiffre** (le lignage).

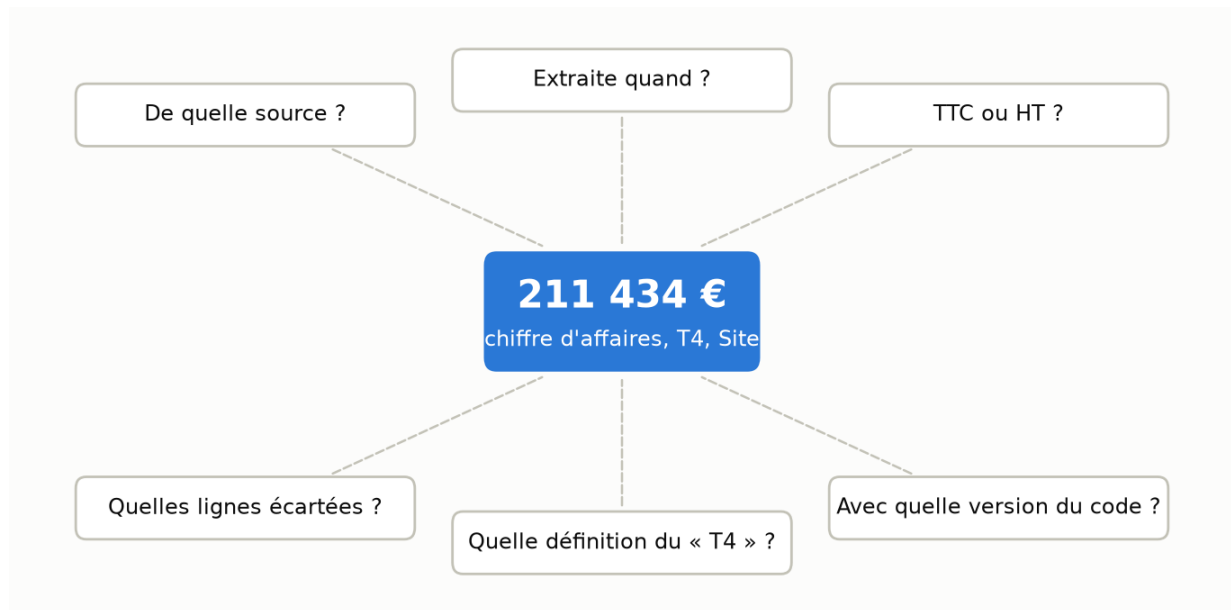


Figure 22: Le chiffre de 211 434 € et les six questions que se pose quelqu'un qui doit le refaire : de quelle source, extraite quand, TTC ou HT, quelles lignes écartées, quelle définition du trimestre, avec quelle version du code.

Pourquoi un analyste documente

Documenter n'est pas de la bureaucratie ; c'est la partie du travail qui **rend le reste réutilisable**. Quatre raisons, que vous rencontrerez dès la première année.

- **Refaire.** Dans trois mois, la gérante vous demandera la même analyse sur le trimestre suivant. Si le travail est écrit, il se **rejoue** en dix minutes ; sinon, il se **refait** en trois jours, avec des résultats légèrement différents sans que l'on sache pourquoi.
- **Comprendre.** Une colonne statut qui contient PAID, paid et Paid ne se comprend pas toute seule. Une colonne total qui change d'unité en septembre non plus. La documentation rend ces pièges **visibles pour la personne suivante**, y compris vous dans six mois.
- **Auditer.** Un chiffre qui va dans un bilan, dans un dossier de banque ou dans une décision de prix peut être contesté. Pouvoir montrer la **chaîne** — la source, les règles, les contrôles — transforme une opinion en démonstration.
- **Transmettre.** Un jour, quelqu'un reprendra votre poste. La documentation est la différence entre une passation de dix minutes et une enquête de dix jours.

Intuition. Documenter, c'est écrire pour **quelqu'un d'intelligent qui n'a pas assisté à la réunion**. Cette personne sait lire du Python, du SQL et des tableaux ; elle ne sait pas ce que **vous** saviez en le faisant. Presque toujours, cette personne, c'est vous dans six mois.

Le chemin de ce chapitre

Le parcours essentiel suit deux étapes, qui correspondent aux deux choses que l'on documente.

- **4.1 Documenter les jeux de données et les transformations** : la fiche d'un jeu de données (d'où vient-il, que représente une ligne, que sait-on de ses défauts), puis le **journal** d'un nettoyage (une règle, une justification, des effectifs avant et après), écrit par le code lui-même.
- **4.2 Construire un dictionnaire de données** : que veut dire chaque colonne (libellé, type, unité, valeurs permises, codage des manquants, sensibilité), comment en fabriquer un squelette automatiquement, comment **vérifier qu'il reste vrai**, et comment s'accorder sur une définition unique de « client actif », de « commande annulée » ou de « panier moyen ».

Une section facultative prolonge ce parcours : **4.3 Lignage des données et pistes d'audit**, pour savoir **d'où vient** un chiffre, prouver qu'une entrée n'a pas changé (les empreintes), tenir un journal d'audit et rejouer un résultat depuis le brut grâce à sa documentation.

Les données du chapitre

Quatre jeux de la boutique. Le chapitre documente des fichiers que vous connaissez déjà ou que vous allez retrouver : `crm_clients.csv` (le CRM, 7 140 lignes, désordonné : il sert à l'exemple de la fiche et du journal), `clients.csv` et `produits.csv` (les référentiels propres du volume I), et `profil_clients.csv` (le profil de 6 000 clients, avec des valeurs manquantes : il sert à l'exemple du dictionnaire). Les commandes (`commandes.csv`, `lignes_commande.csv`) servent à calculer les chiffres que l'on documente. Tous sont **simulés** ; les fichiers `verite_*.csv` ne servent qu'à **juger** nos choix à la fin d'une étude : on ne les ouvre pas pour nettoyer.

Tout le code de ce chapitre est **réellement exécuté** : ce que vous lirez sous un bloc est ce que le code a produit, pas une illustration.

Pour s'entraîner. Le chapitre 4 du cahier propose huit applications (fiches, journaux, dictionnaires, lignage, empreintes) et douze exercices corrigés ; chaque section du livre indique ceux qui la prolongent.

4.1 Documenter les jeux de données et les transformations

Documenter deux choses distinctes : **ce que l'on a reçu** (un jeu de données, avec son histoire et ses défauts) et **ce que l'on en a fait** (une suite de transformations, chacune avec une règle et une justification). La première se consigne dans une **fiche**, la seconde dans un **journal**. Cette section montre les deux, puis comment les ranger pour que quelqu'un d'autre — ou vous, plus tard — puisse tout rejouer depuis le fichier brut.

4.1.1 Ce que l'on documente : une fiche par jeu de données

Un jeu de données n'est pas seulement un tableau : c'est un tableau **avec une histoire**. Deux fichiers identiques en apparence peuvent ne pas vouloir dire la même chose, selon qui les a produits, quand, et avec quel périmètre. La fiche d'un jeu de données (on dit aussi *datasheet*, ou « carte d'identité ») consigne dix informations.

Rubrique	Question à laquelle elle répond	Exemple pour le CRM
Source	Qui l'a produit, avec quel outil ?	export du CRM de la boutique
Date d'extraction	À quelle date l'a-t-on obtenu ?	31 décembre 2025
Périmètre	Qu'est-ce qui est dedans, et qu'est-ce qui n'y est pas ?	clients saisis depuis 2018, doublons compris
Grain	Qu'est-ce qu'une ligne ?	une saisie de fiche, pas un client
Clé	Qu'est-ce qui identifie une ligne ?	id_crm, numéro de saisie
Contenu	Combien de lignes, de colonnes, quelle empreinte ?	mesurés par le code
Limites connues	Quels défauts a-t-on déjà repérés ?	doublons, formats mixtes, lignes de test
Version	Quelle version, quel nom de fichier ?	v1
Propriétaire	Qui répond des questions sur ce jeu ?	le service relation client
Droits et sensibilité	A-t-on le droit de l'utiliser, comment ?	données personnelles, usage interne

Deux de ces rubriques font à elles seules la moitié du travail : le **grain** (qu'est-ce qu'une ligne ?) et la **source** (d'où vient-elle ?). Dans le volume I, vous avez vu qu'oublier le grain est la cause du double comptage ; oublier la source est la cause de la plupart des chiffres qui ne se recoupent pas.

Intuition. La fiche répond à une seule question, posée par quelqu'un qui n'a jamais vu le fichier : « **Puis-je lui faire confiance pour ce que je veux en faire ?** » Si la fiche ne permet pas de répondre, elle est incomplète.

4.1.2 Remplir la fiche du CRM

Une partie de la fiche se **mesure** (le nombre de lignes, de colonnes, l'empreinte du contenu) ; l'autre se **demande** (à la personne qui a fait l'export) ou se **constate** en regardant les données. Mélangeons les deux : la fonction `fiche_jeu` prend les informations saisies à la main et ajoute celles que le tableau livre lui-même.

```
0.fiche_jeu(crm, **{
    "nom": "crm_clients (v1)", "source": "export du CRM de la boutique", "date d'extraction":
        ↪ "2025-12-31",
    "périmètre": "clients saisis depuis 2018, doublons compris", "une ligne =": "une saisie de fiche
        ↪ (pas un client)",
    "clé": "id_crm (numéro de saisie)", "propriétaire": "service relation client",
    "limites connues": "doublons, formats mixtes, lignes de test", "droits": "données personnelles,
        ↪ usage interne"})
```

```
nom          : crm_clients (v1)
source       : export du CRM de la boutique
```

```

date d'extraction      : 2025-12-31
périmètre              : clients saisis depuis 2018, doublons compris
une ligne =            : une saisie de fiche (pas un client)
clé                    : id_crm (numéro de saisie)
propriétaire           : service relation client
limites connues        : doublons, formats mixtes, lignes de test
droits                 : données personnelles, usage interne
lignes                 : 7 140
colonnes               : 11
empreinte du contenu   : 84f642145769

```

Le CRM compte **7 140 lignes** pour **11 colonnes**. L'**empreinte** est un code de douze caractères calculé sur tout le contenu : la même fiche, appliquée plus tard au même fichier, donnera la même empreinte ; si quelqu'un modifie une seule cellule, elle changera (nous y reviendrons en 4.3). C'est un moyen simple de **prouver** qu'on parle du même fichier.

Trois remarques sur cette fiche.

- **La rubrique « limites connues » est la plus utile et la plus négligée.** On y écrit ce que l'on sait déjà, même approximativement : « doublons probables, non mesurés », « montants parfois vides », « dates en formats mixtes ». Une limite signalée est un piège évité pour le lecteur suivant.
- **Le grain n'est pas le client.** La ligne du CRM est une **saisie**, et un client saisi trois fois apparaît trois fois. Écrire « une ligne = une saisie de fiche » évite qu'un collègue compte `len(crm)` comme le nombre de clients.
- **La fiche se range avec le fichier.** Dans le même dossier, sous le nom `crm_clients_v1.md`, ou en tête du dossier du projet : un fichier de données sans sa fiche est un fichier orphelin.

Piège. Ne **déduisez pas** la fiche en regardant le fichier. Le tableau peut vous apprendre le nombre de lignes, pas pourquoi les clients de 2018 y sont ni qui a fait l'export. Posez la question à la source, **par écrit**, et gardez la réponse.

Pour s'entraîner. Cahier, chapitre 4 : application 4.1, exercices 4.1 et 4.2.

4.1.3 Documenter une transformation : le journal des décisions

La fiche décrit le brut. Reste à documenter **ce que l'on en fait**. Un nettoyage de données est une suite de **décisions** : on retire les lignes de test, on met les adresses électroniques en minuscules, on fusionne les doublons. Chacune est discutable, chacune a un effet mesurable. Le **journal des décisions** consigne, pour chaque étape, cinq informations.

1. **La règle** : ce que l'on fait, en une phrase précise (« retirer les lignes dont l'adresse est `test@example.com` »).
2. **La justification** : pourquoi (« ce ne sont pas des clients »). C'est la partie que personne n'écrit, et que tout le monde regrette.
3. **Les effectifs** : combien de lignes **avant**, combien **après**.
4. **Le nombre de lignes touchées** : retirées, ou modifiées sans être retirées.
5. **Une empreinte du résultat** : pour pouvoir vérifier, plus tard, qu'en rejouant les étapes on retombe sur le **même tableau**.

Ces effectifs ne sont pas décoratifs : ils obéissent à une **équation de conservation**.

L'équation de conservation. Pour toute suite d'étapes qui ne fait que retirer ou modifier des lignes, le nombre de lignes lues, moins le nombre de lignes retirées à chaque étape, est le nombre de lignes finales : $N_{\text{lues}} - \sum_k r_k = N_{\text{finales}}$. Si ce calcul ne tombe pas juste, une étape a **perdu** ou **inventé** des lignes sans le dire. C'est le contrôle le plus simple et le plus puissant d'un nettoyage.

Ce qui rend le journal précieux, c'est qu'il transforme une affirmation (« j'ai nettoyé le CRM ») en une **suite vérifiable** : chaque ligne du journal peut être rejouée, contestée et corrigée séparément.

4.1.4 Un journal écrit par le code

Tenir ce journal à la main est fastidieux et fragile : on oublie une étape, on recopie mal un effectif. Mieux vaut que **le code l'écrive lui-même**, à chaque étape, au moment où il agit. Le principe tient en quelques lignes : une petite classe Journal dont la méthode `etape` reçoit le tableau avant, le tableau après, la règle et la justification, et consigne le reste automatiquement.

```
j = 0.Journal("crm", crm)
est_test = crm["email"].fillna("").str.strip().str.lower().eq("test@example.com")
apres = j.etape(crm, crm[~est_test].copy(), "retirer les lignes de test", "ce ne sont pas des
↳ clients")
print(j.table()[["etape", "regle", "avant", "apres", "retirees"]].to_string(index=False))
```

etape	regle	avant	apres	retirees
lecture	lecture du fichier brut, tout en texte	7140	7140	0
1	retirer les lignes de test	7140	7000	140

Dans l'exemple, une seule étape est consignée : 7 140 lignes lues, 140 lignes de test retirées, 7 000 lignes restantes. La fonction `nettoyer_crm`, fournie avec le chapitre, enchaîne **quatre** étapes de ce genre. Voici ce qu'elle consigne.

```
propre, journal = 0.nettoyer_crm(crm)
t = journal.table()
print(t[["etape", "avant", "apres", "retirees", "modifiees", "empreinte"]].to_string(index=False))
print("lues, retirées, finales :", journal.conservation())
```

etape	avant	apres	retirees	modifiees	empreinte
lecture	7140	7140	0	0	84f642145769
1	7140	7000	140	0	2cf9dc57b1dd
2	7000	7000	0	357	a8e64c9664db
3	7000	7000	0	5624	2eeca19cf76d
4	7000	6323	677	0	e729855c52a1

lues, retirées, finales : (7140, 817, 6323)

La dernière ligne est l'équation de conservation : **7 140 lignes lues, 817 retirées (140 + 677), 6 323 lignes finales**. Elle tombe juste. Les quatre étapes sont les suivantes.

Étape	Règle	Justification
1	retirer les lignes de test (adresse test@example.com)	ce ne sont pas des clients
2	adresses e-mail : retirer les espaces, passer en minuscules ; mettre à vide celles qui n'ont pas exactement un @ ou qui contiennent deux points de suite	une adresse n'est comparable que normalisée ; une adresse invalide n'est pas une information
3	téléphones : ne garder que les chiffres	un seul format, pour pouvoir comparer

Étape	Règle	Justification
4	doublons : une seule ligne par adresse normalisée, en gardant le plus petit id_crm	deux saisies de la même adresse sont le même client

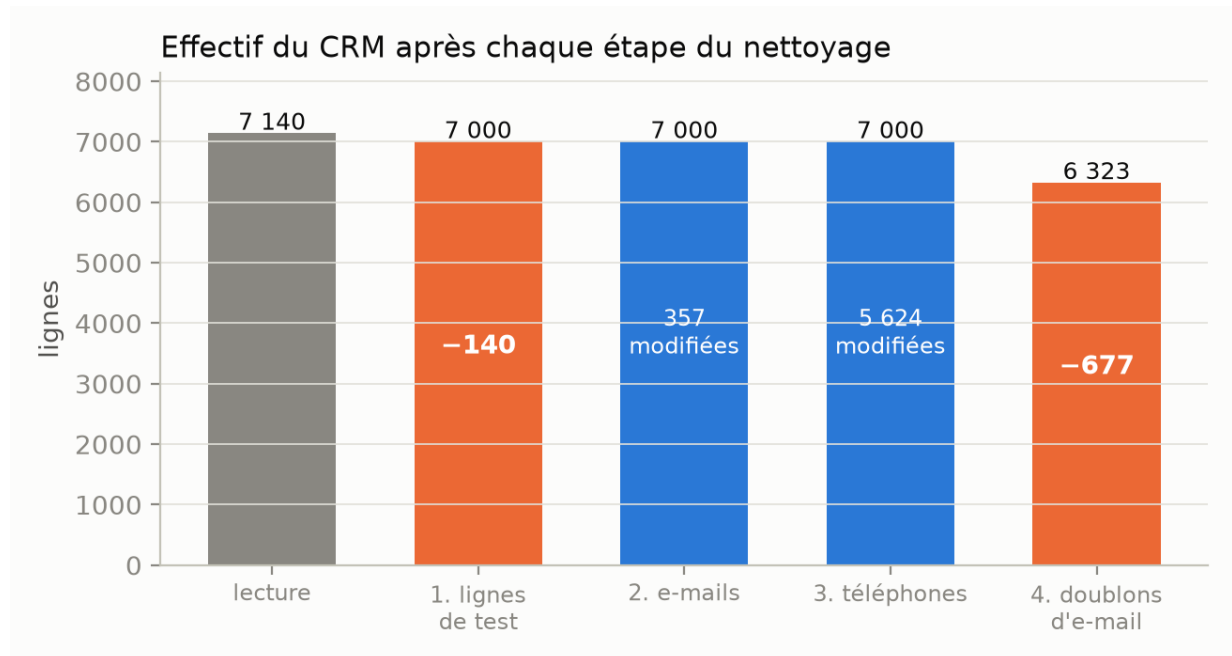


Figure 23: Effectif du CRM après chaque étape : 7 140 lignes lues, 140 lignes de test retirées, 357 adresses et 5 624 téléphones normalisés, puis 677 doublons retirés. Les barres orange sont les étapes qui retirent des lignes, les barres bleues celles qui modifient sans retirer.

Deux colonnes demandent un mot. « **modifiées** » compte les lignes dont au moins la valeur d'une cellule a changé : 357 adresses ont été corrigées ou mises à vide (espaces, majuscules, adresse invalide), 5 624 téléphones ont changé de format. Aucune ligne n'a disparu à ces étapes : l'effectif ne bouge pas, mais le journal montre l'**ampleur** de la transformation. « **empreinte** » est celle du tableau obtenu à la fin de l'étape ; deux personnes qui appliquent les mêmes règles au même fichier obtiendront les mêmes empreintes, étape par étape. Si elles divergent à l'étape 3, le désaccord se **localise** immédiatement.

Intuition. Le journal est un **relevé de compte** : chaque débit est justifié, le solde final est vérifié. On ne demande pas à une banque de nous croire sur parole, on ne devrait pas non plus demander à un analyste de croire son propre chiffre sans relevé.

Pour s'entraîner. Cahier, chapitre 4 : application 4.2, exercice 4.3.

4.1.5 Le journal face à la vérité : ce qu'il montre, ce qu'il ne dit pas

Le journal consigne **ce que l'on a fait**. Il ne dit pas si c'était **juste**. Comme le CRM est simulé, nous pouvons ouvrir le fichier de vérité (`verite_crm.csv`) pour savoir ce que nos quatre étapes ont réellement accompli. C'est un luxe que l'on n'a jamais en vrai ; il nous permet de mesurer la différence entre « j'ai appliqué la règle » et « la règle avait raison ».

lignes de test retirées à l'étape 1	: 140 sur 140
lignes retirées à l'étape 4	: 677 dont 674 vrais doublons et 3 fusions à tort
lignes finales	: 6323 pour 5997 clients distincts (vérité : 6 000)
vrais doublons qui restent	: 326

Quatre enseignements.

1. **L'étape 1 est parfaite** : les 140 lignes de test, et seulement elles, ont été retirées.
2. **L'étape 4 retire presque exclusivement de vrais doublons** (674 sur 677) mais **fusionne à tort trois clients différents** qui avaient la même adresse normalisée. Les trois personnes ont simplement une adresse identique — homonymes dont les adresses coïncident. La règle « une adresse égale un client » est raisonnable, **pas infallible** : le journal l'écrit, la vérité la juge.
3. **Il reste 326 doublons**, puisque la règle ne voit que les adresses **identiques** : les doublons dont l'adresse est absente ou différente passent au travers. Les 6 323 lignes finales décrivent 5 997 clients distincts, alors qu'il y en a 6 000 : **trois clients ont disparu** et 326 lignes sont en trop. Retrouver le reste exige un rapprochement approché des noms, ce que nous verrons en section 2.5.
4. **Le journal n'a rien caché** : c'est précisément parce que la règle, la justification et les effectifs sont écrits que l'on peut, après coup, mesurer ses défauts. Un nettoyage sans journal laisse le même résultat, sans moyen de le discuter.

Piège. Un journal bien tenu ne rend pas un nettoyage correct ; il le rend **contestable**, et c'est mieux. Ne jamais écrire dans le journal « données nettoyées » sans les effectifs : ce n'est pas une documentation, c'est une affirmation.

4.1.6 README, commentaires, noms de fichiers : le reste de la documentation

La fiche et le journal sont le cœur. Autour d'eux, trois habitudes complètent le dispositif.

Un README de projet. C'est le premier fichier qu'ouvre quelqu'un qui découvre le dossier. Il tient en vingt lignes et répond à : *à quoi sert ce projet ? comment relancer ? qu'est-ce qu'il y a dans chaque dossier ? qui contacter ?*

Synthèse du quatrième trimestre 2025 – boutique

****Question**** : chiffre d'affaires, panier moyen et taux de retour par canal, T4 2025 contre T4 2024.
****Livrables**** : ``sortie/synthese_t4.csv``, ``sortie/graphique_t4.png``, ``sortie/message_gerante.md``.

Relancer

1. Placer les fichiers bruts de ``entree/`` (voir leur fiche : ``entree/*.md``).
2. Lancer ``python run.py`` (une seule commande, tout depuis le brut).
3. Comparer ``sortie/journal.csv`` à la version précédente.

Dossiers

``entree/`` fichiers bruts, jamais modifiés · ``travail/`` fichiers intermédiaires · ``sortie/`` livrables
 ↪ · ``docs/`` dictionnaire, glossaire

****Auteur**** : analyse · ****Dernière mise à jour**** : 2026-01-05 · ****Version des données**** : v1

Des commentaires qui disent pourquoi. Un commentaire qui répète le code est du bruit ; un

commentaire qui explique une **décision** est une documentation. Comparez.

```
df = df[df["total"] < 1000] # on garde les totaux inférieurs à 1000      <- inutile : le code le
↳ dit déjà
df = df[df["total"] < 1000] # au-delà : montants en centimes (changement d'unité du site au 15/09)
↳ <- utile : il dit POURQUOI
```

Des noms de fichiers qui portent la date et la version. Pas de `donnees_final.csv`, `donnees_final2.csv` ni `donnees_VRAI_final.csv`. Une convention simple suffit : `nom_AAAA-MM-JJ_vN.ext`, avec la **date de l'extraction** (pas celle où l'on a ouvert le fichier) et un numéro de version qui monte à chaque modification de contenu.

À éviter	À préférer	Pourquoi
<code>clients_final.csv</code>	<code>crm_clients_2025-12-31_v1.csv</code>	la date de l'extraction se lit, et le fichier se retrouve
<code>clients_final2.csv</code> , <code>clients_VRAI.csv</code>	<code>crm_clients_2025-12-31_v2.csv</code>	une version = un numéro, pas un adjectif
Copie de ventes (3).xlsx	<code>ventes_t4_2025_v3.xlsx</code>	pas de copie sans nom
<code>export_05-12.csv</code>	<code>export_caisse_2025-12-05.csv</code>	dates en ISO (année d'abord) : l'ordre alphabétique est l'ordre chronologique

En pratique. On ne modifie **jamais** un fichier brut : on le copie dans `travail/` et l'on travaille sur la copie. Le brut dans `entree/` reste ce qu'il était le jour de la livraison, accompagné de sa fiche et de son empreinte ; c'est lui qui permet de **tout rejouer**.

4.1.7 La reproductibilité : tout refaire depuis le brut

Le test ultime d'une documentation est simple : **quelqu'un d'autre, ou vous dans six mois, peut-il refaire le travail depuis le brut et retrouver le même résultat ?** Concrètement : une seule commande, qui lit les fichiers bruts, applique les étapes dans l'ordre, écrit le journal et les livrables. Pas de manipulation manuelle entre deux étapes, pas d'état caché.

On peut le **vérifier** : relancer tout le nettoyage deux fois, depuis le brut lu à nouveau, et comparer les empreintes.

```
brut2 = 0.charger("crm_clients", dtype=str) # on relit le fichier : aucun état conservé
propre2, journal2 = 0.nettoyer_crm(brut2)
print("même empreinte finale :", 0.empreinte_df(propre2) == 0.empreinte_df(propre))
print("mêmes empreintes à chaque étape :", list(journal2.table()["empreinte"]) ==
↳ list(journal.table()["empreinte"]))
```

```
même empreinte finale : True
mêmes empreintes à chaque étape : True
```

Les deux exécutions produisent exactement le même résultat, étape par étape. C'est la définition pratique de la **reproductibilité** : pas « on devrait retrouver à peu près la même chose », mais « les empreintes sont identiques ».

Piège. Une opération qui dépend du **hasard** (un échantillon tiré sans graine), de l'**heure** (une date du jour écrite dans le résultat) ou de l'**ordre** de lecture d'un dossier (des fichiers lus dans l'ordre où le système les liste) casse la reproductibilité. On fixe la graine, on passe la date en paramètre, on trie les fichiers.

4.1.8 Bonne et mauvaise documentation

Terminons par un portrait-robot. Une documentation est bonne quand elle est **courte, vérifiable et à jour**. Elle est mauvaise quand elle est vague, invérifiable, ou périmée.

Mauvaise documentation	Bonne documentation	Ce qui change
« Données nettoyées. »	« 7 140 lignes lues, 140 lignes de test et 677 doublons d'e-mail retirés, 6 323 lignes finales. »	des effectifs vérifiables
« Export du CRM. »	« Export du CRM du 31/12/2025, tous clients saisis depuis 2018, v1, empreinte dc4918622cc0. »	une date, un périmètre, une version
« Les montants sont en euros. »	« total : euros jusqu'au 14/09/2025, centimes à partir du 15/09 (changement du site). »	l'unité et sa limite
« Voir le notebook. »	« python run.py refait tout depuis entree/ ; résultat attendu : empreinte a4c09a3fa175. »	une commande et un résultat attendu
« Ancien dictionnaire, peut-être encore valable. »	« Dictionnaire v3, vérifié automatiquement contre le fichier le 05/01/2026. »	un contrôle daté

La dernière ligne annonce la section suivante : un dictionnaire de données qui ne se vérifie pas finit toujours par mentir.

4.1.9 Combien documenter ? Proportionner à l'enjeu

Tout documenter autant serait absurde : on ne rédige pas une fiche de six rubriques pour un calcul que l'on jette dans l'heure. La règle est de **proportionner la documentation à l'enjeu** et à la **durée de vie** du travail. Trois niveaux couvrent presque tous les cas.

Niveau	Exemple	Ce que l'on écrit
Exploration jetable	regarder un fichier pour se faire une idée, un calcul à la main	une ligne en tête du notebook : la question, la source, la date
Rapport récurrent	la synthèse du trimestre, un tableau de bord mensuel	la fiche des fichiers d'entrée, le journal des étapes, un README, une commande qui refait tout
Chiffre engageant	un bilan, un dossier de banque, une décision de prix ou de personnel	tout ce qui précède, plus le dictionnaire vérifié, le glossaire, les empreintes, le journal d'audit et une relecture par une autre personne

Un travail change de niveau sans prévenir : l'exploration jetable de lundi devient le chiffre du bilan de vendredi. Le bon réflexe est de se demander, **dès le début**, « cela peut-il me resservir, ou servir à quelqu'un d'autre ? ». Si la réponse est oui, on passe au niveau du dessus avant de produire le chiffre, pas après.

En pratique. Documentez **en travaillant**, pas à la fin. Une règle de nettoyage écrite au moment où on la décide prend dix secondes ; la reconstituer trois semaines plus tard prend une heure et se fait de travers.

À retenir. Une **fiche** par jeu de données (source, extraction, périmètre, grain, clé, limites, version, propriétaire, droits) ; un **journal** par transformation (règle, justification, effectifs avant et après, lignes touchées, empreinte) **écrit par le code** ; l'**équation de conservation** (lignes lues – lignes retirées = lignes finales) comme contrôle de base ; le **brut intact**, une commande qui **refait tout**, et des empreintes identiques d'une exécution à l'autre. Sur le CRM : 7 140 lignes lues, 817 retirées, 6 323 restantes, dont 326 doublons que seul un rapprochement approché trouvera.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 et 4.2, exercices 4.1 à 4.5.

4.2 Construire un dictionnaire de données

La fiche décrit un fichier ; le **dictionnaire de données** décrit ses **colonnes**. C'est le document qui dit ce que veut dire `satisfaction_moy`, en quelle unité est `revenu_annuel`, quelles valeurs sont permises dans `canal_acquisition` et ce que signifie une cellule vide. Cette section montre comment en construire un sans y passer des jours (un squelette automatique, enrichi à la main), comment le **ranger**, comment **vérifier qu'il reste vrai** et comment s'entendre, au-delà des colonnes, sur les **mots** de l'entreprise.

4.2.1 À quoi sert un dictionnaire, et ce qu'il contient

Imaginez que vous receviez le fichier `profil_clients.csv` sans autre explication. Vous voyez une colonne `satisfaction_moy` : est-elle sur 5 ? sur 10 ? calculée sur combien de réponses ? Une colonne `revenu_annuel` : en euros ? du client ou de son foyer ? réel ou estimé ? Un dictionnaire répond à ces questions **une fois pour toutes**, au lieu que chaque lecteur les pose à chaque analyse.

Pour chaque colonne, on documente onze attributs. Les premiers décrivent la colonne, les suivants disent quelles valeurs on peut y rencontrer, les derniers d'où elle vient.

Attribut	Question	Exemple pour <code>revenu_annuel</code>
Nom	Comment s'appelle-t-elle dans le fichier ?	<code>revenu_annuel</code>
Libellé	Que représente-t-elle, en une phrase ?	revenu annuel estimé du foyer
Type	Quel type de valeur, technique et logique ?	décimal (<code>float64</code>)
Unité	En quelle unité ?	€
Valeurs permises	Quel domaine, quelles bornes ?	positif ; modalités listées pour les catégories
Obligatoire	Peut-elle être vide ?	non
Codage des manquants	Que veut dire une cellule vide ?	« non estimé »
Exemple	À quoi ressemble une valeur ?	54 000
Règle de calcul ou source	D'où vient-elle ?	modèle d'estimation externe
Sensibilité	Faut-il la protéger ?	personnelle
Version	Depuis quand, avec quels changements ?	v1

Deux d'entre eux méritent qu'on s'y attarde. Le **type** est double : il y a le type **technique** (ce que

dit l'ordinateur : entier, décimal, texte) et le type **logique** (ce que veut dire la colonne : une date, un identifiant, une catégorie, un montant). Une date lue comme du texte a un type technique `str` et un type logique « date » : le dictionnaire doit donner **les deux**, parce que c'est l'écart entre eux qui annonce un nettoyage à faire. Le **codage des manquants** est le plus oublié : une cellule vide peut vouloir dire « non demandé », « inconnu », « refusé » ou « zéro », et l'analyse change complètement selon la réponse (chapitre 1).

Intuition. Un dictionnaire est un **contrat** entre celui qui produit le fichier et ceux qui l'utilisent : « voici ce que vous trouverez dans chaque colonne ». Quand le contrat est rompu (une colonne change d'unité sans prévenir), c'est le contrat qui permet de le **constater**.

4.2.2 Partir d'un squelette fabriqué par le code

Écrire un dictionnaire à la main, colonne par colonne, est long et source de fautes. Une bonne partie des informations se **lit** pourtant directement dans le tableau : le type technique, la part de valeurs manquantes, le nombre de valeurs distinctes, le minimum, le maximum, un exemple. La fonction `squelette` les rassemble en une ligne par colonne.

```
sq = 0.squelette(profil)
print(sq.to_string(index=False))
```

	colonne	type	manquants_pct	distincts	min	max	exemple
	<code>id_client</code>	<code>int64</code>	0.0	6000	1	6000	1
	<code>age</code>	<code>int64</code>	0.0	68	18	85	33
	<code>canal_acquisition</code>	<code>str</code>	0.0	3	Boutique	Site	Boutique
	<code>revenu_annuel</code>	<code>float64</code>	17.3	555	6900.0	104400.0	54000.0
	<code>nb_commandes_2025</code>	<code>int64</code>	0.0	24	0	25	0
	<code>depense_2025</code>	<code>float64</code>	5.0	2838	0.0	3382.33	0.0
	<code>satisfaction_moy</code>	<code>float64</code>	9.4	312	1.0	5.0	4.61
	<code>minutes_site</code>	<code>float64</code>	0.0	533	0.2	86.4	2.4

En un coup d'œil, ce squelette signale déjà ce qu'il faudra documenter : trois colonnes ont des **valeurs manquantes** (17,3 % des revenus, 5,0 % des dépenses, 9,4 % des satisfactions), `canal_acquisition` ne prend que **trois valeurs** (c'est une catégorie : on listera ses modalités), `satisfaction_moy` va de 1,0 à 5,0 (une note sur 5), et `id_client` a autant de valeurs distinctes que de lignes (c'est une clé).

Mais ce squelette ne sait **pas** dire l'essentiel. Il ne sait pas que `revenu_annuel` est un revenu **estimé** et **du foyer**, que `age` est l'âge **au 31 décembre 2025**, que `depense_2025` est **TTC** et **remises déduites**, que `satisfaction_moy` est une moyenne de **réponses à une enquête** (donc vide pour qui n'a pas répondu). Ni l'unité, ni la règle de calcul, ni la sensibilité ne se déduisent des valeurs. Un squelette automatique fait la partie **la moins difficile** du travail ; le reste demande quelqu'un qui connaît l'activité.

Piège. Ne confondez pas **minimum et maximum observés** avec **valeurs permises**. Le squelette dit que `satisfaction_moy` va de 1,0 à 5,0 *dans ce fichier* ; ce n'est pas lui qui garantit que la note n'a pas le droit de valoir 0 ou 6. La règle est une **décision** (une note de 1 à 5), à écrire à la main.

4.2.3 L'enrichir à la main

On complète le squelette avec ce que seule une personne du métier sait : le libellé, l'unité, les valeurs permises, le caractère obligatoire, le codage des manquants, la règle de calcul, la sensibilité. Le plus simple est de saisir ces informations dans un fichier texte structuré, puis de les **fusionner** avec le squelette : la partie automatique reste toujours à jour, la partie humaine est saisie une fois. Voici, au format YAML, l'entrée complète d'une colonne.

```
import yaml
dico = 0.dico_complet(profil, "profil_clients")
fiche = dico[dico["colonne"] == "revenu_annuel"].iloc[0]
cles = ["colonne", "libelle", "unite", "type", "manquants_pct", "codage_manquant", "regle_ou_source",
        ↪ "sensibilite"]
entree = {k: (None if pd.isna(fiche[k]) else fiche[k]) for k in cles}
print(yaml.safe_dump({k: (v.item() if hasattr(v, "item") else v) for k, v in entree.items()}),
        ↪ allow_unicode=True, sort_keys=False))
```

```
colonne: revenu_annuel
libelle: Revenu annuel estimé du foyer
unite: €
type: float64
manquants_pct: 17.3
codage_manquant: vide = non estimé
regle_ou_source: modèle d'estimation externe (voir limites)
sensibilite: personnelle
```

Le dictionnaire complet du jeu tient ensuite en une table lisible. Voici ses colonnes principales.

```
print(dico[["colonne", "libelle", "unite", "manquants_pct", "sensibilite"]].to_string(index=False))
```

colonne	libelle	unite	manquants_pct	sensibilite
id_client	Identifiant du client		0.0	indirecte
age	Âge au 31/12/2025	années	0.0	personnelle
canal_acquisition	Canal par lequel le client est arrivé		0.0	
revenu_annuel	Revenu annuel estimé du foyer	€	17.3	personnelle
nb_commandes_2025	Nombre de commandes passées en 2025	commandes	0.0	
depense_2025	Dépense totale en 2025, TTC, remises déduites	€	5.0	
satisfaction_moy	Satisfaction moyenne déclarée note de 1 à 5		9.4	
minutes_site	Temps passé sur le site	minutes	0.0	personnelle

La colonne « **sensibilité** » mérite un mot : elle ne sert pas à l'analyse, elle sert à la **protection**. Elle distingue les colonnes qui identifient directement une personne (nom, adresse), celles qui permettent de la retrouver en les croisant (âge, ville, identifiant) et celles qui sont confidentielles pour l'entreprise (le coût d'achat). On la remplit dès maintenant, parce que le chapitre 5 en aura besoin.

En pratique. Faites remplir le dictionnaire par **deux personnes** : celle qui connaît les données (l'analyste) pour les types et les manquants, celle qui connaît le métier (la gérante, un responsable) pour les libellés, les unités et les règles. Une colonne que personne ne sait décrire est une colonne qu'il ne faut pas utiliser.

Pour s'entraîner. Cahier, chapitre 4 : application 4.3, exercices 4.6 et 4.7.

4.2.4 Où ranger le dictionnaire : quelques formats

Un dictionnaire ne sert que s'il est **trouvable, lisible et exploitable par un programme**. Cinq formats courants, aucun n'est le meilleur partout.

Format	Les +	Les –	Quand l'utiliser
Tableur (.xlsx)	tout le monde sait l'ouvrir, saisie facile pour les non-techniciens	peu lisible par un programme, se périmé en silence	dictionnaire rédigé par des métiers
CSV	simple, versionnable, lisible par un programme	pas de mise en forme, pas de structure imbriquée	dictionnaire d'un seul jeu, en entrée d'un contrôle automatique
YAML (ou JSON)	structuré, imbriqué, lisible par un humain et un programme	demande un peu de rigueur de syntaxe	dictionnaire pilotant un traitement
Markdown	se lit tel quel, se range avec le code (README)	pas fait pour être lu par un programme	documentation publiée
Commentaires de la base	le dictionnaire vit dans la base, à côté de la colonne	propre à chaque moteur, souvent ignoré	bases de données d'entreprise

Quel que soit le format, le dictionnaire se range **à côté du fichier** qu'il décrit, avec le même nom, la même version et la même date. Il peut aussi être **publié** sous une forme agréable à lire : voici le dictionnaire de `profil_clients` rendu sous forme d'une page web.

Dictionnaire de données : `profil_clients`

Version 1 · 6 000 lignes · propriétaire : analyse

Colonne	Libellé	Unité	Type	Manquants (%)	Sensibilité
<code>id_client</code>	Identifiant du client		int64	0.0	indirecte
<code>age</code>	Âge au 31/12/2025	années	int64	0.0	personnelle
<code>canal_acquisition</code>	Canal par lequel le client est arrivé		str	0.0	
<code>revenu_annuel</code>	Revenu annuel estimé du foyer	€	float64	17.3	personnelle
<code>nb_commandes_2025</code>	Nombre de commandes passées en 2025	commandes	int64	0.0	
<code>depense_2025</code>	Dépense totale en 2025, TTC, remises déduites	€	float64	5.0	
<code>satisfaction_moy</code>	Satisfaction moyenne déclarée	note de 1 à 5	float64	9.4	
<code>minutes_site</code>	Temps passé sur le site	minutes	float64	0.0	personnelle

Figure 24: Le dictionnaire de `profil_clients` publié en page web : une ligne par colonne, avec libellé, unité, type, part de valeurs manquantes et sensibilité. Capture réelle d'une page produite localement et rendue avec un navigateur sans interface.

En pratique. Pour un analyste isolé, un **CSV par jeu de données** (une ligne par colonne) rangé à côté du fichier est le meilleur rapport simplicité/utilité. Il se lit dans un tableur, se versionne avec le code et se **vérifie par programme**, ce que nous allons faire.

4.2.5 Un dictionnaire qui ment : le vérifier automatiquement

Le pire dictionnaire est celui qui **est faux sans que personne le sache** : la colonne a changé d'unité, une colonne a été ajoutée, une valeur interdite est apparue. Il est alors plus dangereux que pas de dictionnaire du tout, parce qu'il inspire confiance. Un dictionnaire se **vérifie** comme le reste : par un test automatique, rejoué à chaque réception de fichier.

La fonction `verifier_dictionnaire` compare un tableau à son dictionnaire et liste les écarts : colonne absente du dictionnaire ou du fichier, type différent, valeur hors domaine, colonne obligatoire vide. Sur le fichier tel que nous l'avons décrit, tout est en règle ; puis nous le **dégradons** exprès de quatre façons pour voir ce que le test attrape.

```
print("profil_clients :", 0.verifier_dictionnaire(profil, dico) or "aucune anomalie")
profil2 = profil.assign(segment="A").drop(columns="minutes_site")
profil2["age"] = profil2["age"].astype(float)
profil2.loc[0, "canal_acquisition"] = "Magasin"
for a in 0.verifier_dictionnaire(profil2, dico):
    print("-", a)
```

```
profil_clients : aucune anomalie
- colonne absente du dictionnaire : segment
- type de age : attendu int64, trouvé float64
- canal_acquisition : 1 valeurs hors domaine (Boutique|Site|Réseaux)
- colonne du dictionnaire absente du fichier : minutes_site
```

Les quatre dégradations ont été détectées : une colonne apparue, une colonne disparue, un type qui a glissé et une valeur inconnue. Aucune de ces anomalies ne fait planter un calcul, et toutes peuvent fausser un résultat sans qu'on s'en aperçoive. Le même test, appliqué au CRM brut, vérifie son dictionnaire cette fois-ci contre **de vrais défauts**.

```
crm_dico = 0.dico_complet(crm, "crm_clients")
for a in 0.verifier_dictionnaire(crm, crm_dico):
    print("-", a)
```

```
- ville : 2808 valeurs hors domaine (20 valeurs permises)
- consentement_marketing : 2428 valeurs hors domaine (oui|non)
- consentement_marketing : 2161 valeurs vides alors que la colonne est obligatoire
```

Ici, le dictionnaire est **juste** (ce qu'on attend d'une ville, d'un consentement), et c'est **le fichier** qui ne le respecte pas : 2 808 villes écrites autrement que « Ville A » à « Ville T » (casse, faute, arabe), 2 428 consentements écrits autrement que « oui » ou « non » (0, 1, TRUE, OUI...), et 2 161 consentements vides. Le test n'est pas un reproche fait aux données, c'est le **cahier des charges du nettoyage** : exactement ce que le chapitre 1 va corriger.

On branche ce test à la chaîne de traitement de façon qu'il l'arrête quand le fichier ne respecte plus son contrat.

```
anomalies = 0.verifier_dictionnaire(df, dico)
assert not anomalies, "\n".join(anomalies)      # la construction s'arrête : le fichier ne respecte
↳ plus son dictionnaire
```

Piège. Un test qui échoue **sans rien bloquer** est un témoin que personne n'entend. Décidez à l'avance ce que l'échec fait : arrêter la chaîne, envoyer un message, ou classer le fichier en quarantaine. Un contrôle qui ne déclenche rien est une décoration.

Pour s'entraîner. Cahier, chapitre 4 : application 4.4, exercice 4.8.

4.2.6 Dictionnaire et glossaire métier : une définition unique

Le dictionnaire dit ce que contient une **colonne**. Mais bien des désaccords ne portent pas sur une colonne : ils portent sur un **mot**. Qu'est-ce qu'un « client actif » ? Une « commande annulée » ? Un « panier moyen » ? Chacun a sa définition, et les chiffres divergent sans que personne n'ait fait d'erreur — nous l'avons vu avec les quatre chiffres de l'introduction.

Prenons « client actif ». Trois définitions raisonnables, sur les mêmes commandes, au 31 décembre 2025.

```
ref = pd.Timestamp("2025-12-31")
cmd["jour"] = pd.to_datetime(cmd["date_commande"])
def actifs(jours, mini=1):
    r = cmd[cmd["jour"] > ref - pd.Timedelta(days=jours)]
    return int((r.groupby("id_client").size() >= mini).sum())
print("au moins 1 commande sur 12 mois :", actifs(365))
print("au moins 1 commande sur 6 mois :", actifs(183))
print("au moins 2 commandes sur 12 mois:", actifs(365, 2))
print("clients inscrits", len(0.charger("clients")))
```

```
au moins 1 commande sur 12 mois : 3875
au moins 1 commande sur 6 mois : 3148
au moins 2 commandes sur 12 mois: 2654
clients inscrits : 6000
```

Selon la définition, la boutique a **2 654, 3 148 ou 3 875 clients actifs**, sans compter les 6 000 inscrits. L'écart entre la première et la dernière est de **plus de 1 200 clients**, soit près d'un tiers. Une campagne budgétée « par client actif » coûterait près de moitié plus cher (3 875 contre 2 654 clients) selon qui a rédigé la définition. D'où le **glossaire métier** : un document court qui donne **une définition par mot**, avec son calcul exact.

Terme	Définition retenue	Calcul	Contre-exemple
Client actif	client ayant passé au moins une commande payée au cours des 12 derniers mois	id_client distincts dans les commandes du 01/01 au 31/12	un client inscrit sans commande n'est pas actif
Commande annulée	commande dont le statut est cancelled après normalisation de la casse ; exclue du chiffre d'affaires et du nombre de commandes	lower(status) = 'cancelled'	une commande retournée n'est pas annulée : elle a été livrée
Panier moyen	chiffre d'affaires TTC remises déduites divisé par le nombre de commandes distinctes , calculé sur le total	SUM(montant) / COUNT(DISTINCT id_commande)	la moyenne des paniers moyens des canaux n'est pas le panier moyen
Trimestre	trimestre civil : T4 = du 1er octobre au 31 décembre inclus	date >= '2025-10-01' AND date <= '2025-12-31'	une extraction arrêtée le 15/12 n'est pas un trimestre

On voit la même chose avec la **commande annulée** : dans l'export brut du site, le statut s'écrit de **trois manières** selon l'origine de la ligne.

```
so = 0.charger("site_commandes")
print(so["status"].value_counts().to_dict())
```

```
{'paid': 3629, 'PAID': 1537, 'Paid': 907, 'cancelled': 186}
```

Trois écritures du même mot « payée » (paid, PAID, Paid) et 186 lignes cancelled. Une règle qui compterait « les commandes dont le statut vaut paid » ne verrait que 3 629 commandes sur 6 073 payées, soit **60 %** : près de **40 %** des commandes payées disparaîtraient du chiffre d'affaires. C'est le dictionnaire qui doit dire que le statut a deux modalités **après normalisation**, et le glossaire qui dit que l'annulée est exclue.

Intuition. Le dictionnaire dit **ce qu'il y a dans le fichier** ; le glossaire dit **ce que veulent dire les mots de l'entreprise**. Sans le second, deux analystes qui lisent parfaitement le premier peuvent encore annoncer deux chiffres différents.

Trois règles pour qu'un glossaire serve.

1. **Un mot, une définition, un propriétaire.** La personne qui peut trancher en cas de désaccord est nommée.
2. **Le calcul écrit à côté de la phrase.** « Client actif » ne suffit pas ; `COUNT (DISTINCT id_client)` sur une période précise, si.
3. **Les mots changent, pas leur historique.** Si l'on redéfinit « client actif » (de 12 à 6 mois), on **conserve** l'ancienne définition dans une version antérieure et l'on recalcule les anciens chiffres, sans quoi la courbe saute.

4.2.7 Trois dictionnaires pour la boutique

Rassemblons ce que nous venons de construire pour trois jeux : le référentiel clients, le référentiel produits et le CRM brut. Le tableau compte, pour chacun, les colonnes, celles qui sont sensibles, celles qui sont obligatoires, celles dont les valeurs sont **listées** (un domaine), et celles qui ont des manquants.

jeu	colonnes	sensibles (personnelles ou indirectes)	confidentielles	obligatoires	domaine
↪ listé avec manquants					
clients	8	5	0	8	
↪ 4	0				
produits	7	0	1	7	
↪ 1	0				
crm_clients	11	9	0	7	
↪ 3	3				

Le CRM est de loin le jeu le plus **sensible** : 9 colonnes sur 11 sont des données personnelles ou permettent de les retrouver. Le référentiel produits n'a aucune donnée personnelle, mais son `cout_achat` est **confidentiel** pour l'entreprise : le dictionnaire doit le dire pour qu'on ne le publie pas par mégarde. Et c'est le CRM encore qui a **trois colonnes avec des manquants**, que l'équipe devra décider de traiter ou d'accepter. Ce tableau est le début d'un **inventaire** des données de l'entreprise : on y voit où se trouvent les risques et les travaux à venir.

4.2.8 Faire vivre le dictionnaire : versions et changements

Un dictionnaire n'est jamais « terminé » : les colonnes évoluent, les unités changent, des modalités apparaissent. Ce qui compte est de **suivre** ces changements au lieu de les subir. Deux habitudes suffisent.

Un journal des changements (*change log*) en tête du dictionnaire : à chaque modification, la date, le changement, la raison et le nom de la personne. Par exemple : « 2025-09-15 : total passe de l'euro au centime (changement de la plateforme du site) ; ancienne valeur : euros ». C'est ce qui permet, six mois plus tard, de comprendre pourquoi une série **saute** à une date précise.

Un numéro de version qui dit la gravité du changement. On distingue deux sortes de modifications.

Type de changement	Exemple	Effet sur les analyses existantes	Version
Compatible	ajouter une colonne, préciser un libellé, ajouter une modalité à une catégorie	aucun : les analyses d'hier tournent toujours	1.1 → 1.2
Incompatible	changer l'unité d'une colonne, renommer ou supprimer une colonne, redéfinir un mot du glossaire	les analyses d'hier peuvent donner un résultat faux sans erreur visible	1.2 → 2.0

Un changement **incompatible** se **prévient**, avant qu'il n'arrive : qui lit cette colonne ? (le lignage de colonnes de la section 4.3 répond), qui doit adapter son calcul ? À partir de quand ? Le test automatique de la section 4.2.5 attrape les changements incompatibles **après** coup ; le journal des changements et le lignage permettent de les **anticiper**.

Intuition. Un dictionnaire est comme la notice d'un appareil : utile surtout quand l'appareil change. Sa valeur se mesure à la facilité avec laquelle on retrouve **ce qui a changé et quand**.

À retenir. Un dictionnaire décrit chaque colonne (**libellé, type technique et logique, unité, valeurs permises, caractère obligatoire, codage des manquants, règle ou source, sensibilité**) ; on en fabrique un **squelette automatique** (types, manquants, distincts, bornes, exemple) qu'on **enrichit à la main** ; on le range à côté du fichier, en CSV ou en YAML ; on le **vérifie par un test** qui arrête la chaîne ; et on le complète par un **glossaire** qui donne **une définition unique** de chaque mot de l'entreprise : sur les mêmes commandes, « client actif » vaut 2 654, 3 148 ou 3 875.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.3 à 4.5, exercices 4.6 à 4.9.

4.3 ⊕ Pour aller plus loin : lignage des données et pistes d'audit

Section complémentaire. Elle prolonge la documentation d'un jeu de données et d'un nettoyage par la question que pose tout lecteur d'un rapport : **d'où vient ce chiffre ?** Elle montre comment dessiner la chaîne de dépendances (le **lignage**), prouver qu'un fichier n'a pas changé (les **empreintes**), tenir un **journal d'audit** et **rejouer** un résultat depuis le brut. Rien de ce qui suit n'est nécessaire au reste du volume.

4.3.1 D'où vient ce chiffre ?

Le **lignage des données** (*data lineage*) est la carte des dépendances : quelle source alimente quelle table, quelle requête produit quel résultat, quel graphique repose sur quel tableau. On lui pose deux questions, dans les deux sens.

- **En amont** : *d'où vient ce chiffre ?* On remonte du résultat vers ses sources. C'est la question de l'auditeur, de la collègue de la comptabilité, de la gérante qui doute.
- **En aval** : *si ce fichier change, qu'est-ce qui est touché ?* On descend de la source vers ses livrables. C'est la question de l'analyste qui s'apprête à modifier une table et qui veut savoir ce qu'il risque de casser : **l'analyse d'impact**.

Sans lignage, ces deux questions se règlent à la main, en fouillant des dossiers et en interrogeant des collègues : lent, incomplet, et surtout impossible à **vérifier**. Avec un lignage, même simple, elles se règlent en une ligne.

4.3.2 Un lignage à la main, au niveau des fichiers

Pas besoin d'un logiciel pour commencer. Un lignage est un **dictionnaire** : pour chaque élément, la liste des éléments dont il dépend. Voici celui de la synthèse du quatrième trimestre (celle du projet du volume I) : quatre tables sources, trois traitements, un tableau de synthèse, un graphique et le message adressé à la gérante.

```
L = 0.Lignage()
for nom in ["commandes", "lignes_commande", "produits", "retours"]:
    L.ajouter(nom, "source")
L.ajouter("ca_par_canal.sql", "requete", ["commandes", "lignes_commande"])
L.ajouter("marge.py", "script", ["lignes_commande", "produits"])
L.ajouter("taux_retour.sql", "requete", ["lignes_commande", "retours", "commandes"])
L.ajouter("synthese_t4.csv", "table", ["ca_par_canal.sql", "marge.py", "taux_retour.sql"])
L.ajouter("graphique_t4.png", "sortie", ["synthese_t4.csv"])
L.ajouter("message_gerante.md", "sortie", ["synthese_t4.csv", "graphique_t4.png"])
print("amont du message :", L.amont("message_gerante.md"))
print("sources du message :", L.sources("message_gerante.md"))
print("si `produits` change :", L.aval("produits"))
```

```
amont du message : ['ca_par_canal.sql', 'commandes', 'graphique_t4.png', 'lignes_commande',
↳ 'marge.py', 'produits', 'retours', 'synthese_t4.csv', 'taux_retour.sql']
sources du message : ['commandes', 'lignes_commande', 'produits', 'retours']
si `produits` change : ['graphique_t4.png', 'marge.py', 'message_gerante.md', 'synthese_t4.csv']
```

La première réponse dit que le message dépend, directement ou non, de **neuf éléments** ; la deuxième, que tout repose sur **quatre sources** ; la troisième, que modifier la table produits touche **quatre éléments**, dont le message lui-même. Cette dernière réponse est **l'analyse d'impact** : avant de corriger un prix dans produits, on sait déjà quoi relancer et qui prévenir. Le graphe ci-dessous montre la même chose ; les éléments qui ne dépendent pas de produits sont grisés.

Intuition. Le lignage est un **plan de plomberie** : on ne l'ouvre que quand une fuite apparaît ou qu'on veut changer un tuyau. Le moment où il sert est justement celui où l'on n'a pas le temps de le reconstituer : il faut l'avoir dessiné avant.

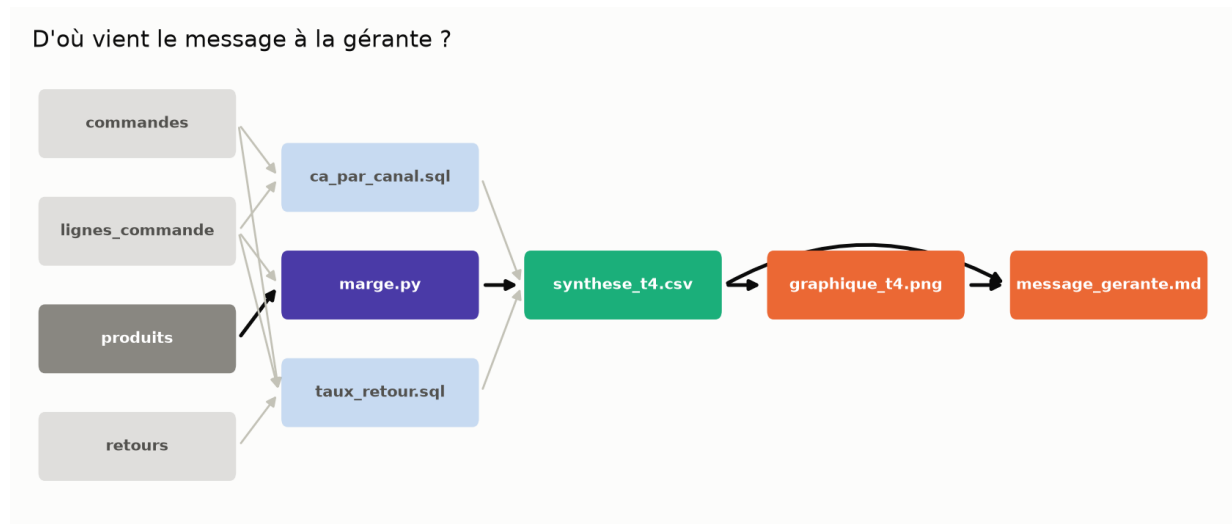


Figure 25: Lignage de la synthèse du quatrième trimestre : de gauche à droite, les sources (gris), les traitements SQL (bleu) et Python (violet), le tableau de synthèse (vert), le graphique et le message (orange). Les éléments qui dépendent de la table produits sont en couleur pleine, les autres sont grisés.

4.3.3 Le lignage des colonnes

Le lignage de fichiers répond à « quel fichier ? ». Pour savoir **quelle colonne** et **quel calcul**, il faut descendre d'un niveau : le **lignage de colonnes**. C'est lui qui répond à une question comme : « le panier moyen de la synthèse change quand on change quoi, exactement ? ».

```

C = 0.Lignage()
for c in ["quantite", "prix_unitaire", "remise_pct", "canal", "date_commande", "id_commande"]:
    C.ajouter(c, "source")
C.ajouter("montant", "table", ["quantite", "prix_unitaire", "remise_pct"])
C.ajouter("ca_t4_canal", "table", ["montant", "canal", "date_commande"])
C.ajouter("nb_commandes_t4", "table", ["id_commande", "canal", "date_commande"])
C.ajouter("panier_moyen", "sortie", ["ca_t4_canal", "nb_commandes_t4"])
print("colonnes sources du panier moyen :", C.sources("panier_moyen"))
print("ce qui dépend de `remise_pct` :", C.aval("remise_pct"))
  
```

```

colonnes sources du panier moyen : ['canal', 'date_commande', 'id_commande', 'prix_unitaire',
↳ 'quantite', 'remise_pct']
ce qui dépend de `remise_pct` : ['ca_t4_canal', 'montant', 'panier_moyen']
  
```

Le panier moyen dépend de **six colonnes sources**. Si la plateforme change l'unité de prix_unitaire (centimes au lieu d'euros), le graphe montre en un coup d'œil que montant, ca_t4_canal et panier_moyen sont touchés, alors que nb_commandes_t4 ne l'est pas. C'est exactement le cas du changement d'unité du total dans l'export du site, en septembre (voir les données du volume) : un lignage de colonnes aurait indiqué les calculs à vérifier.

En pratique. Le lignage de colonnes d'un grand entrepôt ne se dessine pas à la main : il se **déduit** des requêtes (en lisant les SELECT) ou se déclare dans les outils de transformation. À l'échelle d'une analyse, un dictionnaire Python de vingt lignes, tenu à jour dans le dossier du projet, suffit amplement et vaut mieux qu'un outil que personne ne maintient.

Pour s'entraîner. Cahier, chapitre 4 : application 4.6, exercice 4.10.

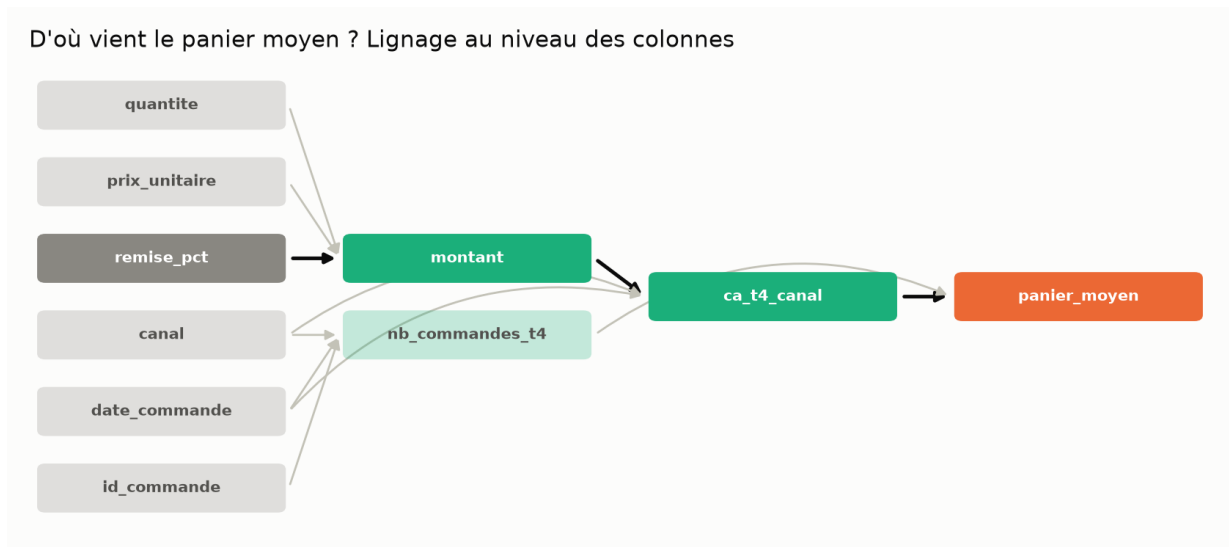


Figure 26: Lignage des colonnes du panier moyen : six colonnes sources (gris), trois colonnes calculées (vert), l'indicateur final (orange). En couleur pleine : ce qui dépend de la remise.

4.3.4 Prouver qu'une entrée n'a pas changé : les empreintes

Le lignage dit **quel fichier** alimente un chiffre. Il ne dit pas si ce fichier est **le même** que celui de l'an dernier. Quelqu'un a pu le remplacer, le corriger, le ré-exporter. Pour le **prouver**, on calcule une **empreinte** : une chaîne de caractères, de longueur fixe, calculée à partir de **tout le contenu** du fichier par une fonction de hachage (ici SHA-256). Propriété essentielle : le même contenu donne **toujours** la même empreinte ; un contenu qui diffère, même d'un seul caractère, en donne une **tout autre**.

```

for nom in ["commandes.csv", "lignes_commande.csv", "clients.csv"]:
    print(f"{nom:22s} {0.empreinte_fichier(os.path.join(0.donnees(), nom))[:16]}")
copie = os.path.join(TMP4, "commandes_copie.csv")
with open(os.path.join(0.donnees(), "commandes.csv"), encoding="utf-8") as f:
    contenu = f.read().replace("Site", "Sitf", 1)          # un seul caractère modifié, une seule
    ↪ fois
    open(copie, "w", encoding="utf-8").write(contenu)
print(f"{'copie modifiée':22s} {0.empreinte_fichier(copie)[:16]}")
  
```

commandes.csv	284247714df1013f
lignes_commande.csv	3320bc35e48b90f6
clients.csv	7c3ef5cbc42f6e5f
copie modifiée	92fe621230c2173d

Un fichier de 36 395 commandes dont **un seul mot** a changé a une empreinte entièrement différente. On garde cette empreinte avec la fiche du fichier : le jour où l'on rejoue l'analyse, on recalcule l'empreinte, et l'on sait **sans rien ouvrir** si l'on part du même fichier.

Piège. Une empreinte prouve que le fichier est **identique**, pas qu'il est **juste**. Deux fichiers faux de la même façon ont la même empreinte. L'empreinte répond à « est-ce bien le fichier que j'ai documenté ? », jamais à « ce fichier est-il correct ? ». La seconde question relève du dictionnaire et des contrôles de qualité (chapitre 3).

4.3.5 Le journal d'audit : qui, quand, quoi

Le lignage dit d'où vient un chiffre ; le journal d'audit dit **ce qui s'est passé** : qui a reçu quel fichier, qui l'a transformé, qui l'a validé, à quelle date. C'est un fichier texte dans lequel on **ajoute** une ligne à chaque événement, **jamais** on n'en modifie une. Cette règle (on ajoute, on ne réécrit pas) lui donne sa valeur : un journal que l'on peut retoucher ne prouve plus rien.

```
audit = os.path.join(TMP4, "audit.jsonl")
O.consigner(audit, "2026-01-05 09:12", "analyste", "réception", "crm_clients_v1.csv",
↳ O.empreinte_df(crm))
O.consigner(audit, "2026-01-05 09:40", "analyste", "nettoyage", "crm_propre_v1.csv",
↳ O.empreinte_df(propres))
O.consigner(audit, "2026-01-06 14:05", "gérante", "validation", "crm_propre_v1.csv",
↳ O.empreinte_df(propres))
print(O.lire_audit(audit).to_string(index=False))
```

	quand	qui	action	objet	empreinte
	2026-01-05 09:12	analyste	réception	crm_clients_v1.csv	84f642145769
	2026-01-05 09:40	analyste	nettoyage	crm_propre_v1.csv	e729855c52a1
	2026-01-06 14:05	gérante	validation	crm_propre_v1.csv	e729855c52a1

On lit l'histoire d'un coup d'œil : l'analyste a reçu le CRM brut (empreinte dc49...), l'a transformé en un fichier propre (empreinte a4c0...), et la gérante a **validé exactement ce fichier-là**, puisque son empreinte est la même. Si, le lendemain, quelqu'un modifie le fichier propre, son empreinte changera et la validation ne portera plus sur le fichier en circulation. Cette phrase simple est ce qu'un audit demande.

Quatre règles pour un journal d'audit utile : **ajouter seulement** ; consigner **qui, quand, quoi, sur quel objet, avec quelle empreinte** ; noter **les validations** autant que les modifications ; ne jamais y mettre de données personnelles (on y met des **noms de fichiers** et des empreintes, pas leur contenu).

Pour s'entraîner. Cahier, chapitre 4 : applications 4.7, exercice 4.11.

4.3.6 Versionner des données, sans outil lourd

Les outils de gestion de versions (Git) conviennent au code, pas aux gros fichiers de données. À l'échelle d'une analyse, trois pratiques légères suffisent.

1. **Un dossier par livraison**, daté : entree/2025-12-31/, entree/2026-03-31/. On ne remplace jamais un fichier : on ajoute un dossier.
2. **Un manifeste** : un petit fichier qui liste, pour chaque dossier de livraison, les fichiers et leur empreinte. Il se versionne avec le code.
3. **Une vérification à l'ouverture** : le programme compare les empreintes des fichiers qu'il va lire à celles du manifeste, et s'arrête s'il y a un écart.

```
manifeste = {nom: O.empreinte_fichier(os.path.join(O.donnees(), nom))[:12] for nom in
↳ ["commandes.csv", "lignes_commande.csv"]}
chemin = os.path.join(TMP4, "MANIFESTE_2025-12-31.json")
json.dump(manifeste, open(chemin, "w"), indent=1)
print(open(chemin).read())
```

```
{
  "commandes.csv": "284247714df1",
  "lignes_commande.csv": "3320bc35e48b"
}
```

Ce petit manifeste est la **carte d'identité d'une livraison**. Il pèse quelques octets, se lit dans

n'importe quel éditeur et suffit à répondre à la question « partons-nous des mêmes fichiers ? ». Pour de grosses volumétries ou de nombreux fichiers, des outils spécialisés de versionnement de données existent ; ils reprennent la même idée (une empreinte par fichier, un fichier de description versionné avec le code).

4.3.7 Les outils existants (non exécutés)

Tout ce que nous venons de faire à la main, des outils le font à l'échelle d'une entreprise. Nous ne les installons pas ici : voici ce qu'ils apportent, **sans prétendre les avoir exécutés**.

- **Les catalogues de données** rassemblent, pour tous les jeux d'une entreprise, leur fiche, leur dictionnaire, leur propriétaire et leur sensibilité, avec une recherche. Ils remplacent « le classeur partagé que personne ne retrouve ».
- **Les outils de transformation par fichiers SQL** (dbt est le plus connu) déclarent les dépendances entre tables dans les requêtes elles-mêmes, en **déduisent** le graphe de lignage et **publient** un site de documentation. Ils permettent aussi d'écrire les tests (valeurs permises, absence de vide, unicité) à côté de la description de la colonne.
- **Des standards ouverts de lignage** (OpenLineage, par exemple) décrivent comment un outil de traitement signale « j'ai lu ceci, j'ai produit cela », pour que les graphes de plusieurs outils s'assemblent.

À titre d'exemple de ce qu'on y écrit, voici le dictionnaire d'une colonne avec ses tests, dans la syntaxe de l'un de ces outils (non exécuté ; la syntaxe varie selon l'outil et la version, **à vérifier dans sa documentation**).

```
version: 2
models:
  - name: synthese_t4
    description: Synthèse du quatrième trimestre par canal (CA TTC remises déduites)
    columns:
      - name: canal
        description: Canal de vente
        tests:
          - not_null
          - accepted_values: {values: [Boutique, Site, Réseaux]}
```

En pratique. Un outil de lignage ne vaut que s'il est **tenu à jour** par ceux qui produisent les données. Avant d'en adopter un, commencez par un dictionnaire Python et un manifeste : s'ils restent à jour chez vous pendant six mois, vous saurez ce que vous attendez d'un outil.

4.3.8 Mini-projet : rejouer un chiffre depuis le brut

Réunissons tout. La question de l'introduction était : *comment refaire le chiffre de 211 434 € du Site au quatrième trimestre ?* Si la documentation est bonne, la réponse tient dans un dictionnaire qui dit **quelles sources** (avec leur empreinte), **quels filtres** et **quelle mesure**, et dans une fonction rejouer qui exécute cette description.

```
doc = {
  "question": "CA TTC du canal Site au quatrième trimestre 2025",
  "sources": {"commandes": {"fichier": "commandes.csv", "empreinte": manifeste["commandes.csv"]},
    "lignes_commande": {"fichier": "lignes_commande.csv", "empreinte":
      ↳ manifeste["lignes_commande.csv"]}},
  "filtres": [{"canal", "=", "Site"}, {"date_commande", ">=", "2025-10-01"}, {"date_commande",
    ↳ "<=", "2025-12-31"}],
  "mesure": "montant"}
res = 0.rejouer(doc)
print(res)
```

```
{'empreintes_ok': True, 'valeur': 211433.79, 'lignes': 4911}
```

La documentation redonne **211 433,79 €**, calculé sur les 4 911 lignes de commande du canal Site du 1er octobre au 31 décembre, et confirme que les fichiers lus sont ceux de la livraison documentée (`empreintes_ok`). Un collègue qui lit ce dictionnaire sait **quoi** calculer, **sur quoi**, et **comment vérifier** qu'il part des mêmes fichiers. Les quatre chiffres de l'introduction (211 434, 176 195, 214 993, 166 108) se distinguent désormais par une ligne du dictionnaire : le filtre, le taux de taxe, l'étendue des dates.

Reste à voir ce que fait la fonction quand la documentation et la réalité **divergent** : on modifie l'empreinte attendue du fichier des commandes, comme si quelqu'un avait remplacé le fichier.

```
doc["sources"]["commandes"]["empreinte"] = "000000000000"
print(0.rejouer(doc)["empreintes_ok"])
```

```
False
```

Le calcul se fait toujours, mais le drapeau passe à `False` : on **sait** que l'on n'est plus sur la livraison documentée. C'est exactement ce qu'on attend d'une documentation vivante : elle ne garantit pas que tout est juste, elle garantit qu'on **saura** quand quelque chose a changé.

À retenir. Le **lignage** donne la carte des dépendances, dans les deux sens (**amont** : d'où vient ce chiffre ; **aval** : qu'est-ce qui est touché si ceci change), au niveau des fichiers et des colonnes. Les **empreintes** prouvent qu'un fichier n'a pas changé (pas qu'il est juste). Le **journal d'audit** consigne qui, quand, quoi et avec quelle empreinte, en **ajoutant** seulement. Un **manifeste** versionne une livraison. Avec ces quatre outils légers, un chiffre se **rejoue** depuis le brut : celui du Site au quatrième trimestre, 211 433,79 €, se retrouve à partir d'une documentation d'une dizaine de lignes.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.6 à 4.8, exercices 4.10 à 4.12.

Bilan du chapitre 4

Vous savez maintenant :

- **expliquer pourquoi on documente** (refaire, comprendre, auditer, transmettre) et **reconnaître ce qui manque** quand un chiffre ne se retrouve pas : la source, la date d'extraction, le périmètre, la taxe, les remises, la définition des mots ;
- **remplir la fiche d'un jeu de données** (source, date d'extraction, périmètre, grain, clé, contenu, limites connues, version, propriétaire, droits) et savoir pourquoi le **grain** et la **source** en sont les deux rubriques décisives ;
- **tenir le journal d'un nettoyage** : une règle, une justification, des effectifs avant et après, les lignes retirées et modifiées, une empreinte du résultat, et le **faire écrire par le code** ; vérifier l'**équation de conservation** (lignes lues – lignes retirées = lignes finales) ;
- **juger un nettoyage** autrement que sur la foi de son propre journal : un journal le rend **contestable**, une vérité (quand on en a une) le **mesure** ;
- **ranger la documentation** : README, commentaires qui disent *pourquoi*, noms de fichiers datés et versionnés, brut jamais modifié, une commande qui refait tout, et un test de reproductibilité par les empreintes ;

- **construire un dictionnaire de données** : un squelette automatique (types, manquants, valeurs distinctes, bornes, exemple) enrichi à la main (libellé, unité, valeurs permises, obligatoire, codage des manquants, règle ou source, sensibilité), rangé à côté du fichier en CSV ou en YAML ;
- **vérifier qu'un dictionnaire reste vrai** par un test automatique qui arrête la chaîne, et s'accorder sur **une définition unique** des mots de l'entreprise grâce à un glossaire ;
- (en option) **dessiner un lignage** au niveau des fichiers et des colonnes, remonter en amont et mesurer l'impact en aval, **prouver qu'un fichier n'a pas changé** par son empreinte, tenir un **journal d'audit**, **versionner** une livraison par un manifeste et **rejouer un chiffre depuis le brut**.

Le chapitre a mis des chiffres sur des idées qui restent souvent abstraites. Tous viennent de calculs réellement exécutés sur les données de la boutique :

Question	Résultat mesuré
Un même « chiffre d'affaires du T4 pour le Site », quatre calculs honnêtes Journal du nettoyage du CRM	211 434 € (TTC), 176 195 € (HT), 214 993 € (avant remises), 166 108 € (extraction arrêtée au 15/12) 7 140 lignes lues, 140 de test et 677 doublons d'e-mail retirés, 6 323 lignes finales (équation de conservation vérifiée)
Ce que ce nettoyage a réellement accompli (vérité)	les 140 lignes de test sont toutes bien retirées ; sur 677 doublons retirés, 674 sont vrais et 3 sont des fusions à tort ; 326 vrais doublons restent ; 5 997 clients distincts sur 6 000
Reproductibilité	deux exécutions depuis le brut : mêmes empreintes à chaque étape
Le dictionnaire de profil_clients face à un fichier dégradé	4 anomalies détectées sur 4 (colonne ajoutée, colonne disparue, type glissé, valeur hors domaine)
Le dictionnaire du CRM face au CRM brut	2 808 villes hors domaine, 2 428 consentements hors domaine, 2 161 consentements vides alors qu'ils sont obligatoires
Combien de « clients actifs » ?	2 654, 3 148 ou 3 875 selon la définition (sur 6 000 inscrits)
Statut des commandes du site, avant normalisation	paid : 3 629 ; PAID : 1 537 ; Paid : 907 ; cancelled : 186 : filtrer sur paid ferait perdre 40 % des commandes payées
Lignage de la synthèse du T4	9 éléments en amont du message, 4 sources ; modifier produits touche 4 éléments
Rejouer le chiffre du Site depuis le brut	211 433,79 € sur 4 911 lignes, avec contrôle des empreintes

Le fil conducteur du chapitre tient en une phrase : **un chiffre est une conclusion, sa documentation est la preuve**. Presque tous les désaccords d'analyse — « elle ne retrouve pas mon chiffre » — viennent de ce qui n'a pas été écrit : une taxe, un filtre, une définition, une date. On ne documente pas par scrupule, on documente pour que le chiffre **survive** à celui qui l'a produit.

Rappel d'honnêteté. Aucun des outils de lignage ou de catalogue évoqués en 4.3.7 n'a été exécuté ici : ils sont décrits, pas démontrés. Le CRM, les commandes et les profils sont **si-mulés** ; la « vérité » qui a servi à juger le nettoyage du CRM n'existe pas dans un cas réel, où l'on dispose seulement du journal.

Le chapitre 5 ferme le volume par une question qui touche toute la documentation que nous

venons de construire : une colonne marquée « **sensible** » dans un dictionnaire change ce que l'on a le droit de **garder, partager et publier**. C'est le sujet de la **confidentialité et de l'anonymisation des données**.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 à 4.8 (fiche d'un jeu, journal d'un nettoyage, dictionnaires, test de dictionnaire, glossaire, lignage, empreintes, rejouer un chiffre) et exercices 4.1 à 4.12.

Chapitre 5 : ⊕ Confidentialité et anonymisation des données

« Une donnée n'est pas anodine parce qu'elle est dans un tableau : elle parle encore de quelqu'un. »

Chapitre complémentaire. Il est entièrement facultatif : le reste du volume ne le suppose pas. Il est pourtant celui que l'on regrette le plus de ne pas avoir lu le jour où un fichier part par courriel. Il suppose le chapitre 1 (nettoyer), la section 2.5 (rapprocher des enregistrements) et un peu de pandas (volume I, chapitre 4).

La gérante de la boutique vous écrit un lundi matin. Un prestataire lui a proposé d'analyser sa clientèle : « Il veut un fichier avec nos clients, leur âge, leur ville, ce qu'ils ont acheté, s'ils sont contents. Je peux lui envoyer le CRM tel quel ? Ou si j'enlève les noms, c'est bon ? » Elle ajoute, un peu inquiète : « Un client m'a aussi demandé de supprimer toutes ses données. Combien de temps ça prend, au juste ? »

Ces deux questions n'ont rien de statistique, et pourtant ce sont des questions d'analyste. Vous êtes la personne qui **manipule** les fichiers : vous savez quelles colonnes existent, combien de lignes décrivent une même personne, ce qui se retrouve en croisant deux tableaux. Les juristes écrivent les règles ; c'est à vous de savoir **ce qu'un fichier contient vraiment** et ce que l'on peut en faire sortir. Ce chapitre vous donne les repères pour répondre à la gérante avec des chiffres plutôt qu'avec des impressions.

Deux idées le traversent. La première est que **retirer les noms ne suffit presque jamais** : une personne se reconnaît à son association de caractéristiques (une ville, une année de naissance, un canal, une carte de fidélité) bien avant son nom, et à plus forte raison à ses habitudes d'achat. La seconde est qu'il n'existe pas de procédé magique : chaque protection a un **prix** en information perdue, et l'on choisit un équilibre, que l'on documente.

Ce que ce chapitre n'est pas. Ce n'est pas un conseil juridique. Les règles de protection des données personnelles dépendent du pays, du secteur et de la date ; nous parlerons de principes **communs à la plupart des cadres** et nous citerons, à titre d'exemple seulement, le type de règles que l'on trouve dans les textes régionaux ou internationaux. Pour une décision réelle, consultez la personne qui, dans votre organisation, est chargée de la protection des données, ou un juriste.

Le chemin de ce chapitre

Section	Question	Idée centrale
5.1	Qu'est-ce qu'une donnée personnelle, et que doit-on en faire ?	Un identifiant direct n'est que la partie visible : les quasi-identifiants identifient aussi ; des principes simples guident l'analyste
5.2	Comment remplacer un identifiant sans le perdre ni le trahir ?	Pseudonymiser n'est pas anonymiser ; un hachage sans clé se retrouve par dictionnaire
5.3	Comment mesurer le risque de reconnaître une personne ?	Unicité, k-anonymat, recoupement, l-diversité ; la confidentialité différentielle en une page
5.4	Que faire concrètement, de lundi à vendredi ?	Minimiser, séparer, agréger, ne pas publier de petits groupes, vérifier avant d'envoyer

Les données du chapitre

Données simulées. Tous les fichiers sont **simulés** (graines fixes, générateurs `build/donnees_a1.py` et `build/donnees_a2.py`) : la boutique est fictive, ses clients aussi, et leurs **noms sont inventés** à partir de syllabes, sans origine particulière. Nous connaissons donc la **vérité** (qui est qui) et nous nous en servons pour mesurer ce qu'un attaquant pourrait retrouver : en pratique, on ne la connaît pas, et c'est bien le problème.

- `crm_clients.csv` : le CRM de la boutique, **7 140 lignes** pour 6 000 clients (des doublons, 140 lignes de test), avec prénom, nom, e-mail, téléphone, ville, code postal, date de naissance, consentement (sections 5.1, 5.2).
- `verite_crm.csv` et `verite_identites.csv` : la vérité (à quel client correspond chaque ligne, quelle est la vraie identité), utilisée pour **juger** les attaques et les nettoyages, jamais comme une entrée de traitement.
- `clients.csv` et `profil_clients_verite.csv` : les 6 000 clients avec leur ville, leur année de naissance, leur canal d'acquisition, leur carte de fidélité, leur revenu estimé, leur dépense et leur satisfaction : c'est la table que l'on voudrait confier au prestataire (sections 5.3 et 5.4).
- `commandes.csv` et `lignes_commande.csv` : les commandes de 2023 à 2025, pour montrer qu'un **montant et une date** suffisent à reconnaître une commande (section 5.3).

5.1 Données personnelles et principes

Avant de savoir comment protéger un fichier, il faut savoir **ce qui, dans ce fichier, est à protéger**. Cette section pose le vocabulaire (identifiants, quasi-identifiants, données sensibles), les principes que partagent la plupart des cadres de protection des données, puis les applique au CRM de la boutique : quelles colonnes sont à risque, comment lire un consentement écrit de sept façons, et pourquoi supprimer une personne suppose de savoir **combien de lignes** la décrivent.

5.1.1 Ce qu'est une donnée personnelle

Une **donnée personnelle** est une information qui se rapporte à une personne **identifiée ou identifiable**. Le mot décisif est le second : une donnée n'a pas besoin de porter un nom pour être personnelle, il suffit qu'on puisse, **avec des moyens raisonnables**, retrouver de qui elle parle. Un numéro de client, une adresse électronique, un numéro de téléphone, un identifiant de carte de fidélité, une adresse IP, un historique d'achats suffisamment précis sont des données personnelles. Un chiffre d'affaires par canal ne l'est pas : il ne parle de personne.

Deux conséquences pratiques. La première : la qualification dépend **du contexte et de ce que l'on peut recouper**. Une année de naissance seule ne désigne personne ; associée à une ville, un canal d'achat et une carte de fidélité, elle peut désigner une seule personne parmi 6 000 (nous le mesurerons en 5.3). La seconde : la question n'est pas « ce fichier contient-il un nom ? » mais « **quelqu'un pourrait-il retrouver la personne ?** » — et cette question se pose à chaque transformation, pas seulement au moment de l'envoi.

Intuition. Pensez à un fichier comme à une foule masquée. Retirer les noms, c'est retirer les badges ; mais si chacun porte un manteau d'une couleur rare, une écharpe et une canne, la foule reste reconnaissable. Plus un participant est « particulier » (rare dans le tableau), plus il est facile à retrouver.

5.1.2 Identifiants directs, quasi-identifiants, données sensibles

On range habituellement les colonnes d'un fichier en trois familles, qui n'appellent pas la même vigilance.

- Les **identifiants directs** désignent une personne à eux seuls : nom, prénom, adresse électronique, numéro de téléphone, adresse postale complète, numéro de client ou de carte (lorsqu'il circule hors de l'organisation), photographie.
- Les **quasi-identifiants** ne désignent personne séparément mais **identifient en se combinant** : ville, code postal, date ou année de naissance, sexe, profession, canal d'acquisition, date d'inscription, carte de fidélité, mais aussi un montant et une date de commande.
- Les **données sensibles** relèvent d'une protection renforcée dans la plupart des cadres : santé, origine, opinions politiques ou religieuses, vie sexuelle, données biométriques, condamnations. Une boutique de maison et de décoration n'en collecte pas officiellement ; mais un historique d'achats peut en **révéler** (un article de santé, un article religieux) : l'inférence compte autant que la collecte.

Voici le classement des colonnes du CRM de la boutique, tel que vous le feriez avant d'envoyer quoi que ce soit.

Colonne	Famille	Risque	Conduite habituelle
prenom, nom	identifiant direct	élevé	retirer, ou remplacer par un pseudonyme
email, telephone	identifiant direct	élevé	retirer, ou pseudonymiser avec une clé secrète (5.2)
id_crm	identifiant interne	moyen	remplacer par un pseudonyme qui ne se déduit pas
ville, code_postal	quasi-identifiant	moyen	généraliser (ville → région, code postal → département)

Colonne	Famille	Risque	Conduite habituelle
date_naissance	quasi-identifiant	élevé	ramener à l'année, puis à une tranche (5.3)
date_inscription	quasi-identifiant	moyen	ramener à l'année
consentement_marketing	donnée administrative	faible en soi	normaliser (5.1.4) et en tenir compte dans l'usage
source_saisie	donnée technique	faible	conserver si utile

Un tel tableau n'est pas définitif : il se discute avec la personne chargée de la protection des données, et il se **refait** quand le fichier change. Mais il oblige à regarder chaque colonne, ce que l'on omet trop souvent quand on exporte « tout le CRM ».

5.1.3 Les principes communs aux cadres de protection

Les textes qui protègent les données personnelles varient d'un pays à l'autre, mais ils reposent presque tous sur les mêmes principes. À titre d'**exemple de cadre** (qui ne remplace pas un conseil juridique, et dont le vôtre peut différer), le règlement européen sur la protection des données énonce la plupart d'entre eux ; vous retrouverez ces idées sous d'autres noms dans la loi de votre pays.

1. **Finalité.** On collecte des données pour un but **précis et annoncé** (livrer une commande, envoyer une offre), et on ne les réutilise pas pour un but incompatible. Envoyer le CRM à un prestataire pour une analyse est un **nouvel usage** : il faut se demander si les clients s'y attendaient.
2. **Minimisation.** On ne garde et on ne transmet que les données **nécessaires** au but. C'est le principe qui compte le plus pour un analyste : la question « ai-je besoin de cette colonne ? » vaut mieux que n'importe quel outil de masquage.
3. **Base légale.** Chaque traitement repose sur une justification admise : le contrat (livrer la commande), l'obligation légale (conserver une facture), l'intérêt légitime, ou le **consentement** de la personne (envoyer des offres commerciales, dans beaucoup de cadres).
4. **Exactitude.** Les données doivent être justes et tenues à jour : un doublon, une adresse périmée, un consentement contradictoire sont des **défauts de qualité qui deviennent des défauts de conformité**.
5. **Limitation de la conservation.** On ne garde pas indéfiniment : une durée est fixée, et les données sont supprimées ou anonymisées ensuite.
6. **Sécurité.** Accès restreint, chiffrement, journaux : les données sont protégées contre la perte et la divulgation.
7. **Droits des personnes.** Une personne peut en général **accéder** à ses données, les faire **rectifier**, demander leur **effacement**, s'**opposer** à certains usages, parfois les **emporter** (portabilité).
8. **Responsabilité.** L'organisation doit pouvoir **démontrer** qu'elle respecte ces règles : tenir un registre de ses traitements, documenter ses choix.

En pratique. Avant tout export, posez-vous quatre questions, à écrire dans votre note de transmission : *Pour quoi faire ?* (finalité) ; *Quelles colonnes sont nécessaires ?* (minimisation) ; *Sur quelle base peut-on le faire ?* (base légale) ; *Qui y aura accès, combien de temps ?* (sécurité, conservation). Si l'une des réponses est « je ne sais pas », le fichier ne part pas.

Pour l'analyste, l'effet de ces principes est très concret. Ils transforment la préparation des données en une **responsabilité** : le travail de nettoyage du chapitre 1, la détection de doublons de la section

1.3 ou la réconciliation du chapitre 3 sont aussi des travaux de **conformité**, et inversement un fichier propre se protège plus facilement qu'un fichier désordonné.

5.1.4 Le consentement marketing et ses codages

Un exemple montre que la qualité des données et la protection des données sont les deux faces d'un même travail. Le CRM de la boutique contient une colonne `consentement_marketing`, qui dit si le client a accepté de recevoir des offres. Lisons ce que contient réellement cette colonne.

```
print(crm["consentement_marketing"].fillna("(vide)").value_counts().to_string())
```

```
consentement_marketing
oui      2411
(vide)   2161
Oui       703
1         687
0         369
TRUE      338
OUI       331
non       140
```

Sept façons d'écrire « oui » ou « rien » (et « non » pour les lignes de test). Pour une campagne d'envoi, la règle de lecture à retenir est celle de la **précaution** : seules les valeurs qui expriment clairement un accord comptent comme un consentement ; **le vide n'est pas un oui**. Normalisons, puis comptons sur les 7 000 lignes qui ne sont pas des lignes de test. (Pour savoir quelles lignes sont des tests et lesquelles appartiennent à quel client, nous utilisons ici le fichier de **vérité** ; dans une situation réelle, c'est le dédoublonnage du chapitre 1 et de la section 2.5 qui fournirait ce regroupement.)

```
reelles = crm.merge(vcrm, on="id_crm").query("id_client > 0").copy()
reelles["consent"] = reelles["consentement_marketing"].isin(0.OUI).astype(int)
print("lignes avec consentement :", round(reelles["consent"].mean() * 100, 1), "% | lignes vides :",
      ↪ round(reelles["consentement_marketing"].isna().mean() * 100, 1), "%")
```

```
lignes avec consentement : 69.1 % | lignes vides : 30.9 %
```

Reste un piège que seuls les doublons révèlent. Quand un client apparaît sur plusieurs lignes (le chapitre 1, section 1.3, et la section 2.5 en ont montré l'origine), ses lignes peuvent **se contredire** : une ligne dit oui, l'autre est vide. Combien de clients sont concernés, et que change la règle que l'on choisit pour trancher ?

```
par_client = reelles.groupby("id_client")["consent"].agg(["min", "max", "size"])
print("clients sur plusieurs lignes :", int((par_client["size"] > 1).sum()), "| clients aux lignes
      ↪ contradictoires :", int(((par_client["min"] == 0) & (par_client["max"] == 1)).sum()))
print("consentement si 'au moins une ligne' :", round((par_client["max"] == 1).mean() * 100, 1), "% |
      ↪ si 'toutes les lignes' :", round((par_client["min"] == 1).mean() * 100, 1), "%")
```

```
clients sur plusieurs lignes : 950 | clients aux lignes contradictoires : 397
consentement si 'au moins une ligne' : 72.4 % | si 'toutes les lignes' : 65.8 %
```

Les deux règles donnent des résultats différents : 72,4 % des clients seraient joignables avec la règle « au moins une ligne » (qui **peut** contacter une personne qui a refusé ailleurs), 65,8 % avec la règle de précaution « toutes les lignes ». L'écart de **6,6 points** représente des clients que l'on contacterait **sans être sûr de leur accord**. Le choix n'est pas statistique mais éthique et juridique ; l'analyste doit en revanche **le faire apparaître**, documenter la règle retenue (chapitre 4) et mesurer son effet.

Piège. Un vide n'est pas un accord, mais ce n'est pas non plus forcément un refus : c'est une **absence d'information**. Pour l'envoi, on le traite comme un non ; pour l'analyse (« quelle part des clients accepte ? »), on le traite comme une **valeur manquante** que l'on signale.

5.1.5 Les droits des personnes : supprimer, c'est d'abord retrouver

La gérante parlait d'un client qui demande l'effacement de ses données. Supprimer une personne suppose de **retrouver toutes les lignes qui la concernent**, dans toutes les sources. Or nous avons vu que le CRM contient des doublons mal écrits : le même client peut figurer sur deux ou trois lignes, avec des e-mails différents, en majuscules ou absents.

Simulons une demande d'effacement pour les 950 clients qui ont plusieurs lignes dans le CRM. Première méthode : chercher les lignes dont l'e-mail est **exactement** celui que le client a communiqué. Deuxième méthode : normaliser avant de comparer (minuscules, espaces retirés).

```
vrai = ident.set_index("id_client")["email"]
dupl = reelles[reelles["id_client"].isin(par_client[par_client["size"] > 1].index)].copy()
dupl["email_vrai"] = dupl["id_client"].map(vrai)
exact = dupl["email"] == dupl["email_vrai"]
norm = dupl["email"].fillna("").str.strip().str.lower() == dupl["email_vrai"].str.lower()
print("lignes concernées :", len(dupl), "| retrouvées par e-mail exact :", int(exact.sum()), "| après
↳ normalisation :", int(norm.sum()))
resume = dupl.assign(ex=exact, no=norm).groupby("id_client").agg(n=("ex", "size"), ex=("ex", "sum"),
↳ no=("no", "sum"))
print("clients dont TOUTES les lignes sont retrouvées : exact", int((resume["n"] ==
↳ resume["ex"]).sum()), "| normalisé", int((resume["n"] == resume["no"]).sum()), "sur", len(resume))
```

```
lignes concernées : 1950 | retrouvées par e-mail exact : 1362 | après normalisation : 1624
clients dont TOUTES les lignes sont retrouvées : exact 371 | normalisé 628 sur 950
```

Avec la recherche exacte, seuls 371 clients sur 950 sont **entièrement** effacés ; la normalisation en retrouve 628. Les autres lignes (e-mail absent, invalide ou différent) ne se retrouvent qu'avec les méthodes de **rapprochement approximatif** de la section 2.5, ou avec une clé interne commune. Moralité : **le dédoublonnage est aussi une obligation de conformité**. Une organisation qui efface « la ligne » d'une personne en laissant deux copies de son nom dans le fichier n'a pas honoré sa demande.

En pratique. Pour pouvoir répondre à une demande d'accès ou d'effacement, tenez une **cartographie** : quelles tables contiennent des données personnelles, comment elles se relient (une clé commune, de préférence), où se trouvent les **copies** (exports Excel, sauvegardes, fichiers envoyés à des prestataires). Une donnée que l'on ne sait pas localiser est une donnée que l'on ne sait pas protéger.

À retenir. Une donnée personnelle est une donnée qu'on peut rattacher à une personne **avec des moyens raisonnables**, pas seulement une donnée qui porte un nom. Classez les colonnes (identifiants, quasi-identifiants, sensibles) avant tout export. Les principes de finalité, de minimisation, de base légale, d'exactitude, de conservation limitée, de sécurité et de droits des personnes guident le travail ; la qualité des données (consentements normalisés, doublons repérés) est une condition de leur respect.

Pour s'entraîner. Cahier, chapitre 5 : application 5.1, exercices 5.1 à 5.3.

5.2 Pseudonymisation et anonymisation

On croit souvent qu'il suffit de « rendre le fichier anonyme » en remplaçant les noms par des codes. Cette section montre pourquoi cette croyance est fausse, ce que la **pseudonymisation** protège réellement, comment la faire correctement (une clé secrète, une table de correspondance gardée à part), et ce qu'il faut de plus pour parler d'**anonymisation**. Elle passe en revue les techniques usuelles (suppression, masquage, généralisation, bruit, données synthétiques) en mesurant, à chaque fois, **ce qu'elles font perdre**.

5.2.1 Pseudonomiser : remplacer l'identifiant par un code

Pseudonomiser, c'est remplacer un identifiant direct (un nom, un e-mail, un numéro de client) par un code, le **pseudonyme**, de sorte que le fichier ne montre plus qui est qui. Les lignes restent **liées entre elles** : le client « c_0412 » est le même d'une commande à l'autre, ce qui permet de reconstituer un parcours d'achats, de calculer une fréquence de commande, de mesurer un taux de retour par client. C'est précisément ce qui rend la pseudonymisation utile pour l'analyse.

La bonne façon de procéder tient en trois règles. On **tire des pseudonymes aléatoires** (qui ne se déduisent pas de l'identifiant) ; on **range la table de correspondance** (identifiant ↔ pseudonyme) **ailleurs**, avec un accès restreint ; on **envoie le fichier pseudonymisé seul**. Voici le principe sur la table des clients.

```
rng = np.random.default_rng(42)
pseudo = pd.Series([f"c_{v:08d}" for v in rng.choice(10**8, size=len(x), replace=False)],
    ↪ index=x["id_client"])
correspondance = pseudo.rename("pseudonyme").reset_index()           # à conserver à part, sous clé
envoi = x.assign(pseudonyme=x["id_client"].map(pseudo)).drop(columns=["id_client"])
print(envoi[["pseudonyme", "ville", "annee_naissance",
    ↪ "canal_acquisition"]].head(3).to_string(index=False))
```

pseudonyme	ville	annee_naissance	canal_acquisition
c_37077585	Ville K	1992	Boutique
c_53823529	Ville B	1991	Boutique
c_71344719	Ville Q	1985	Boutique

Le fichier d'envoi ne contient plus d'`id_client`. Mais gardez deux idées en tête pour la suite. D'abord, la table de correspondance est **le point faible** : celui qui la possède défait la pseudonymisation d'un geste, donc elle se protège comme un secret. Ensuite, et surtout, les colonnes restantes (`ville`, `annee_naissance`, `canal_acquisition`...) sont des **quasi-identifiants** : le pseudonyme ne les protège pas (section 5.3).

5.2.2 Le hachage sans clé ne protège pas

Une tentation fréquente : fabriquer le pseudonyme **à partir de l'identifiant**, par une fonction de **hachage** (SHA-256, par exemple). Un hachage transforme n'importe quel texte en une empreinte de 64 caractères, sans qu'on puisse « remonter » de l'empreinte au texte par un calcul inverse : c'est ce qu'on appelle une fonction à sens unique. Le même texte donne toujours la même empreinte, ce qui conserve les liens entre lignes. Cela semble parfait, et cela ne l'est pas.

Le défaut est que le hachage est **déterministe et public** : n'importe qui peut calculer l'empreinte d'un texte qu'il imagine. Pour retrouver l'e-mail qui se cache derrière une empreinte, il suffit de **hacher tous les e-mails plausibles** et de comparer : c'est une **attaque par dictionnaire**. Les adresses des clients de la boutique ont la forme `prenom.nom@domaine`. Imaginons qu'un attaquant dispose d'un annuaire de noms (ici, la liste des identités) et connaisse trois domaines usuels : il fabrique les adresses candidates, les hache, et compare aux empreintes du fichier « protégé ».

```

empreintes = ident["email"].map(0.sha256)                                # fichier « anonymisé » par hachage
dico = 0.annuaire(ident)                                                # empreinte -> e-mail, pour toutes les
↳ adresses plausibles
retrouvees = empreintes.isin(dico.keys())
print("adresses candidates :", len(dico), "| empreintes retrouvées :", int(retrouvees.sum()), "sur",
↳ len(empreintes), f"({retrouvees.mean() * 100:.1f} %)")

```

```
adresses candidates : 17985 | empreintes retrouvées : 5142 sur 6000 (85.7 %)
```

L'attaque retrouve **plus de 85 %** des adresses. Les autres portent un numéro (prenom.nom63@...) que le dictionnaire n'avait pas prévu ; un dictionnaire plus riche les retrouverait aussi. Le calcul est **instantané** : on teste 17 985 adresses en une fraction de seconde.

Le cas des **identifiants numériques** est pire encore. Si le pseudonyme est le hachage du numéro de client (1, 2, 3...), l'espace à explorer est minuscule : on essaie tous les entiers jusqu'à un million.

```

table_ids = {0.sha256(i): i for i in range(1, 100_001)}
trouvees = ident["id_client"].map(0.sha256).isin(table_ids.keys())
print("identifiants retrouvés :", int(trouvees.sum()), "sur", len(ident), "avec", len(table_ids),
↳ "essais")

```

```
identifiants retrouvés : 6000 sur 6000 avec 100000 essais
```

Tous les identifiants sont retrouvés. La leçon est générale : **un hachage protège un secret imprévisible, pas un identifiant prévisible**. Dès que l'ensemble des entrées possibles est petit (des numéros), structuré (prenom.nom@domaine) ou devinable (dates de naissance, numéros de téléphone), le hachage nu n'est qu'un déguisement.

Piège. « Nous avons haché les e-mails, donc le fichier est anonyme » est l'une des erreurs les plus répandues. Un hachage **sans clé** n'est **pas** une protection suffisante pour des identifiants prévisibles.

5.2.3 Le hachage à clé, et la table de correspondance séparée

La parade tient en un mot : **la clé**. Un hachage à clé (par exemple HMAC avec SHA-256) mélange à l'identifiant une **clé secrète** avant de hacher. Sans la clé, l'attaque par dictionnaire est impossible : l'attaquant peut calculer toutes les empreintes qu'il veut, aucune ne correspondra à celles du fichier.

```

cle = b"cle-de-demonstration-a-garder-hors-du-fichier"
a_cle = ident["email"].map(lambda e: 0.hmac256(e, cle))
print("empreintes à clé retrouvées par le même dictionnaire :", int(a_cle.isin(dico.keys()).sum()),
↳ "sur", len(a_cle))

```

```
empreintes à clé retrouvées par le même dictionnaire : 0 sur 6000
```

Aucune. Le hachage à clé conserve par ailleurs la propriété utile du hachage : **le même e-mail donne toujours la même empreinte**, donc deux fichiers pseudonymisés avec la même clé peuvent être **rapprochés** sans que personne ne voie l'adresse. Vérifions-le en rapprochant le CRM et l'export du site par leurs adresses (après normalisation en minuscules, sans espaces).

```

site = pd.read_csv(os.path.join(0.D, "site_commandes.csv"))
nm = lambda s: s.dropna().str.strip().str.lower()
pc = set(nm(crm["email"]).map(lambda e: 0.hmac256(e, cle)))
ps = set(nm(site["customer_email"]).map(lambda e: 0.hmac256(e, cle)))
print("adresses communes, vues par clé :", len(pc & ps), "| vues en clair :",
↳ len(set(nm(crm["email"])) & set(nm(site["customer_email"]))))

```

adresses communes, vues par clé : 2845 | vues en clair : 2845

Le rapprochement par empreintes à clé donne exactement le même résultat que le rapprochement en clair : la pseudonymisation a **préservé l'utilité** (on peut relier les sources) tout en retirant l'adresse du fichier. Trois précautions : la clé est **longue, aléatoire et rangée hors du fichier** (un coffre ou un gestionnaire de secrets, jamais dans le code partagé) ; on **normalise avant de hacher** (sinon Jean@... et jean@... donnent deux pseudonymes) ; on **change la clé** d'un projet à l'autre si l'on ne veut pas qu'un prestataire puisse relier ses fichiers entre eux.

Intuition. Le hachage nu est un cadenas dont tout le monde connaît le code à quatre chiffres ; le hachage à clé est un cadenas dont le code est long et secret. Et la **table de correspondance** (5.2.1) est la même idée sous une autre forme : le secret, c'est le registre qui relie le pseudonyme à la personne.

5.2.4 Autres techniques : ce qu'elles protègent, ce qu'elles coûtent

La pseudonymisation ne traite que les **identifiants**. Pour les quasi-identifiants et les valeurs, on dispose d'autres techniques, qui retirent de l'information en échange de protection. Chaque ligne du tableau suivant est un choix.

Technique	Exemple	Ce qu'elle protège	Ce qu'elle coûte
Suppression	retirer prenom, nom, telephone	l'identité directe	l'information de la colonne
Masquage	z***@courrier.test	la lecture à l'œil	peu : le domaine reste utile
Généralisation	année de naissance → tranche de dix ans ; ville → région	la reconnaissance par recoupement	la finesse de l'analyse (5.3)
Perturbation	ajouter un bruit à un revenu, arrondir	la valeur exacte	la précision des statistiques
Agrégation	ne fournir que des totaux par groupe	l'individu	tout détail individuel
Échantillonnage	ne transmettre qu'une partie des lignes	la certitude qu'une personne figure dans le fichier	la précision, par la taille
Chiffrement	transformer avec une clé, réversible	la lecture sans clé	rien pour le détenteur de la clé ; ne protège pas une fois déchiffré
Données synthétiques	fabriquer des lignes qui ressemblent aux vraies	les individus réels (en principe)	les relations non reproduites

Mesurons deux de ces coûts sur le revenu annuel estimé, qui est lié à l'âge (corrélation de 0,36).

```
rng = np.random.default_rng(42)
bruite = x["revenu_annuel"] + rng.normal(0, 5000, len(x))
arrondi = (x["revenu_annuel"] / 1000).round() * 1000
r = lambda s: round(float(np.corrcoef(x["age"], s)[0, 1]), 3)
print("corrélation âge-revenu : brut", r(x["revenu_annuel"]), "| bruité", r(bruite), "| arrondi à 1  
↪ 000", r(arrondi))
print("moyenne : brut", round(x["revenu_annuel"].mean()), "| bruité", round(bruite.mean()), "|  
↪ écart-type brut", round(x["revenu_annuel"].std()), "| bruité", round(bruite.std()))
```

```
corrélation âge-revenu : brut 0.36 | bruité 0.332 | arrondi à 1 000 0.36
moyenne : brut 28322 | bruité 28280 | écart-type brut 11212 | bruité 12305
```

Un bruit gaussien d'écart-type 5 000 € **ne change presque pas la moyenne** (le bruit se compense en moyenne), mais il **gonfle l'écart-type** et **affaiblit la corrélation** (de 0,36 à environ 0,33) : c'est le compromis typique, d'autant plus visible que l'on regarde de petits groupes. L'arrondi à 1 000 € est beaucoup moins coûteux pour l'analyse, mais il protège aussi beaucoup moins : le revenu reste connu à 500 € près.

Reste la tentation des **données synthétiques** : puisqu'on ne peut pas partager les vraies lignes, on en fabrique de fausses. La méthode la plus simple consiste à tirer chaque colonne séparément selon sa distribution. Les moyennes sont alors préservées, mais **les liens entre colonnes disparaissent**.

```
synth = pd.DataFrame({c: x[c].sample(frac=1, random_state=i).values for i, c in enumerate(["age",
↪ "revenu_annuel"])}))
print("corrélation âge-revenu : vraie", r(x["revenu_annuel"]), "| synthétique (colonnes tirées
↪ indépendamment)", round(float(np.corrcoef(synth["age"], synth["revenu_annuel"])[0, 1]), 3))
```

```
corrélation âge-revenu : vraie 0.36 | synthétique (colonnes tirées indépendamment) -0.003
```

La corrélation de 0,36 est devenue nulle : le jeu synthétique **ne permettrait plus de retrouver la relation entre l'âge et le revenu**, qui est justement ce que le prestataire voulait étudier. Des méthodes plus fines (modèles génératifs) reproduisent mieux les liens, mais plus elles reproduisent fidèlement les lignes réelles, plus elles risquent de **recopier** des individus : un jeu synthétique n'est pas automatiquement anonyme et se teste comme les autres (5.3).

5.2.5 Pseudonymisation n'est pas anonymisation

La différence est **juridique et pratique**. Un fichier **pseudonymisé** reste, dans la plupart des cadres, un fichier de **données personnelles** : tant que quelqu'un détient la clé ou la table de correspondance, ou qu'on peut retrouver la personne par recoupement, les règles de protection s'appliquent (finalité, sécurité, droits des personnes). Un fichier **anonymisé** est un fichier dont on ne peut **plus** retrouver les personnes par aucun moyen raisonnable, y compris pour celui qui l'a produit : les règles ne s'appliquent plus, mais la barre est haute.

Pour juger si un jeu de données est vraiment anonyme, on se pose trois questions, que beaucoup d'autorités de protection reprennent sous des formes voisines :

1. **L'individualisation.** Peut-on isoler une personne dans le fichier (une ligne qui n'a pas de jumeaux) ?
2. **Le recoupement.** Peut-on relier cette ligne à une autre source (un autre fichier, un registre public, la mémoire de quelqu'un) ?
3. **L'inférence.** Peut-on déduire une information sur une personne **sans même l'identifier**, parce que son groupe est homogène ?

Si l'on répond « oui » à l'une des trois, le fichier n'est pas anonyme. Ces questions sont exactement celles que la section 5.3 transforme en **mesures** : l'unicité (individualisation), le k-anonymat et l'attaque par recoupement (recoupement), la l-diversité (inférence).

	Pseudonymisé	Anonymisé
Identité directe retirée	oui	oui
Retour possible vers la personne	oui, avec la clé ou par recoupement	non, par aucun moyen raisonnable

	Pseudonymisé	Anonymisé
Données personnelles au sens des cadres de protection	oui	non
Utilité pour l'analyse	élevée (lignes liées, valeurs exactes)	réduite (généralisations, bruit, agrégats)
Erreur la plus fréquente	croire qu'on a anonymisé	croire qu'on a anonymisé sans le tester

À retenir. Pseudonymiser, c'est remplacer l'identifiant par un code en gardant les liens entre lignes : c'est utile, mais cela **n'anonymise pas**. Un hachage sans clé se retrouve par dictionnaire (plus de 85 % des e-mails de la boutique, tous les numéros de client) ; un hachage à clé et une table de correspondance gardée à part résistent. Toute technique (généralisation, bruit, synthèse) retire de l'information : on mesure ce que l'on perd. Un fichier est anonyme quand on ne peut ni isoler, ni recouper, ni inférer : cela se **teste**.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.2 et 5.3, exercices 5.4 à 5.6.

5.3 Mesurer le risque de réidentification

La section précédente a montré qu'un fichier sans noms n'est pas pour autant anonyme. Il faut maintenant **mesurer** ce risque, avec des chiffres, pour décider ce que l'on peut envoyer. Nous suivrons les trois questions de 5.2.5 : l'**individualisation** (combien de personnes sont uniques ?) avec l'unicité et le **k-anonymat**, le **recoupement** (peut-on relier une ligne à une autre source ?) avec l'exemple des commandes, et l'**inférence** (apprend-on quelque chose d'un groupe homogène ?) avec la **l-diversité**. Nous terminerons par la **confidentialité différentielle**, qui change de point de vue : au lieu de modifier les lignes, on bruite les résultats.

5.3.1 Quasi-identifiants et unicité

La table que la gérante voudrait confier au prestataire contient, pour les 6 000 clients, quatre colonnes banales : la ville, l'année de naissance, le canal d'acquisition et la carte de fidélité ; et des mesures plus intimes : le revenu estimé, la dépense, la satisfaction. Aucun nom, aucun e-mail. Combien de clients sont **uniques** sur ces quatre colonnes, c'est-à-dire seuls de leur espèce dans le fichier ?

Pour un client donné, on compte les clients qui partagent exactement les mêmes valeurs : c'est la **taille de son groupe**. Un client dont le groupe est de taille 1 est **unique** : quiconque connaît ses quatre caractéristiques le retrouve avec certitude. Ajoutons les colonnes une à une.

```

QI = ["ville", "annee_naissance", "canal_acquisition", "fidelite"]
lignes = []
for k in range(1, 5):
    s = 0
    stats_k(x, QI[:k])
    lignes.append(["", ".join(QI[:k]), s["groupes"], s["uniques"], round(s["uniques"] / len(x) * 100,
↪ 1)])
print(pd.DataFrame(lignes, columns=["colonnes connues", "groupes", "clients uniques", "%
↪ uniques"]).to_string(index=False))

```

colonnes connues	groupes	clients uniques	% uniques
ville	20	0	0.0
ville, annee_naissance	1036	203	3.4
ville, annee_naissance, canal_acquisition	2088	830	13.8
ville, annee_naissance, canal_acquisition, fidelite	2968	1581	26.4

Le résultat est brutal. La **ville seule** ne désigne personne (aucun client unique) ; la ville et l'année de naissance isolent déjà 3,4 % des clients ; en ajoutant le canal, 13,8 % ; avec la carte de fidélité, **plus d'un client sur quatre (26,4 %)** est unique. Quatre informations que n'importe quel proche, voisin ou collègue peut connaître suffisent à désigner un client sur quatre.

Voyons ce que cela donne concrètement. Imaginons un employé du prestataire qui sait qu'une connaissance habite la Ville K, est née en 1992, a été attirée par la boutique physique et possède la carte de fidélité. Il cherche dans la table reçue.

```
cible = x.query("ville == 'Ville K' and annee_naissance == 1992 and canal_acquisition == 'Boutique'
↳ and fidelite == 1")
print(len(cible), "ligne trouvée\n" + cible[["revenu_annuel", "depense_2025",
↳ "satisfaction_moy"]].to_string(index=False))
```

```
1 ligne trouvée
revenu_annuel  depense_2025  satisfaction_moy
54000.0         0.0         4.61
```

Une seule ligne correspond : il vient d'apprendre le **revenu estimé**, la **dépense** et la **satisfaction** de la personne, sans que son nom ait jamais figuré dans le fichier. C'est la définition d'une **ré-identification** : les valeurs sensibles se sont raccrochées à une personne connue, par le seul jeu des quasi-identifiants.

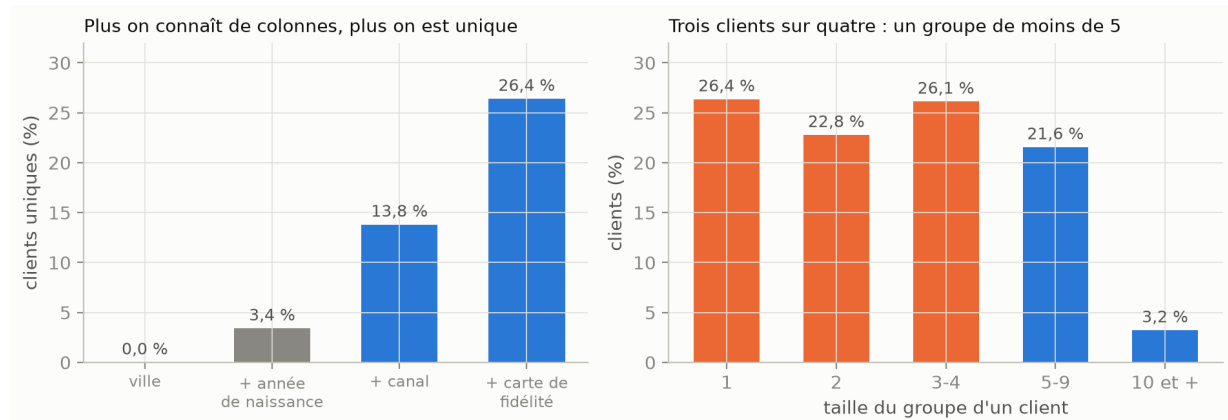


Figure 27: Part des clients uniques selon le nombre de colonnes connues (à gauche) et répartition des tailles de groupes pour les quatre quasi-identifiants (à droite) ; les groupes de moins de 5 personnes sont en orange.

Piège. On répond parfois : « mais l'attaquant ne connaît pas ces quatre informations ». Le danger est justement que **vous ne savez pas ce qu'il connaît**. Le calcul d'unicité ne dit pas que l'attaque aura lieu : il dit **combien de personnes seraient exposées si elle avait lieu**. C'est un test de prudence, comme on teste la résistance d'un pont à une charge qu'on espère ne jamais voir passer.

5.3.2 Le k-anonymat

Le **k-anonymat** est la mesure de ce risque. On dit qu'un fichier est **k-anonyme** pour un ensemble de quasi-identifiants si **chaque combinaison de valeurs apparaît au moins k fois** : chaque personne est alors indiscernable d'au moins $k - 1$ autres. Le plus petit groupe donne la valeur de k du fichier. Dans notre table, le plus petit groupe est de taille 1 : le fichier est **1-anonyme**, ce qui veut dire qu'il n'est pas anonyme du tout.

Le calcul est un simple comptage de groupes : on regroupe par quasi-identifiants et on prend la taille de chaque groupe (`groupby(...).size()`, ou ici `taille_groupes`, qui recopie la taille du groupe sur chaque ligne). La répartition des tailles dit à quel point le fichier est exposé.

```
tg = 0.taille_groupes(x, QI)
classes = pd.cut(tg, [0, 1, 2, 4, 9, 100], labels=["1", "2", "3-4", "5-9", "10 et
↳ +"]).value_counts().sort_index()
print(pd.DataFrame({"clients": classes, "%": (classes / len(x) * 100).round(1)}).to_string())
print("clients dans un groupe de moins de 5 :", int((tg < 5).sum()), f"({(tg < 5).mean() * 100:.1f}
↳ %)")
```

	clients	%
ville		
1	1581	26.4
2	1366	22.8
3-4	1567	26.1
5-9	1293	21.6
10 et +	193	3.2
clients dans un groupe de moins de 5 : 4514 (75.2 %)		

Pour un seuil usuel de $k = 5$, **plus de trois clients sur quatre** (75,2 %) sont dans un groupe de moins de 5 personnes. Le seuil de 5 n'a rien de sacré : plus k est grand, plus la protection est forte et plus l'information se dégrade. On le choisit selon la sensibilité des données et l'environnement (qui recevra le fichier ?), et on le **note** dans la documentation du jeu de données.

5.3.3 Généraliser et supprimer pour atteindre k

Pour augmenter k , on dispose de deux leviers. La **généralisation** remplace une valeur précise par une valeur plus large : une ville par une région, une année de naissance par une tranche de dix ans. Elle fusionne des groupes minuscules en groupes plus gros. La **suppression** retire les lignes qui restent isolées une fois la généralisation faite.

Ici, nous regroupons les 20 villes en **4 régions** de 5 villes, et les années de naissance en **tranches de dix ans** (« 1985-1994 »). Comparons plusieurs recettes.

```
g = 0.generaliser(x)
recettes = {"brut": QI, "ville, tranche de 10 ans, canal, carte": ["ville", "tranche",
↳ "canal_acquisition", "fidelite"],
            "région, tranche, canal, carte": ["region", "tranche", "canal_acquisition", "fidelite"],
            "région, tranche": ["region", "tranche"]}
res = pd.DataFrame({nom: 0.stats_k(g, cols) for nom, cols in recettes.items()}).T
res["% à supprimer pour k = 5"] = (res["sous_k"] / len(g) * 100).round(1)
print(res.to_string())
```

	groupes	k_min	uniques	sous_k	% à supprimer pour k = 5
brut	2968	1	1581	4514	75.2
ville, tranche de 10 ans, canal, carte	710	1	142	741	12.4
région, tranche, canal, carte	176	1	16	79	1.3
région, tranche	31	4	0	4	0.1

La généralisation fait tomber très vite le risque. En passant de la ville à la région et de l'année à la tranche, les clients uniques passent de 1 581 à 16, et les clients dans un petit groupe de 4 514 à

79. En dehors de la recette la plus généralisée (région et tranche seulement, qui n'a aucun client unique mais renonce au canal et à la carte), la recette « région, tranche, canal, carte » **garde les quatre colonnes** et ne laisse que 79 clients à supprimer pour atteindre $k = 5$, soit **1,3 %**.

```
ng = 0.taille_groupes(g, recettes["région, tranche, canal, carte"])
k5 = g[ng >= 5]
print("lignes conservées :", len(k5), "| supprimées :", len(g) - len(k5), "| k obtenu :",
      ↪ 0.stats_k(k5, recettes["région, tranche, canal, carte"])["k_min"])
```

```
lignes conservées : 5921 | supprimées : 79 | k obtenu : 5
```

Après suppression, le plus petit groupe compte 5 personnes : le fichier est **5-anonyme** pour ces quatre colonnes. Remarquez que la suppression a retiré des clients **rare**s, donc potentiellement atypiques : elle n'est pas neutre, et il faut dire combien de lignes on a retirées et lesquelles (par exemple, les très jeunes ou les très âgés d'une région peu peuplée).

5.3.4 Ce que l'on perd

Aucune protection n'est gratuite. Mesurons ce qu'a coûté la généralisation sur trois usages de la table : la corrélation entre l'âge et le revenu, la moyenne du revenu, et la **géographie** du revenu.

```
mid = 2025 - (g["annee_naissance"] // 10 * 10 + 4.5)                # âge approché par le milieu
↪ de la tranche
print("corrélation âge-revenu : exacte", round(float(np.corrcoef(g["age"], g["revenu_annuel"])[0,
↪ 1]), 3), "| avec la tranche", round(float(np.corrcoef(mid, g["revenu_annuel"])[0, 1]), 3))
print("revenu moyen : avant", round(g["revenu_annuel"].mean()), "| après suppression des lignes
↪ rares", round(k5["revenu_annuel"].mean()))
vm, rm = g.groupby("ville")["revenu_annuel"].mean(), g.groupby("region")["revenu_annuel"].mean()
print("écart entre la ville la plus riche et la plus pauvre :", round(vm.max() - vm.min()), "€ | entre
↪ régions :", round(rm.max() - rm.min()), "€")
```

```
corrélation âge-revenu : exacte 0.36 | avec la tranche 0.354
revenu moyen : avant 28322 | après suppression des lignes rares 28281
écart entre la ville la plus riche et la plus pauvre : 7638 € | entre régions : 3854 €
```

Trois résultats. La **corrélation** âge-revenu est presque intacte (0,360 contre 0,354) : la tranche de dix ans suffit pour étudier la relation globale. La **moyenne** du revenu bouge à peine (28 322 € puis 28 281 €) : supprimer 1,3 % de lignes ne change pas le portrait d'ensemble. En revanche, la **géographie** se perd : l'écart de revenu moyen entre la ville la plus riche et la plus pauvre est de 7 638 €, alors que l'écart entre régions n'est que de 3 854 €. Si le prestataire voulait savoir quelles villes ont les revenus les plus élevés, la généralisation lui a **retiré la réponse**.

Intuition. La protection se paie en **résolution** : on voit moins fin. Le bon réglage dépend de la question que se pose le destinataire : si elle porte sur des tendances globales (âge, canal), la généralisation coûte peu ; si elle porte sur les cas particuliers (un quartier, une ville), elle coûte cher et il faut soit renoncer à l'envoi, soit accepter un niveau de risque, soit organiser un **accès contrôlé** plutôt qu'un envoi (5.4).

5.3.5 Le recoupement : un montant et une date suffisent

L'unicité ne concerne pas que les profils. Les **données de transactions** sont les plus difficiles à protéger, parce qu'une commande est presque toujours unique. Supposons qu'on veuille confier au prestataire les commandes du site en 2025, **sans identité** (un pseudonyme par client), avec la date et le montant de chaque commande. Un tiers qui connaît **une seule** commande d'un client (parce qu'il l'a vue sur un reçu, sur un écran, dans un courriel) peut-il retrouver ce client dans le fichier ?

```
cs = cmd[(cmd["canal"] == "Site") & (cmd["date_commande"] >= "2025-01-01")].copy()
cs["total"] = cs["id_commande"].map(lig.groupby("id_commande")["montant"].sum().round(2))
cs["mois"], cs["euro"] = cs["date_commande"].str[:7], cs["total"].round(0)
savoir = {"le montant seul": ["total"], "le mois et le montant": ["mois", "total"], "la date et le
↳ montant arrondi à l'euro": ["date_commande", "euro"], "la date et le montant exact":
↳ ["date_commande", "total"]}
print(pd.Series({k: round((cs.groupby(c)["total"].transform("size") == 1).mean() * 100, 1) for k, c
↳ in savoir.items()}), name="% de commandes uniques").to_string())
```

le montant seul	17.1
le mois et le montant	54.3
la date et le montant arrondi à l'euro	90.5
la date et le montant exact	97.2

Sur les 6 078 commandes du site en 2025, la **date et le montant exact** désignent **97,2 %** d'entre elles de façon unique ; avec le montant arrondi à l'euro, 90,5 % ; le mois et le montant, 54,3 % ; le montant seul, déjà 17,1 %. Et une fois la commande retrouvée, on apprend **tout le reste de l'historique** du client sous son pseudonyme.

```
connue = cs.iloc[100]
trouvee = cs[(cs["date_commande"] == connue["date_commande"]) & (cs["total"] == connue["total"])]
histo = cs[cs["id_client"] == trouvee["id_client"].iloc[0]]
print("commande connue :", connue["date_commande"], connue["total"], "€ | commandes correspondantes
↳ :", len(trouvee))
print("autres commandes du client :", len(histo) - 1, "| dépense totale du client :",
↳ round(histo["total"].sum(), 2), "€")
```

```
commande connue : 2025-01-07 99.81 € | commandes correspondantes : 1
autres commandes du client : 3 | dépense totale du client : 439.73 €
```

Une seule commande connue (le 7 janvier, 99,81 €) correspond à **une seule ligne** du fichier ; en la retrouvant, l'attaquant découvre que ce client a passé 3 autres commandes et dépensé 439,73 € en tout. Aucun nom n'était dans le fichier : le motif « date, montant » a suffi à relier la ligne à une personne connue par une autre source. C'est l'attaque par **recoupement**.

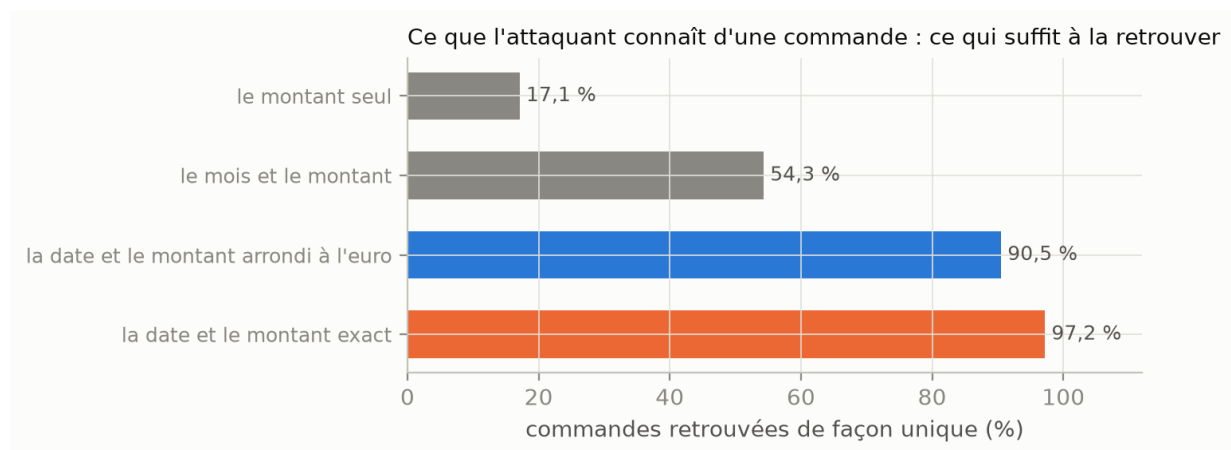


Figure 28: Part des commandes du site retrouvées de façon unique selon ce que l'attaquant connaît : un montant et une date identifient presque toute commande.

Les parades existent mais elles **coûtent** : retirer la date exacte (garder le mois), arrondir les montants, ou, mieux, **ne pas partager de transactions** et fournir à leur place des **indicateurs par client** (nombre de commandes, panier moyen, catégorie préférée), plus difficiles à utiliser comme clé de recoupement. Dans tous les cas, on **mesure** de nouveau l'unicité après transformation, comme nous venons de le faire.

5.3.6 La l-diversité : quand un groupe homogène trahit

Le k-anonymat protège contre l'**identification** de la ligne, pas contre l'**inférence** : si les k personnes d'un groupe partagent la même valeur d'un attribut sensible, savoir à quel groupe appartient quelqu'un suffit pour connaître cette valeur, sans le retrouver parmi les autres. La **l-diversité** demande que, dans chaque groupe, l'attribut sensible prenne **au moins l valeurs distinctes**.

Illustrons-le sur la table 5-anonyme obtenue en 5.3.3. Prenons comme attribut « sensible » le fait d'être **insatisfait** (satisfaction moyenne de 3 ou moins), qui concerne 14,9 % des clients.

```
k5 = k5.assign(insatisfait=(k5["satisfaction_moy"] <= 3).astype(int))
grp = k5.groupby(recettes["région", "tranche", "canal", "carte"]).agg(n=("insatisfait", "size"),
    ↳ part=("insatisfait", "mean"))
print("taux d'insatisfaits :", round(k5["insatisfait"].mean() * 100, 1), "% | groupes :", len(grp),
    ↳ "| groupes sans aucun insatisfait :", int((grp["part"] == 0).sum()), "| clients concernés :",
    ↳ int(grp.loc[grp["part"] == 0, "n"].sum()))
print("groupe le plus exposé :", grp.sort_values("part").iloc[-1].round(3).to_dict())
```

```
taux d'insatisfaits : 14.9 % | groupes : 136 | groupes sans aucun insatisfait : 13 | clients concernés
↳ : 114
groupe le plus exposé : {'n': 7.0, 'part': 0.429}
```

Sur les 136 groupes, **13** ne comptent aucun client insatisfait, et leurs 114 membres sont donc connus pour **ne pas** l'être, rien qu'en connaissant leur groupe. À l'inverse, dans le groupe le plus exposé (7 personnes), 43 % sont insatisfaits, alors que la proportion générale est de 15 % : appartenir à ce groupe **triple** la probabilité d'être mécontent. Ici l'attribut n'est pas vraiment sensible ; remplacez-le par une information sur la santé ou les finances et la fuite devient sérieuse. Le k-anonymat est donc **nécessaire mais pas suffisant** : on vérifie aussi la **diversité** des valeurs sensibles dans les groupes, et l'on fusionne les groupes homogènes.

5.3.7 La confidentialité différentielle, en une page

Les méthodes précédentes modifient les **lignes**. La **confidentialité différentielle** change de point de vue : on laisse les données intactes à l'intérieur de l'organisation et l'on publie seulement des **résultats bruités** (comptages, moyennes), avec un bruit calculé de sorte que **la présence ou l'absence d'une personne ne change presque pas** ce que l'on publie. Un paramètre, ϵ (epsilon), règle l'équilibre : plus ϵ est petit, plus le bruit est fort et plus la protection est grande.

Pour un **comptage**, une personne change le résultat d'au plus 1. On ajoute alors un bruit de **Laplace d'échelle $1/\epsilon$** : l'erreur typique (médiane de la valeur absolue du bruit) vaut $\ln 2/\epsilon \approx 0,69/\epsilon$, **quel que soit** le comptage. Simulons-le pour trois tailles de groupes.

```
rng = np.random.default_rng(0)
err = {e: np.median(np.abs(0.69*laplace(0, e, rng, 100_000))) for e in (0.1, 0.5, 1, 5)} #
    ↳ erreur absolue médiane
tab = pd.DataFrame({"comptage de {n}": {f"ε = {e}": f"{v:.2f} ({v / n * 100:.1f} %)" for e, v in
    ↳ err.items()} for n in (5, 50, 500)})
print(tab.to_string())
```

	comptage de 5	comptage de 50	comptage de 500
$\epsilon = 0.1$	6.92 (138.4 %)	6.92 (13.8 %)	6.92 (1.4 %)
$\epsilon = 0.5$	1.38 (27.5 %)	1.38 (2.8 %)	1.38 (0.3 %)
$\epsilon = 1$	0.70 (13.9 %)	0.70 (1.4 %)	0.70 (0.1 %)
$\epsilon = 5$	0.14 (2.8 %)	0.14 (0.3 %)	0.14 (0.0 %)

L'erreur absolue ne dépend que de ϵ (environ 7 pour $\epsilon = 0,1$, 0,7 pour $\epsilon = 1$) ; l'**erreur relative**, elle, dépend de la taille du groupe : pour un comptage de 5, un bruit de $\epsilon = 1$ représente environ 14 % ; pour $\epsilon = 0,1$, il dépasse 100 %, et le résultat est inutilisable. C'est la leçon centrale : **les petits groupes ne se protègent pas par le bruit, ils se masquent** (section 5.4).

Ce que garantit ϵ se voit mieux avec une attaque. Imaginons que l'on publie le nombre d'insatisfaits d'un groupe, **puis** le même nombre sans une personne donnée (par exemple, après son départ). Avec des comptages exacts, la différence révèle la valeur de cette personne à coup sûr : c'est une **attaque par différence**. Avec du bruit de Laplace, l'attaquant devine « cette personne est insatisfaite » quand la différence dépasse 0,5. Quelle est sa réussite ?

```
rng = np.random.default_rng(1)
for e in (0.1, 1, 5):
    vrai = 1 + rng.laplace(0, 1 / e, 100_000) - rng.laplace(0, 1 / e, 100_000)    # la personne est
    ↪ insatisfaite
    faux = rng.laplace(0, 1 / e, 100_000) - rng.laplace(0, 1 / e, 100_000)    # elle ne l'est pas
    print(f"ε = {e} : devine « insatisfait » dans {(vrai > 0.5).mean() * 100:.0f} % des cas si elle
    ↪ l'est, {(faux > 0.5).mean() * 100:.0f} % si elle ne l'est pas")
```

$\epsilon = 0.1$: devine « insatisfait » dans 51 % des cas si elle l'est, 48 % si elle ne l'est pas

$\epsilon = 1$: devine « insatisfait » dans 62 % des cas si elle l'est, 38 % si elle ne l'est pas

$\epsilon = 5$: devine « insatisfait » dans 91 % des cas si elle l'est, 9 % si elle ne l'est pas

Pour $\epsilon = 0,1$, l'attaquant devine « insatisfait » dans 51 % des cas quand la personne l'est et dans 48 % quand elle ne l'est pas : **autant pile ou face**, il n'apprend rien. Pour $\epsilon = 5$, il a raison 91 % du temps pour 9 % de fausses alertes : la protection est faible. Avec des comptages exacts, la réussite serait de 100 % pour 0 % de fausses alertes. Le paramètre ϵ est donc une **quantité de protection**, que l'on dépense à chaque résultat publié : publier dix fois le même comptage bruité revient à publier un comptage moyen très précis, d'où la notion de **budget**.

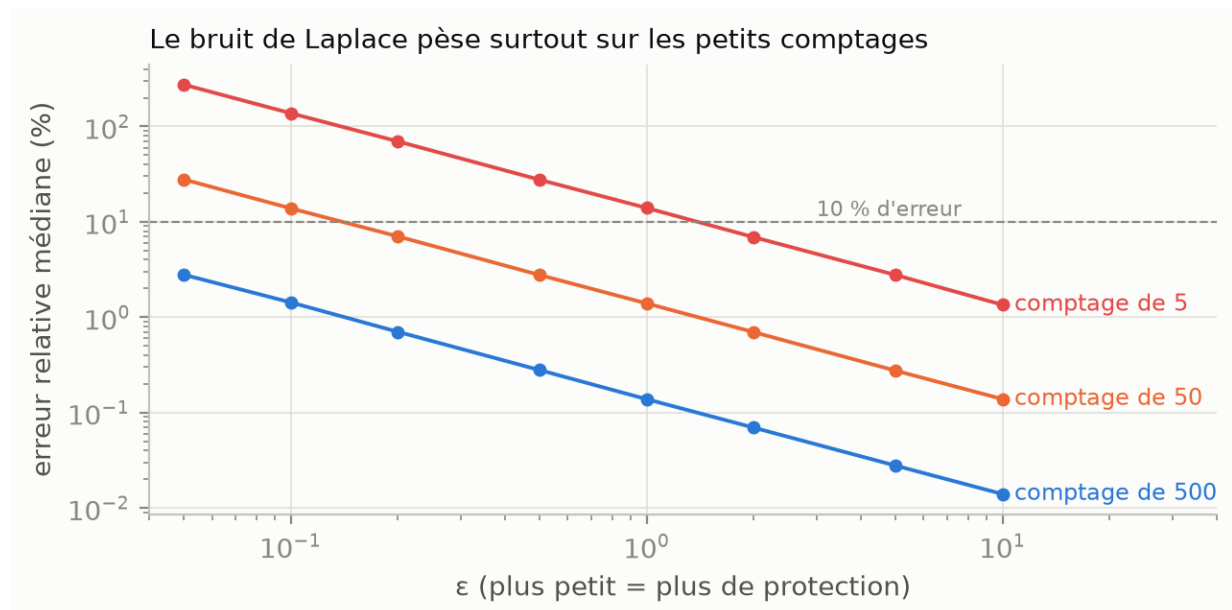


Figure 29: Erreur relative médiane d'un comptage bruité selon ϵ , pour trois tailles de groupes (échelles logarithmiques) : pour un comptage de 5, l'erreur dépasse 10 % dès que ϵ est inférieur à environ 1,4.

Piège. La confidentialité différentielle protège contre les attaques **sur les résultats publiés**, pas contre une fuite du fichier source ni contre une mauvaise gestion des accès. Elle suppose aussi un **budget total** : chaque question posée au fichier consomme de la protection. Pour un analyste de PME, retenez l'idée (bruit calibré, petits groupes inutilisables) et laissez les systèmes complets aux équipes spécialisées.

À retenir. On mesure le risque : **unicité** (un client sur quatre est unique avec quatre colonnes banales), **k-anonymat** (taille du plus petit groupe), **recoupement** (une date et un montant retrouvent 97 % des commandes), **1-diversité** (un groupe homogène trahit). La généralisation (ville → région, année → tranche) et la suppression de quelques lignes rares font passer la table de 1-anonyme à 5-anonyme pour 1,3 % de lignes perdues, au prix d'une perte de résolution géographique. La confidentialité différentielle bruite les résultats : les petits groupes ne se protègent pas, ils se masquent.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.4 à 5.6, exercices 5.7 à 5.10.

5.4 Les bonnes pratiques de l'analyste

Les trois sections précédentes ont donné les notions et les mesures. Celle-ci les traduit en **gestes** : préparer un fichier à partager (minimiser, séparer, pseudonymiser, généraliser), ne jamais publier de petits groupes, soigner les sorties de modèles et de graphiques, contrôler les accès et savoir quoi faire en cas de fuite. Elle se termine par la **liste de contrôle** que vous pouvez coller dans votre carnet avant d'envoyer le moindre fichier, et par le rôle de la personne chargée de la protection des données.

5.4.1 Minimiser et séparer : préparer le fichier du prestataire

La gérante voulait envoyer « le CRM tel quel ». Voici, pas à pas, ce qu'un analyste fait à la place. Le prestataire veut comprendre la relation entre l'âge, le revenu, le canal d'achat et la satisfaction : il n'a besoin ni des noms, ni des e-mails, ni des adresses, ni de la date de naissance exacte, ni de la ville précise.

1. **Minimiser** : on ne garde que les colonnes utiles à la question (ici : région, tranche d'âge, canal, carte, revenu, satisfaction).
2. **Séparer** : l'identité (nom, e-mail) reste dans le CRM de la boutique ; le fichier d'analyse n'a qu'un **pseudonyme** à clé (5.2.3).
3. **Généraliser** : ville → région, année de naissance → tranche de dix ans (5.3.3) ; revenu arrondi au millier (5.2.4).
4. **Supprimer les petits groupes** : les lignes des groupes de moins de 5 personnes sont retirées (5.3.3).
5. **Contrôler** : on mesure le résultat (unicité, k) avant d'envoyer.

```
cle = b"cle-d-un-projet-a-garder-hors-du-fichier"
envoi, retirees = 0.preparer_envoi(x, cle, k=5)
print(envoi.head(3).to_string(index=False))
print("lignes envoyées :", len(envoi), "sur", len(x), "| lignes retirées (groupes de moins de 5) :",
      ↵ retirees)
```

pseudonyme	region	tranche	canal_acquisition	fidelite	revenu_arrondi	satisfaction_moy
c_5690fc60ba	Région 3	1985-1994	Boutique	1	54000.0	4.61
c_df00f34023	Région 1	1985-1994	Boutique	1	30000.0	3.94
c_4f1efe30da	Région 4	1985-1994	Boutique	0	42000.0	3.92

lignes envoyées : 5921 sur 6000 | lignes retirées (groupes de moins de 5) : 79

Le fichier d'envoi compte 5 921 lignes sur 6 000 : les 79 clients des groupes trop petits ont été retirés. Il n'a plus ni nom, ni e-mail, ni identifiant interne, ni ville, ni année de naissance ; les quatre quasi-identifiants restants sont généralisés. Relisez-le avec les trois questions de 5.2.5 : on **ne peut**

plus isoler un client (le plus petit groupe compte 5 personnes) ; on ne peut **recouper** qu'avec des informations déjà très générales ; reste l'**inférence**, qu'on a mesurée en 5.3.6. Ce n'est pas un fichier anonyme au sens strict (une personne qui connaîtrait la clé retrouverait les pseudonymes), mais le risque est **fortement réduit** et la démarche est **documentée**.

En pratique. Gardez le **code** qui prépare le fichier d'envoi (comme la fonction ci-dessus) plutôt que le fichier lui-même : si les données changent, on refait l'envoi à l'identique, et la façon dont il a été fabriqué fait partie de la documentation du chapitre 4. Notez la valeur de *k*, la clé utilisée (son nom, pas sa valeur), la date, le destinataire et la finalité.

5.4.2 Les petits effectifs : ne pas publier un groupe de trois personnes

La protection ne concerne pas que les fichiers de lignes : un **tableau de synthèse** peut aussi exposer des personnes quand une cellule compte très peu d'individus. « Deux clientes de la Ville E sans carte de fidélité achètent par les Réseaux » est un renseignement presque nominatif pour qui connaît la ville. La règle courante est de **masquer ou regrouper les cellules d'effectif inférieur à un seuil** (5 ou 10 selon les organisations).

Prenons le nombre de clients par ville, canal d'acquisition et carte de fidélité : $20 \times 3 \times 2 = 120$ cellules.

```
ct = pd.crosstab(x["ville"], [x["canal_acquisition"], x["fidelite"]])
petites = ct < 5
print("cellules :", ct.size, "| cellules de moins de 5 clients :", int(petites.sum().sum()), "| plus
↳ petit effectif :", int(ct.min().min()))
print(ct.where(~petites, "<5").loc[petites.any(axis=1)].head(3).to_string())
```

```
cellules : 120 | cellules de moins de 5 clients : 7 | plus petit effectif : 2
canal_acquisition Boutique      Réseaux      Site
fidelite              0    1          0    1      0    1
ville
Ville L                55   30         17   <5    41   20
Ville N                47   19         10   <5    41   24
Ville 0                34   22          9   <5    32   13
```

Sept cellules sont masquées, avec des effectifs aussi bas que 2. Mais **masquer ne suffit pas** si l'on publie aussi les totaux : une ligne dont on masque une seule cellule se retrouve par **soustraction**. C'est le problème de la **divulcation complémentaire**.

```
total_ligne = ct.sum(axis=1)
visible = ct.where(~petites, 0).sum(axis=1)
retrouve = (total_ligne - visible)[petites.sum(axis=1) == 1]
vrai_masque = ct.where(petites, 0).sum(axis=1)[petites.sum(axis=1) == 1]
print("lignes avec exactement une cellule masquée :", len(retrouve), "| valeurs retrouvées par
↳ soustraction :", int((retrouve == vrai_masque).sum()))
```

```
lignes avec exactement une cellule masquée : 7 | valeurs retrouvées par soustraction : 7
```

Les sept valeurs masquées se retrouvent exactement. La parade est la **suppression complémentaire** (masquer aussi une autre cellule de la ligne et de la colonne) ou, plus simplement, le **regroupement** : fusionner des modalités (deux villes voisines, deux canaux) jusqu'à ce qu'aucune cellule ne soit trop petite. Retenez la règle d'or : **ne publiez que des groupes assez gros pour que personne ne s'y reconnaisse, et vérifiez que les totaux ne trahissent pas ce que vous masquez**.

Piège. Les petits effectifs se cachent dans les **filtres**. Un tableau de bord interactif où l'on peut filtrer par ville, tranche d'âge, canal et carte descend, en quelques clics, jusqu'à un groupe

d'une personne. Si vous construisez un tableau de bord, appliquez le seuil **dans le calcul**, pas dans l'affichage.

5.4.3 Les sorties : modèles, graphiques et captures d'écran

On pense aux fichiers de données, rarement aux **sorties** de l'analyse, qui en contiennent pourtant aussi.

- Un **graphique en nuage de points** dont chaque point est un client est un fichier de données déguisé. Un graphique d'**agrégats** (moyennes par groupe) est plus sûr, **à condition** que les groupes soient assez gros et que l'effectif soit visible.
- Une **moyenne par groupe** de trois personnes est un renseignement individuel. Dans la table de la boutique, croiser la ville et la tranche de dix ans crée 150 groupes dont 29 comptent moins de 5 clients : une carte ou un classement « par ville et âge » en exposerait une partie.
- Un **modèle** peut **mémoriser** des cas particuliers : un arbre très profond ou un modèle de plus proches voisins reproduit presque des lignes d'entraînement. Partager un modèle, ses règles ou ses sorties détaillées revient parfois à partager des données ; on partage des **métriques d'ensemble** et, si besoin, un modèle simple.
- Les **captures d'écran** (un tableau Excel, une requête, un tableau de bord) contiennent souvent des noms, des e-mails ou des identifiants que l'on n'a pas vus. Relisez chaque capture avant de l'envoyer ou de la coller dans une présentation, et conservez les captures **dans le même coffre** que les données.
- Les **exports intermédiaires** (un CSV laissé sur le bureau, une feuille « test » dans un classeur, un notebook avec la sortie d'un head() du CRM) sont l'endroit où les fuites naissent le plus souvent. Un notebook partagé conserve les **sorties** de ses cellules : effacez-les ou nettoyez avant de partager.

5.4.4 Accès, journaux et conduite à tenir en cas de fuite

Quelques principes de sécurité, que vous n'avez pas à implémenter seul mais qu'il faut connaître et réclamer.

- **Moindre privilège** : chaque personne n'accède qu'aux données dont elle a besoin. L'analyse de campagne n'a pas besoin des noms ; la relation client n'a pas besoin du revenu.
- **Séparation** : l'identité, la table de correspondance et la clé ne sont **jamais** dans le même endroit que les données d'analyse (5.2.3).
- **Journaux** : savoir qui a accédé à quoi, et quand, est la condition pour enquêter après un incident ; un fichier qui circule sans trace ne se retrouve pas.
- **Transmission** : jamais de fichier de données personnelles en pièce jointe non protégée ; on utilise un espace de partage contrôlé, avec une date d'expiration, ou un fichier chiffré dont le mot de passe passe par un **autre canal**.
- **Prestataires** : un contrat ou une clause précise ce que le prestataire peut faire des données, où elles sont conservées, quand elles sont **détruites**, et s'il peut les confier à un tiers.
- **Conservation** : on supprime les copies de travail dès que la finalité est atteinte ; une extraction qui traîne depuis un an est un risque sans bénéfice.

Et si, malgré tout, **un fichier est parti à la mauvaise adresse** ou qu'un classeur a été publié par erreur ? Les cadres de protection prévoient en général une **obligation de réagir vite** (informer la personne chargée de la protection des données, parfois l'autorité et les personnes concernées, dans un délai court : à **vérifier dans votre pays**). Pour l'analyste, la conduite est simple, et se répète à l'avance :

1. **Arrêter la diffusion** (retirer l'accès, rappeler le message, révoquer le lien).

2. **Prévenir immédiatement** la personne chargée de la protection des données (ou la direction), sans attendre d'avoir « tout compris » ni de s'être fait une idée de la gravité.
3. **Décrire** ce qui est parti : quel fichier, quelles colonnes, combien de personnes, vers qui, depuis quand ; la liste des colonnes et la mesure d'unicité des sections précédentes servent ici.
4. **Conserver les preuves** (journaux, messages), sans rien effacer.
5. **Laisser décider** : c'est à la personne responsable, avec les juristes, de décider des notifications ; l'analyste fournit les faits.

5.4.5 La liste de contrôle avant d'envoyer un fichier

Voici la liste que nous vous conseillons de recopier. Elle est volontairement courte : on s'en sert vraiment si elle tient sur une page.

#	Question	Comment vérifier
1	Quelle est la finalité ? Est-elle compatible avec la collecte ?	une phrase écrite, validée par le responsable
2	Chaque colonne est-elle nécessaire ?	classement des colonnes (5.1.2), retrait des autres
3	Reste-t-il un identifiant direct (nom, e-mail, téléphone, identifiant interne) ?	relecture des noms et des contenus de colonnes
4	Les identifiants sont-ils remplacés par un pseudonyme à clé ?	pas de hachage sans clé (5.2.2)
5	Les quasi-identifiants sont-ils généralisés ?	ville, âge, date réduits
6	Quel est le k du fichier ? Combien de lignes sont uniques ?	taille_groupes, stats_k (5.3.2)
7	Y a-t-il des petits groupes dans les tableaux ? Les totaux trahissent-ils un masquage ?	seuil appliqué, divulgation complémentaire (5.4.2)
8	Un recoupement est-il possible (date et montant, par exemple) ?	test d'unicité sur les colonnes de transaction (5.3.5)
9	Les valeurs sensibles sont-elles diversifiées dans les groupes ?	l-diversité (5.3.6)
10	Qui reçoit le fichier, comment , jusqu'à quand ?	canal sécurisé, durée, clause de destruction
11	La clé et la table de correspondance sont-elles ailleurs ?	pas dans le même dossier ni le même envoi
12	Cette démarche est-elle documentée ?	note de transmission : date, destinataire, valeur de k, règle de masquage

Une partie des vérifications (3, 6) s'automatise. La fonction `controle_avant_envoi` cherche les colonnes dont le **nom** ou le **contenu** évoque un identifiant direct (e-mail, téléphone) et calcule k et le nombre de lignes uniques. Appliquons-la au CRM brut, à la table de 5.3 et au fichier d'envoi de 5.4.1.

```
cas = {"CRM brut": (crm, ["ville", "date_naissance", "code_postal"]), "table clients + profil": (x,
↪ QI), "fichier d'envoi": (envoi, 0.QI_ENVOI)}
for nom, (df, qi) in cas.items():
```

```
c = 0.controle_avant_envoi(df, qi)
print(f"{nom:24s} suspectes : {c['colonnes_suspectes']} or 'aucune' | k = {c['k_min']} | lignes
↳ uniques : {c['uniques']} | prêt à envoyer : {c['ok']}")
```

```
CRM brut          suspectes : ['id_crm', 'prenom', 'nom', 'email', 'telephone',
↳ 'date_naissance'] | k = 1 | lignes uniques : 6308 | prêt à envoyer : False
table clients + profil suspectes : ['id_client'] | k = 1 | lignes uniques : 1581 | prêt à envoyer :
↳ False
fichier d'envoi    suspectes : aucune | k = 5 | lignes uniques : 0 | prêt à envoyer : True
```

Le CRM brut échoue avec six colonnes suspectes et 6 308 lignes uniques ; la table clients + profil échoue à cause de l'identifiant interne et de ses 1 581 lignes uniques ; seul le fichier d'envoi passe. Ce contrôle automatique est un **filet**, pas un **juge** : il ne voit ni la finalité, ni le recoupement avec une source qu'il ne connaît pas, ni un identifiant déguisé sous un nom de colonne anodin. La relecture par une personne reste obligatoire.

5.4.6 La personne chargée de la protection des données

Beaucoup d'organisations désignent une personne ou une équipe chargée de la protection des données (dans certains cadres, un **délégué à la protection des données**, obligatoire selon la taille ou la nature des traitements ; le nom et les obligations varient selon les pays). Ses missions sont en général : **conseiller** sur les traitements, tenir le **registre** des traitements, être le **point de contact** des personnes et de l'autorité de contrôle, aider à traiter les **demandes de droits** et les **incidents**.

Pour l'analyste, la bonne conduite tient en trois habitudes. **Poser la question tôt** : à chaque nouveau projet, nouvel usage, nouveau destinataire externe, ou nouvelle sorte de donnée, on la consulte avant de construire, pas après. **Apporter des faits** : la liste des colonnes, l'unicité, le k, l'inventaire des copies, ce que ce chapitre apprend à produire. **Garder une trace** : les choix de minimisation, de généralisation et de seuil se notent dans la documentation du jeu de données (chapitre 4), pour pouvoir les expliquer, les contrôler et les refaire.

À retenir. Un fichier d'envoi se **fabrique** : minimiser, séparer (pseudonyme à clé), généraliser, supprimer les petits groupes, contrôler. Les tableaux ont leurs propres risques : ne publiez pas de petits effectifs et vérifiez que les totaux ne les révèlent pas. Soignez les sorties (graphiques, modèles, captures, notebooks). Prévoyez la conduite en cas de fuite avant qu'elle n'arrive. Avant tout envoi, la liste de contrôle : finalité, colonnes, identifiants, pseudonymes, k, petits groupes, recoupement, diversité, destinataire, clés, documentation.

Pour s'entraîner. Cahier, chapitre 5 : application 5.7, exercices 5.11 et 5.12.

Bilan du chapitre 5

Vous savez maintenant :

- **reconnaître une donnée personnelle** (une donnée qu'on peut rattacher à une personne avec des moyens raisonnables), **classer les colonnes** d'un fichier en identifiants directs, quasi-identifiants et données sensibles, et énoncer les **principes communs** des cadres de protection : finalité, minimisation, base légale, exactitude, conservation limitée, sécurité, droits des personnes ;
- **voir la protection des données comme un problème de qualité** : un consentement écrit de sept façons, des lignes contradictoires pour un même client, des doublons qui font échouer un effacement (371 clients sur 950 entièrement retrouvés par l'e-mail exact) ;
- **pseudonymiser correctement** : pseudonymes aléatoires, table de correspondance gardée à part, hachage à clé plutôt que hachage nu (le hachage sans clé a laissé retrouver plus de 85 % des e-mails et tous les numéros de client), et savoir que **pseudonymiser n'est pas anonymiser** ;
- **choisir et chiffrer une technique de protection** (suppression, masquage, généralisation, bruit, synthèse) en **mesurant ce qu'elle fait perdre** (une corrélation de 0,36 réduite à 0,33 par le bruit, annulée par une synthèse colonne par colonne) ;
- **mesurer un risque de réidentification** : unicité (un client sur quatre est unique avec quatre colonnes banales), **k-anonymat** (taille du plus petit groupe), **recoupement** (une date et un montant retrouvent 97 % des commandes), **l-diversité** (un groupe homogène trahit) ;
- **ramener un fichier à $k = 5$** par généralisation et suppression de quelques lignes rares (1,3 % ici), et dire ce que l'on perd (la géographie fine) ;
- (en option, dans ce chapitre) comprendre la **confidentialité différentielle** : bruit de Laplace d'échelle $1/\epsilon$, erreur relative d'autant plus grande que le groupe est petit, attaque par différence ;
- **préparer un fichier d'envoi, ne pas publier de petits effectifs** (et vérifier que les totaux ne les révèlent pas), soigner les sorties, savoir quoi faire en cas de fuite, et **dérouler la liste de contrôle** avant tout envoi.

Le chapitre a mis des chiffres sur des craintes que l'on garde d'habitude vagues :

Question	Ce que nous avons mesuré
Combien de clients sont uniques sur ville, année de naissance, canal et carte ?	1 581 sur 6 000 (26,4 %) ; $k = 1$
Combien de clients sont dans un groupe de moins de 5 ?	4 514 (75,2 %)
Que coûte le passage à $k = 5$ (région, tranche de dix ans, canal, carte) ?	79 lignes supprimées (1,3 %) ; corrélation âge-revenu $0,360 \rightarrow 0,354$; écart géographique de revenu $7\,638\,€ \rightarrow 3\,854\,€$
Un hachage sans clé protège-t-il les e-mails ?	5 142 sur 6 000 retrouvés (85,7 %) ; avec une clé : 0
Une date et un montant d'une commande du site suffisent-ils ?	97,2 % des commandes sont uniques ; 90,5 % avec le montant arrondi à l'euro
Un groupe de 5 peut-il trahir ?	13 groupes sur 136 sans aucun insatisfait ; un groupe de 7 avec 43 % d'insatisfaits (15 % en général)
Que fait un bruit de Laplace sur un comptage de 5 ?	environ 14 % d'erreur pour $\epsilon = 1$, plus de 100 % pour $\epsilon = 0,1$
Masquer les petites cellules suffit-il ?	non : 7 valeurs masquées sur 7 retrouvées par soustraction des totaux

Le fil conducteur du chapitre tient en une phrase : **retirer les noms ne protège pas, c'est la combinaison des caractéristiques qui identifie, et toute protection se mesure et se paie**. L'analyste n'a pas à trancher les questions de droit, mais il doit **apporter les faits** (colonnes, unicité, k , copies) qui permettent à d'autres de trancher, et **documenter** ses choix, comme le chapitre 4 l'enseigne pour tout le reste.

Rappel. Ce chapitre ne remplace pas un conseil juridique. Les règles de protection des données dépendent du pays et évoluent : les seuils ($k = 5$), les délais (notification d'une fuite) et les obligations (désignation d'une personne chargée de la protection) donnés ici sont des **exemples** et des ordres de grandeur, à vérifier dans votre contexte.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 à 5.7 (classer les colonnes et normaliser le consentement, attaque par dictionnaire et hachage à clé, coût du bruit et de la synthèse, unicité et k -anonymat, recoupement par date et montant, l -diversité et bruit de Laplace, fichier d'envoi et petits effectifs) et exercices 5.1 à 5.12.

Ce chapitre complémentaire prolonge les quatre premiers du volume (nettoyer, transformer, contrôler, documenter) : une fois les données propres et documentées, il reste à décider **à qui on les montre, et sous quelle forme**.

Points clés

« Un chiffre propre n'est pas un chiffre corrigé : c'est un chiffre dont on connaît chaque transformation. »

Ce court chapitre fixe l'essentiel du volume, chapitre par chapitre, puis les idées qui les relient. Le **projet du volume** (nettoyer et réconcilier la caisse et le site en une table fiable) et une **auto-évaluation** de quarante questions se trouvent dans le cahier d'exercices.

Pour s'entraîner. Cahier, projet du volume et auto-évaluation : dix étapes, de l'inventaire des sources à la décision de livrer, une variante sur le dédoublonnage du CRM, puis quarante questions avec corrigés.

Les points clés, chapitre par chapitre

Chapitre	Ce que vous devez emporter
1. Nettoyage des données	Une absence a un mécanisme (MCAR, MAR, MNAR) qui décide de ce qu'une suppression déforme ; un zéro, un vide et un code (9999, « ND ») sont trois choses différentes. Une valeur aberrante n'est pas toujours une erreur : la règle métier repère mieux que la distance à la moyenne. Des lignes identiques ne sont pas toutes des doublons. Les formats et unités changent en cours de route (le passage aux centimes du 15 septembre, la dérive de schéma de la caisse). ☉ Texte, accents, mojibake , encodage ; aucune imputation ne répare un MNAR.
2. Transformation et fusion	Une variable dérivée est une décision (à documenter). Avant et après une jointure, on compte : une clé qui n'est pas unique (deux produits de même nom) multiplie les lignes. On empile des fichiers avec une fonction de lecture paramétrée, on change de grain avec une somme de contrôle. ☉ Passage du large au long ; appariement approximatif : normaliser, mesurer, bloquer, trois zones (accepter, revoir, rejeter), et le coût des erreurs décide du seuil.

Chapitre	Ce que vous devez emporter
3. Qualité et réconciliation	La qualité se mesure par dimensions (complétude, validité, unicité, cohérence, exactitude, actualité) pour un usage . Un contrôle est une règle qui renvoie ses échecs . Réconcilier : comparer les effectifs , comparer les totaux , expliquer l'écart jusqu'à zéro ; la définition du chiffre (annulations comprises ?) est une décision. ⊗ Tolérances, rapport d'exceptions trié par montant ; pandera et Great Expectations pour un pipeline récurrent.
4. Documentation	Pas de confiance sans trace : fiche du jeu de données, journal des décisions avec effectifs avant et après, dictionnaire de données testé contre le fichier réel, définitions uniques dans un glossaire (« client actif »). Refaire depuis le brut est la meilleure documentation. ⊗ Lignage ; une empreinte prouve l'identité d'un fichier, pas sa justesse.
⊗ 5. Confidentialité	Identifiants directs, quasi-identifiants , données sensibles ; finalité, minimisation, conservation, droits. Un hachage sans clé se retrouve par dictionnaire ; pseudonymiser n'est pas anonymiser . Le k-anonymat mesure le risque (un quart de clients uniques sur quatre colonnes), la généralisation le réduit au prix d'une perte d'information ; masquer de petits effectifs ne sert à rien si l'on publie les totaux.
Projet (cahier)	Inventaire → lecture paramétrée de douze fichiers → nettoyage de la caisse (montants vides, doublons de scan prouvés par la base) → nettoyage du site (doublons, tests, annulations, unités) → réconciliation avec la base et contrôle de couverture (un canal absent) → table unique → contrôles → journal et dictionnaire → décision, puis ouverture de la vérité.

Six idées qui traversent tout le volume

Les six idées.

1. **Une donnée est propre pour un usage.** Aucun nettoyage n'est neutre : chaque règle (supprimer, corriger, imputer, fusionner) répond à une question et s'écrit.
2. **Compter avant et après.** Une jointure, un filtre, un dédoublonnage : si l'on ne sait pas combien de lignes ont bougé, on ne sait pas ce qu'on a fait.
3. **Le total de contrôle est le meilleur ami de l'analyste.** Une source de référence (une base, un total imprimé) transforme « ça a l'air juste » en « l'écart est de 0,00 € ».
4. **Ne jamais supprimer ce qu'on ne peut pas prouver.** Un drop_duplicates aveugle a supprimé 87 vraies ventes ; la règle prudente a retiré 67 doubles sur 68.
5. **Une décision de définition n'est pas un écart de données.** Les commandes annulées, le TTC et le HT, le « client actif » : on tranche, on écrit, on s'y tient.
6. **Tout se rejoue et tout se documente.** Un script depuis le brut, un journal, un dictionnaire à jour : la transmission fait partie du travail.

Et maintenant ?

Vous savez maintenant lire des sources désordonnées, les nettoyer, les fusionner, les contrôler et les documenter, et vous savez où s'arrête ce que l'on peut prouver. Le volume III passe à l'analyse : exploration, tests d'hypothèses, tests A/B, régression, segmentation, séries temporelles et indicateurs. Pour vous entraîner d'ici là, reprenez le projet avec le canal Réseaux, que les deux exports ne couvrent pas.

À retenir, tout simplement. Préparer les données, c'est rendre chaque chiffre explicable : d'où il vient, ce qu'on lui a fait, de combien il peut se tromper et ce qu'il ne dit pas.